



Team #3
AEROAUTOMATORS

Task 4

Autopilot

Name

Section/B.N

Ahmed Essam Mohammed Ammar

1/9

Mohammed Zakaria Sayed Sayed

2/16

Mohammed Shehta Gharib Mohammed

2/18

Mahmoud Mohamed Sief El-Nasr

2/24

Mohammed Jamal Abd-Elhakim Aljendy

2/9 | Team Leader



Dr. Osama Saaid Mohamady Abdel Razek
T.A Mahmoud GamalEldin Mohammad Elewa



Mar. 17, 2023



StelaXVision@gmail.com

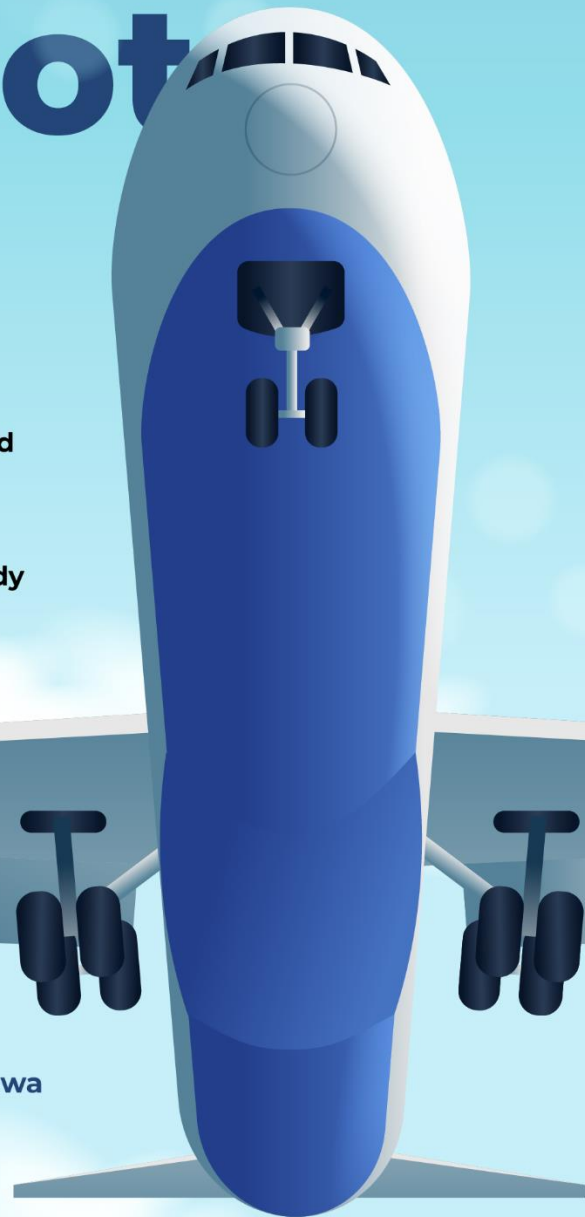


Table of Contents

Research Questions.....	4
a) “What is an Autopilot?”, i.e. define its main objective.....	4
b) When were the first autopilots invented, and what was their function?	5
c) What are the inputs & outputs of the Autopilot system onboard an airplane?	6
d) What would be the role of the pilot in an airplane equipped with an autopilot?	7
e) What is the difference between an Autopilot & SAS (stability augmentation system)?	8
f) What is the role of the onboard sensors like (GPS, gyroscopes, ..etc.)? i.e. Mention the sensors used by the autopilot, what they measure, and their typical sample rates.....	10
g) What can we do if we need to know a state (i.e. to use it in the controller) if it is not directly measured by a sensor?.....	11
h) What is a fly-by-wire flight control system?	12
i) Find 2 open-source autopilots that can be used on UAVs	13
Flight Mechanics Review	14
a) State the general rigid body dynamics (RBD) equations in 3D space	14
b) Classify the above equations into (Kinetics & Kinematics) equations.....	15
c) What are the assumptions introduced while deriving those equations?.....	15
d) State the set of equations added to the (RBD) equations to form the Fixed wing Airplanes (EOM)	16
e) Classify the airplane EOM equations mathematically	16
f) What is the difference between the (Body axes) and the (earth or inertial axes)?.....	17
g) What is the difference between the pitch angle (θ) and the angle of attack (α), and between the sideslip angle (β) and the heading angle (ψ)?	18
h) There are different attitude representations other than Euler angles such as Direction Cosine Matrix, quaternions, and axis-angle representation. Explain them, and tell what're their advantages/disadvantages	20
Numerical Solution of ODEs.....	22
a) State some of the numerical solving algorithms for ODEs. Showing the advantages and disadvantages of each.....	22
b) Choose one algorithm for solving the Airplane EOM, and clearly state the (Initial conditions needed, Inputs needed in each iteration, and Outputs calculated in each iteration)	3
c) Solve the system of first-order ODEs	3
Airplane Simulator Part I: RBD Equations Solver	5
a) Introduction.....	5
b) RBD Equations	6
c) Runge-Kutta Method, Inputs, Initial conditions, Outputs & Procedure	7
d) Validation.....	4

e) Results	5
f) Conclusion	12
g) Bonus	13
Airplane Simulator Part II.....	15
a) Introduction.....	15
b) Airplane forces and moments calculation.....	16
c) Trim Conditions	18
d) Body axes of the airplane.....	19
e) Simulink Schematic	22
f) Benchmark test (B747-F5).....	24
g) Our Airplane (JETSTAR-F3).....	33
References.....	78
Appendix.....	117

Research Questions

a) “What is an Autopilot?”, i.e. define its main objective

An autopilot is a system designed to assist pilots in controlling an aircraft by automating some or all of the flight control functions. Its main objective is to maintain a predetermined flight path by controlling and stabilizing the aircraft's speed, altitude, and direction. The autopilot continuously receives data from various sensors such as GPS, airspeed indicators, altimeters, and gyroscopes, and uses this information to adjust the aircraft's control surfaces, including the ailerons, elevators, and rudder. Autopilots can also perform other functions such as takeoff and landing and handling emergencies such as engine failure or loss of cabin pressure. [1]

The primary goal of an autopilot system is to reduce the workload on the pilots and improve the safety and efficiency of flight operations by ensuring that the aircraft stays on the desired flight path. Autopilots are particularly useful for long-haul flights, where pilots may become fatigued or distracted by other tasks. It's important to note that the autopilot is not a substitute for human pilots, as they are still responsible for monitoring the system and taking control of the aircraft if necessary. With the assistance of an autopilot, pilots can concentrate on other essential tasks, such as communicating with air traffic control, monitoring weather conditions, and maintaining situational awareness, making flights safer and more efficient. Autopilots are a key component of aircraft safety, as they help to ensure that the aircraft is operating within its design limits at all times. [1]

b) When were the first autopilots invented, and what was their function?

The first autopilots were invented in the early 20th century, with the first successful autopilot flight taking place in 1912 [1]. One of the earliest known autopilot systems was developed by the Sperry Corporation in the same year for use in a Curtiss C-2 biplane. The system used gyroscopes to maintain the aircraft's altitude and direction and was controlled by a pendulum that was connected to the aircraft's control surfaces [1][3]. This early form of autopilot allowed aircraft to fly straight and steady without requiring the pilot's undivided attention, reducing fatigue during long flights [2].

In the 1930s, autopilots became more advanced with the introduction of new technologies such as radio navigation aids and electronic instruments [1]. These advancements allowed autopilot systems to maintain more precise flight paths, and also enabled the development of automatic landing systems. Early autopilots were only able to maintain a constant heading and altitude, but modern autopilots are capable of controlling every part of the flight envelope from just after take-off to landing.

During World War II, autopilots became an essential part of military aviation, with many aircraft being equipped with advanced autopilot systems that could handle complex flight maneuvers [1]. Since then, more improvements have been made to the autopilot, such as improved control algorithms and hydraulic servomechanisms [1][3].

Today, autopilots are an integral part of modern aviation and are used in a wide range of aircraft. They have evolved to include advanced features such as GPS navigation, flight management systems, and automated collision avoidance systems.

c) What are the inputs & outputs of the Autopilot system onboard an airplane?

The autopilot system onboard an airplane is a complex system that takes input from various sensors and avionics systems to determine the current state of the aircraft and outputs control commands to the aircraft's control surfaces and engines to maintain a predetermined flight path. The inputs to the autopilot system include attitude sensors, airspeed sensors, altitude sensors, navigation systems, and control surface position sensors. These sensors measure the pitch, roll, and yaw of the aircraft, and provide information about the aircraft's speed, height above the ground, position, heading, and the current state of the aircraft's control surfaces [4].

The outputs of the autopilot system consist of flight control servos, engine control systems, and displays. The autopilot system sends control commands to the aircraft's servos, which move the control surfaces to maintain the desired flight path. It can also control the aircraft's engines to maintain a specific airspeed or altitude. Additionally, the autopilot system provides information to the pilots through various displays, such as the primary flight display (PFD) and the multifunction display (MFD) [4].

Overall, the inputs and outputs of the autopilot system work together to maintain a safe and stable flight path for the aircraft [4].

d) What would be the role of the pilot in an airplane equipped with an autopilot?

Autopilots can do almost everything a pilot can do, except for takeoff, steering the airplane on the ground, and taxiing at the gate [5]. In some newer aircraft models, autopilot systems can even land the plane [5]. Though the role of the pilot in an airplane equipped with an autopilot system is critical to ensuring the safety of the flight.

Side Note: Designing an autopilot system for takeoff would require handling a high level of turbulence and real-time adjustments, making it more costly than its potential advantages. Therefore, the pilot's decision to take control during this phase is a safer and more practical option.

Firstly, the autopilot system is capable of controlling many aspects of the flight, such as maintaining the altitude, speed, and heading of the aircraft [5][6]. However, it cannot replace the decision-making abilities of the pilot. The pilot is responsible for monitoring the autopilot's performance, making critical decisions, and taking manual control of the aircraft if necessary [5][6][7]. The pilot must also be well-versed in the operational details of the plane to make any necessary adjustments to the autopilot's settings or to implement a different autopilot system in the event of a malfunction [5][6][7].

Secondly, the pilot is responsible for navigating the aircraft, controlling its desired speed and altitude, and communicating with air traffic control [5][6]. The pilot may also be responsible for programming the autopilot, selecting navigation routes, and making adjustments as needed [5][6][7].

Finally, the pilot must ensure that all safety protocols are followed and be aware of any limitations of the autopilot system [5][6][7]. In emergency situations or system malfunctions, the pilot must take over the controls, analyze the situation, and take appropriate action to ensure the safety of the flight [5][7].

In summary, an autopilot system is a valuable tool for the pilot to manage the aircraft, but it cannot replace the pilot's decision-making abilities and situational awareness. The pilot's role is to monitor the autopilot system, maintain situational awareness, and make critical decisions based on the information available. The pilot's role is crucial to ensuring that the aircraft operates safely and efficiently [5][6][7].

e) What is the difference between an Autopilot & SAS (stability augmentation system)? ^{[8][9][10][11]}

- **Definitions:** Autopilot is an automatic flight control system that can follow a pre-programmed flight plan or pilot-inputted instructions. ^{[1][10]}. On the other hand, SAS is designed to provide short-term attitude and attitude rate stabilization for use during hands-on flying or autopilot flying ^{[8][10]}.

- Let's start our answer with a bird's-eye view of the **difference in the mechanics of working** for autopilot and SAS systems:

Autopilot:

1. The autopilot system receives input signals from various sensors on the aircraft, including GPS, airspeed sensors, altimeters, and attitude and heading reference systems (AHRS).
2. The autopilot system uses these inputs to determine the current position, speed, altitude, and orientation of the aircraft, as well as the desired flight path.
3. The autopilot system then uses sophisticated control algorithms to adjust the aircraft's control surfaces, such as the ailerons, elevators, and rudder, to maintain the desired flight path and flight parameters.
4. The autopilot system continually adjusts the control surfaces to ensure that the aircraft stays on the desired flight path, and can switch between different control modes, such as altitude hold, heading hold, and airspeed hold, depending on the pilot's inputs and flight conditions.

Stability Augmentation System (SAS):

1. The SAS system receives input signals from sensors on the aircraft, such as accelerometers and gyroscopes, that detect the aircraft's motion and orientation.
2. The SAS system uses these inputs to determine the aircraft's stability and any deviations from the desired flight path.
3. The SAS system then makes small adjustments to the aircraft's control surfaces, such as the ailerons, elevators, or rudder, to counteract any disturbances that could cause the aircraft to deviate from its desired flight path. A SAS uses a limit function to restrict movement of the total surface deflection and a rate limit function to contain transient response by imposing a maximum actuator slew rate demand ^[9].
4. The SAS system continually monitors the aircraft's stability and makes adjustments as needed to maintain stable flight, but does not actively control the aircraft's flight path or parameters in the same way that an autopilot system does.

The differences:

1. Functionality: Autopilot systems are designed to automate the control of the aircraft and can manage various aspects of flight, such as altitude, airspeed, heading, and navigation. They usually have multiple control modes, including altitude hold, heading hold, and airspeed hold. In contrast, SAS is primarily focused on improving the stability and handling of the aircraft, and it provides stability augmentation in different flight conditions. While autopilot systems control the aircraft, SAS stabilizes it.

2. Inputs: Autopilot systems use inputs from the aircraft's navigation and flight control systems, such as GPS, airspeed sensors, altimeters, Weather Radar systems, Inertial Navigation Systems (INS), and attitude and heading reference systems (AHRS). These inputs are used to determine the aircraft's current position, speed, altitude, and orientation, as well as the desired flight path. While SAS systems use inputs from sensors that detect the aircraft's motion and orientation, such as accelerometers and gyroscopes that measure changes in the aircraft's pitch, roll, and yaw movements, Angle of Attack (AoA) sensors that measure the angle between the aircraft's wings and the oncoming airflow, helping to prevent stalls, and Airspeed indicators that measure the aircraft's speed relative to the air. These sensors provide information about the aircraft's stability and any deviations from the desired flight path. The differences in the sensors used by the autopilot and SAS systems reflect the different functions and priorities of each system. Autopilot systems are designed to provide precise control over the aircraft's flight path, while SAS systems are designed to provide stability and prevent deviations from the desired flight path.

3. Outputs: Autopilot systems provide outputs to the aircraft's flight control surfaces, such as the ailerons, elevators, and rudder. These outputs are used to control the aircraft's heading, altitude, airspeed, and other parameters. While SAS systems also provide outputs to the aircraft's flight control surfaces, but these outputs are focused on providing stability augmentation. For example, SAS systems may make small adjustments to the ailerons, elevators, or rudder to counteract turbulence or other disturbances that could cause the aircraft to deviate from its desired flight path.

4. Operation Phases: Autopilot systems are primarily used during the cruise phase of flight, but can be disengaged by the pilot if they need to make a manual adjustment, such as changing altitude to avoid turbulence. However, during takeoff and sometimes during descent and approach, autopilot is turned off. On the other hand, SAS is typically engaged throughout the entire flight, from takeoff to landing. It provides continuous stability augmentation to ensure a smooth and controlled flight. However, there may be situations where the SAS is disengaged if the pilot needs to make a manual adjustment to the aircraft's stability.

5. Control strategy: Autopilot systems use a closed-loop control strategy, which means that they continuously measure the aircraft's actual state and compare it to the desired state, and then adjust the control outputs to minimize the error between the two. This requires sophisticated control algorithms and feedback loops to ensure that the aircraft follows the desired flight path accurately. In contrast, SAS systems typically use a simpler open-loop control strategy, where the control outputs are pre-programmed based on the expected behavior of the aircraft in different flight conditions.

In summary, the main difference between Autopilot and SAS is that autopilot systems are mainly used to automate the control of the aircraft, while SAS is designed to improve the stability and handling of the aircraft, especially in turbulent conditions or during maneuvers [8][10].

f) What is the role of the onboard sensors like (GPS, gyroscopes, ..etc.)? i.e. Mention the sensors used by the autopilot, what they measure, and their typical sample rates ^{[12][13][14][15]}

The onboard sensors used by an autopilot system play a critical role in determining the aircraft's position, attitude, and velocity. These sensors provide the necessary data to the autopilot system to help it make the necessary control adjustments to maintain the desired flight path. Here are some of the sensors commonly used in autopilot systems, along with their functions, what they measure, and their typical sample rates:

- 1. GPS (Global Positioning System)** - GPS is a satellite-based navigation system that provides precise location information. Autopilot systems use GPS to determine the aircraft's position, altitude, speed and time. GPS provides a sample rate of up to 10 Hz or more, depending on the type of GPS unit used.
- 2. Gyroscopes** - Gyroscopes are used to measure the aircraft's orientation or attitude. They measure the aircraft's pitch, roll, and yaw rates, which are used to stabilize the aircraft and maintain its orientation. The sample rates for gyroscopes typically range of 1000 Hz or more, depending on the sensitivity and accuracy of the sensors.
- 3. Accelerometers** - Accelerometers are used to measure the aircraft's linear acceleration. They are typically used in conjunction with gyroscopes to provide more accurate measurements of the aircraft's motion. Accelerometers typically have a sample rate of 1000 Hz or more.
- 4. Airspeed sensors** - Airspeed sensors are used to measure the aircraft's speed relative to the air around it. Autopilot systems use this data to maintain a constant airspeed. Airspeed sensors typically have a sample rate up to 100 Hz or more.
- 5. Altitude sensors (Altimeters)** - Altimeters are used to measure the aircraft's altitude above sea level. Autopilot systems use this data to maintain a constant altitude. Altitude sensors typically have a sample rate up to 100 Hz or more.
- 6. Magnetometers** - Magnetometers are used to measure the Earth's magnetic field, which is used by autopilot systems to determine the aircraft's heading. Magnetometers typically have a sample rate up to 100 Hz or more.

In conclusion, the onboard sensors used by an autopilot system play a critical role in ensuring the aircraft's stability and maintaining its desired flight path. These sensors provide accurate and precise measurements of the aircraft's position, attitude, and velocity, which the autopilot system uses to make the necessary control adjustments. The sample rates of these sensors vary depending on their type and sensitivity. However, among the various sensors used in autopilot systems, gyroscopes and accelerometers typically have the highest sample rates, ranging from 1000 Hz or more. On the other hand, GPS typically has a lower sample rate of up to 10 Hz or more. Airspeed sensors, altitude sensors, and magnetometers typically fall in between with sample rates up to 100 Hz or more.

g) What can we do if we need to know a state (i.e. to use it in the controller) if it is not directly measured by a sensor? ^{[16][17][18][19][20]}

When controlling an aircraft, it may be necessary to know the state of the aircraft, even if it is not directly measured by a sensor. In such cases, state estimation or observer design techniques can be used to estimate the values of the states that are not directly measured.

State estimation involves using available sensor measurements and mathematical models of the aircraft to estimate the required states. Several techniques are available for state estimation, including **Kalman filters, extended Kalman filters, and nonlinear observers**. These techniques use a mathematical model of the aircraft dynamics and the available sensor measurements to estimate the states in real-time.

The estimated states can then be used as feedback signals in the controller to improve the aircraft's performance and stability. State estimation is widely used in aircraft control systems, especially in autopilot systems, where the accurate estimation of the aircraft's states is essential for maintaining a safe and stable flight.

The Kalman filter is an algorithm used for state estimation that produces estimates of unknown variables based on multiple measurements over time. The filter uses a system's dynamic model, known control inputs to that system, and multiple sequential measurements to form an estimate of the system's varying quantities (its state) that is better than the estimate obtained by using only one measurement. The extended Kalman filter is a variant of the Kalman filter that is computationally cheaper and more efficient in dealing with nonlinear models. Nonlinear observers are also used in cases where the system is nonlinear and the measurements are noisy. Each method has its own strengths and weaknesses, and their suitability should be carefully considered before implementation. The suitability of which filter to use depends on the non-linearity indices of the process and observation model.

It is important to note that the accuracy of the state estimation depends on the quality of the sensor measurements and the mathematical model used for the estimation. Therefore, the design of the state estimation algorithm should take into consideration the sensor characteristics and the model uncertainties to ensure estimation accuracy and reliability.

h) What is a fly-by-wire flight control system? ^{[21][22][23]}

A fly-by-wire (FBW) flight control system is a type of aircraft control system that uses electronic signals to transmit control commands from the pilot's inputs to the aircraft's control surfaces, rather than the traditional mechanical or hydraulic systems. The system consists of three parts: sensors that convert the physical pilot control movements, computers that process the signals from the sensors, and actuators that move the aircraft's control surfaces.

In a fly-by-wire system, the pilot's inputs are detected by sensors, which are then transmitted to a flight control computer that interprets and processes the signals before transmitting them to the appropriate control surface actuators. The computer monitors sensors throughout the aircraft to make automatic adjustments to forces as they fly, creating a closed feedback loop.

The fly-by-wire system allows for greater flexibility in aircraft design, as it can be used to control a variety of different aircraft configurations, from fighter jets to commercial airliners. It also allows for more precise and efficient control of the aircraft, as the electronic signals can be processed and adjusted more quickly and accurately than mechanical or hydraulic systems.

In addition, fly-by-wire systems provide a level of automation that can enhance safety by limiting the ability of the pilot to make dangerous or extreme maneuvers, and by providing built-in protections against stalls, overspeeds, and other hazardous flight conditions. However, these systems can also be complex and may require significant training for pilots to operate effectively.

Overall, fly-by-wire flight control systems represent a significant technological advancement in aircraft control and have become increasingly common in modern aircraft designs.

i) Find 2 open-source autopilots that can be used on UAVs. ^{[24][25]}

There are several open-source autopilot systems available for use on unmanned aerial vehicles (UAVs). Here are two examples:

1. ArduPilot: ArduPilot is a popular open-source autopilot system that can be used on a wide range of UAVs, including fixed-wing aircraft, multirotor drones, and even boats. It is based on Arduino hardware and can be customized for specific applications using its open-source code. ArduPilot provides a range of features, including autonomous mission planning, object avoidance, and support for various sensors.

2. PX4: PX4 is another popular open-source autopilot system that is used on a wide range of UAVs, including drones, VTOLs, and even air taxis. It is based on NuttX RTOS and provides a range of features, including autonomous flight, advanced navigation, and support for various sensors. PX4 also provides a flexible software architecture that allows developers to customize it for specific applications.

Both ArduPilot and PX4 are widely used in the UAV industry and have active developer communities that continue to improve and expand their features.

Flight Mechanics Review

a) State the general rigid body dynamics (RBD) equations in 3D space ^[26]

The general rigid body dynamics (RBD) equations describe the motion of a rigid body in three-dimensional space.

The equations can be expressed in scalar form as follows:

Kinetics Equations

$$\begin{aligned} X - mg \sin \theta &= m(\ddot{u} + q\dot{w} - r\dot{v}) \\ Y + mg \cos \theta \sin \phi &= m(\ddot{v} + r\dot{u} - p\dot{w}) \\ Z + mg \cos \theta \cos \phi &= m(\ddot{w} + p\dot{v} - q\dot{u}) \end{aligned}$$

$$\begin{aligned} L &= I_x \ddot{p} - I_{yz}(q^2 - r^2) - I_{zx}(\dot{r} + pq) - I_{xy}(\dot{q} - rp) - (I_y - I_z)qr \\ M &= I_y \ddot{q} - I_{zx}(r^2 - p^2) - I_{xy}(\dot{p} + qr) - I_{yz}(\dot{r} - pq) - (I_z - I_x)rp \\ N &= I_z \ddot{r} - I_{xy}(p^2 - q^2) - I_{yz}(\dot{q} + rp) - I_{zx}(\dot{p} - qr) - (I_x - I_y)pq \end{aligned}$$

Kinematics Equations

$$\begin{aligned} \dot{\phi} &= p + (q \sin \phi + r \cos \phi) \tan \theta \\ \dot{\theta} &= q \cos \phi - r \sin \phi \\ \dot{\psi} &= (q \sin \phi + r \cos \phi) \sec \theta \end{aligned}$$

Path Equations

$$\begin{aligned} \dot{x}_E &= u \cos \theta \cos \psi + v(\sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi) \\ &\quad + w(\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ \dot{y}_E &= u \cos \theta \sin \psi + v(\sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi) \\ &\quad + w(\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ \dot{z}_E &= -u \sin \theta + v \sin \phi \cos \theta + w \cos \phi \cos \theta \end{aligned}$$

where:

X, Y, Z are total forces acting on the body along body axes

L, M, N are the total torques acting on the body

u, v, w are the velocities along body axes

$\dot{u}, \dot{v}, \dot{w}$ are the accelerations along body axes

p, q, r are the angular velocities along body axes

$\dot{p}, \dot{q}, \dot{r}$ are the angular accelerations along body axes

I_x, I_y, I_z are the body's inertia, which describes the body's resistance to rotation

x_E, y_E, z_E is CG position relative to Earth axes

ψ, θ, ϕ are yaw, pitch, and roll angles (body's orientation angles)

$\dot{\psi}, \dot{\theta}, \dot{\phi}$ are yaw, pitch and roll rates

m is the mass of the body

b) Classify the above equations into (Kinetics & Kinematics) equations ^[26]

In physics, the terms "kinetic" and "kinematic" refer to different concepts related to motion. Kinematics is the study of motion without considering the forces causing it, while kinetics is the study of the forces causing motion. So, the first 6 Equations in the previous page are kinetics, and the next three equations are kinematics.

c) What are the assumptions introduced while deriving those equations? ^[26]

The following are some of the assumptions that are typically introduced while deriving the general RBD equations in 3D space:

- 1. Flat-Earth approximation:** The flat-Earth approximation assumes that the Earth's surface is flat, which is often reasonable for small regions of the Earth's surface. This simplifies the equations of motion for aircraft or spacecraft, as it allows them to be treated as operating in a two-dimensional plane rather than a three-dimensional space.
- 2. Body axes:** The body axes assumption assumes that the body-fixed coordinate system is aligned with the principal axes of the body. This simplifies the equations of motion, as the moment of inertia matrix can be diagonalized.
- 3. No wind:** The no-wind assumption assumes that there is no wind or other external disturbances acting on the rigid body. This simplifies the equations of motion, as it eliminates the need to consider the effects of aerodynamic forces or other environmental factors.

If these equations are being used to model the motion of a fixed-wing aircraft, then in addition to the assumptions we have already discussed, the following assumption may also be made:

- 4. Aerodynamic symmetry:** The aircraft is assumed to be symmetric about its longitudinal axis, meaning that the left and right sides of the aircraft have the same aerodynamic properties. This allows the aerodynamic forces and moments to be simplified. (x - z is a plane of symmetry $I_{xy} = I_{yz} = 0$).

It is important to note that these assumptions are often used in simplifying the equations of motion for a rigid body, but they may not always be applicable in real-world situations. For instance, the flat-Earth approximation may not be valid for long-range aircraft or spacecraft flights, and the no-wind assumption may not be valid for aircraft operating in the presence of strong winds or turbulence. In such cases, more complex equations may be required to accurately describe the motion of the rigid body.

d) State the set of equations added to the (RBD) equations to form the Fixed wing Airplanes (EOM) ^[27]

The equations added to the Rigid Body Dynamics (RBD) equations to form the Equations of Motion (EOM) for a fixed-wing airplane are known as the Aerodynamic Forces and Moments equations. These equations describe the aerodynamic forces and moments acting on an aircraft due to its motion through the air.

$$\begin{aligned}
 \Delta X &= \frac{\partial X}{\partial u} \Delta u + \frac{\partial X}{\partial w} \Delta w + \frac{\partial X}{\partial \delta_e} \Delta \delta_e + \frac{\partial X}{\partial \delta_T} \Delta \delta_T \\
 \Delta Y &= \frac{\partial Y}{\partial v} \Delta v + \frac{\partial Y}{\partial p} \Delta p + \frac{\partial Y}{\partial r} \Delta r + \frac{\partial Y}{\partial \delta_r} \Delta \delta_r \\
 \Delta Z &= \frac{\partial Z}{\partial u} \Delta u + \frac{\partial Z}{\partial w} \Delta w + \frac{\partial Z}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial Z}{\partial q} \Delta q \\
 &\quad + \frac{\partial Z}{\partial \delta_e} \Delta \delta_e + \frac{\partial Z}{\partial \delta_T} \Delta \delta_T \\
 \Delta L &= \frac{\partial L}{\partial v} \Delta v + \frac{\partial L}{\partial p} \Delta p + \frac{\partial L}{\partial r} \Delta r + \frac{\partial L}{\partial \delta_r} \Delta \delta_r + \frac{\partial L}{\partial \delta_a} \Delta \delta_a \\
 \Delta M &= \frac{\partial M}{\partial u} \Delta u + \frac{\partial M}{\partial w} \Delta w + \frac{\partial M}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial M}{\partial q} \Delta q \\
 &\quad + \frac{\partial M}{\partial \delta_e} \Delta \delta_e + \frac{\partial M}{\partial \delta_T} \Delta \delta_T \\
 \Delta N &= \frac{\partial N}{\partial v} \Delta v + \frac{\partial N}{\partial p} \Delta p + \frac{\partial N}{\partial r} \Delta r + \frac{\partial N}{\partial \delta_r} \Delta \delta_r + \frac{\partial N}{\partial \delta_a} \Delta \delta_a
 \end{aligned}$$

where:

$\partial X / \partial u, \partial X / \partial w, \partial X / \partial \delta_e$, and $\partial X / \partial \delta_T$, called stability derivatives, that are evaluated at the reference flight condition. The variables δ_e and δ_T are the change in elevator angle and throttle setting, respectively.

e) Classify the airplane EOM equations mathematically ^[26]

Mathematically, the EOM equations for an airplane can be classified as **1st-order non-linear coupled ODEs** in the independent variable t .

f) What is the difference between the (Body axes) and the (earth or inertial axes)?

The body axes are a set of coordinates fixed to the aircraft's body, where the x-axis points out of the nose of the aircraft, the y-axis points out of the right wing, and the z-axis points out of the bottom of the aircraft. These axes move with the aircraft and are useful for describing the aircraft's attitude and movement relative to its own body [28].

On the other hand, the earth or inertial axes are a set of coordinates fixed to the earth's surface or an inertial reference frame as shown in figure 1, which is a non-accelerating frame of reference [29]. In this coordinate system, the x-axis points towards the vernal equinox, the y-axis points towards the east, and the z-axis points towards the earth's north pole. These axes do not move with the aircraft and are useful for describing the aircraft's position and movement relative to the earth or an inertial reference frame which is defined by three angles known as Euler Angles [30].

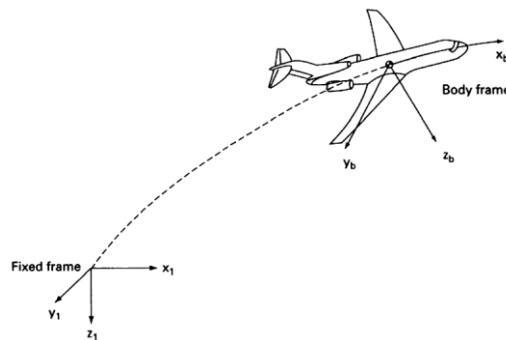


Figure 1 Inertial & Body Axes

g) What is the difference between the pitch angle (θ) and the angle of attack (α), and between the sideslip angle (β) and the heading angle (ψ)?

The pitch angle (θ) is the angle between the aircraft's longitudinal axis (i.e. the direction of travel) and its horizontal plane [31]. In other words, it describes whether the aircraft is pitched up or down relative to its direction of travel. The pitch angle can be controlled by adjusting the aircraft's elevators.

The angle of attack (α), on the other hand, is the angle between the aircraft's chord line (i.e. the line connecting the leading and trailing edges of the wing) and the relative wind [32] as shown in figure 2. This angle describes how much the air is "attacking" the wing and is an important factor in determining the lift and drag forces on the wing. The angle of attack can be controlled by adjusting the aircraft's flaps and by changing the pitch angle.

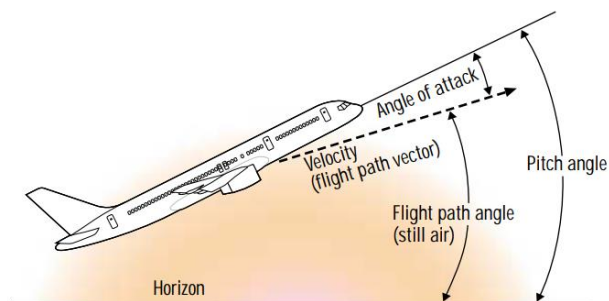


Figure 2 Pitch angle & AoA

The sideslip angle (β) is the angle between the aircraft's longitudinal axis and the relative wind in the lateral (sideways) direction [33] as shown in figure 3. In other words, it describes whether the aircraft is sliding to the left or right relative to its direction of travel. The sideslip angle can be controlled by adjusting the aircraft's rudder.

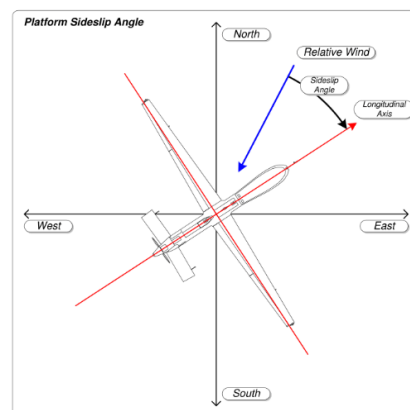


Figure 3 Sideslip angle

The heading angle (ψ), on the other hand, is the angle between the aircraft's longitudinal axis and true north [31] as shown in figure 4. It describes the direction in which the aircraft is pointing. The heading angle can be controlled by adjusting the aircraft's ailerons and by using the rudder to initiate turns.

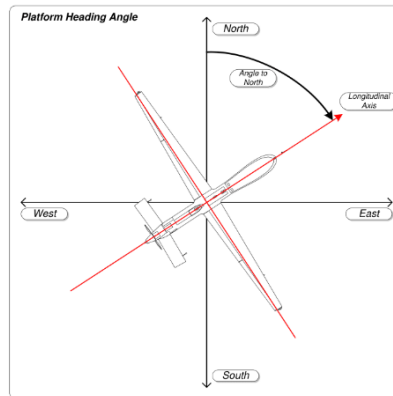


Figure 4 Heading angle

In general, a higher sideslip angle can cause the aircraft to experience a yawing motion (i.e. rotation around its vertical axis), which can be uncomfortable for passengers and can cause problems with directional stability. The heading angle is important for navigation and for maintaining the desired flight path.

h) There are different attitude representations other than Euler angles such as Direction Cosine Matrix, quaternions, and axis-angle representation. Explain them, and tell what're their advantages/disadvantages ^{[34][35][36][37]}

1. Euler Angles: Euler angles are a set of three angles that define the orientation of an object relative to a fixed coordinate system. The three angles are typically referred to as roll, pitch, and yaw angles.

2. Direction Cosine Matrix (DCM): A direction cosine matrix is a 3x3 matrix that defines the transformation between two coordinate frames. In the context of attitude representation, the DCM describes the rotation from a body frame (the frame attached to the vehicle) to an inertial frame (a fixed, non-rotating frame).

3. Quaternions: A quaternion is a four-dimensional number that can be used to represent a rotation in three-dimensional space. Quaternions have a scalar component (real part) and a vector component (imaginary part), and can be used to perform rotations by multiplying them with a vector.

4. Axis-Angle Representation: An axis-angle representation defines a rotation as an axis (a unit vector) and an angle of rotation about that axis.

Table 1 Advantages and disadvantages of attitude representations

Representation	Advantages	Disadvantages
Euler Angles	- Simple to understand and can be easily visualized.	- Prone to gimbal lock¹ Interpolating between two orientations is problematic. - Not well-suited for numerical calculations, and can be prone to numerical instability.²
Direction Cosine Matrix (DCM)	- More numerically stable than Euler angles. - Can be easily inverted to obtain the inverse transformation.	- Can be computationally expensive to calculate and update, especially in real-time applications. - Prone to singularities when the rotation angle approaches 180 degrees.³
Quaternions	- Highly numerically stable and are not prone to gimbal lock or other singularities. - Can be easily interpolated to smoothly transition between two different orientations.	- Can be difficult to understand and visualize. - May require more computational resources to calculate and update than other attitude representations. - Quaternions are not unique.⁴
Axis-Angle Representation	- Relatively easy to understand. - Can be easily converted to other representations. - Numerically stable and not prone to singularities.	- Can be less efficient to compute than other representations, especially when dealing with large numbers of orientations. - Can be more difficult to interpolate smoothly between two orientations compared to quaternions.

¹ Occurs when two of the three axes align and the third axis becomes ambiguous. This can cause significant errors in attitude estimation and control. For example, if an aircraft is pitching straight up or down, the yaw angle becomes undefined, which can lead to numerical instability in the control system.

² This is because small errors in the Euler angles can result in large errors in the calculated orientation of the object.

³ This is because the matrix becomes ill-conditioned, meaning that small changes in the input can result in large changes in the output.

⁴ Meaning that there are multiple quaternions that can represent the same rotation. This can lead to confusion and errors if not properly accounted for in the control system.

Overall, each of these attitude representations has its own advantages and disadvantages, and the choice of representation depends on the specific application and the trade-offs between computational efficiency, numerical stability, and ease of use.

Quaternions are a popular choice for autopilot systems due to their numerical stability and lack of singularities. They are also easy to interpolate, which is important for smooth transitions between different orientations. Additionally, they can be easily converted to other representations, such as Euler angles or DCMs, when necessary.

Direction Cosine Matrix (DCM) and Axis-Angle representations are also used in some autopilot systems, but may be less common than quaternions. DCMs can be computationally expensive to calculate and update, especially in real-time applications, but they are more numerically stable than Euler angles. Axis-Angle representations are relatively easy to understand and not prone to singularities, but may be less efficient to compute than other representations.

Numerical Solution of ODEs ^{[38][39]}

a) State some of the numerical solving algorithms for ODEs. Showing the advantages and disadvantages of each.

There are several numerical methods used to solve ordinary differential equations (ODEs). Some of the commonly used methods, along with their advantages and disadvantages, are as follows:

1. Euler's method: Euler's method is based on a Taylor series expansion of the solution

$$y_{i+1} = y_i + h * f(x_i, y_i) + O(h^2)$$

This is a basic numerical method for solving first-order ODEs. It is easy to implement and requires only simple arithmetic operations. However, it has a large truncation error, which means that the error between the exact solution and the numerical approximation can be significant. This method is also not very accurate for stiff ODEs. One of the disadvantages of Euler's method is its lack of accuracy, often requiring an extremely small-time step.

2. Modified Euler method: A first guess via the Euler method is required to compute $f(x_{i+1}, y_{i+1})$ term.

$$y_{i+1} = y_i + \frac{h}{2} * [f(x_i, y_i) + f(x_{i+1}, y_{i+1})] + O(h^3)$$

This is a modified version of Euler's method that uses the midpoint between the current and next time step to improve accuracy. It is also known as the Heun's method. It is more accurate than Euler's method, but less accurate than the Runge-Kutta method. It is easy to implement and computationally efficient, but may not be stable for stiff ODEs. This method makes the Taylor expansion of order three rather than two.

3. Runge-Kutta method: This is a family of numerical methods that are widely used for solving ODEs. The fourth-order Runge-Kutta method is the most commonly used, and it is more accurate than Euler's method. It has a smaller truncation error and is more stable for stiff ODEs. However, it requires more computation than Euler's method. As we have just shown, the accuracy of numerical solutions of ordinary differential equations can be improved by adding more terms to the Taylor expansion. The Runge-Kutta method builds upon this idea, just as the modified Euler method did.

The fourth-order Runge-Kutta scheme:

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2 * k_2 + 2 * k_3 + k_4)$$

where:

$$\left. \begin{aligned} k_1 &= h * f(x_i, y_i) \\ k_2 &= h * f\left(x_i + \frac{1}{2} * h, y_i + \frac{1}{2} * k_1\right) \end{aligned} \right| \begin{aligned} k_3 &= h * f\left(x_i + \frac{1}{2} * h, y_i + \frac{1}{2} * k_2\right) \\ k_4 &= h * f(x_i + h, y_i + k_3) \end{aligned}$$

Example:

$$x' = x + t, \quad x(0) = 1$$

The exact solution is $x_{exact}(t) = 2e^t - t - 1$

Using the MATLAB script that we've created "**Euler_ModifiedEuler_RK4.m**", Both the Euler, modified Euler and RK4 methods have been used to numerically integrate the ODE, compare solution to the exact solution and the absolute value of the relative error is plotted in the figure 5 as a function of time for various time steps.

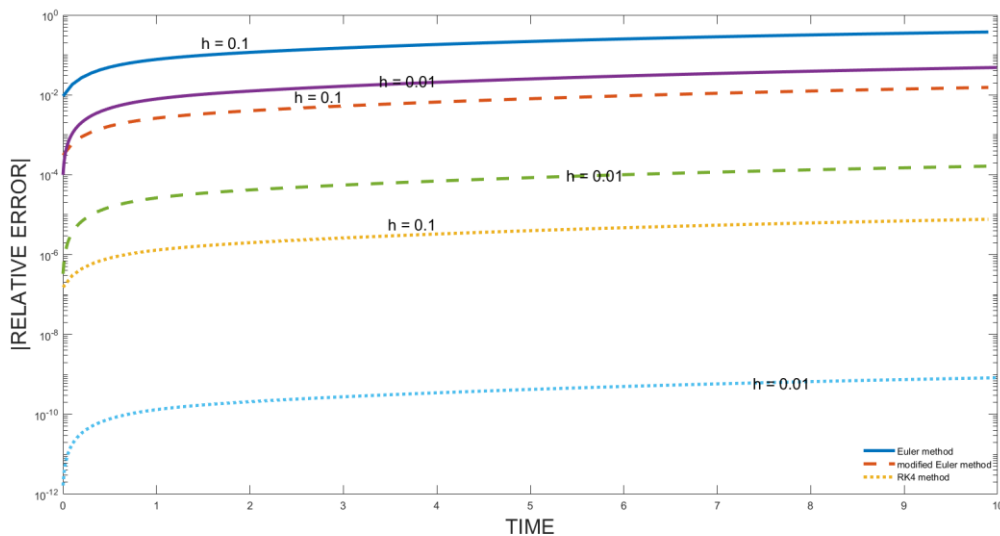


Figure 5 Euler, Modified Euler and Runge-Kutta methods

In general, the error grows with time. The decrease of error with smaller time steps, as predicted in our analysis, is quite apparent

All of the methods presented so far (Euler, modified Euler, Runge-Kutta) are single point methods, the solution at $i+1$ depends solely on a single point i . A popular alternative to these schemes is multistep methods that compute y_{i+1} by reusing previously obtained values of y_n like Multi-step Adams methods. In Adams, we used backward derivative approximation to find higher order derivatives terms rather than forward derivative approximation used in Euler and Runge-Kutta

4. Adams-Bashforth (Open Adams) method: This is a family of numerical methods that use previous values of the function to approximate the next value. They are more accurate than Euler's method and require less computation than Runge-Kutta methods. However, they can be unstable for stiff ODEs and may require more memory to store previous values.

The fourth-order formula is the most widely used:

$$y_{i+1} = y_i + \frac{h}{24} * [55 * f_i - 59 * f_{i-1} + 37 * f_{i-2} - 9 * f_{i-3}] + O(h^5)$$

5. Adams-Moulton (Closed Adams) method: This is a family of numerical methods that use both previous and current values of the function to approximate the next value. They are implicit methods therefore, Iterations are needed. They are more accurate than Adams-Bashforth methods, but require more computation and may be unstable for stiff ODEs.

6. Predictor–Corrector method

The two methods presented above (closed and open Adams formulas) can be combined into a method known as the predictor–corrector method. The idea is to begin by finding an estimated value $y_{i+1}^{(0)}$ (the predictor that is close to the final value y_{i+1}). This value is computed using the open version of the formula then injected into the closed method of the same order. This combined approach **accelerates the convergence**, allowing us to fully exploit the advantages of the Adams method (reduced computation times and significantly decreased estimated truncation errors).

In conclusion, each numerical method has its own advantages and disadvantages. The choice of method depends on the specific problem being solved, the desired level of accuracy, and the computational resources available.

Side Note: The built-in features offered by MATLAB can solve almost any problem phrased in terms of ordinary differential equations of the following type and with the following initial conditions:

$$\frac{dy}{dx} = f(x, y), \quad y(x_0) = y_0$$

MATLAB has several differential equation solvers: some designed for classical problems, and others designed for the so-called “stiff” problems.

The following solvers are classical solvers:

- **ode45** uses an explicit one-step Runge–Kutta method. This solver is the best choice for most problems.
- **ode23** uses an explicit one-step Runge–Kutta method. It can be more efficient than ode45 in some cases.
- **ode113** uses an Adams–Bashforth–Moulton method. This is a multi-step solver.

Also, there are four solvers for “stiff” problems: **ode15s**, **ode23s**, **ode23t** and **ode23tb**.

b) Choose one algorithm for solving the Airplane EOM, and clearly state the (Initial conditions needed, Inputs needed in each iteration, and Outputs calculated in each iteration)

Chosen Algorithm: For aircraft simulation, the most commonly used method is the Runge-Kutta method, particularly the **fourth-order Runge-Kutta method**, due to its accuracy, stability, and ease of implementation. It can handle both stiff and non-stiff ODEs, and can be used for multi-step simulations. Moreover, it can handle different types of forces and moments acting on the aircraft, such as aerodynamic forces, gravity, and thrust.

States: $(u, v, w), (p, q, r), (x, y, z), (\phi, \theta, \psi)$
 Constants of airplane: mass (m), Inertia matrix (I).

The initial conditions needed for the Runge-Kutta method are the values of the state variables at a given initial time, such as the initial position, velocity, and attitude of the aircraft. In addition, the initial conditions may also include any input values that affect the EOM, such as the aircraft's initial mass, engine thrust, and control surface positions.

Initial conditions for all states are needed
 $(u_0, v_0, w_0), (p_0, q_0, r_0), (x_0, y_0, z_0), (\phi_0, \theta_0, \psi_0)$

In each iteration of the Runge-Kutta method, **the inputs needed** include the current state of the aircraft and any external forces or moments acting on the aircraft. These forces and moments may include aerodynamic forces, gravity, and engine thrust. The algorithm also requires a time step value, which determines the accuracy of the numerical approximation.

For the fourth-order Runge-Kutta scheme

$$x_{i+1} = x_i + \frac{1}{6} (k_1 + 2 * k_2 + 2 * k_3 + k_4)$$

Where

$$\left. \begin{aligned} k_1 &= h * f(t_i, x_i) \\ k_2 &= h * f\left(t_i + \frac{1}{2} * h, x_i + \frac{1}{2} * k_1\right) \end{aligned} \right| \begin{aligned} k_3 &= h * f\left(t_i + \frac{1}{2} * h, x_i + \frac{1}{2} * k_2\right) \\ k_4 &= h * f(t_i + h, x_i + k_3) \end{aligned}$$

And

$$h = \Delta t, \quad f(t, x) = \dot{x}, \quad x = \begin{Bmatrix} u \\ v \\ w \\ p \\ q \\ r \\ \emptyset \\ \theta \\ \Psi \\ x \\ y \\ z \end{Bmatrix}$$

Each iteration depends on **previous iteration states and states rates values**

Thus, **Inputs for iteration (n) are:**

- Previous iteration States values (x_{n-1}).
- Previous iteration States rates values ($\dot{x}_{n-1}$).

The Runge-Kutta method outputs the values of the state variables at the next time step. These outputs include the new position, velocity, and attitude of the aircraft, as well as any other relevant state variables, such as fuel consumption or control surface deflections.

Outputs from each iteration (n) are **states** (x_{n+1})

The Runge-Kutta method is a numerical method that iteratively approximates the solution to the EOM by evaluating the derivatives of the state variables at multiple intermediate time steps. The method uses a weighted sum of these intermediate values to obtain a more accurate approximation of the state variables at the next time step. The order of the method determines the accuracy of the numerical approximation, with higher-order methods providing more accurate solutions at the cost of increased computational complexity.

c) Solve the system of first-order ODEs

$$\frac{dy_1}{dt} = \sin(t) + \cos(y_1) + \sin(y_2)$$
$$\frac{dy_2}{dt} = \cos(t) + \sin(y_2)$$

The Initial conditions are:

$$\text{At } t = 0, \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

End of solution interval:

$$t = 20$$

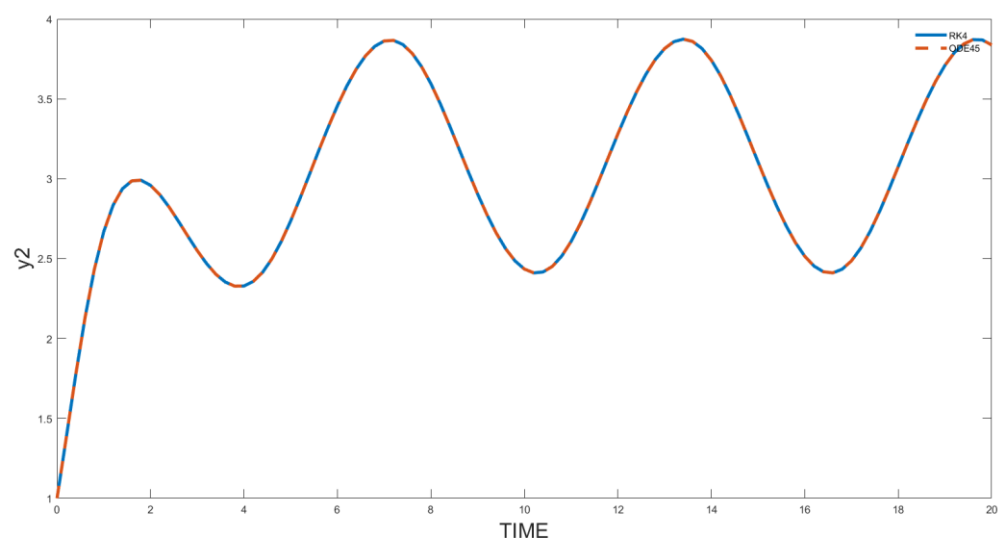
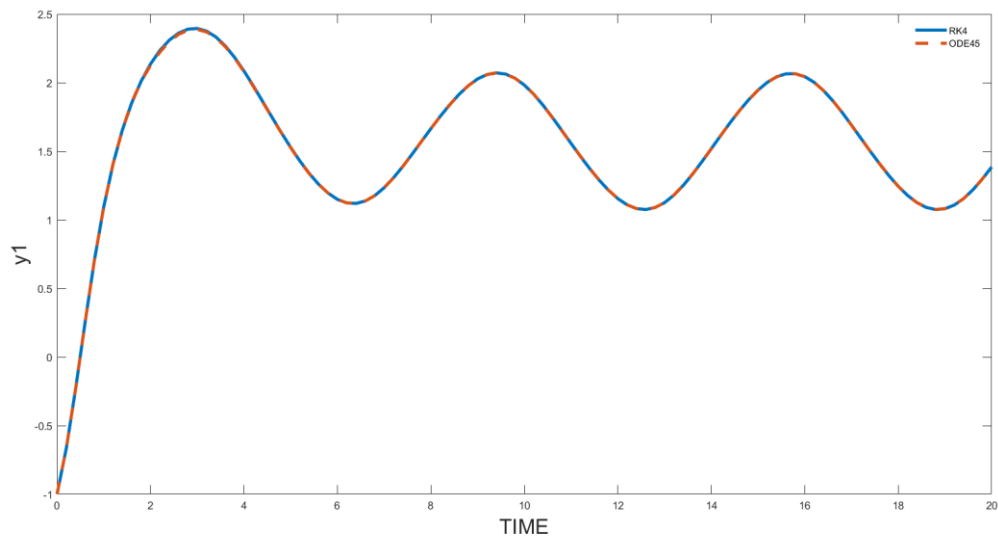
Number of intervals:

$$n = 100$$

Thus, time step

$$\Delta t = \frac{t}{n} = 0.2$$

We'll solve it using Runge-Kutta method and compare the results with (ode45) function on MATLAB.



Airplane Simulator Part I: RBD Equations Solver

a) Introduction

This documentation describes the second task in the autopilot course as the last course of our control engineering courses, focused on implementing the Runge-Kutta 4th order method to solve the 12 RBD equations of an airplane. The previous task involved reviewing the RBD equations of the airplane and solving a simple ODE using different methods, including the Runge-Kutta 4th order method. The current task aims to verify the correctness of the MATLAB code developed for solving the RBD equations of the airplane, by comparing its results with the Simulink's "6 DOF Euler Angles" block. Visual comparison of the time histories of the Euler angles obtained from the two simulations was used to verify the correctness of the MATLAB code. The goal is to ensure that the MATLAB code is accurate and suitable for use in building a non-linear airplane simulator.

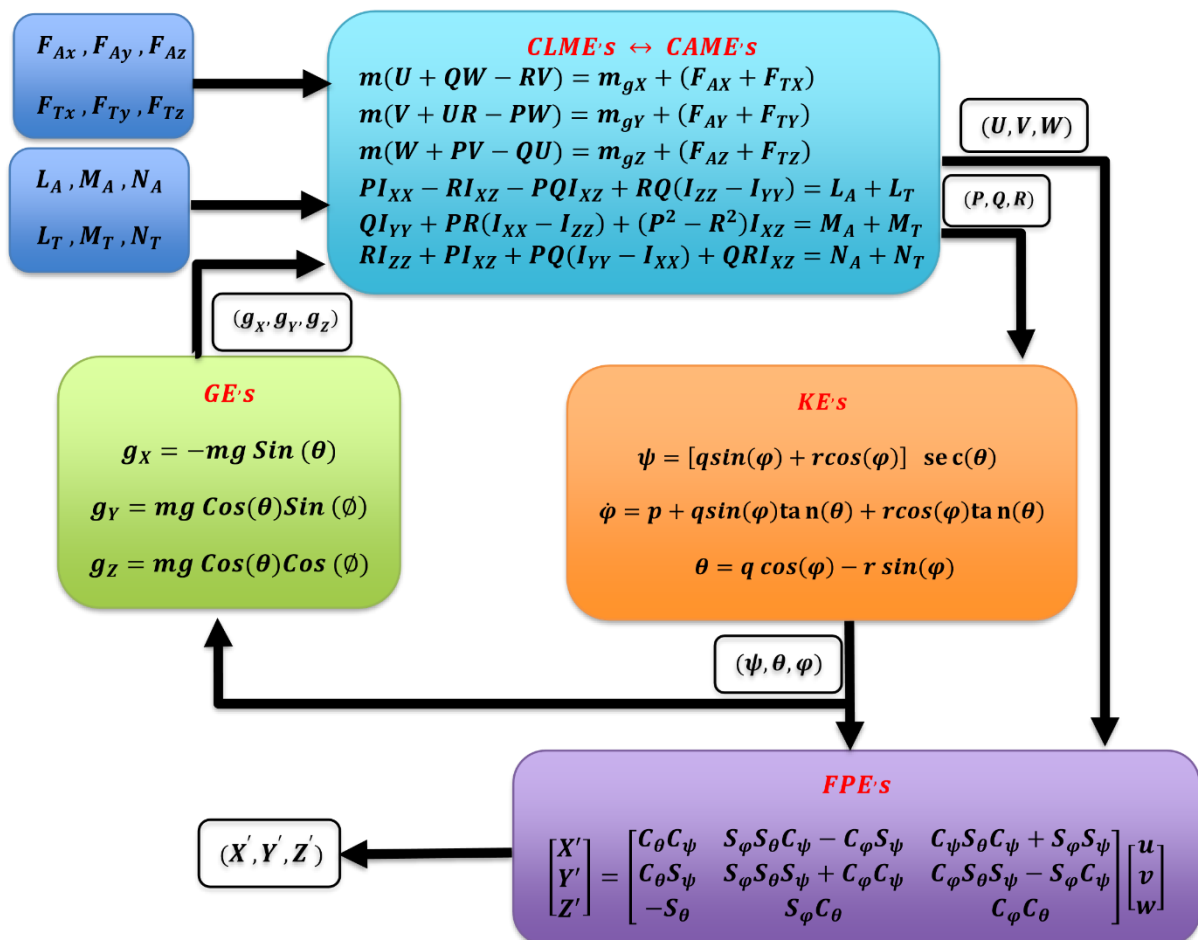
b) RBD Equations ^[26]

First, we will review the 12 equations that describe the rigid body dynamics (RBD) of the airplane. These equations are essential for simulating the behavior of the airplane and are the basis for building a non-linear airplane simulator.

To provide an overview of these equations, we will present them in a chart that summarizes the 12 RBD equations. This chart will serve as a reference for understanding and solving the RBD equations of the airplane. With a clear understanding of these equations, we can proceed to implement the Runge-Kutta 4th order method for solving them.

where:

Equations Name
CLME'S: Conservation of Linear Momentum Equations
CAME'S: Conservation of Angular Momentum Equations
KE's: Kinematics Equations
GE's: Gravitational Equations
FPE's: Flight Path Equations



c) Runge-Kutta Method, Inputs, Initial conditions, Outputs & Procedure ^[40]

The Runge-Kutta 4th order method is a numerical method used to solve ordinary differential equations (ODEs). This method is a widely used technique for simulating the dynamics of an airplane, and it was chosen as the most suitable method for this task.

The method's scheme:

$$x_{i+1} = x_i + \frac{1}{6} (k_1 + 2 * k_2 + 2 * k_3 + k_4)$$

where:

$$\left. \begin{aligned} k_1 &= h * f(t_i, x_i) \\ k_2 &= h * f\left(t_i + \frac{1}{2} * h, x_i + \frac{1}{2} * k_1\right) \end{aligned} \right| \begin{aligned} k_3 &= h * f\left(t_i + \frac{1}{2} * h, x_i + \frac{1}{2} * k_2\right) \\ k_4 &= h * f(t_i + h, x_i + k_3) \end{aligned}$$

for Airplane:

$$h = \Delta t, \quad f(t, x) = \dot{x}, \quad x = \begin{Bmatrix} u \\ v \\ w \\ p \\ q \\ r \\ \phi \\ \theta \\ \Psi \\ x \\ y \\ z \end{Bmatrix}$$

Our procedure:

To implement the Runge-Kutta 4th order method using **MATLAB⁵**, we need to define the 12 equations of motion of the in a form suitable for the method like, $\dot{y} = f(t, x, y, z, \dots)$.

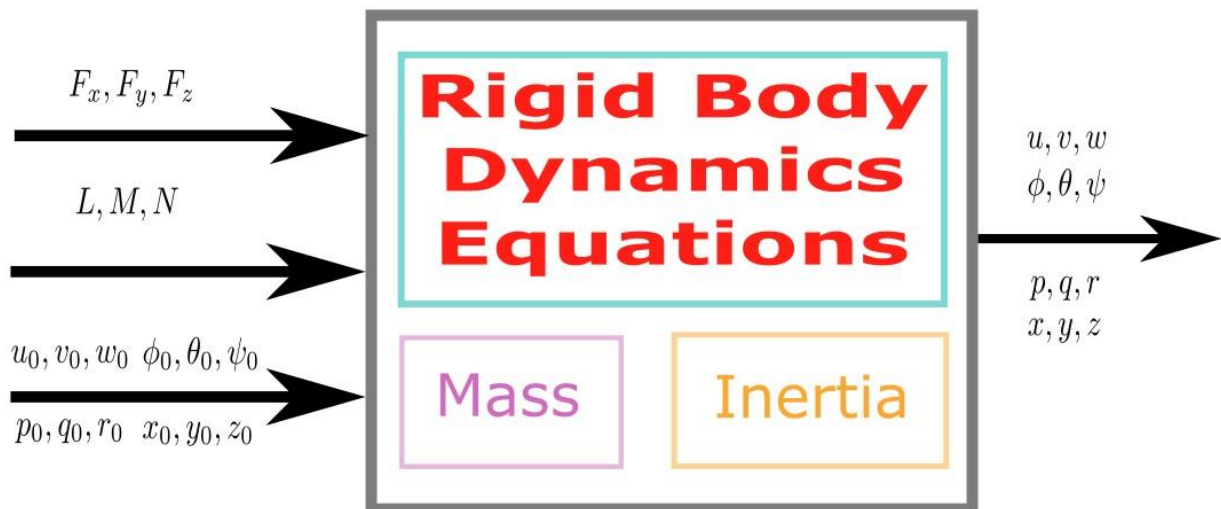
Euler angles rate & the position rates is ready for the method's form, but the force & moment equations need to be re-arranged as below:

$$\begin{bmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{bmatrix} = \begin{bmatrix} X - mgS_\theta \\ Y + mgC_\theta S_\phi \\ Z + mgC_\theta C_\phi \end{bmatrix} / m - \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} u \\ v \\ w \end{bmatrix} \right) \rightarrow \text{Forces (Re - arranged)}$$

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = I^{-1} \begin{bmatrix} L \\ M \\ N \end{bmatrix} - \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix} \right) \rightarrow \text{Moments (Re - arranged)}$$

Then, we get the constants $\{k_1, k_2, k_3, k_4\}$ simultaneously using the initial conditions & time step Δt . After the constants have been calculated we will use them to calculate the next point of each 12 states and so on.

Solver Overview:



Constants of airplane

Mass (m), Inertia matrix (I), Time interval (t_{final} & $timestep$)

Inputs for each iteration (n)

Forces and Moments $[F_x, F_y, F_z, L, M, N]$

Initial conditions for all 12 states $[u_0, v_0, w_0, \phi_0, \dots]$

Outputs from each iteration (n)

All 12 states at the next timestep $[u, v, w, \phi, \dots]$

⁵ MATLAB codes have been attached with this documentation in Appendix.

Our problem gives:

From Task 2 Statement:

GIVEN DATA	
$t_{final} = 25 \text{ sec}$	
$Forces = [10 \ 5 \ 3] \text{ N}$	
$Moments = [10 \ 15 \ 20] \text{ Nm}$	
$Mass = 30 \text{ Kg}$	
$I = \begin{bmatrix} I_x & I_{xy} & I_{xz} \\ I_{yx} & I_y & I_{yz} \\ I_{zx} & I_{zy} & I_z \end{bmatrix} = \begin{bmatrix} 1 & -2 & -1 \\ -2 & 5 & -4 \\ -1 & -4 & 0.2 \end{bmatrix}^6$	

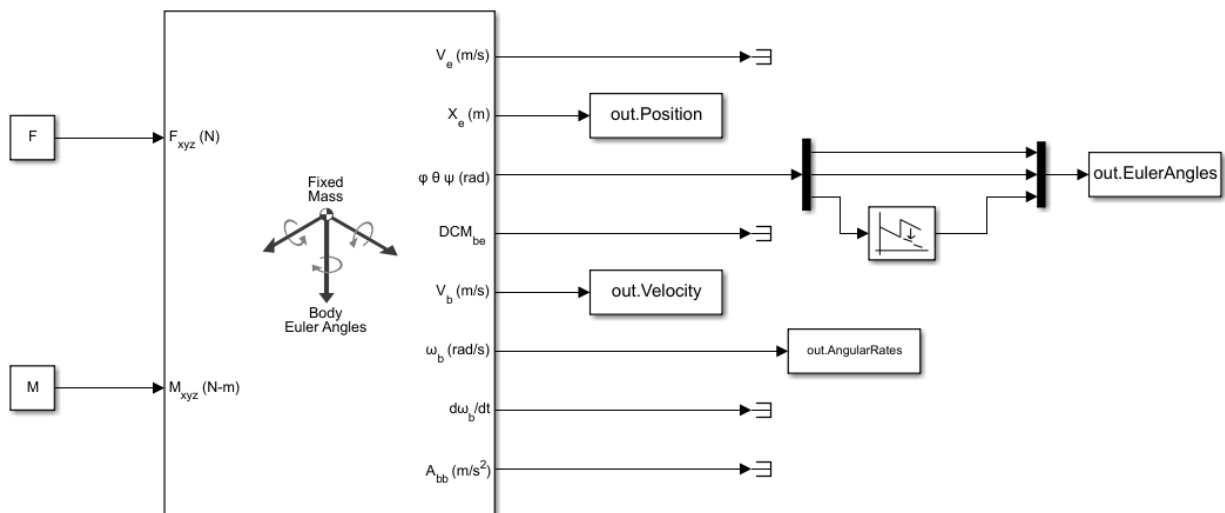
INITIAL CONDITIONS	
$\begin{Bmatrix} u \\ v \\ w \\ p \\ q \\ r \\ \phi \\ \theta \\ \Psi \\ x \\ y \\ z \end{Bmatrix} = \begin{Bmatrix} 10 \\ 2 \\ 0 \\ \frac{2\pi}{180} \\ \frac{\pi}{180} \\ 0 \\ \frac{20\pi}{180} \\ \frac{15\pi}{180} \\ \frac{30\pi}{180} \\ 2 \\ 4 \\ 7 \end{Bmatrix}$	

⁶ Since $(I_{xy} \neq I_{yz} \neq 0)$, so x-z isn't a plane of symmetry, therefore this isn't equations for an airplane.

d) Validation ^[41]

To validate our implementation of the Runge-Kutta 4th order method for solving the RBD equations of the airplane, we used the 6 DOF Euler angles block in **Simulink**⁷. This block is designed to solve the RBD equations of the airplane and is a built-in feature of Simulink.

The Simulink schematic used for this comparison is shown below. The input to the 6 DOF Euler angles block is the initial state of the airplane, and the output is the state of the airplane at each time step. We compared this output with the output of our MATLAB code and ensured that they were consistent.



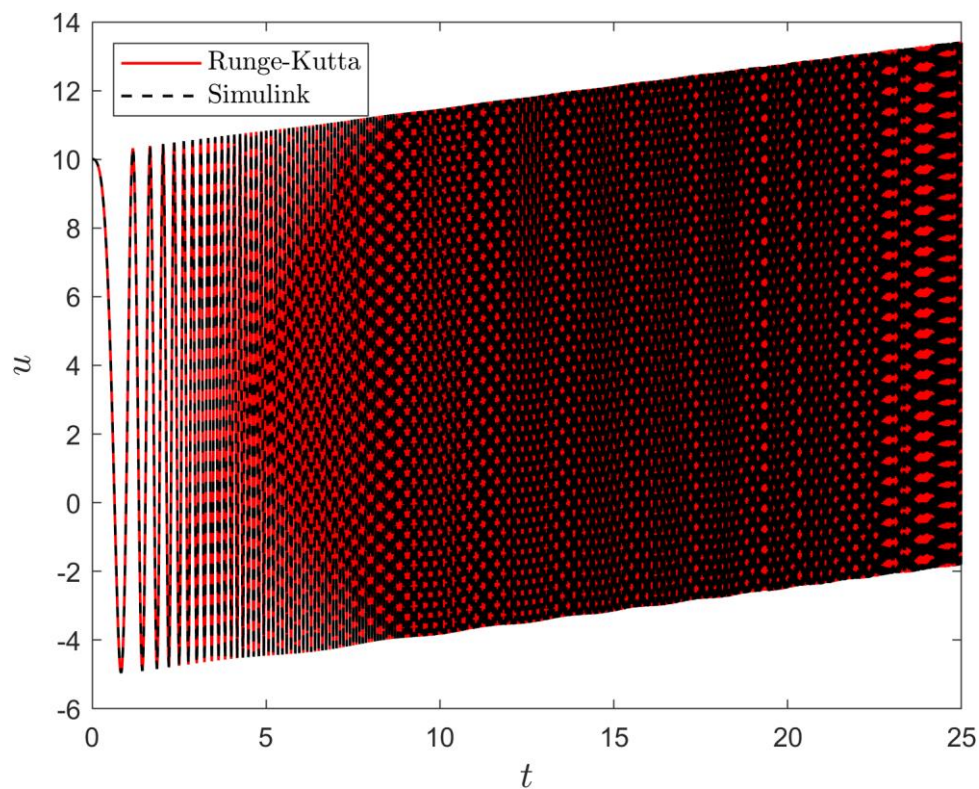
⁷ Simulink files have been submitted with this documentation.

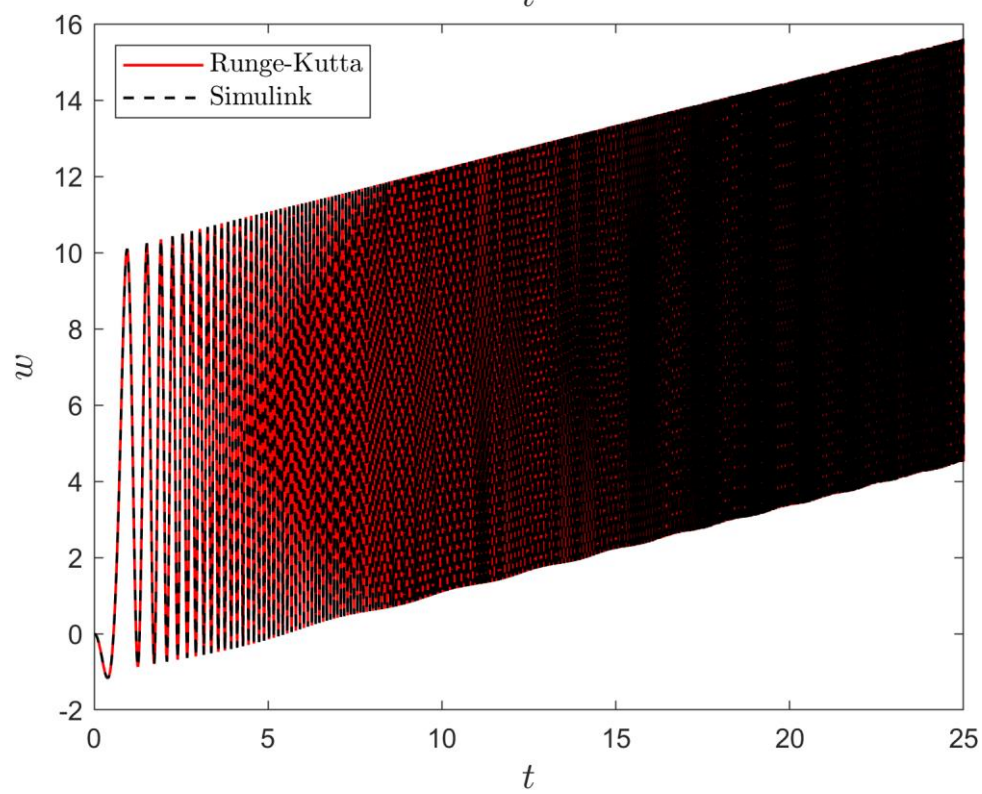
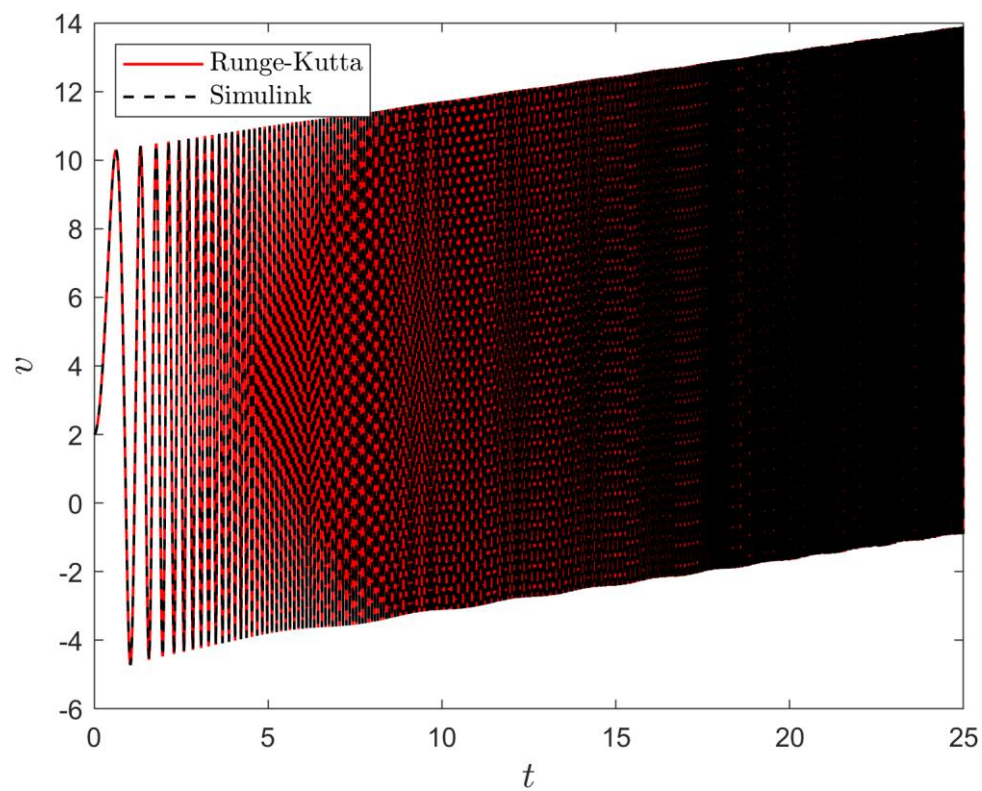
e) Results

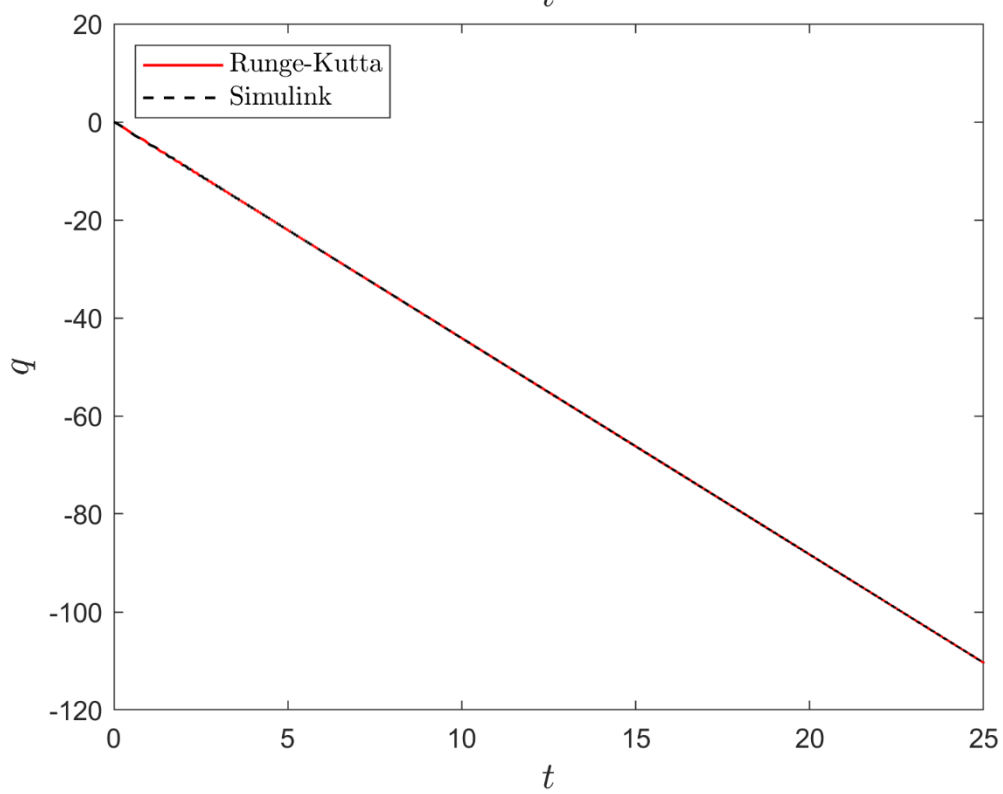
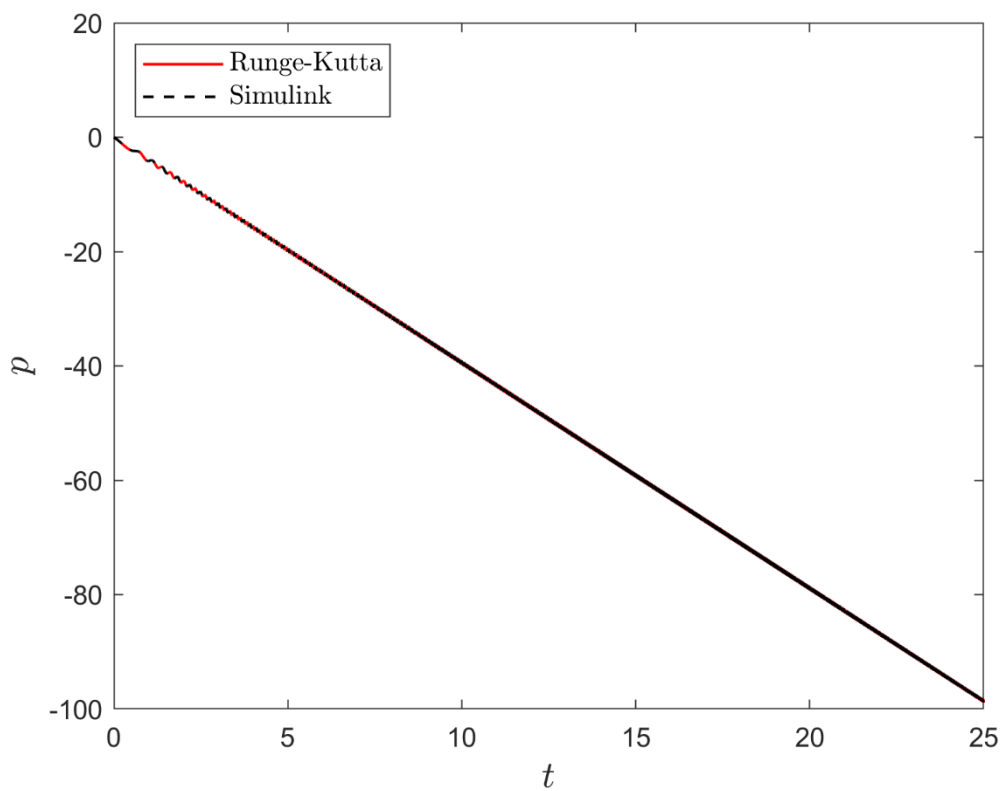
In this section, we present the results of our implementation of the Runge-Kutta 4th order method for solving the RBD equations. To visually compare the results of the MATLAB code and the Simulink “6 DOF Euler angles” block, we have generated 12 graphs, each graph corresponding to one of the 12 state variables of the airplane.

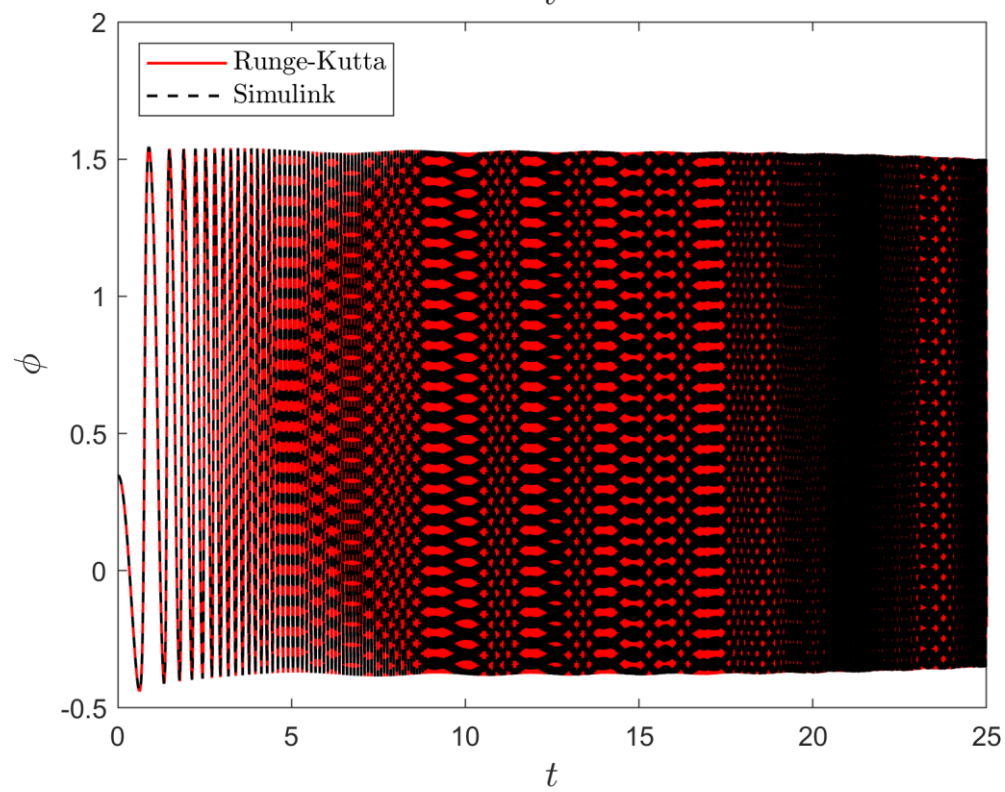
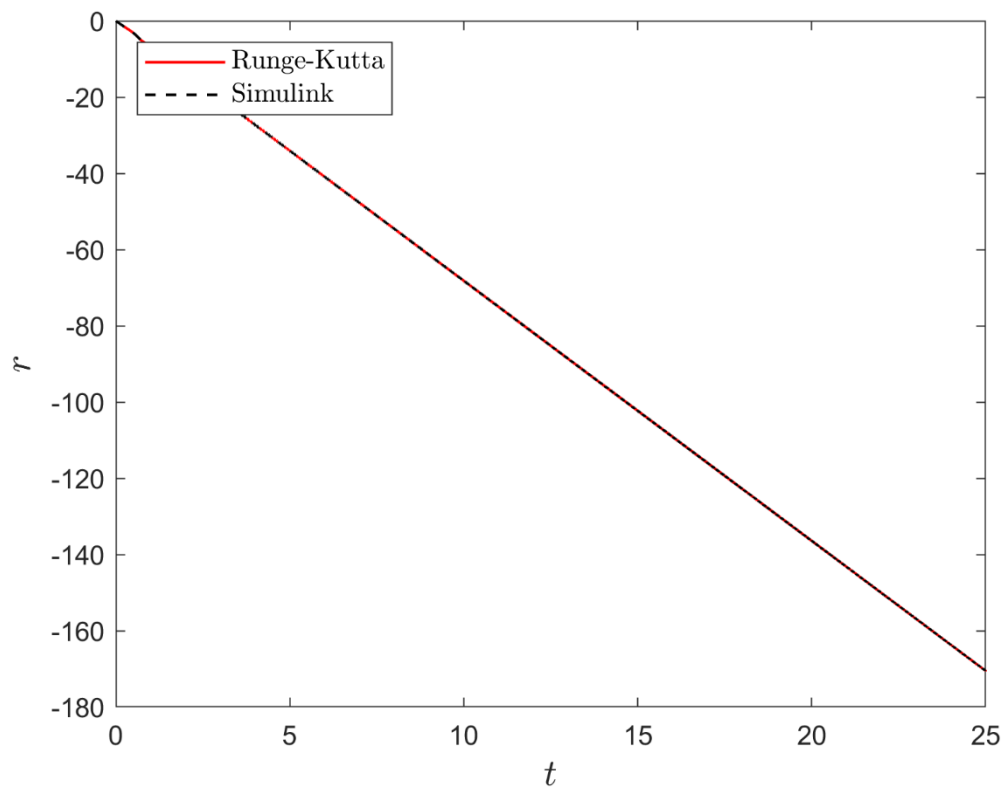
Each graph displays the results of both the MATLAB code and the Simulink Euler angles block for the corresponding state variable, allowing for a direct comparison between the two methods. The graphs provide an insightful representation of the behavior of the airplane and help to verify the correctness of our MATLAB code.

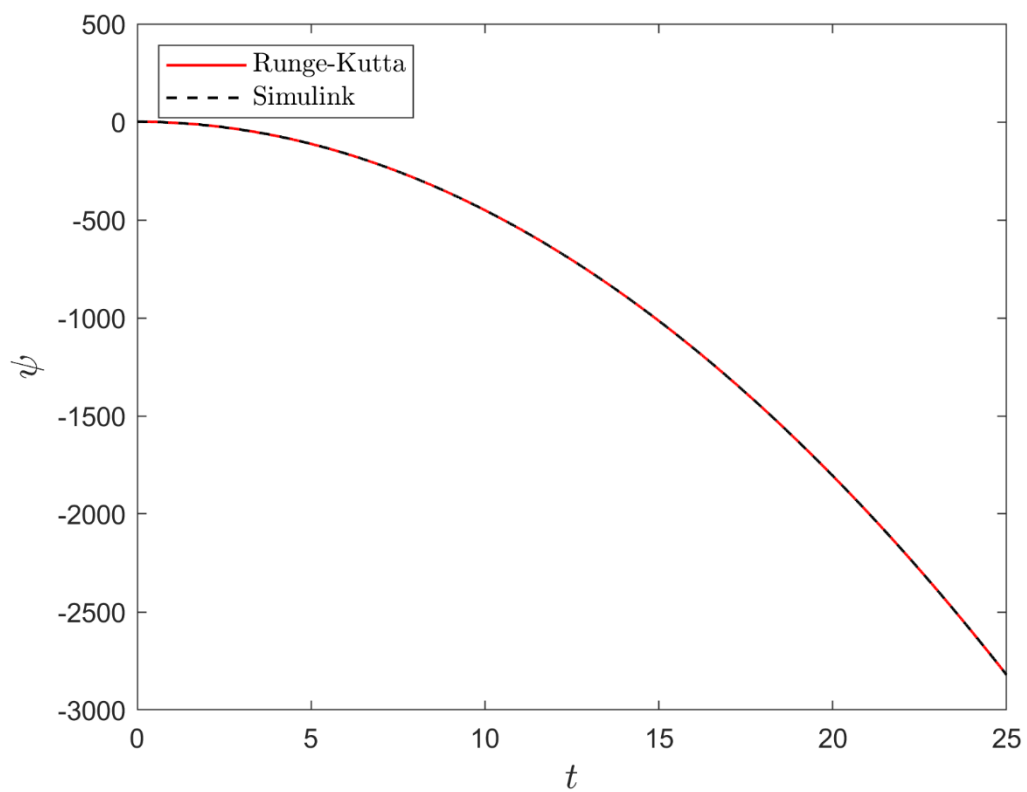
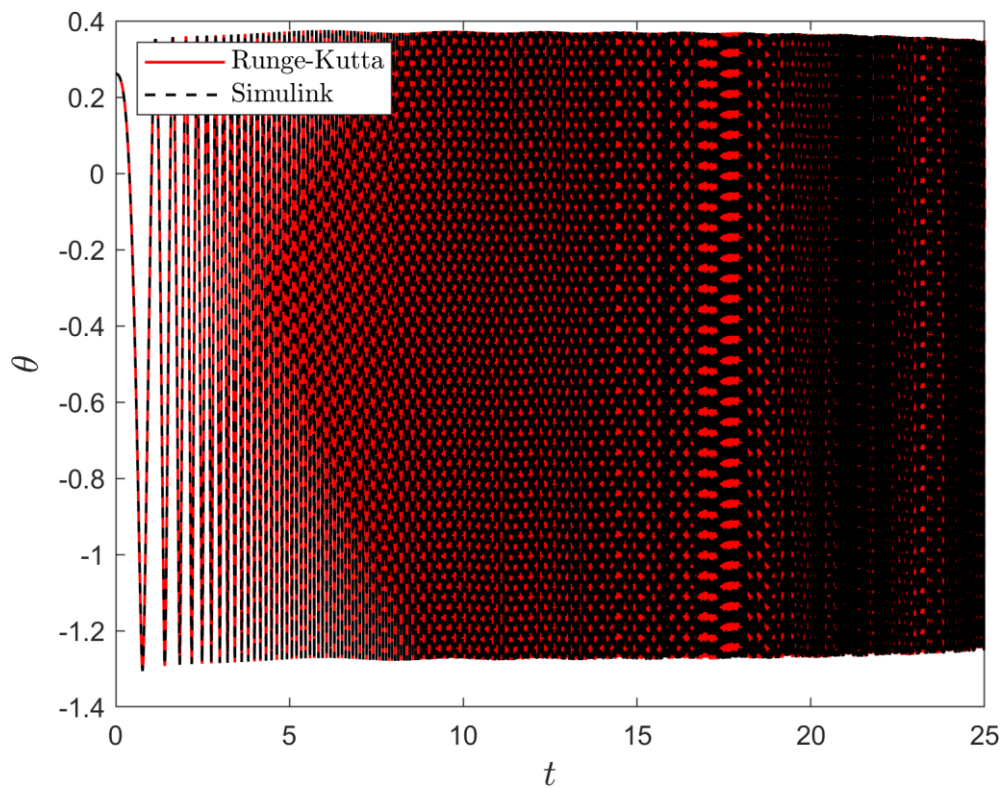
The comparison between the MATLAB code and Simulink results shows good agreement for all state variables, confirming the correctness of our implementation of the Runge-Kutta 4th order method.

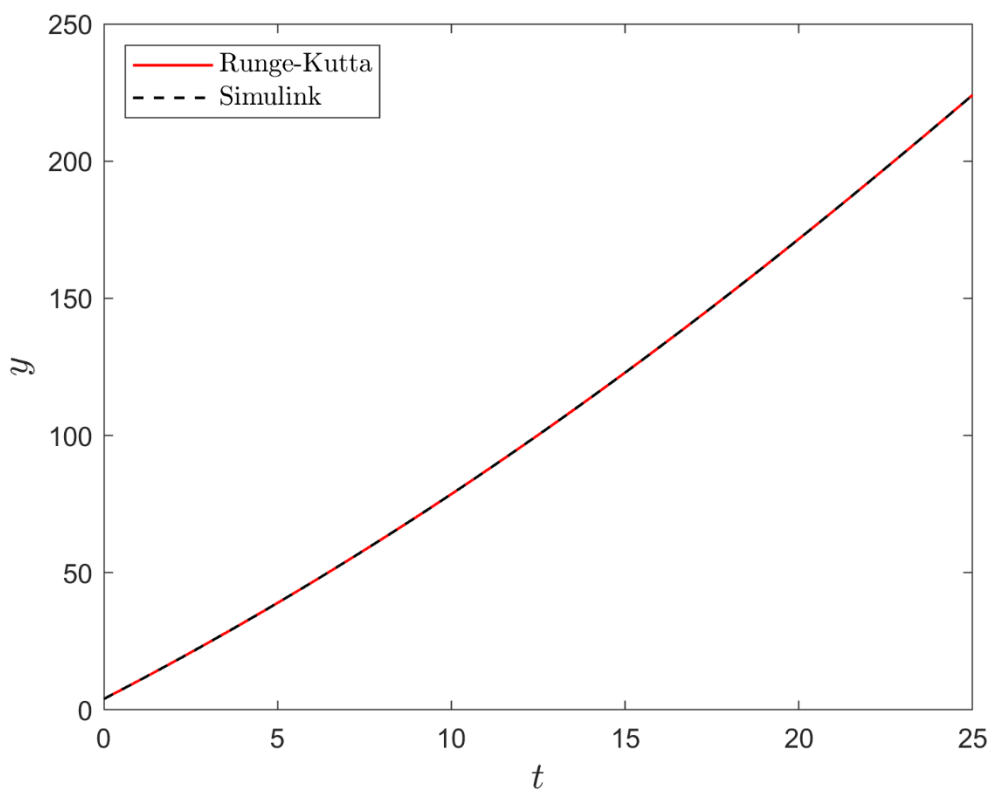
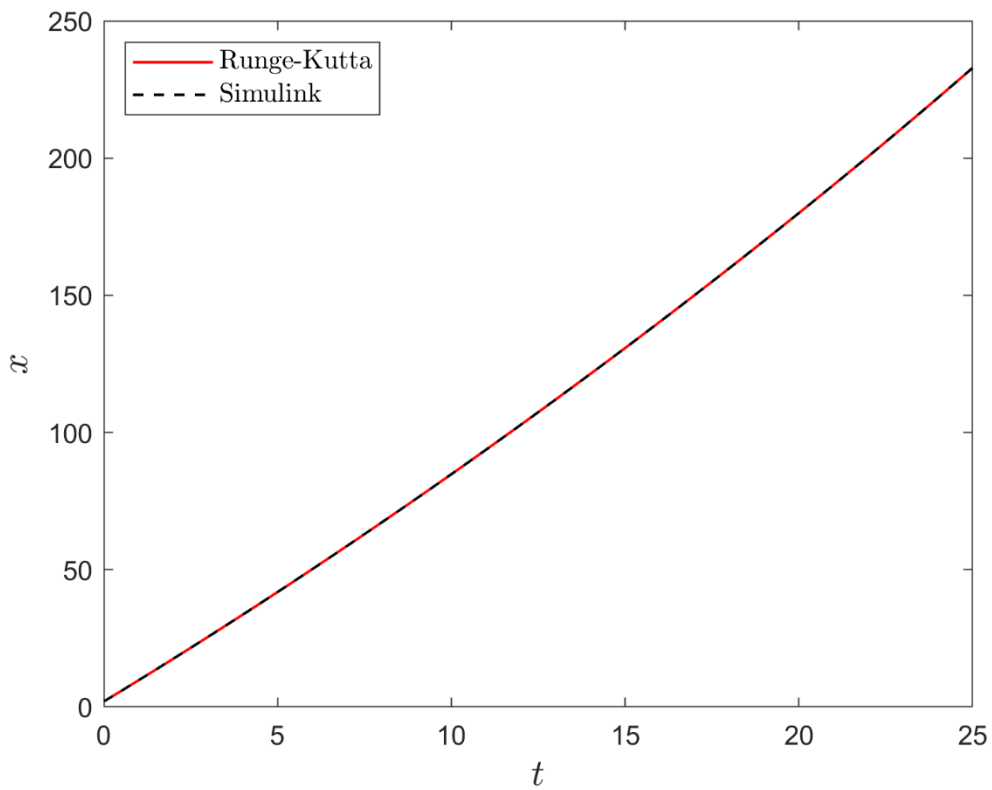


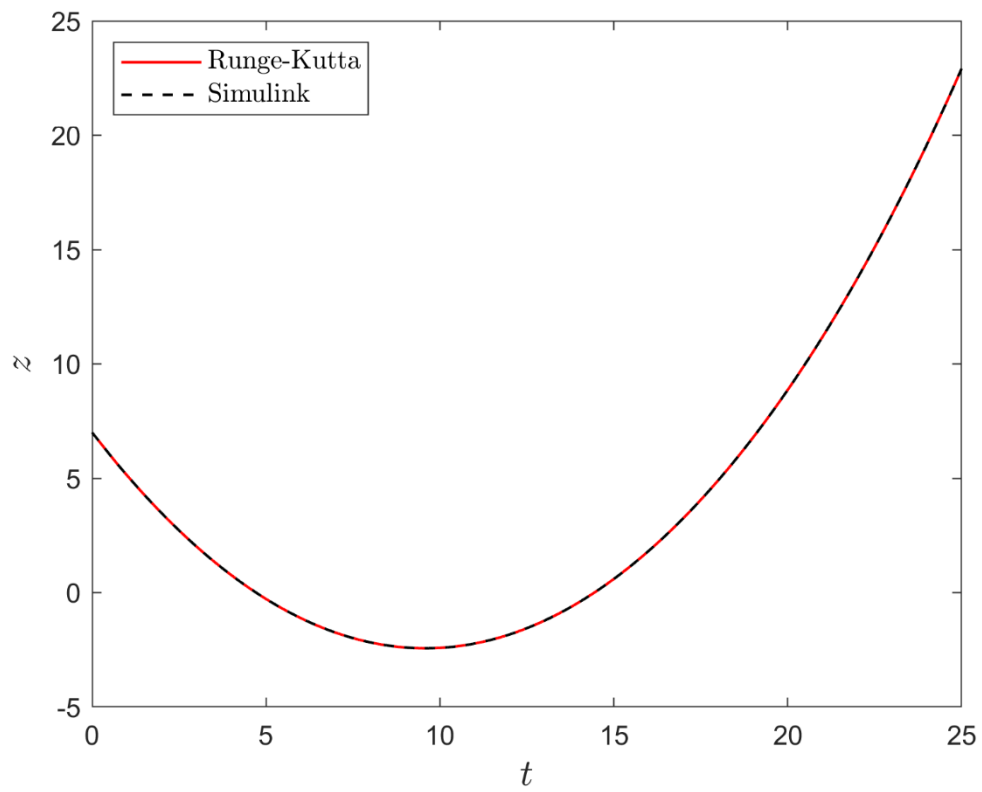












f) Conclusion

In conclusion, the task of implementing the Runge-Kutta 4th order method to solve the RBD equations of an airplane and comparing the results with the Simulink Euler angles block was successfully completed. The visual comparison of the two sets of results provided confidence in the accuracy of the MATLAB code developed for solving the RBD equations. Any discrepancies or differences between the two sets of results were investigated and resolved. This MATLAB code can now be used as a part of a non-linear airplane simulator.

g) Bonus

Question Statement:

You want to compare two signals (vectors), what mathematical expression can you use to express the error between the two vectors?

Answer:

- **Root-Mean-Square Error:** Root Mean Square Error (RMSE) is the standard deviation of the residuals (prediction errors). Residuals are a measure of how far from the regression line data points are; RMSE is a measure of how spread out these residuals are. In other words, it tells you how concentrated the data is around the line of best fit. Root mean square error is commonly used in climatology, forecasting, and regression analysis to verify experimental results. [43]

For a forecast array F and actual array, A made up of n scalar observations, the root-mean-square error is defined as

$$E = \sqrt{\frac{1}{n} \sum_{i=1}^n |A_i - F_i|^2}$$

with the summation performed along the specified dimension. [42]

- **Weighted Root-Mean-Square Error:** For a forecast array F and actual array, A made up of n scalar observations and weighting scheme W , the weighted root-mean-square error is defined as

$$E_W = \sqrt{\frac{\sum_{i=1}^n W_i |A_i - F_i|^2}{\sum_{i=1}^n W_i}}$$

with the summation performed along the specified dimension. [42]

Notes:

- These functions exist in MATLAB: `rsme()`
- Should RMSE be high or low?
RMSE is a metric which ranges from 0 to infinity, where the closer the score is to 0 the better performing the model is. So, the RMSE value should be as low as possible. [44]
- What is a good RMSE value?
The closer RMSE is to 0, the more accurate the model is. But RMSE is returned on the same scale as the target you are predicting for and therefore there isn't a general rule for what is considered a 'good' value. How good your metric value is can only be evaluated within the dataset context you are working. [44]

Airplane Simulator Part II

a) Introduction

The aim of this task is to develop a non-linear simulator for an airplane by combining an airframe model code with an RBD solver code. The simulator is designed to calculate the change in states of the airplane due to changes in control surface deflection values at specific flight conditions. The code was implemented in MATLAB and solved using the RK4 method and Simulink.

To validate the accuracy and reliability of the simulator, we performed a benchmark test on **Boeing 747 in flight condition 5**, for various inputs including aileron, rudder, elevator, and thrust. The results of the benchmark test were compared with a previously published data set to verify the correctness of our simulator.

Subsequently, we applied our simulator to our own airplane model, **JETSTAR_FC3**, and tested it for similar inputs. The results were then verified with **TEAM #10** to ensure the validity of our simulator.

This section presents the benchmark test results, and the results of our simulator on JETSTAR_FC3. The codes and Simulink model used in this task will be attached too.

b) Airplane forces and moments calculation

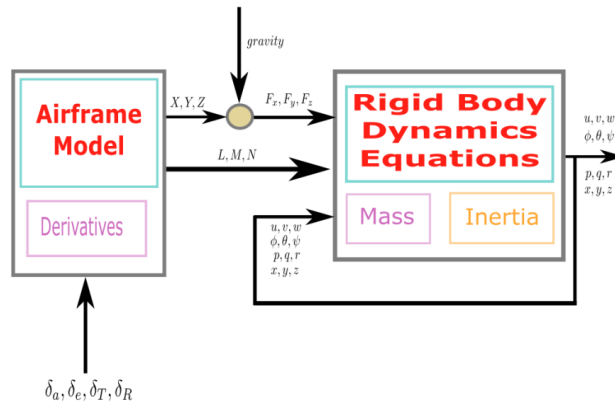


Figure 6 Nonlinear Simulator

The following set of **linear equations** represents the **change** in the Aerodynamic & thrust forces and moments, they are function of:

- Stability Derivatives.
- The perturbation change in the states and the control surfaces deflections from their values at the trim condition.

Forces

$$\begin{aligned}\Delta X &= \frac{\partial X}{\partial u} \Delta u + \frac{\partial X}{\partial w} \Delta w + \frac{\partial X}{\partial \delta_e} \Delta \delta_e + \frac{\partial X}{\partial \delta_T} \Delta \delta_T \\ \Delta Y &= \frac{\partial Y}{\partial v} \Delta v + \frac{\partial Y}{\partial p} \Delta p + \frac{\partial Y}{\partial r} \Delta r + \frac{\partial Y}{\partial \delta_r} \Delta \delta_r \\ \Delta Z &= \frac{\partial Z}{\partial u} \Delta u + \frac{\partial Z}{\partial w} \Delta w + \frac{\partial Z}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial Z}{\partial q} \Delta q + \frac{\partial Z}{\partial \delta_e} \Delta \delta_e + \frac{\partial Z}{\partial \delta_T} \Delta \delta_T\end{aligned}$$

Moments

$$\begin{aligned}\Delta L &= \frac{\partial L}{\partial v} \Delta v + \frac{\partial L}{\partial p} \Delta p + \frac{\partial L}{\partial r} \Delta r + \frac{\partial L}{\partial \delta_r} \Delta \delta_r + \frac{\partial L}{\partial \delta_a} \Delta \delta_a \\ \Delta M &= \frac{\partial M}{\partial u} \Delta u + \frac{\partial M}{\partial w} \Delta w + \frac{\partial M}{\partial \dot{w}} \Delta \dot{w} + \frac{\partial M}{\partial q} \Delta q + \frac{\partial M}{\partial \delta_e} \Delta \delta_e + \frac{\partial M}{\partial \delta_T} \Delta \delta_T \\ \Delta N &= \frac{\partial N}{\partial v} \Delta v + \frac{\partial N}{\partial p} \Delta p + \frac{\partial N}{\partial r} \Delta r + \frac{\partial N}{\partial \delta_r} \Delta \delta_r + \frac{\partial N}{\partial \delta_a} \Delta \delta_a\end{aligned}$$

Note: $\Delta X, \Delta Y, \Delta Z, \Delta L, \Delta M, \Delta N$ are the changes in the forces & moments, i.e. These are not the absolute values of the forces and moments. They should be added to the reference values at the trim condition $X_0, Y_0, Z_0, L_0, M_0, N_0$ to calculate the absolute values X, Y, Z, L, M, N .

Similarly, $\Delta u, \Delta v, \Delta w, \dots$ are the changes in the states values from their values at the reference condition $\Delta u = u - u_0, \Delta v = v - v_0, \Delta w = w - w_0, \dots$

Hence:

- Inputs and outputs of the (Airframe Model) are perturbations from the reference values.
- Inputs and outputs of the RBD are absolute values.

So that: Perturbation values resulting from the (Airframe Model) should be added to the reference values before passing them to the (RBD) and the absolute values resulting from the (RBD) should be converted to perturbation values by subtracting the reference values from them.

• How to get the reference values? (Which are fixed in the Simulation)

The total forces acting on an airplane are:

1. Aerodynamic forces.
2. Thrust force.
3. Gravity force.

$$\begin{aligned} X - mg \sin \theta &= m(\dot{u}^E + qw^E - rv^E) \\ Y + mg \cos \theta \sin \phi &= m(\dot{v}^E + ru^E - pw^E) \\ Z + mg \cos \theta \cos \phi &= m(\dot{w}^E + pv^E - qu^E) \end{aligned}$$

Initially at the **reference flight condition**, the airplane is in an **equilibrium state**, which means:

$$\sum \text{Forces} = 0 \quad \& \quad \sum \text{Moments} = 0$$

c) Trim Conditions

We will choose **symmetric, and steady state** flight condition.

$$\begin{aligned}
 u &= u_0, \quad v = 0, \quad w = w_0 \\
 p &= q = r = 0 \\
 \phi &= 0 \\
 \theta &= \theta_0 \\
 \psi &\text{ has no effect}
 \end{aligned}$$

Substitute by these conditions in the total forces equations at equilibrium state:

$$\begin{aligned}
 X_0 - mg \sin \theta_0 &= 0 \rightarrow X_0 = mg \sin \theta_0 \\
 Y_0 - mg \cos \theta_0 \sin \phi_0 &= 0 \rightarrow Y_0 = -mg \cos \theta_0 \sin \phi_0 \\
 Z_0 - mg \cos \theta_0 \cos \phi_0 &= 0 \rightarrow Z_0 = -mg \cos \theta_0 \cos \phi_0
 \end{aligned}$$

$$\begin{aligned}
 \therefore X &= X_0 + \Delta X = \Delta X + mg \sin \theta_0 \\
 Y &= Y_0 + \Delta Y = \Delta Y - mg \cos \theta_0 \sin \phi_0 \\
 Z &= Z_0 + \Delta Z = \Delta Z - mg \cos \theta_0 \cos \phi_0
 \end{aligned}$$

And the total force acting on the airplane (this value is the input which you will give to the RBD)

$$\begin{aligned}
 F_x &= X - mg \sin \theta = \Delta X + mg \sin \theta_0 - mg \sin \theta \\
 F_y &= Y + mg \cos \theta \sin \phi = \Delta Y + mg \cos \theta \sin \phi \\
 F_z &= Z + mg \cos \theta \cos \phi = \Delta Z - mg \cos \theta_0 \cos \phi_0 + mg \cos \theta \sin \phi
 \end{aligned}$$

$$\begin{aligned}
 \begin{pmatrix} F_x \\ F_y \\ F_z \\ L \\ M \\ N \end{pmatrix} &= \begin{pmatrix} \Delta X \\ \Delta Y \\ \Delta Z \\ \Delta L \\ \Delta M \\ \Delta N \end{pmatrix} + \begin{pmatrix} X_0 \\ Y_0 \\ Z_0 \\ L_0 \\ M_0 \\ N_0 \end{pmatrix} + \begin{pmatrix} -mg \sin \theta \\ mg \cos \theta \sin \phi \\ mg \cos \theta \cos \phi \\ 0 \\ 0 \\ 0 \end{pmatrix} \\
 \begin{pmatrix} X_0 \\ Y_0 \\ Z_0 \end{pmatrix} &= \begin{pmatrix} mg \sin \theta_0 \\ -mg \cos \theta_0 \sin \phi_0 \\ -mg \cos \theta_0 \cos \phi_0 \end{pmatrix}, \quad \begin{pmatrix} L_0 \\ M_0 \\ N_0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}
 \end{aligned}$$

d) Body axes of the airplane

It's any axes which are fixed in the airplane, with origin at the CG and because most aircraft are very nearly symmetrical, it is usual to assume exact symmetry

So, **XZ** plane is the plane of symmetry ($I_{xy} = I_{zy} = 0$), but for now there is no constraint on X and Z axes, as they can take any position as long, they are orthogonal and achieve RHL.

coinciding X axis to a certain position, we get specific type of Body axis they are mainly Three:

1. Principal Axes

a. Definition

Coinciding X and Z Axes with the principal axes of the vehicle, so that the remaining product of inertia I_{xz} vanishes.

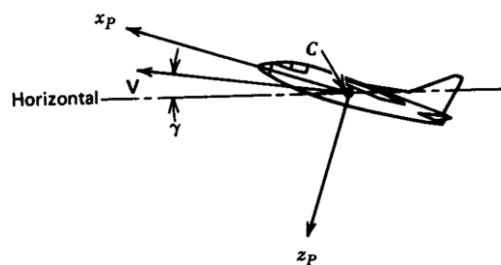


Figure 7 Principal Axes.

b. Cons

Simplifying equations as I_{xz} vanishes.

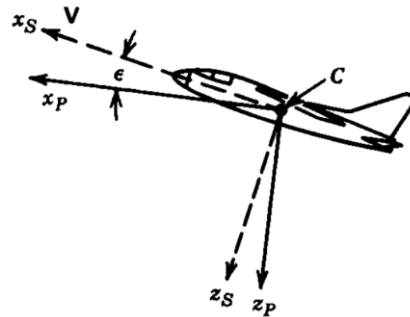
c. Pros

Any change in payload we need to set the angle of these axes.

2. Stability Axes

a. Definition

Aligning X axis with V_{tot} in a reference condition of steady symmetric flight.



$$\epsilon > 0$$

b. Cons

Simplifying equations as v and w vanish.

c. Pros

for different initial flight conditions, the axes are differently oriented in the airplane, and hence the values of I_x , I_z , and I_{xz} will vary from problem to problem.

$$I_x = I_{xp} \cos^2 \epsilon + I_{zp} \sin^2 \epsilon$$

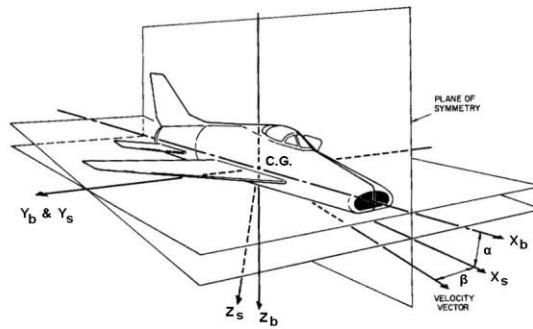
$$I_z = I_{xp} \sin^2 \epsilon + I_{zp} \cos^2 \epsilon$$

$$I_{zx} = \frac{1}{2} (I_{zp} - I_{xp}) \sin 2\epsilon$$

3. Body Axes

a. Definition

When the axes are neither principal axes nor stability axes, they are usually simply called body axes. In this case the x axis is usually fixed to a longitudinal reference line in the airplane.



Definition of Body and Stability Axes

Figure 8 difference between Body and Stability axes

b. Cons

These axes may be the most convenient ones to use if the aerodynamic data have been measured by a wind-tunnel balance that resolves the forces and moments into body-fixed axes rather than tunnel-fixed axes.

c. Pros

Using equation without Simplification due to axes.

e) Simulink Schematic

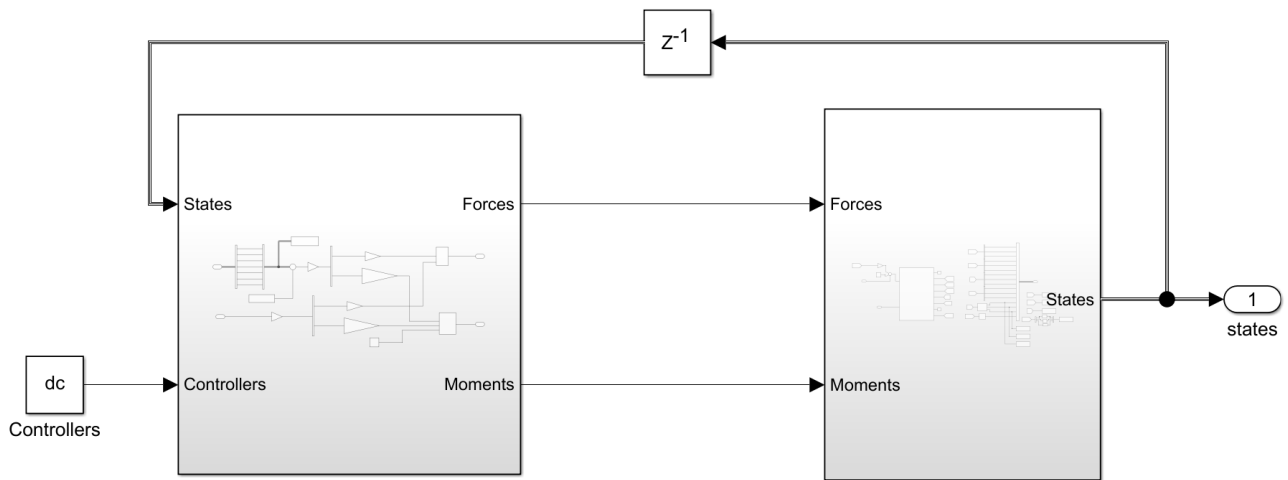


Figure 9 Simulator Schematic

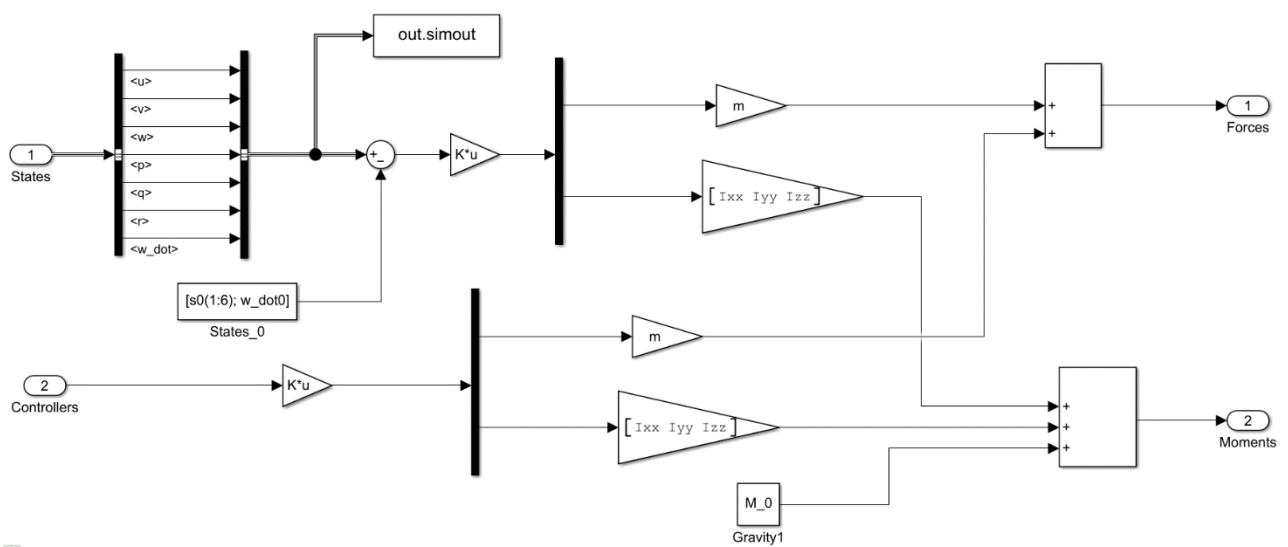


Figure 10 Subsystem-1 Schematic

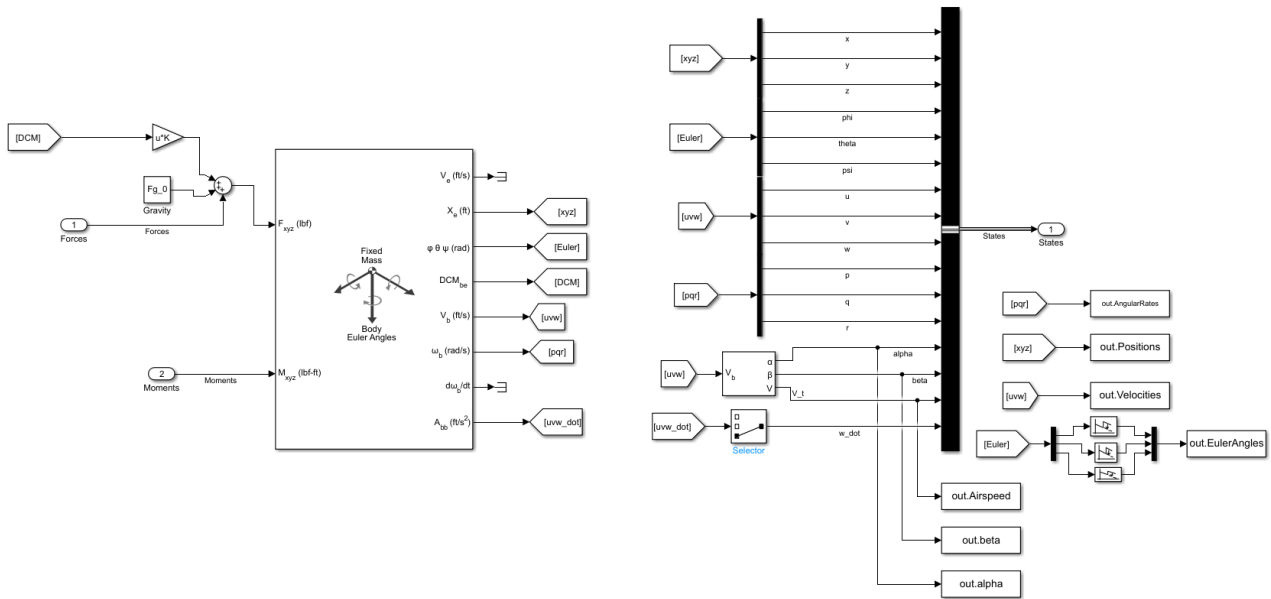
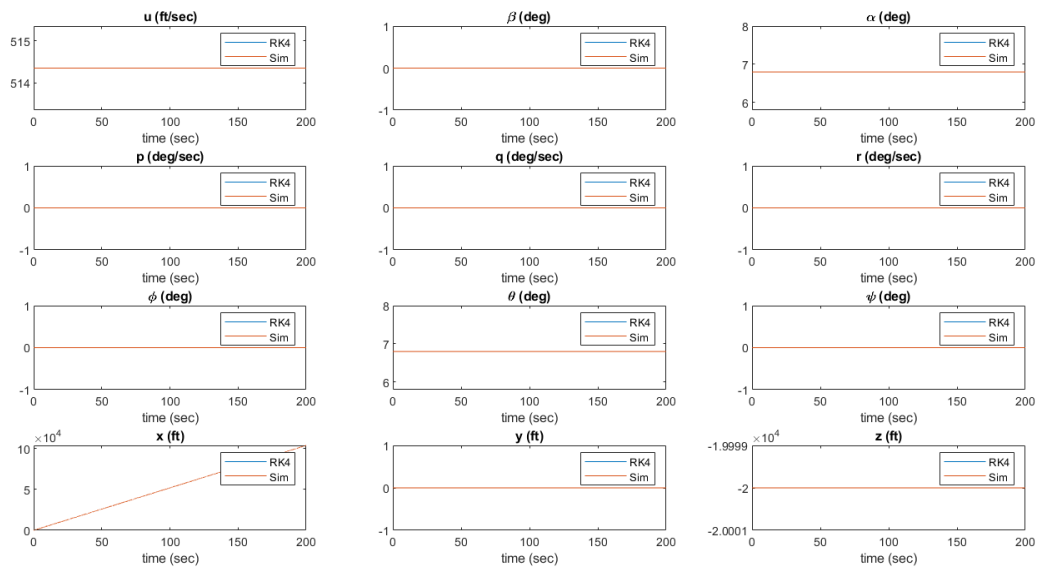
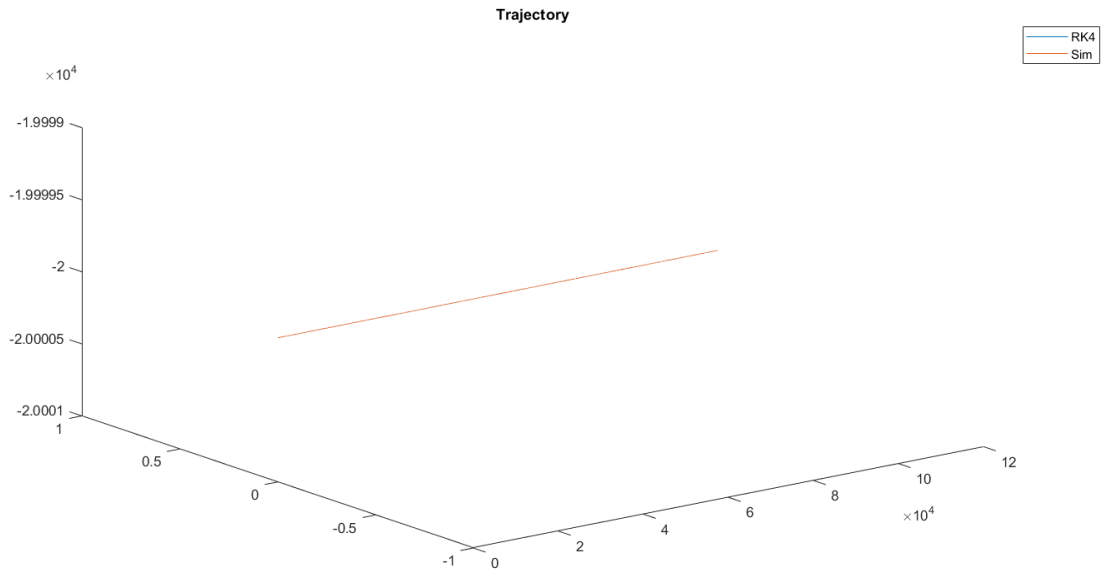


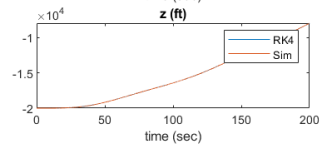
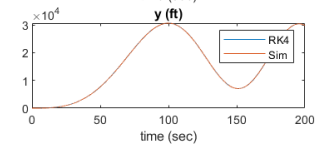
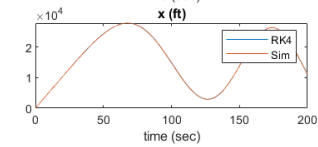
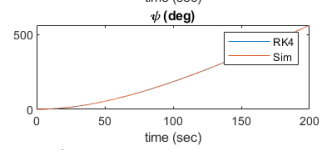
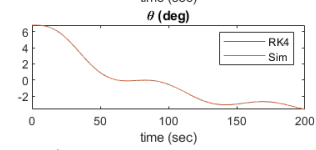
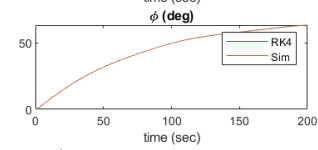
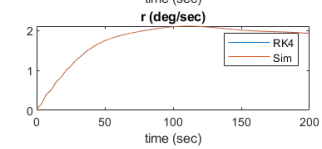
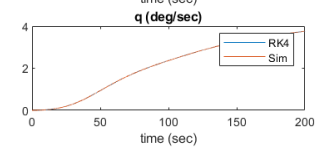
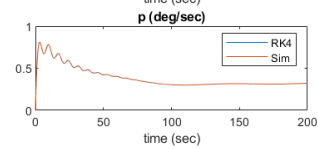
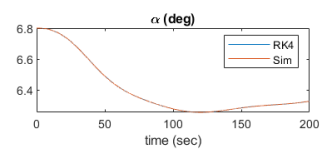
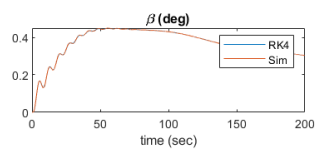
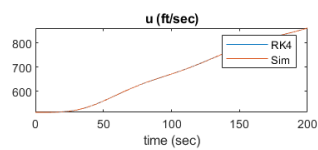
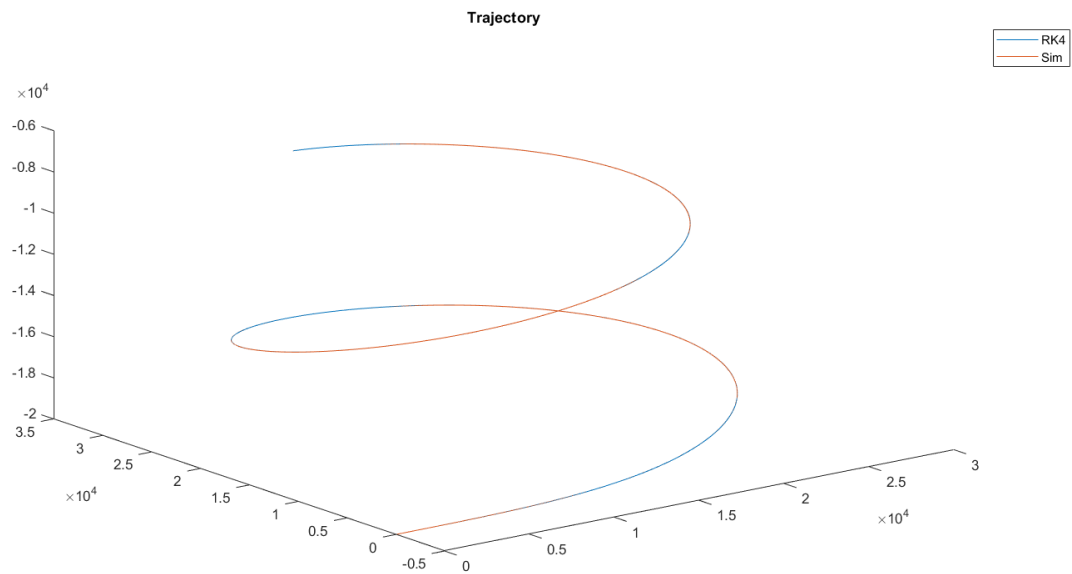
Figure 11 Subsystem-2 Schematic

f) Benchmark test (B747-F5)

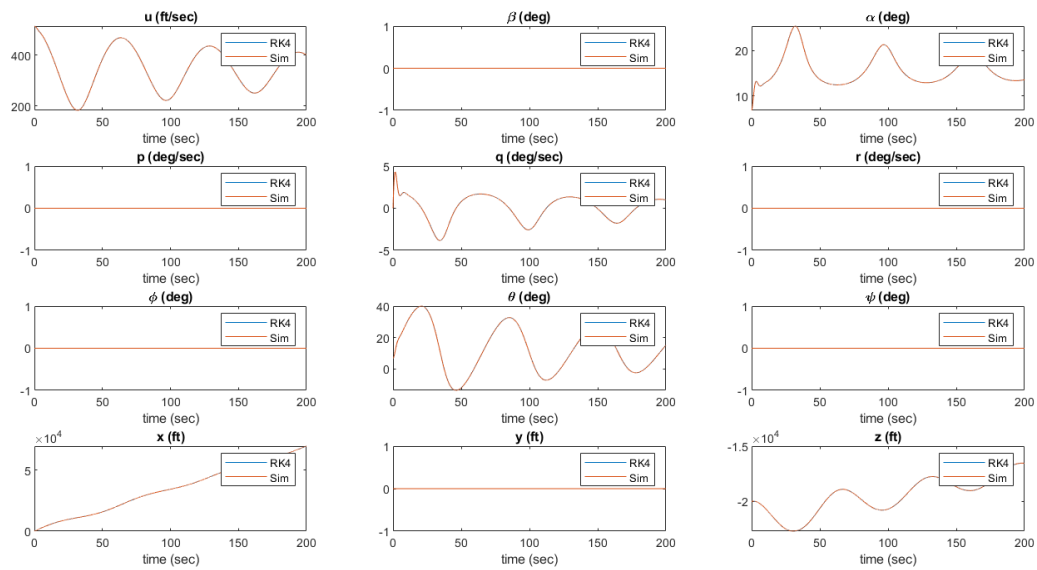
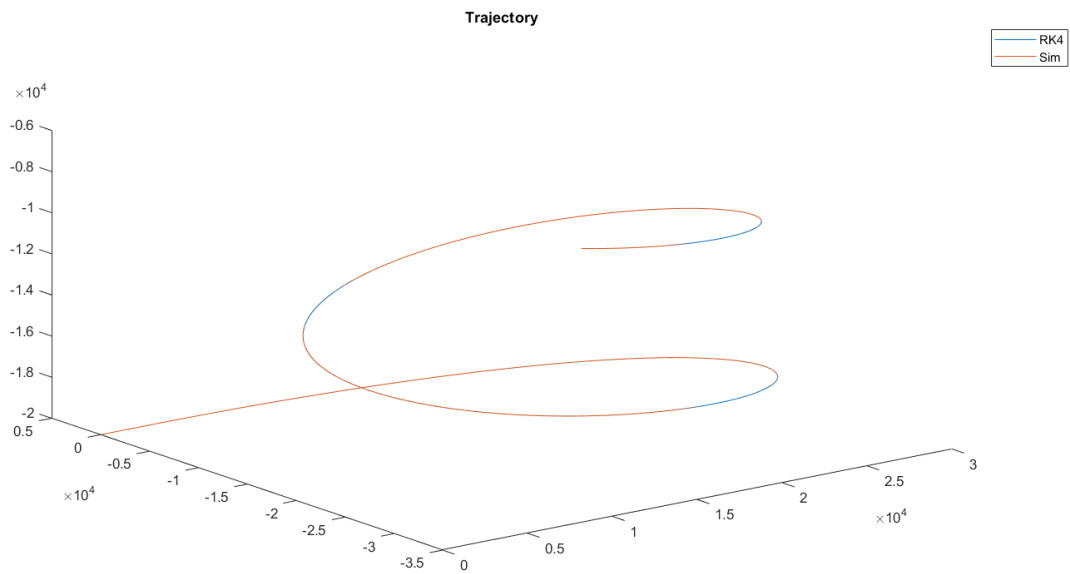
For No Input



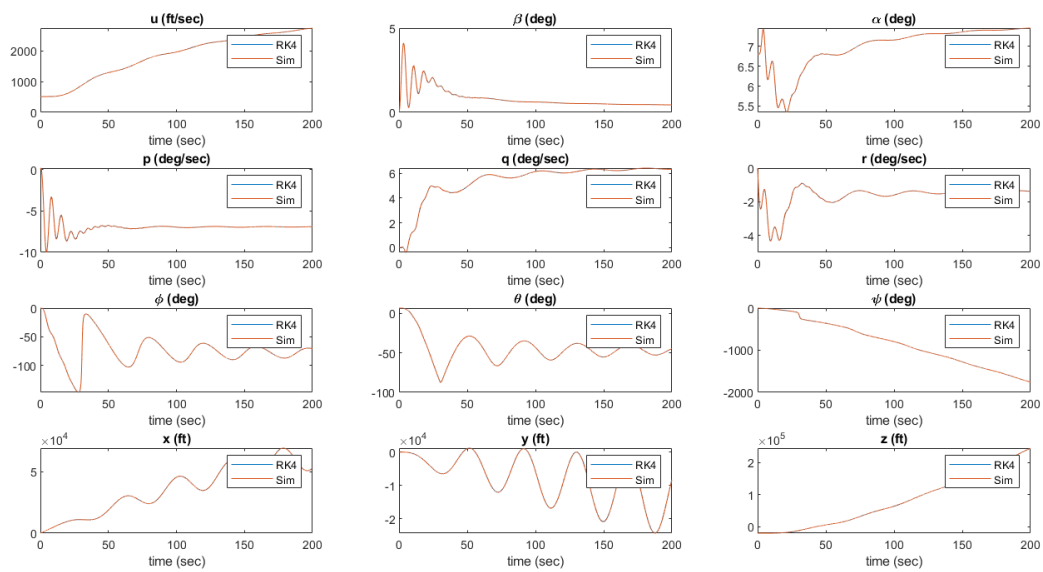
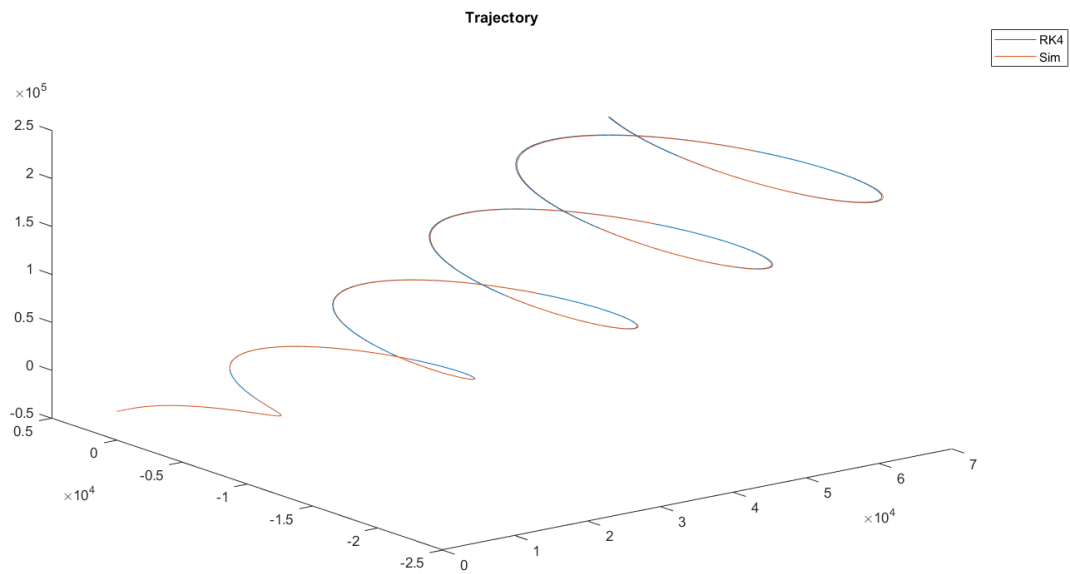
For +ve 5 degree aileron



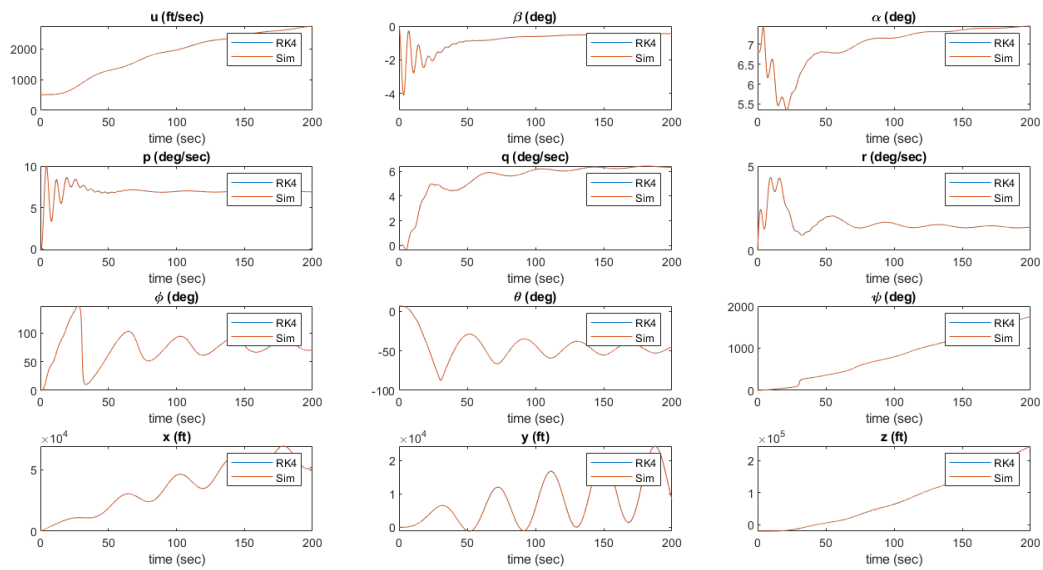
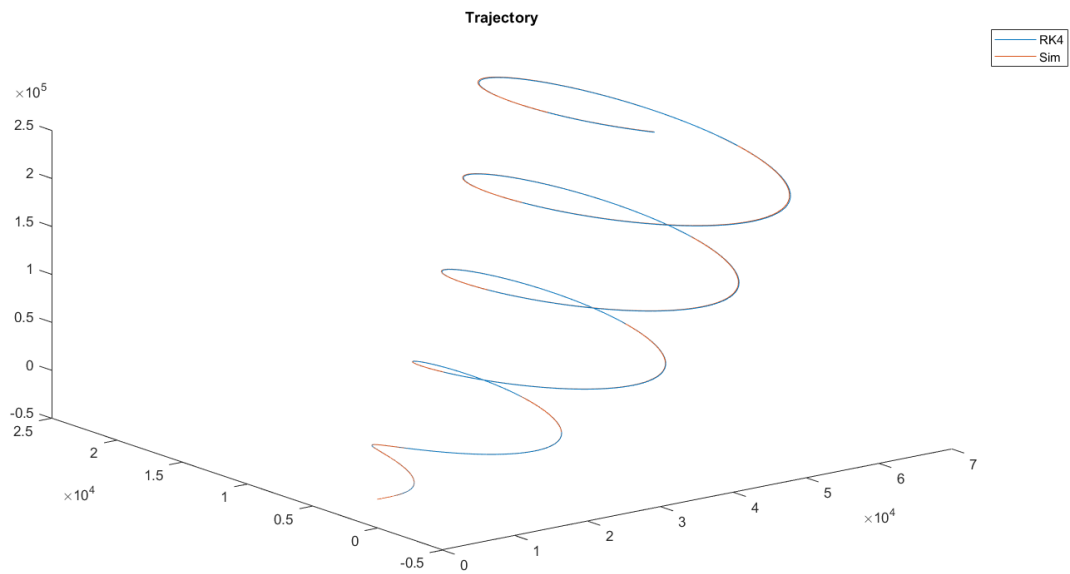
For -ve 5 degree aileron



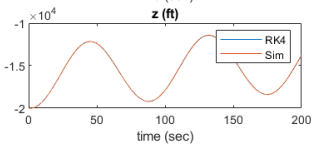
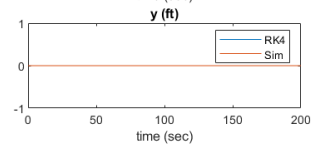
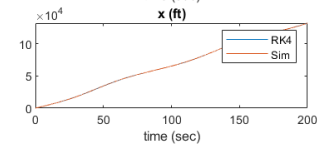
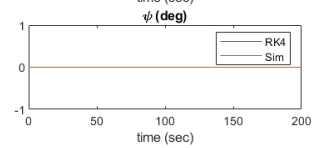
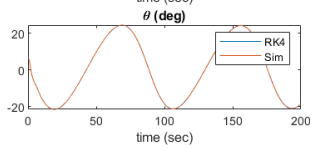
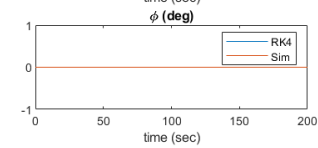
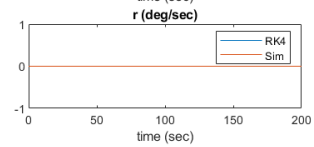
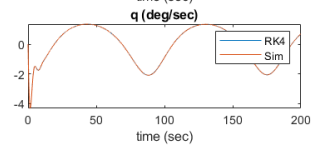
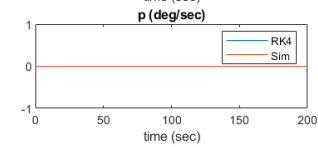
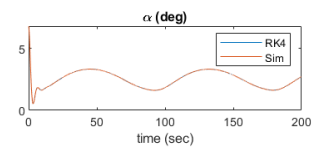
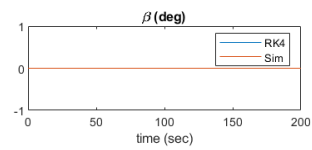
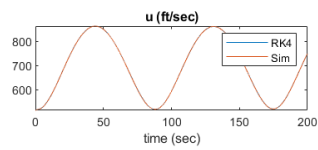
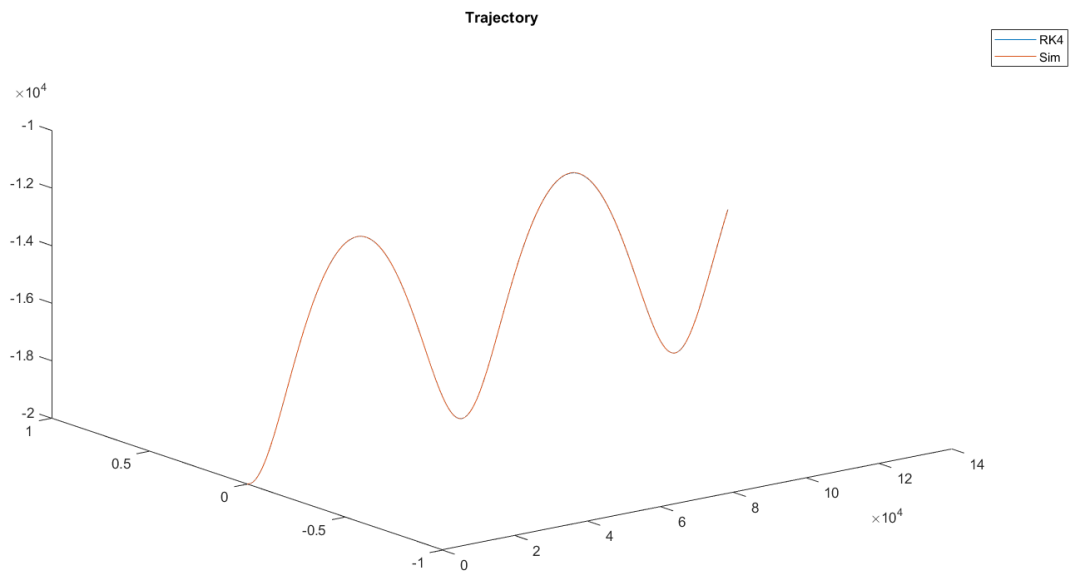
For +ve 5 degree rudder



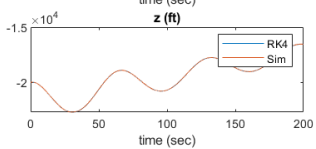
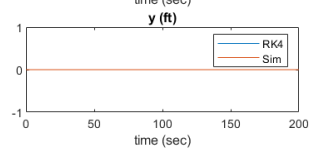
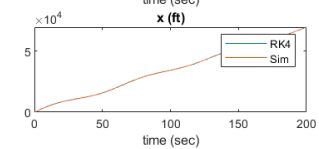
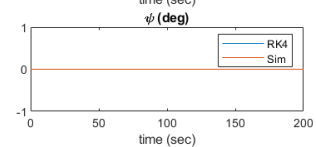
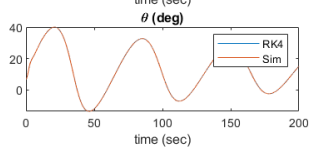
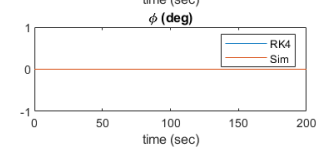
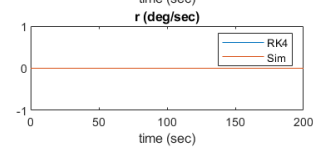
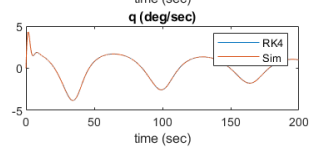
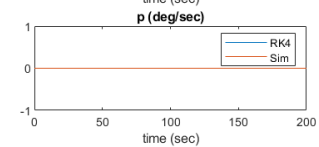
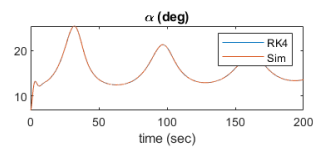
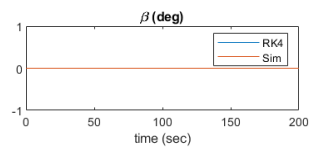
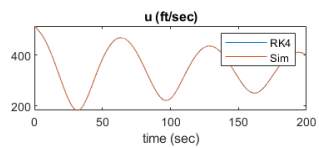
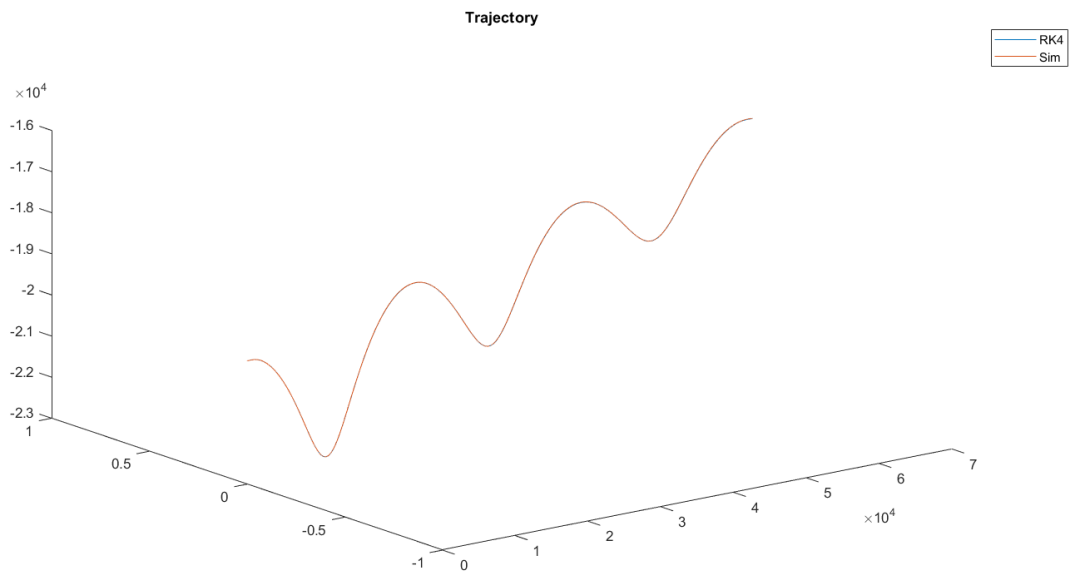
For -ve 5 degree rudder



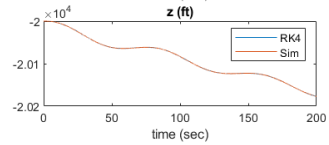
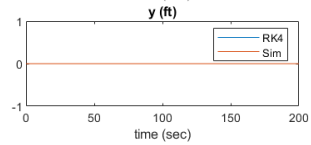
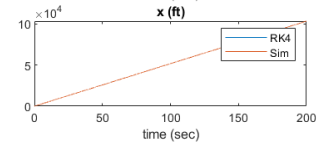
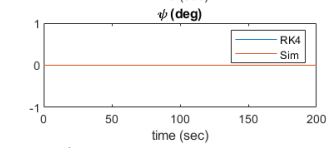
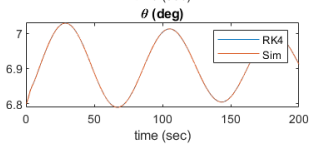
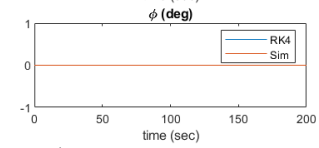
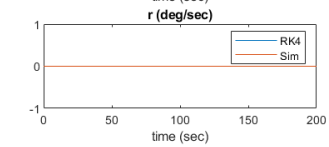
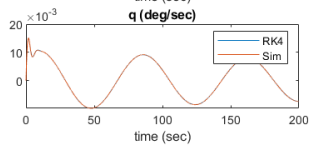
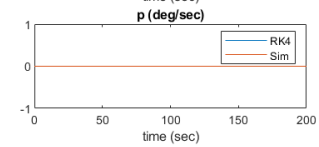
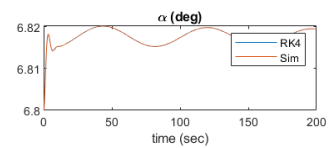
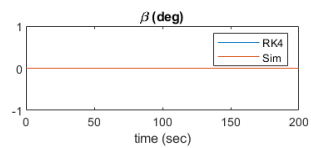
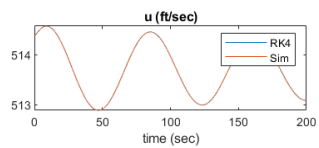
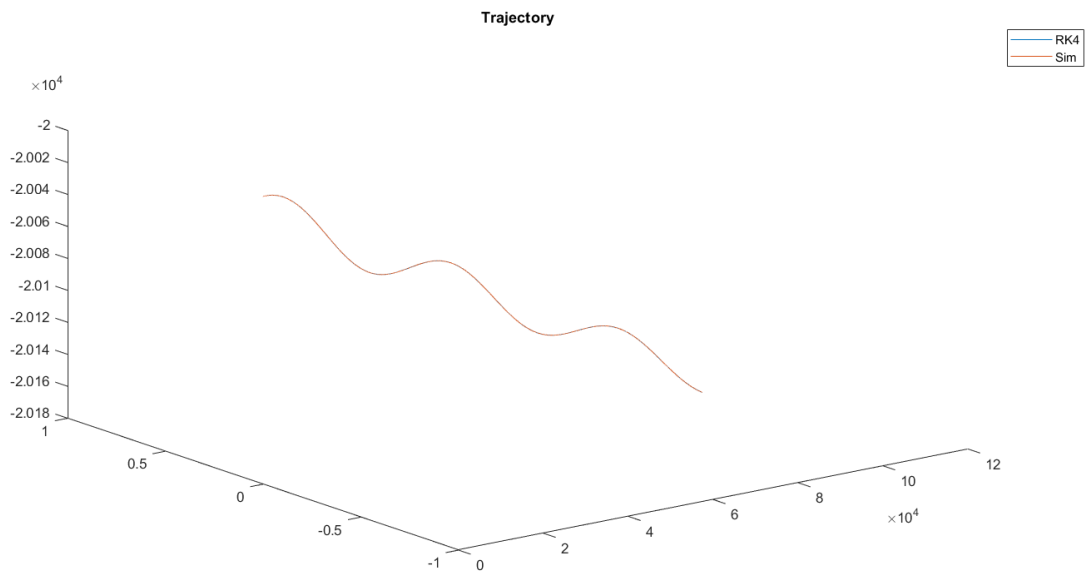
For +ve 5 degree elevator



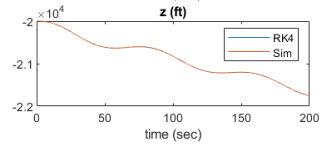
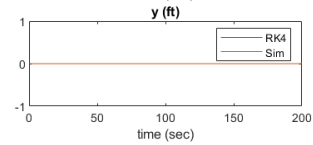
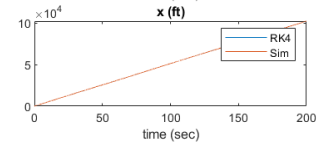
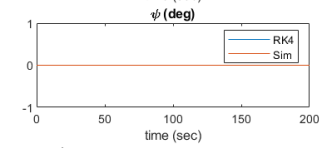
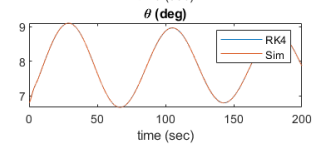
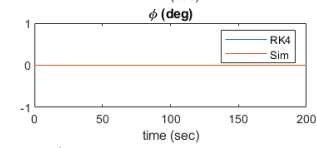
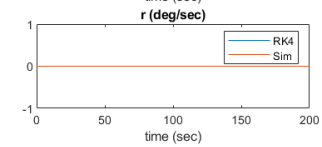
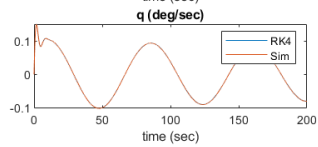
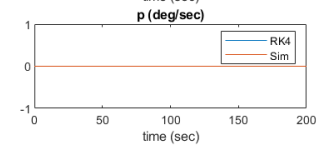
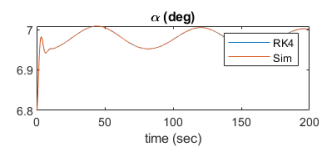
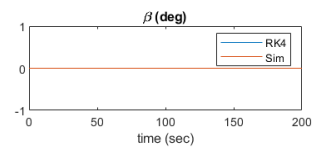
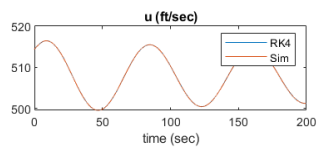
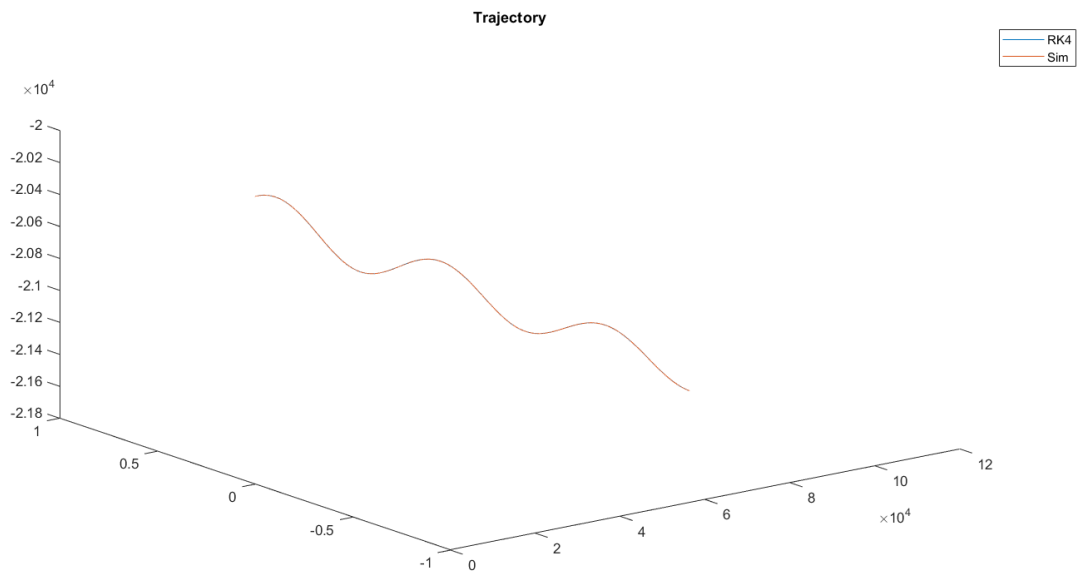
For -ve 5 degree elevator



For 1000 in thrust

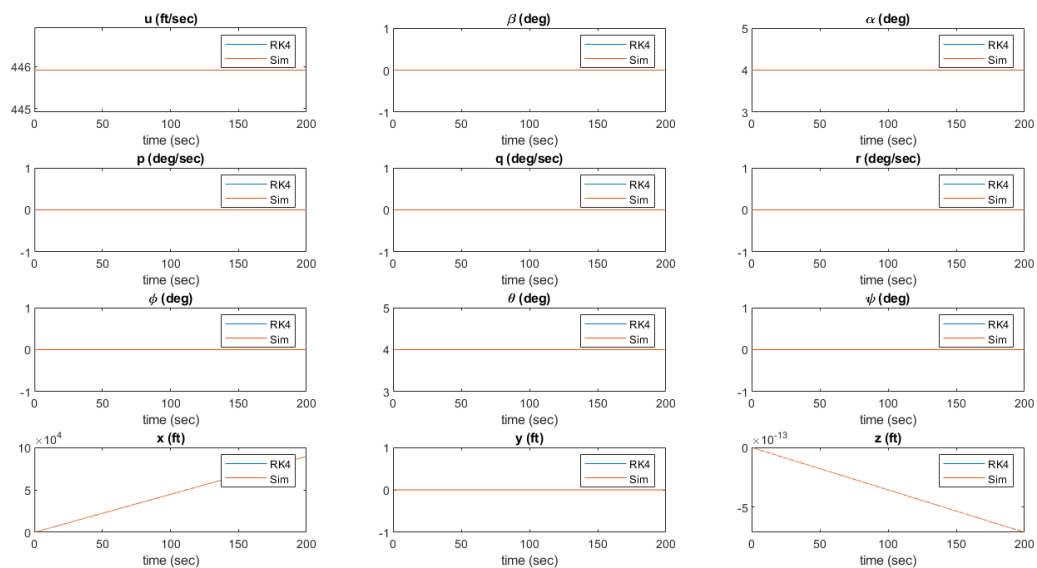
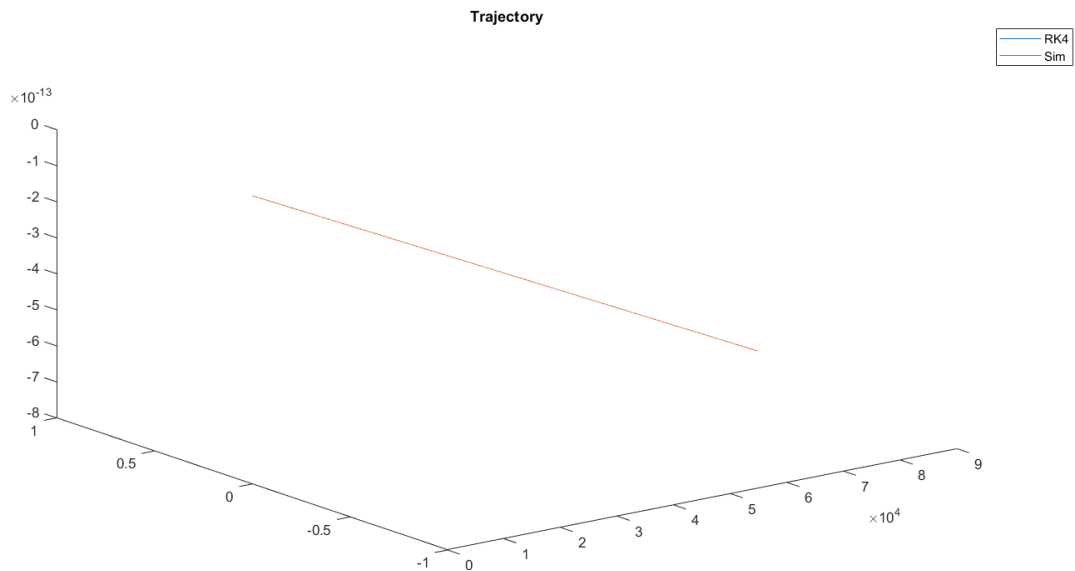


For 10000 in thrust

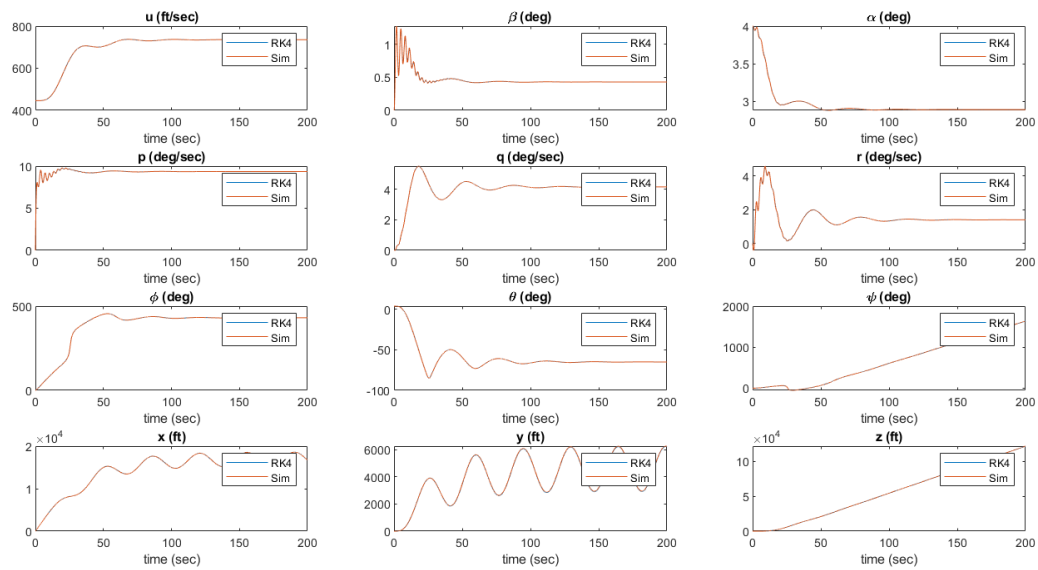
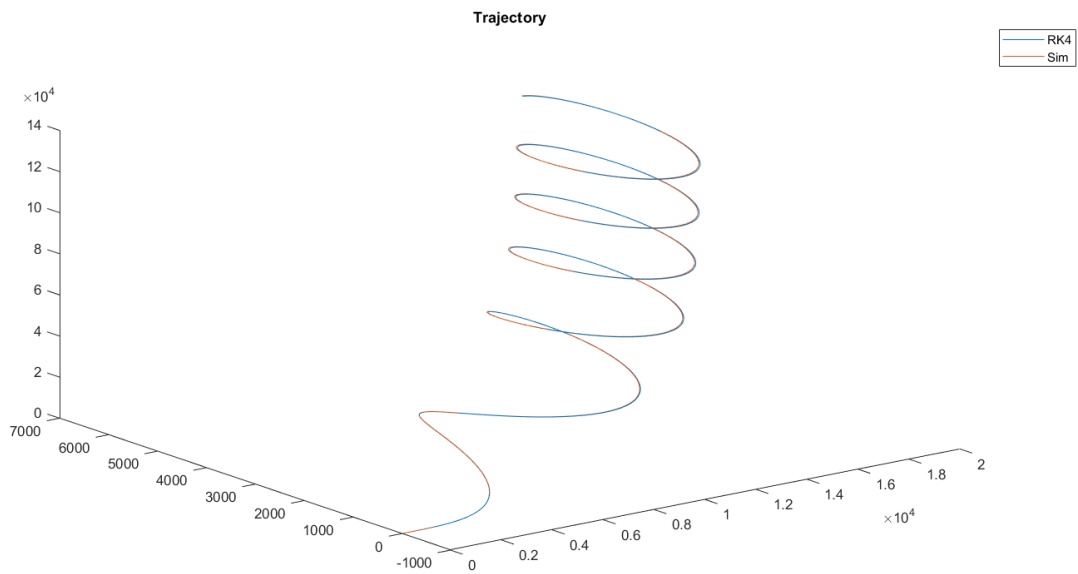


g) Our Airplane (JETSTAR-F3)

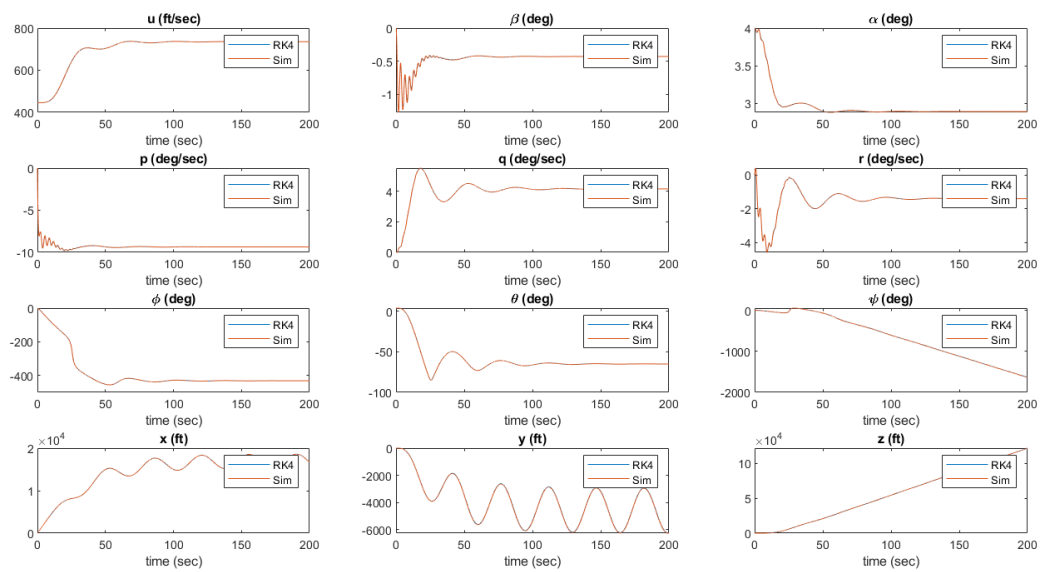
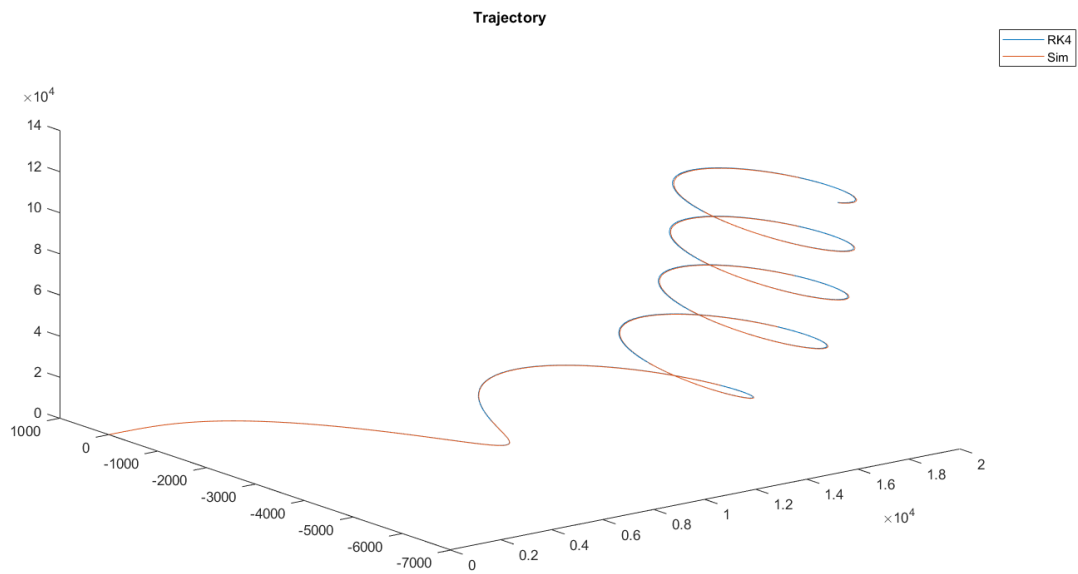
For No Input



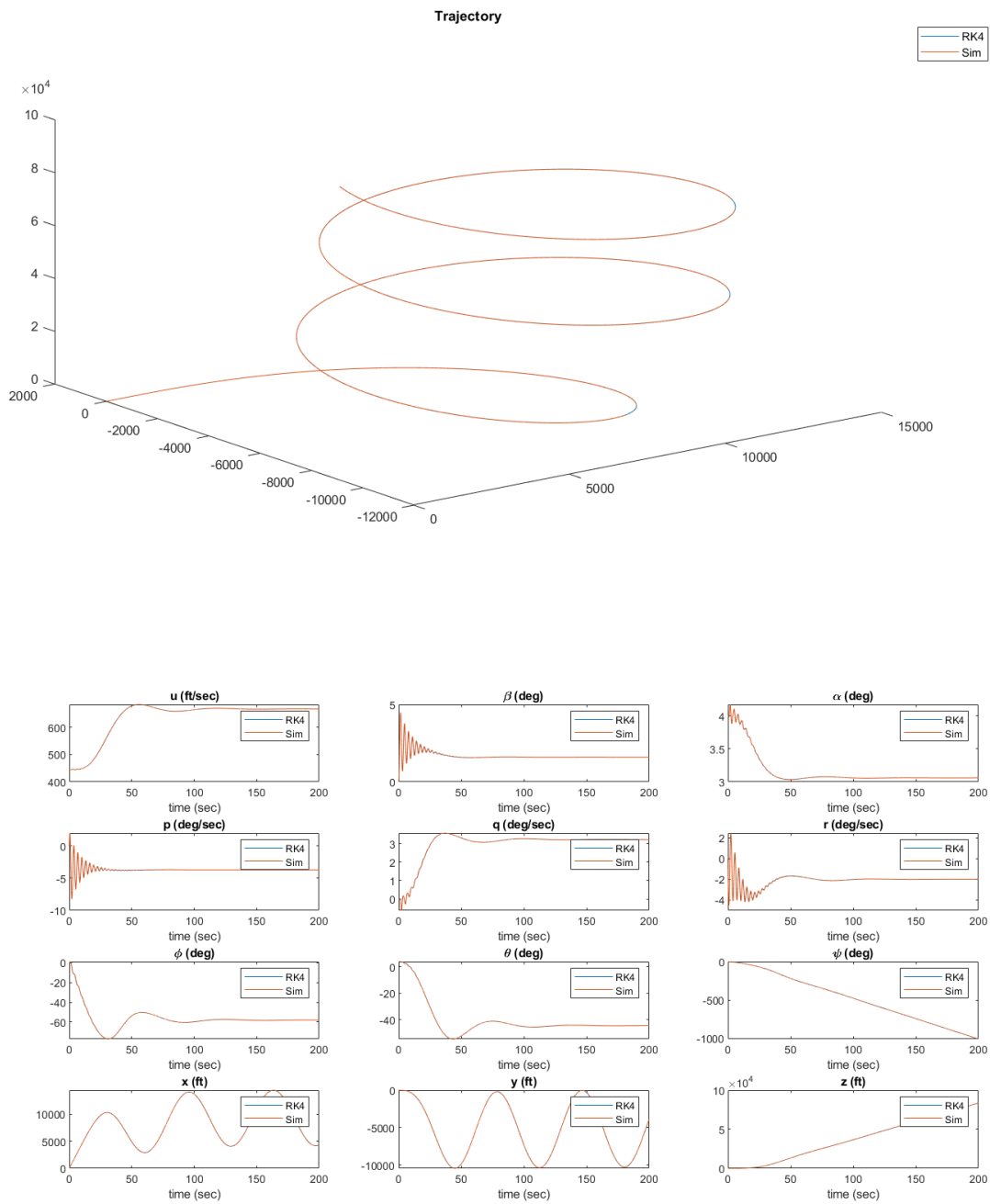
For +ve 5 degree aileron



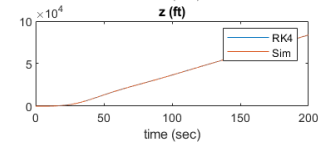
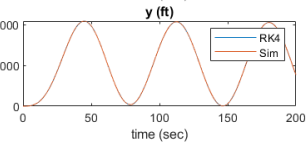
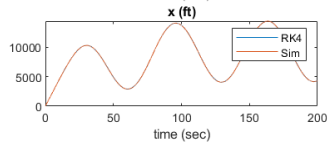
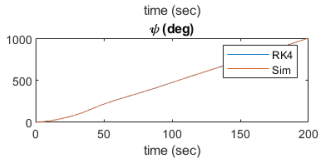
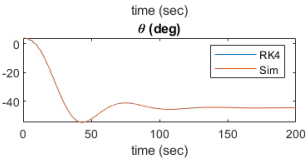
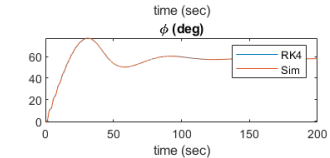
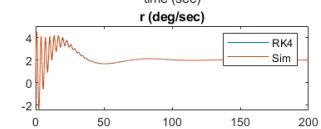
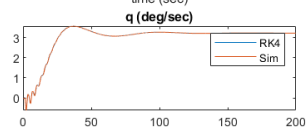
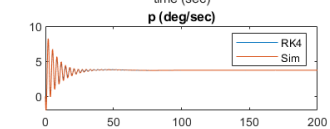
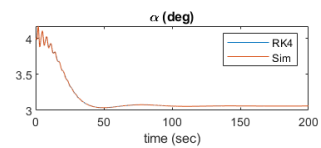
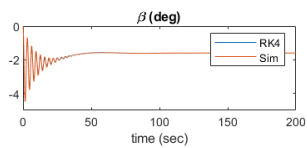
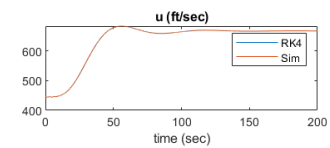
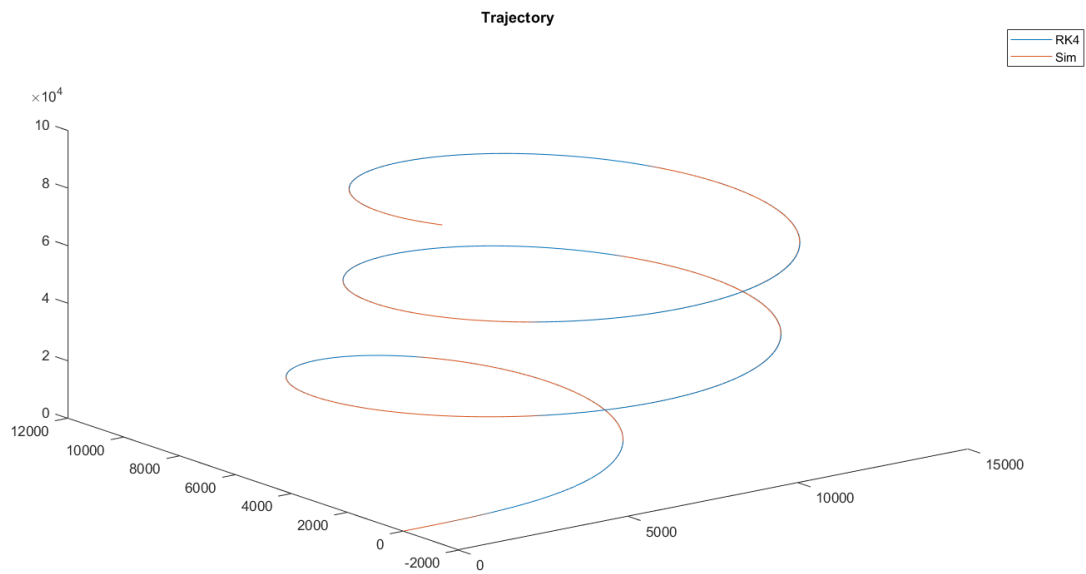
For -ve 5 degree aileron



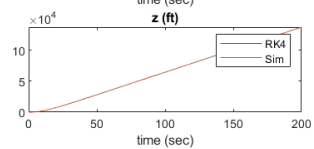
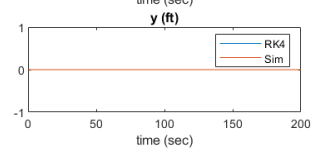
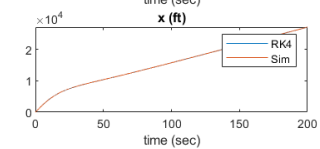
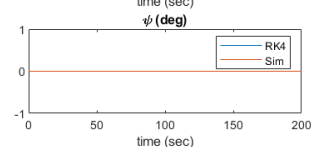
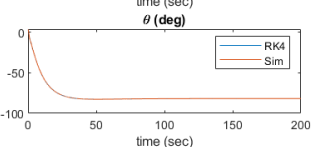
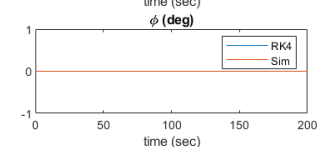
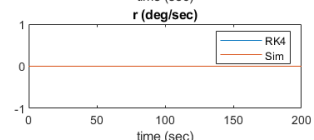
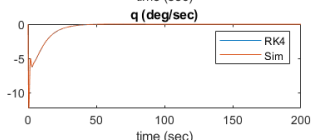
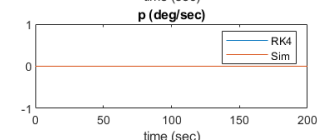
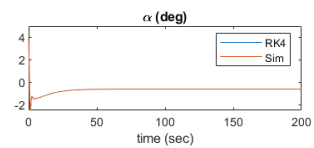
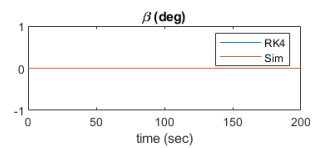
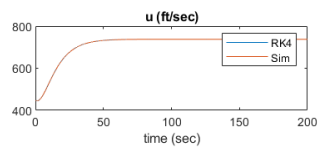
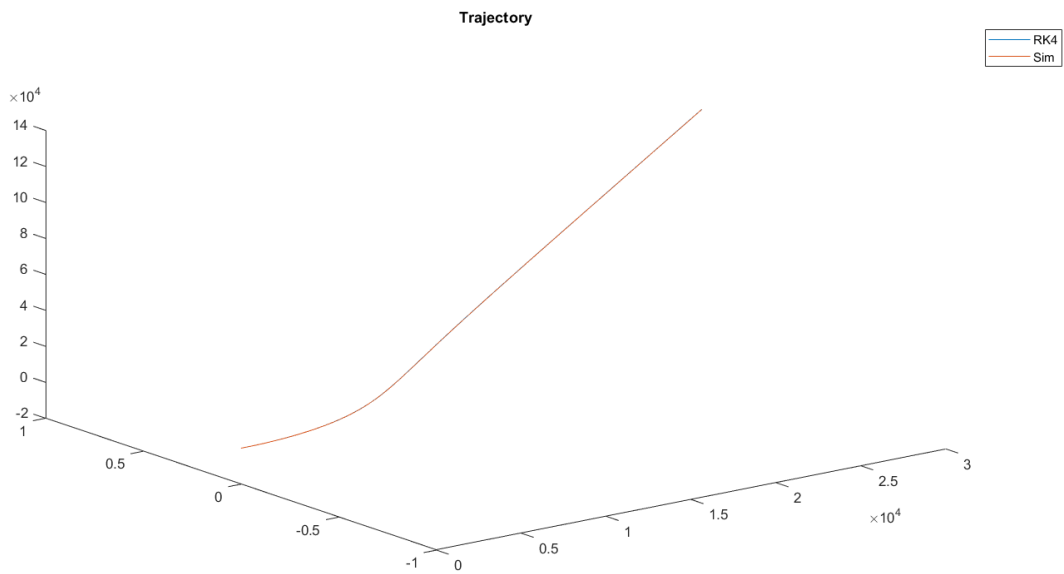
For +ve 5 degree rudder



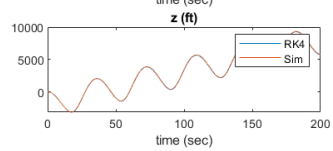
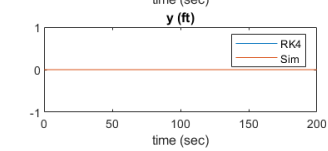
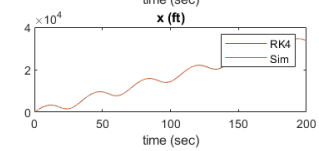
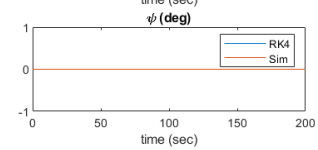
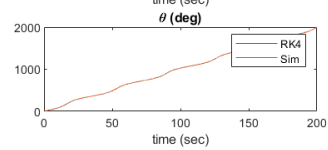
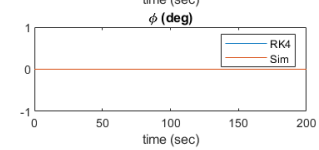
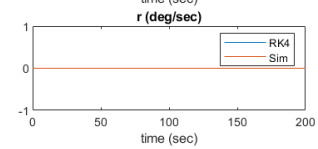
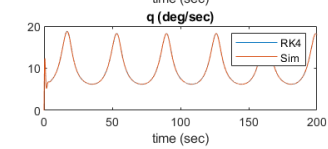
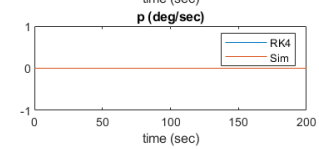
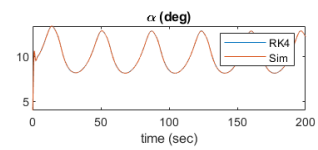
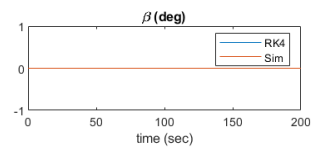
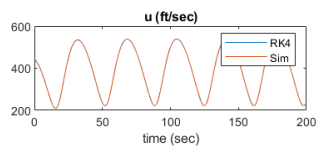
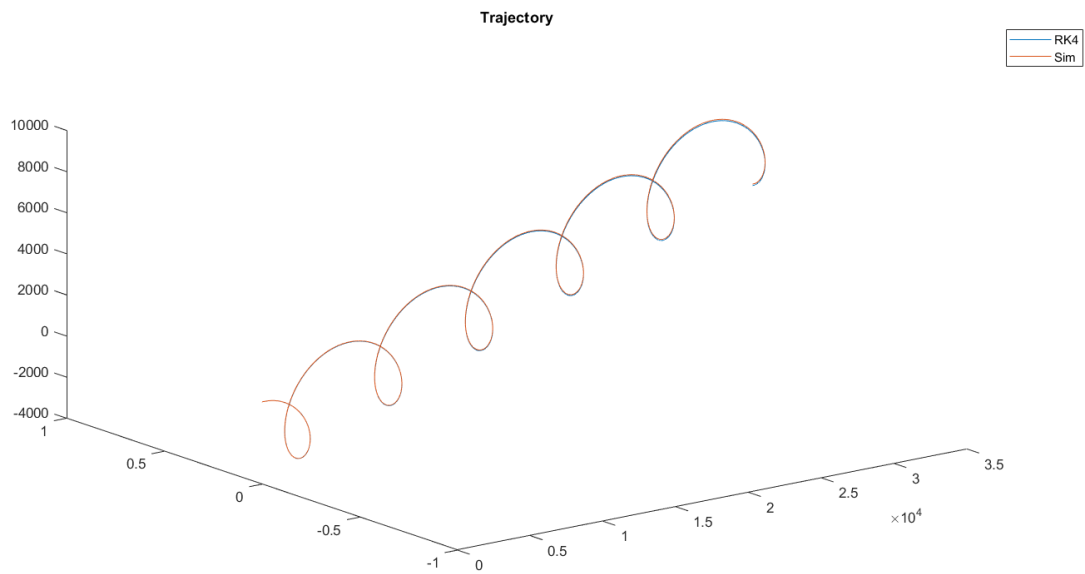
For -ve 5 degree rudder



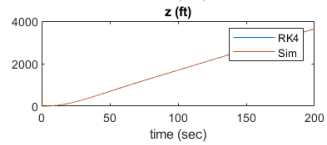
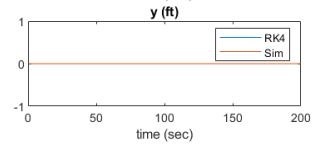
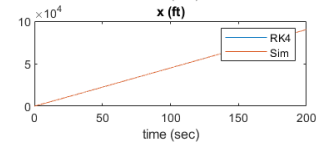
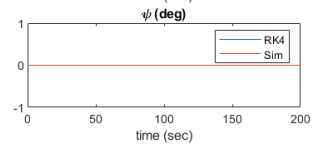
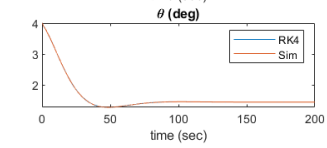
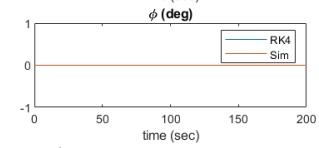
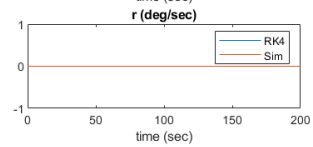
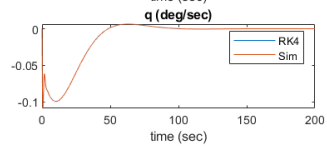
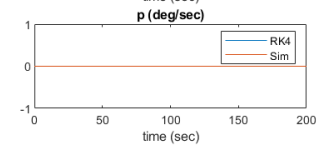
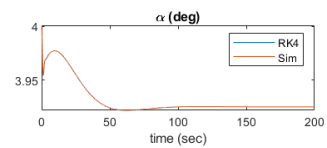
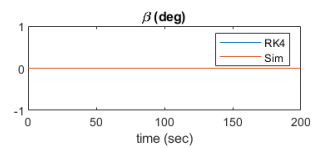
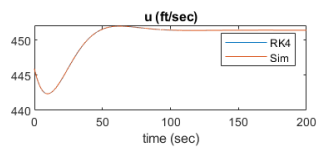
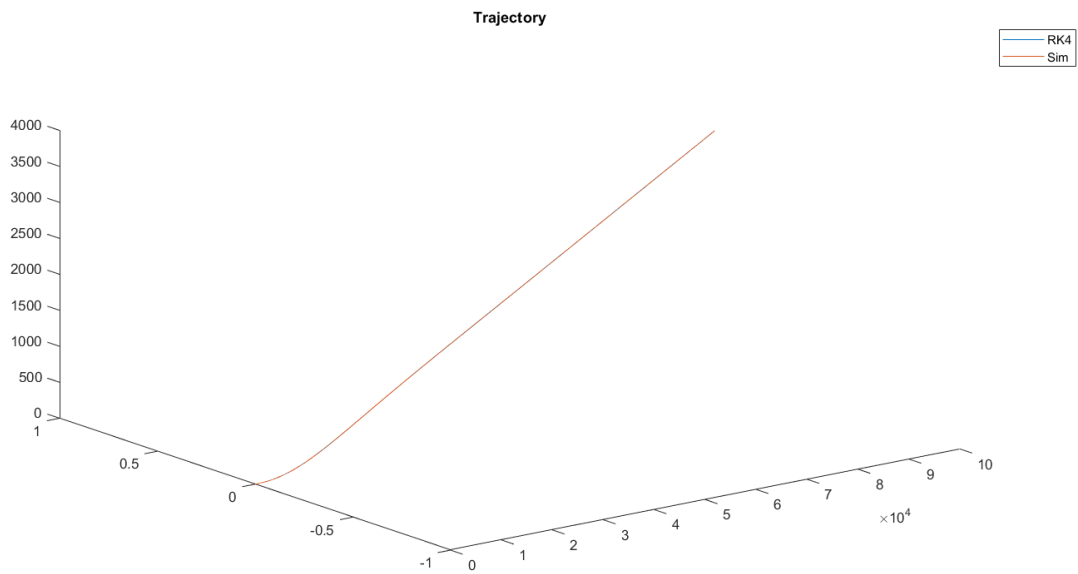
For +ve 5 degree elevator



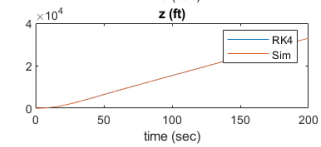
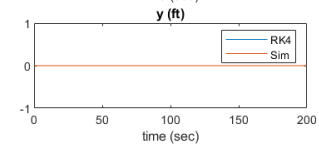
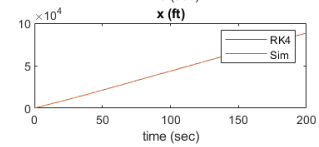
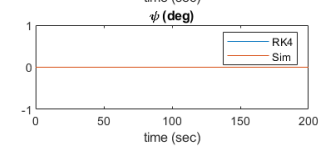
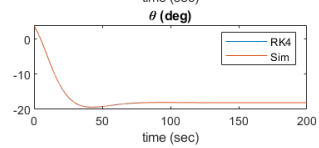
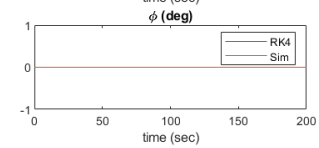
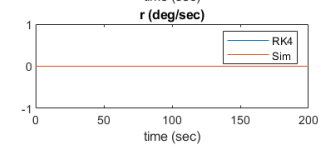
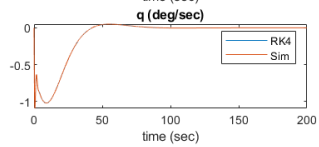
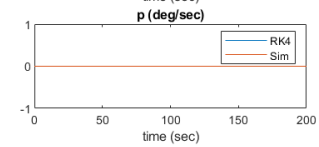
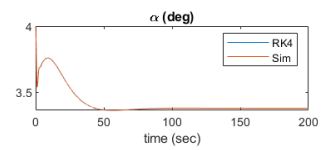
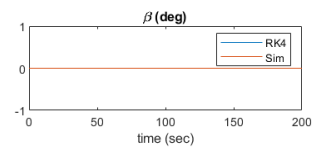
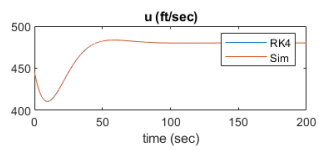
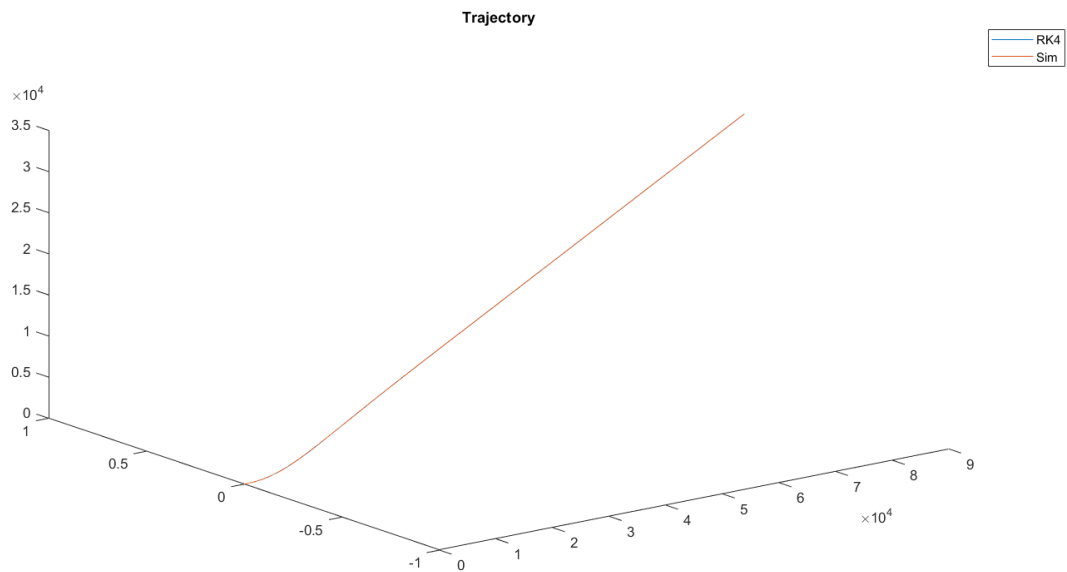
For -ve 5 degree elevator



For 1000 in thrust



For 10000 in thrust



Linearization and Approximations Part I

h) Linearization of 12 EOM for a fixed wing A/C

$$\left. \begin{aligned}
 X - mg S_\theta &= m(\dot{u} + qw - rv) \\
 Y + mg C_\theta S_\varphi &= m(\dot{v} + ru - pw) \\
 Z + mg C_\theta C_\varphi &= m(\dot{w} + pv - qu) \\
 L &= I_x \dot{p} - I_{xz} \dot{r} + qr(I_z - I_y) - I_{xz} pq \\
 M &= I_y \dot{q} + rp(I_x - I_z) - I_{xz}(p^2 - r^2) \\
 N &= -I_{xz} \dot{p} + I_z \dot{r} + pq(I_y - I_x) + I_{xz} qr \\
 \dot{\theta} &= q C_\varphi - r S_\varphi \\
 \dot{\varphi} &= p + q S_\varphi T_\theta + r C_\varphi T_\theta \\
 \dot{\psi} &= (q S_\varphi + r C_\varphi) \sec(\theta) \\
 \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} &= \begin{bmatrix} C_\theta C_\psi & S_\varphi S_\theta C_\psi - C_\varphi S_\psi & C_\psi S_\theta C_\psi + S_\varphi S_\psi \\ C_\theta S_\psi & S_\varphi S_\theta S_\psi + C_\varphi C_\psi & C_\varphi S_\theta S_\psi - S_\varphi C_\psi \\ -S_\theta & S_\varphi C_\theta & C_\varphi C_\theta \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix}
 \end{aligned} \right\} \text{12 EOM for a fixed wing A/C}$$

Applying Small disturbance theory ($A = A_o + \Delta A$)

$u = u_o + \Delta u$	$p = p_o + \Delta p$	$X = X_o + \Delta X$	$L = L_o + \Delta L$	$= u_o + \Delta u$
$v = v_o + \Delta v$	$q = q_o + \Delta q$	$Y = Y_o + \Delta Y$	$M = M_o + \Delta M$	$\theta = \theta_o + \Delta \theta$
$w = w_o + \Delta w$	$r = r_o + \Delta r$	$Y = Y_o + \Delta Y$	$N = N_o + \Delta N$	$\psi = \psi_o + \Delta \psi$

$v_o = p_o = q_o = r_o = \varphi_o = \psi_o = 0$ but $u_o, w_o, \theta_o \neq 0$ for Cruise condition

Change in propulsive forces

$$\begin{aligned}
 \Delta X/m &= (X_u \Delta u + X_w \Delta w + X_{\delta_e} \Delta \delta_e + X_{\delta_{th}} \Delta \delta_{th}) \\
 \Delta Y/m &= (Y_\beta \Delta \beta + Y_r \Delta r + Y_p \Delta p + Y_a \Delta \delta_a + Y_{\delta_r} \Delta \delta_r) \\
 \Delta Z/m &= (Z_u \Delta u + Z_w \Delta w + Z_{\dot{w}} \Delta \dot{w} + Z_q \Delta q + Z_{\delta_e} \Delta \delta_e + Z_{\delta_{th}} \Delta \delta_{th}) \\
 \Delta L/I_{xx} &= (L_\beta \Delta \beta + L_p \Delta p + L_r \Delta r + L_{\delta_r} \Delta \delta_r + L_{\delta_a} \Delta \delta_a) \\
 \Delta M/I_{yy} &= (M_u \Delta u + M_w \Delta w + M_{\dot{w}} \Delta \dot{w} + M_q \Delta q + M_{\delta_e} \Delta \delta_e + M_{\delta_{th}} \Delta \delta_{th}) \\
 \Delta N/I_{zz} &= (N_\beta \Delta \beta + N_p \Delta p + N_r \Delta r + N_{\delta_r} \Delta \delta_r + N_{\delta_a} \Delta \delta_a)
 \end{aligned}$$

where:

- $X_u = (\partial X / \partial u)/m$, $X_{\delta_T} = (\partial X / \partial \delta_T)/m$, $L_v = (\partial L / \partial v)/I_{xx}$, $M_u = (\partial M / \partial u)/I_{yy}$...
- $\Delta \delta_a, \Delta \delta_r, \Delta \delta_e, \Delta \delta_T$ are the change in aileron, rudder, elevator angles & throttle setting (thrust)
- IC's that came from applying the equilibrium equations at steady flight condition:

$$\begin{Bmatrix} X_o \\ Y_o \\ Z_o \end{Bmatrix} = \begin{Bmatrix} mg S_{\theta_o} \\ -mg C_{\theta_o} S_{\varphi_o} \\ -mg C_{\theta_o} C_{\varphi_o} \end{Bmatrix} = \begin{Bmatrix} mg S_{\theta_o} \\ 0 \\ -mg C_{\theta_o} \end{Bmatrix}, \begin{Bmatrix} L_o \\ M_o \\ N_o \end{Bmatrix} = \{0\}, \begin{Bmatrix} \dot{x}_o \\ \dot{y}_o \\ \dot{z}_o \end{Bmatrix} = \begin{Bmatrix} u_o C_{\theta_o} \\ 0 \\ -u_o S_{\theta_o} \end{Bmatrix}$$

Some approximations & trigonometric will be used in linearization:

$$\begin{aligned}\Delta(\text{variable}) * \Delta(\text{variable}) &\simeq 0 \\ \sin(\Delta(\text{variable})) &\simeq 0, \cos(\Delta(\text{variable})) \simeq \Delta(\text{variable}) \\ \sin(a + b) &= \sin(a) \cos(b) + \sin(b) \cos(a) \\ \cos(a + b) &= \cos(a) \cos(b) - \sin(a) \sin(b)\end{aligned}$$

i. Longitudinal calculations:

$$1) X - mg \sin(\theta) = m(\dot{u} + qw - rv)$$

i. Recast each variable in terms of a steady-state value and a perturbed value.

$$\begin{aligned}X_o + \Delta X - mg \sin(\theta_o + \Delta\theta) \\ = m(\dot{u}_o + \Delta\dot{u} + (q_o + \Delta q)(w_o + \Delta w) - (r_o + \Delta r)(v_o + \Delta v)) \\ \sin(a + b) = \sin(a) \cos(b) + \sin(b) \cos(a) \\ X_o + \Delta X - mg (\sin(\theta_o) \cos(\Delta\theta) + \sin(\Delta\theta) \cos(\theta_o)) \\ = m(\Delta\dot{u} + (\Delta q)(w_o + \Delta w) - (\Delta r)(\Delta v))\end{aligned}$$

ii. Apply the small-angle assumption to trig functions of perturbed angles.

$$\begin{aligned}\sin(a \rightarrow 0) &= a, \cos(a \rightarrow 0) = 1 \\ X_o + \Delta X - mg \sin(\theta_o) - mg \cos(\theta_o) \Delta\theta &= m(\Delta\dot{u} + \Delta q * w_o + \Delta q * \Delta w - \Delta r * \Delta v)\end{aligned}$$

iii. Assume products of small perturbations are negligible.

$$X_o + \Delta X - mg \sin(\theta_o) - mg \cos(\theta_o) \Delta\theta = m(\Delta\dot{u} + \Delta q * w_o)$$

iv. Remove the steady-state equation from the perturbed equation

$$\begin{aligned}\therefore \Delta X - mg \cos(\theta_o) \Delta\theta &= m(\Delta\dot{u} + \Delta q * w_o) \\ \therefore \Delta\dot{u} &= \frac{\Delta X}{m} - g \cos(\theta_o) \Delta\theta - \Delta q * w_o, \quad \frac{\Delta X}{m} = X_u \Delta u + X_w \Delta w + X_{\delta_e} \Delta\delta_e + X_{\delta_{th}} \Delta\delta_{th} \\ \therefore \Delta\dot{u} &= X_u \Delta u + X_w \Delta w + X_{\delta_e} \Delta\delta_e + X_{\delta_{th}} \Delta\delta_{th} - g \cos(\theta_o) \Delta\theta - \Delta q * w_o\end{aligned}$$

$$2) Z + mg \cos(\theta) * \cos(\varphi) = m(\dot{w} + pv - qu)$$

Same as (2)

$$\begin{aligned}\cos(a + b) &= \cos(a) \cos(b) - \sin(a) \sin(b) \\ \therefore (1 - Z_{\dot{w}}) \Delta\dot{w} &= Z_u \Delta u + Z_w \Delta w + (Z_q + u_o) \Delta q + Z_{\delta_e} \Delta\delta_e + Z_{\delta_{th}} \Delta\delta_{th} - g \sin(\theta_o) \Delta\theta \\ \therefore \Delta\dot{w} &= \frac{Z_u}{(1 - Z_{\dot{w}})} \Delta u + \frac{Z_w}{(1 - Z_{\dot{w}})} \Delta w + \frac{(Z_q + u_o)}{(1 - Z_{\dot{w}})} \Delta q + \frac{Z_{\delta_e}}{(1 - Z_{\dot{w}})} \Delta\delta_e + \frac{Z_{\delta_{th}}}{(1 - Z_{\dot{w}})} \Delta\delta_{th} \\ &\quad - \frac{g \sin(\theta_o)}{(1 - Z_{\dot{w}})} \Delta\theta\end{aligned}$$

$$3) \quad M = I_y \dot{q} + rp(I_x - I_z) - I_{xz}(p^2 - r^2)$$

$$\begin{aligned} \therefore \Delta \dot{q} &= M_u \Delta u + M_w \Delta w + M_{\dot{w}} \\ &\quad * \left(\frac{Z_u}{(1-Z_{\dot{w}})} \Delta u + \frac{Z_w}{(1-Z_{\dot{w}})} \Delta w + \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} \Delta q + \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} \Delta \delta_e \right. \\ &\quad \left. + \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \Delta \delta_{th} - \frac{g \sin(\theta_o)}{(1-Z_{\dot{w}})} \Delta \theta \right) + M_q \Delta q + M_{\delta_e} \Delta \delta_e + M_{\delta_{th}} \Delta \delta_{th} \end{aligned}$$

$$\begin{aligned} \therefore \Delta \dot{q} &= \left(M_u + M_{\dot{w}} \frac{Z_u}{(1-Z_{\dot{w}})} \right) \Delta u + \left(M_w + M_{\dot{w}} \frac{Z_w}{(1-Z_{\dot{w}})} \right) \Delta w \\ &\quad + \left(M_q + M_{\dot{w}} \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} \right) \Delta q + \left(M_{\delta_e} + M_{\dot{w}} \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} \right) \Delta \delta_e \\ &\quad + \left(M_{\delta_{th}} + M_{\dot{w}} \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \right) \Delta \delta_{th} - M_{\dot{w}} * \frac{g \sin(\theta_o)}{(1-Z_{\dot{w}})} \Delta \theta \end{aligned}$$

$$4) \quad \dot{\theta} = q C_\varphi - r S_\varphi$$

$$\therefore \Delta \dot{\theta} = \Delta q$$

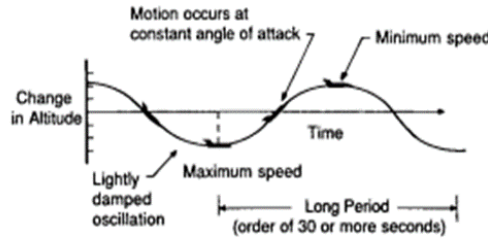
ii. Longitudinal State-Space Model:

- $\Delta \dot{u} = X_u \Delta u + X_w \Delta w + X_{\delta_e} \Delta \delta_e + X_{\delta_{th}} \Delta \delta_{th} - g \cos(\theta_o) \Delta \theta - \Delta q * w_o$
- $\therefore \Delta \dot{w} = \frac{Z_u}{(1-Z_{\dot{w}})} \Delta u + \frac{Z_w}{(1-Z_{\dot{w}})} \Delta w + \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} \Delta q + \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} \Delta \delta_e + \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \Delta \delta_{th} - \frac{g \sin(\theta_o)}{(1-Z_{\dot{w}})} \Delta \theta$
- $\therefore \Delta \dot{q} = \left(M_u + M_{\dot{w}} \frac{Z_u}{(1-Z_{\dot{w}})} \right) \Delta u + \left(M_w + M_{\dot{w}} \frac{Z_w}{(1-Z_{\dot{w}})} \right) \Delta w + \left(M_q + M_{\dot{w}} \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} \right) \Delta q +$
 $\left(M_{\delta_e} + M_{\dot{w}} \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} \right) \Delta \delta_e + \left(M_{\delta_{th}} + M_{\dot{w}} \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \right) \Delta \delta_{th} - M_{\dot{w}} * \frac{g \sin(\theta_o)}{(1-Z_{\dot{w}})} \Delta \theta$
- $\therefore \Delta \dot{\theta} = \Delta q$

$$\begin{aligned} \begin{bmatrix} \Delta \dot{u} \\ \Delta \dot{w} \\ \Delta \dot{q} \\ \Delta \dot{\theta} \end{bmatrix} &= \begin{bmatrix} X_u & X_w & -w_o & -g \cos(\theta_o) \\ \frac{Z_u}{(1-Z_{\dot{w}})} & \frac{Z_w}{(1-Z_{\dot{w}})} & \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} & \frac{-g \sin(\theta_o)}{(1-Z_{\dot{w}})} \\ M_u + M_{\dot{w}} \frac{Z_u}{(1-Z_{\dot{w}})} & M_w + M_{\dot{w}} \frac{Z_w}{(1-Z_{\dot{w}})} & M_q + M_{\dot{w}} \frac{(Z_q + u_o)}{(1-Z_{\dot{w}})} & -M_{\dot{w}} * \frac{g \sin(\theta_o)}{(1-Z_{\dot{w}})} \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \Delta u \\ \Delta w \\ \Delta q \\ \Delta \theta \end{bmatrix} \\ &\quad + \begin{bmatrix} X_{\delta_e} & X_{\delta_{th}} \\ \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} & \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \\ M_{\delta_e} + M_{\dot{w}} \frac{Z_{\delta_e}}{(1-Z_{\dot{w}})} & M_{\delta_{th}} + M_{\dot{w}} \frac{Z_{\delta_{th}}}{(1-Z_{\dot{w}})} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta \delta_e \\ \Delta \delta_{th} \end{bmatrix} \end{aligned}$$

For simple calculation of the aircraft response t we can use the next approximations.

Long period approximation (mode)



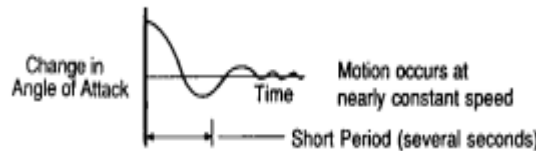
When the aircraft is exposed to some disturbances in the longitudinal direction a change in the altitude is formed due to this disturbance assuming that the angle of attack is constant the approximation to the long period mode is obtained by neglecting the pitching moment equation and assuming that the changes in angle of attack is zero.

$$\Delta \alpha = 0 \quad \Delta \dot{w} = 0$$

$$\Delta \alpha = \frac{\Delta w}{U_o} \quad \Delta \dot{q} = 0$$

$$\begin{bmatrix} \dot{u} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} X_u + \frac{w_o Z_u}{Z_q + u_o} & -g \cos(\theta_o) - \frac{w_o g \sin(\theta_o)}{Z_q + u_o} \\ \frac{-Z_u}{Z_q + u_o} & \frac{g \sin(\theta_o)}{Z_q + u_o} \end{bmatrix} \begin{bmatrix} u \\ \theta \end{bmatrix} + \begin{bmatrix} X_{\delta_e} + \frac{w_o Z_{\delta_e}}{Z_q + u_o} & X_{\delta_{th}} + \frac{w_o Z_{\delta_{th}}}{Z_q + u_o} \\ \frac{-Z_{\delta_e}}{Z_q + u_o} & \frac{-Z_{\delta_{th}}}{Z_q + u_o} \end{bmatrix} \begin{bmatrix} \Delta \delta_e \\ \Delta \delta_{th} \end{bmatrix}$$

Short period approximation (mode)



The same definition which we applied in the previous mode we will use here but with taking in consideration that the disturbance will damp quickly and but there will be a change in angle of attack and neglecting the change in velocity.

$$\Delta u = 0 \quad \Delta \dot{\theta} = 0$$

$$M_\alpha = \frac{1}{I_y} * \frac{\partial M}{\partial \alpha} = \frac{1}{I_y} * \frac{\partial M}{\partial \left(\frac{\Delta w}{u_o} \right)} = \frac{u_o}{I_y} * \frac{\partial M}{\partial w} = u_o M_w$$

$$Z_\alpha = u_o Z_w$$

$$M_{\dot{\alpha}} = u_o M_{\dot{w}}$$

$$\begin{bmatrix} \dot{\alpha} \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \frac{Z_\alpha}{u_o} & 1 \\ M_\alpha + \frac{M_{\dot{\alpha}} Z_\alpha}{u_o} & M_q + M_{\dot{\alpha}} \end{bmatrix} \begin{bmatrix} \alpha \\ q \end{bmatrix} + \begin{bmatrix} Z_{\delta_e} & Z_{\delta_{th}} \\ M_{\delta_e} + \frac{M_\alpha Z_{\delta_e}}{u_o} & M_{\dot{\alpha}} + \frac{M_{\dot{\alpha}} Z_{\delta_{th}}}{u_o} \end{bmatrix} \begin{bmatrix} \Delta \delta_e \\ \Delta \delta_{th} \end{bmatrix}$$

and by solving the characteristic equation of the system and getting its Eigen values we finally reach that;

$$w_n = \sqrt{(M_q \frac{Z_\alpha}{u_o} - M_\alpha)}$$

$$\zeta = \frac{\frac{Z_\alpha}{u_o} + M_q + M_\alpha}{-2w_n}$$

iii. Lateral calculations:

$$1) Y + mg C_\theta S_\varphi = m(\dot{v} + ru - pw)$$

$$\Delta \dot{v} = Y_\beta \Delta \beta + Y_r \Delta r + Y_p \Delta p + Y_{\delta a} \Delta \delta_a + Y_{\delta r} \Delta \delta_r + g \cos(\theta_o) \Delta \varphi - \Delta r * u_o + w_o * \Delta p$$

$$\Delta \dot{\beta} = \frac{\Delta \dot{v}}{V_{to}}$$

$$\therefore \Delta \dot{\beta} = \frac{Y_\beta}{V_{to}} \Delta \beta + \frac{Y_r - u_o}{V_{to}} \Delta r + \frac{Y_p + w_o}{V_{to}} \Delta p + \frac{Y_{\delta a}}{V_{to}} \Delta \delta_a + \frac{Y_{\delta r}}{V_{to}} \Delta \delta_r + \frac{g}{V_{to}} \cos(\theta_o) \Delta \varphi$$

$$2) L = I_x \dot{p} - I_{xz} \dot{r} + qr(I_z - I_y) - I_{xz} pq$$

$$eq(1): \Delta \dot{p} - \frac{I_{xz}}{I_x} * \Delta \dot{r} = \frac{\Delta L}{I_x} = L_\beta \Delta \beta + L_p \Delta p + L_r \Delta r + L_{\delta r} \Delta \delta_r + L_{\delta a} \Delta \delta_a$$

$$3) N = -I_{xz} \dot{p} + I_z \dot{r} + pq(I_y - I_x) + I_{xz} qr$$

$$eq(2): \Delta \dot{r} - \frac{I_{xz}}{I_z} * \Delta \dot{p} = \frac{\Delta N}{I_z} = N_\beta \Delta \beta + N_p \Delta p + N_r \Delta r + N_{\delta r} \Delta \delta_r + N_{\delta a} \Delta \delta_a$$

Solve eq(1) and eq(2) in $(\Delta \dot{r} \quad \Delta \dot{p})$ we get:

$$\therefore \Delta \dot{p} = \left(\frac{\Delta L}{I_x} + \frac{I_{xz}}{I_x} * \frac{\Delta N}{I_z} \right) * G, \text{ where } G = \frac{I_x I_z}{I_x I_z - (I_{xz})^2}$$

$$\therefore \Delta \dot{r} = \left(\frac{I_{xz}}{I_z} * \frac{\Delta L}{I_x} + \frac{\Delta N}{I_z} \right) * G, \text{ where } G = \frac{I_x I_z}{I_x I_z - (I_{xz})^2}$$

$$\frac{\Delta L}{I_x} = \sum L_i \Delta i, \quad \frac{\Delta N}{I_z} = \sum N_i \Delta i$$

$$\therefore \Delta \dot{p} = \sum G * \left(L_i + \frac{I_{xz}}{I_x} * N_i \right) \Delta i = \sum L_i^* \Delta i = L_\beta^* \Delta \beta + L_p^* \Delta p + L_r^* \Delta r + L_{\delta r}^* \Delta \delta_r + L_{\delta a}^* \Delta \delta_a$$

$$\therefore \Delta \dot{r} = \sum G * \left(N_i + \frac{I_{xz}}{I_z} * L_i \right) \Delta i = \sum N_i^* \Delta i = N_\beta^* \Delta \beta + N_p^* \Delta p + N_r^* \Delta r + N_{\delta r}^* \Delta \delta_r + N_{\delta a}^* \Delta \delta_a$$

$$4) \dot{\varphi} = p + q \sin(\varphi) \tan(\theta) + r \cos(\varphi) \tan(\theta)$$

$$\dot{\varphi} \cos(\theta) = p \cos(\theta) + q \sin(\varphi) \sin(\theta) + r \cos(\varphi) \sin(\theta)$$

$$\therefore \Delta \dot{\varphi} = \Delta p + \Delta r * \tan(\theta_o)$$

$$5) \dot{\psi} = (q S_{\varphi} + r C_{\varphi}) \sec(\theta)$$

$$\therefore \Delta \dot{\psi} = \Delta r * \sec(\theta_0)$$

Lateral State-Space Model:

- $\therefore \Delta \dot{\beta} = \frac{Y_{\beta}}{V_{to}} \Delta \beta + \frac{Y_r - u_o}{V_{to}} \Delta r + \frac{Y_p + w_o}{V_{to}} \Delta p + \frac{Y_{\delta a}}{V_{to}} \Delta \delta_a + \frac{Y_{\delta r}}{V_{to}} \Delta \delta_r + \frac{g}{V_{to}} \cos(\theta_o) \Delta \varphi$
- $\therefore \Delta \dot{p} = \sum G * \left(L_i + \frac{I_{xz}}{I_x} * N_i \right) \Delta i = \sum L'_i \Delta i = L'_{\beta} \Delta \beta + L'_p \Delta p + L'_r \Delta r + L'_{\delta_r} \Delta \delta_r + L'_{\delta_a} \Delta \delta_a$
- $\therefore \Delta \dot{r} = \sum G * \left(N_i + \frac{I_{xz}}{I_z} * L_i \right) \Delta i = \sum N'_i \Delta i = N'_{\beta} \Delta \beta + N'_p \Delta p + N'_r \Delta r + N'_{\delta_r} \Delta \delta_r + N'_{\delta_a} \Delta \delta_a$
- $\therefore \Delta \dot{\varphi} = \Delta p + \Delta r * \tan(\theta_0)$
- $\therefore \Delta \dot{\psi} = \Delta r * \sec(\theta_0)$

$$\begin{bmatrix} \Delta \dot{\beta} \\ \Delta \dot{p} \\ \Delta \dot{r} \\ \Delta \dot{\varphi} \\ \Delta \dot{\psi} \end{bmatrix} = \begin{bmatrix} \frac{Y_{\beta}}{V_{to}} & \frac{Y_p + w_o}{V_{to}} & \frac{Y_r - u_o}{V_{to}} & \frac{g \cos(\theta_o)}{V_{to}} & 0 \\ L'_{\beta} & L'_p & L'_r & 0 & 0 \\ N'_{\beta} & N'_p & N'_r & 0 & 0 \\ 0 & 1 & \tan(\theta_0) & 0 & 0 \\ 0 & 0 & \sec(\theta_0) & 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta \beta \\ \Delta p \\ \Delta r \\ \Delta \varphi \\ \Delta \psi \end{bmatrix} + \begin{bmatrix} Y_{\delta a}^* & Y_{\delta r}^* \\ L'_{\delta a} & L'_{\delta r} \\ N'_{\delta a} & N'_{\delta r} \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta \delta_a \\ \Delta \delta_r \end{bmatrix}$$

where:

$Y_{\delta a}^* = Y_{\delta a} / V_{to}$ $\dot{L}_{\beta} = (L_{\beta} + (I_{xz} N_{\beta}) / I_x) G$ $\dot{L}_p = (L_p + (I_{xz} N_p) / I_x) G$ $\dot{L}_r = (L_r + (I_{xz} N_r) / I_x) G$ $\dot{L}_{\delta a} = (L_{\delta a} + (I_{xz} N_{\delta a}) / I_x) G$ $\dot{L}_{\delta r} = (L_{\delta r} + (I_{xz} N_{\delta r}) / I_x) G$	$Y_{\delta r}^* = Y_{\delta r} / V_{to}$ $\dot{N}_{\beta} = (N_{\beta} + (I_{xz} L_{\beta}) / I_z) G$ $\dot{N}_p = (N_p + (I_{xz} L_p) / I_z) G$ $\dot{N}_r = (N_r + (I_{xz} L_r) / I_z) G$ $\dot{N}_{\delta a} = (N_{\delta a} + (I_{xz} L_{\delta a}) / I_z) G$ $\dot{N}_{\delta r} = (N_{\delta r} + (I_{xz} L_{\delta r}) / I_z) G$
$G = \frac{I_x I_z}{I_x I_z - (I_{xz})^2} = 1 / (1 - ((I_{xz})^2 / I_x I_z))$	

For simple calculation of the aircraft response t we can use the next approximations.

3 DOF Spiral approximation

As it is clear in the attached figure how does the airplane behave when it is in the spiral mode.

The side force equation of motion will be neglected and will put $\Delta \beta = 0$.

Therefore, the 3 DOF spiral approximation will be as shown in the following:

$$\begin{bmatrix} \Delta \dot{p} \\ \Delta \dot{r} \\ \Delta \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \dot{L}_p & \dot{L}_r & 0 \\ \dot{N}_p & \dot{N}_r & 0 \\ 1 & \tan(\theta_o) & 0 \end{bmatrix} \begin{bmatrix} \Delta p \\ \Delta r \\ \Delta \varphi \end{bmatrix} + \begin{bmatrix} \dot{L}_{\delta a} & \dot{L}_{\delta r} \\ \dot{N}_{\delta a} & \dot{N}_{\delta r} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \Delta \delta_a \\ \Delta \delta_r \end{bmatrix}$$

3 DOF Dutch Roll approximation

The following terms will be neglected according to certain assumptions:

- $\frac{1}{V_{to}} g \cos \theta_o$, for moderate and high speeds.
- $L'_r r = 0$, Effects of yaw rate on rolling acceleration.
- $N'_p p = 0$, Effects of roll rate on yawing acceleration.

And by taking the previous mentioned assumptions into consideration we reach the following relations:

$$\begin{bmatrix} \Delta \dot{\beta} \\ \Delta \dot{p} \\ \Delta \dot{r} \end{bmatrix} = \begin{bmatrix} Y_v & \frac{(Y_p + w_0)}{V_{to}} & \frac{(Y_r - u_0)}{V_{to}} \\ \dot{L}_\beta & \dot{L}_p & \dot{L}_r \\ \dot{N}_\beta & \dot{N}_p & \dot{N}_r \end{bmatrix} \begin{bmatrix} \Delta \beta \\ \Delta p \\ \Delta r \end{bmatrix} + \begin{bmatrix} Y_{\delta_a}^* & Y_{\delta_r}^* \\ \dot{L}_{\delta_a} & \dot{L}_{\delta_r} \\ \dot{N}_{\delta_a} & \dot{N}_{\delta_r} \end{bmatrix} \begin{bmatrix} \Delta \delta_a \\ \Delta \delta_r \end{bmatrix}$$

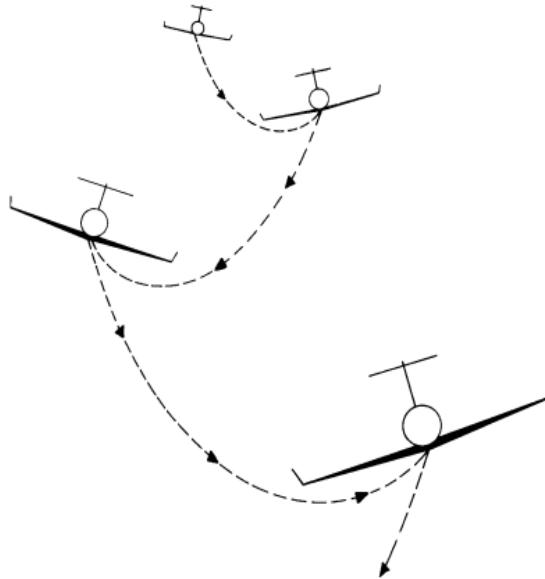
2 DOF Dutch Roll approximation

The combination of the roll and yaw approximations is called the Dutch roll Motion.

In this equation the rolling moment equation is neglected, and we put $\Delta \varphi = 0$.

Therefore, the 2 DOF spiral approximation will be as shown in the following:

$$\begin{bmatrix} \Delta \dot{\beta} \\ \Delta \dot{r} \end{bmatrix} = \begin{bmatrix} Y_v & \frac{Y_r - u_0}{V_{to}} - \frac{Y_r + w_0}{V_{to}} \tan(\theta) \\ \dot{N}_\beta & \dot{N}_r - N'_p \tan(\theta) \end{bmatrix} \begin{bmatrix} \Delta \beta \\ \Delta r \end{bmatrix} + \begin{bmatrix} Y_{\delta_a}^* & Y_{\delta_r}^* \\ \dot{N}_{\delta_a} & \dot{N}_{\delta_r} \end{bmatrix} \begin{bmatrix} \Delta \delta_a \\ \Delta \delta_r \end{bmatrix}$$



DOF Roll approximation.

It has been seen that the rolling mode almost corresponds to the pure rolling. Thus it is reasonable to neglect all equations except the rolling moment equation and all perturbations except p . So we will reach the following relation;

$$\dot{p} - L'_p p = L_{\delta_a}' \delta_a$$

i) Simulink Schematic

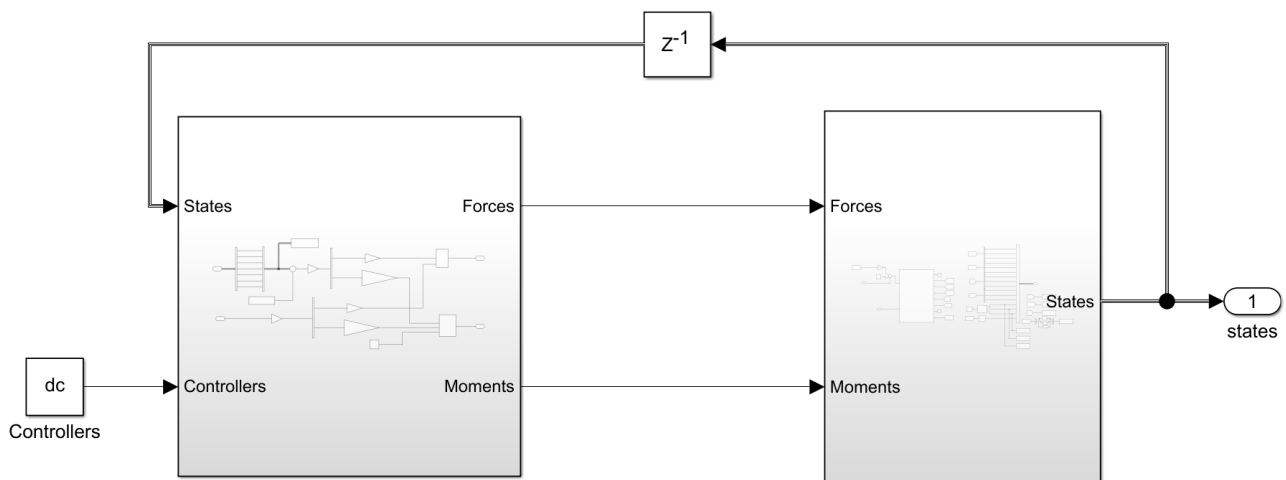


Figure 12 Simulator Schematic.

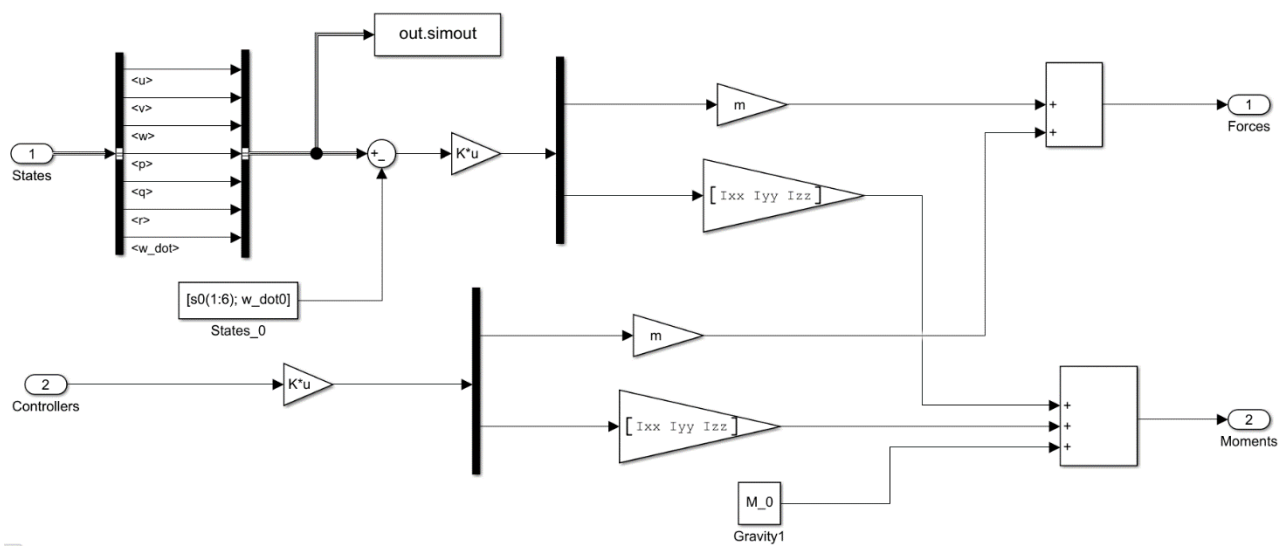


Figure 13 Subsystem-1 Schematic

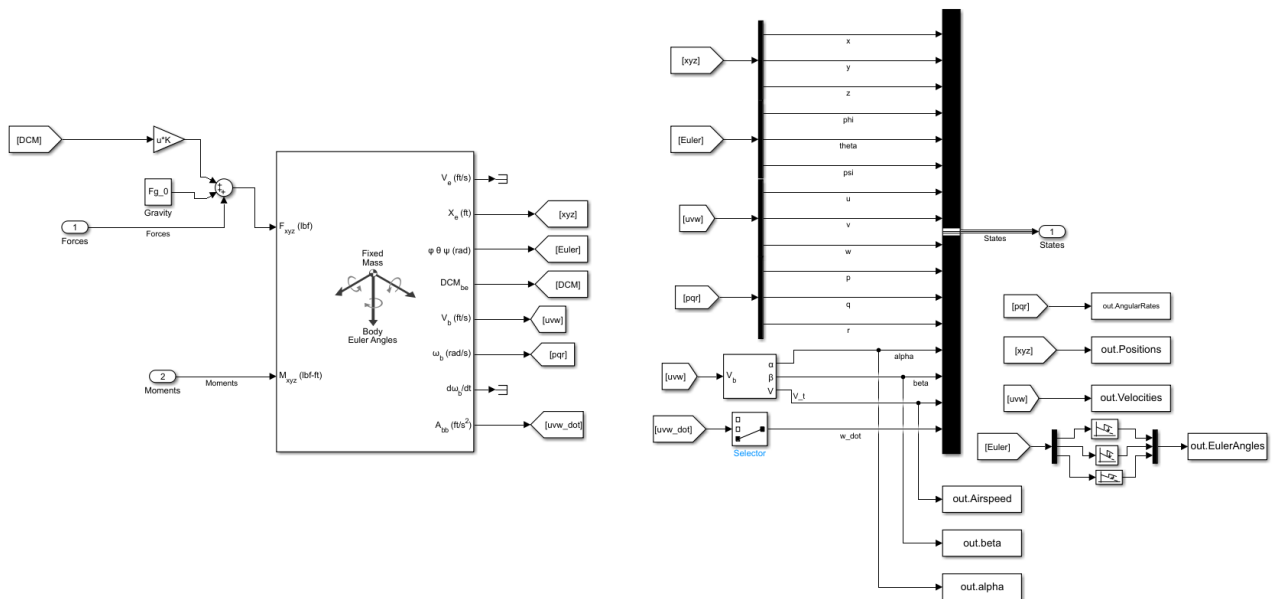


Figure 14 Subsystem-2 Schematic.

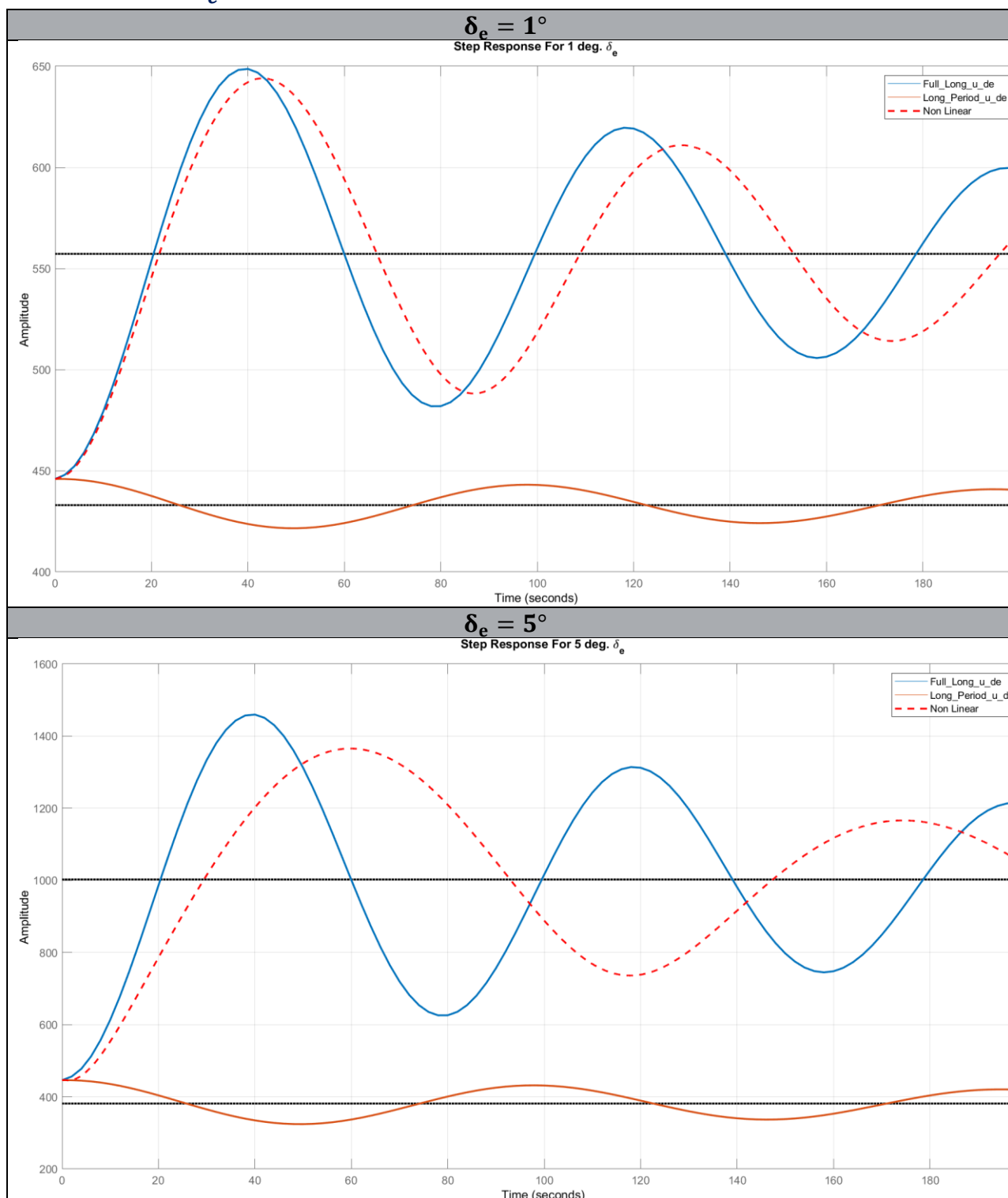
Longitudinal Results

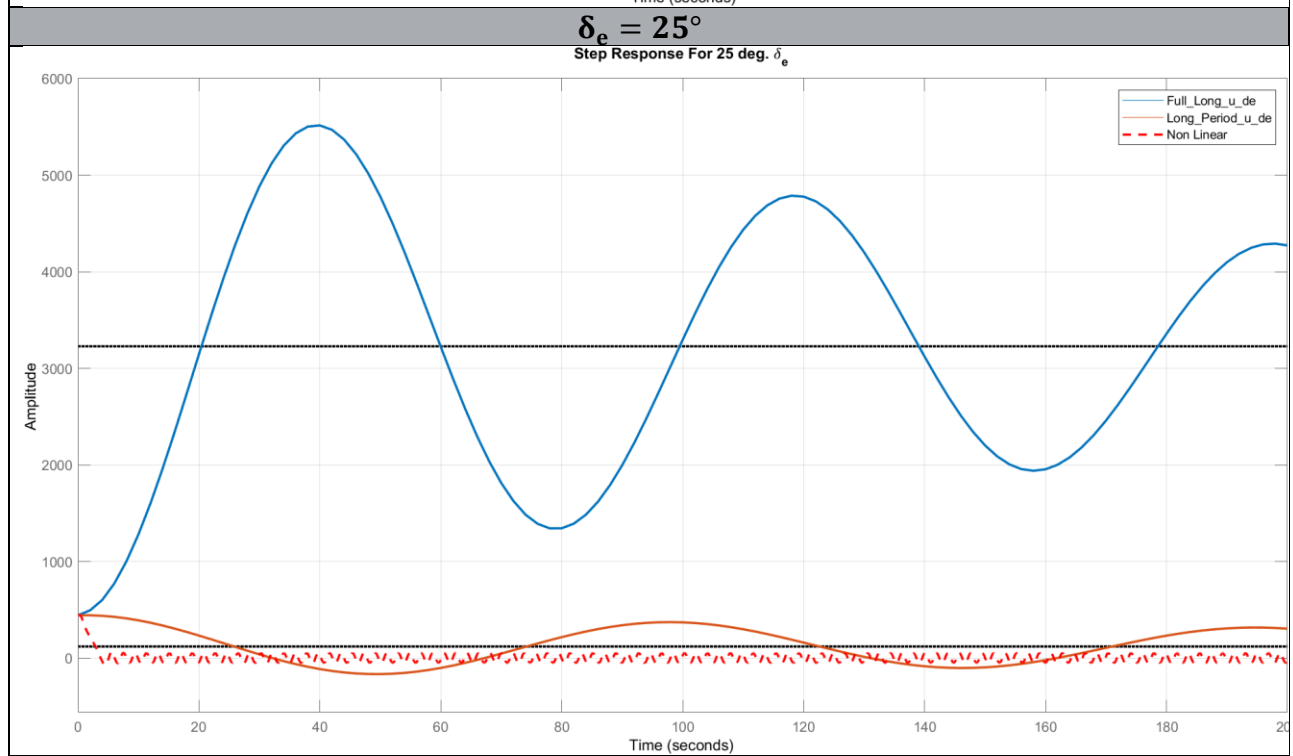
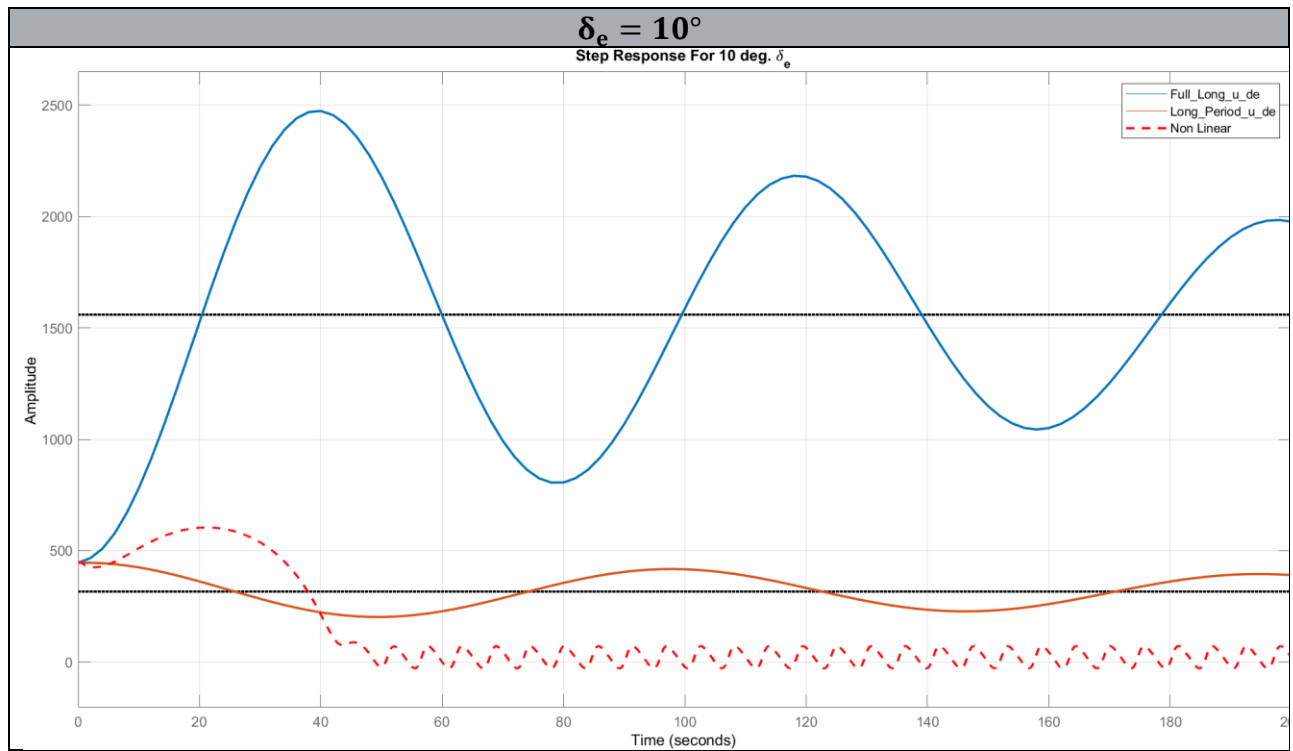
k) Response of state u

i. Transfer Functions

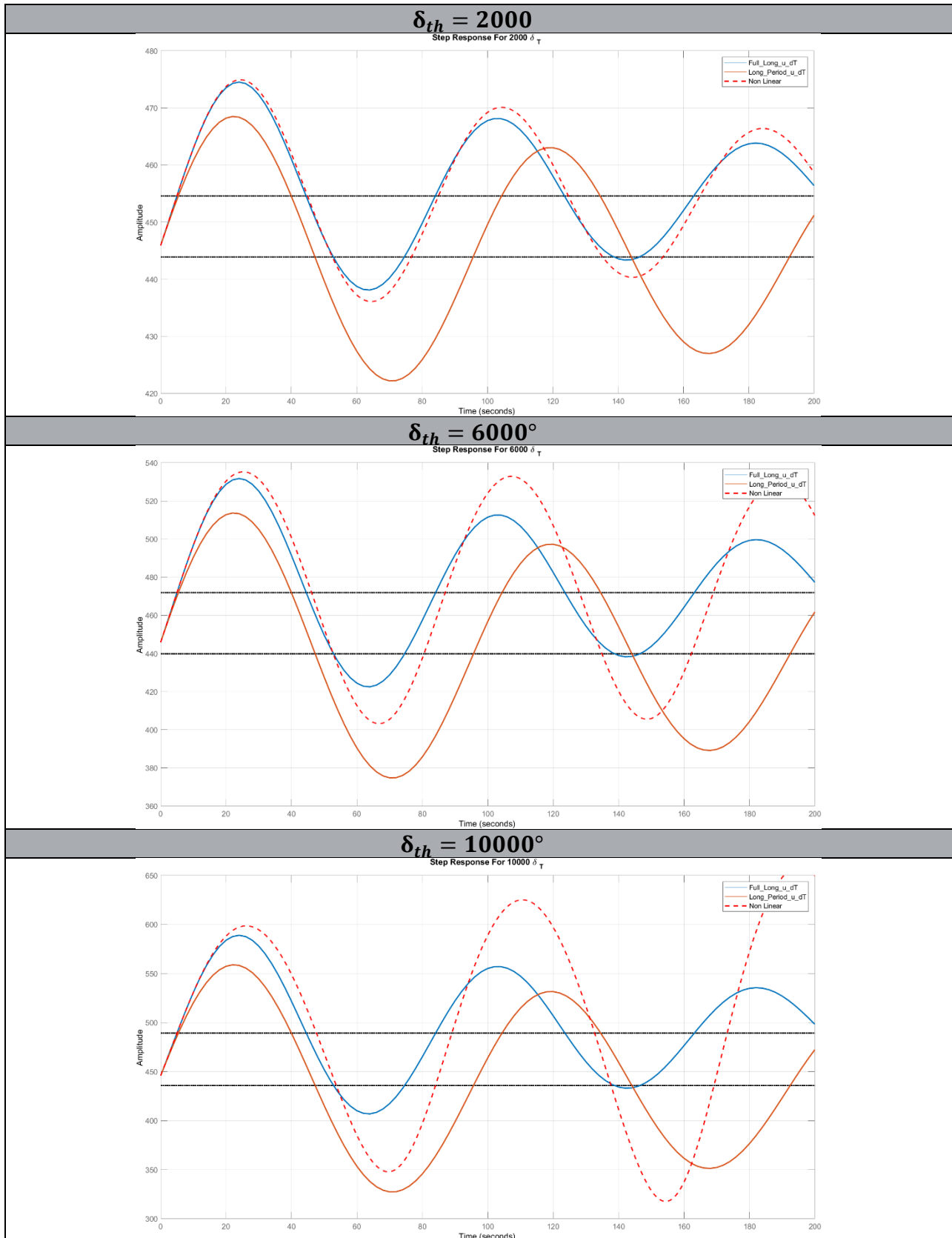
TF	Full Linear	Long Period Mode
u/δ_e	$\frac{3.02s^3 + 263.1s^2 + 149.5s + 315.4}{s^4 + 2.658s^3 + 7.838s^2 + 0.0924s + 0.0495}$	$\frac{3.02s - 3.125}{s^2 + 0.005167s + 0.004217}$
u/δ_{th}	$\frac{0.000842s^3 + 0.002418s^2 + 0.006665s + 0.0002144}{s^4 + 2.658s^3 + 7.838s^2 + 0.0924s + 0.04945}$	$\frac{0.000842s - 4.238e-06}{s^2 + 0.005167s + 0.004217}$

ii. Various δ_e





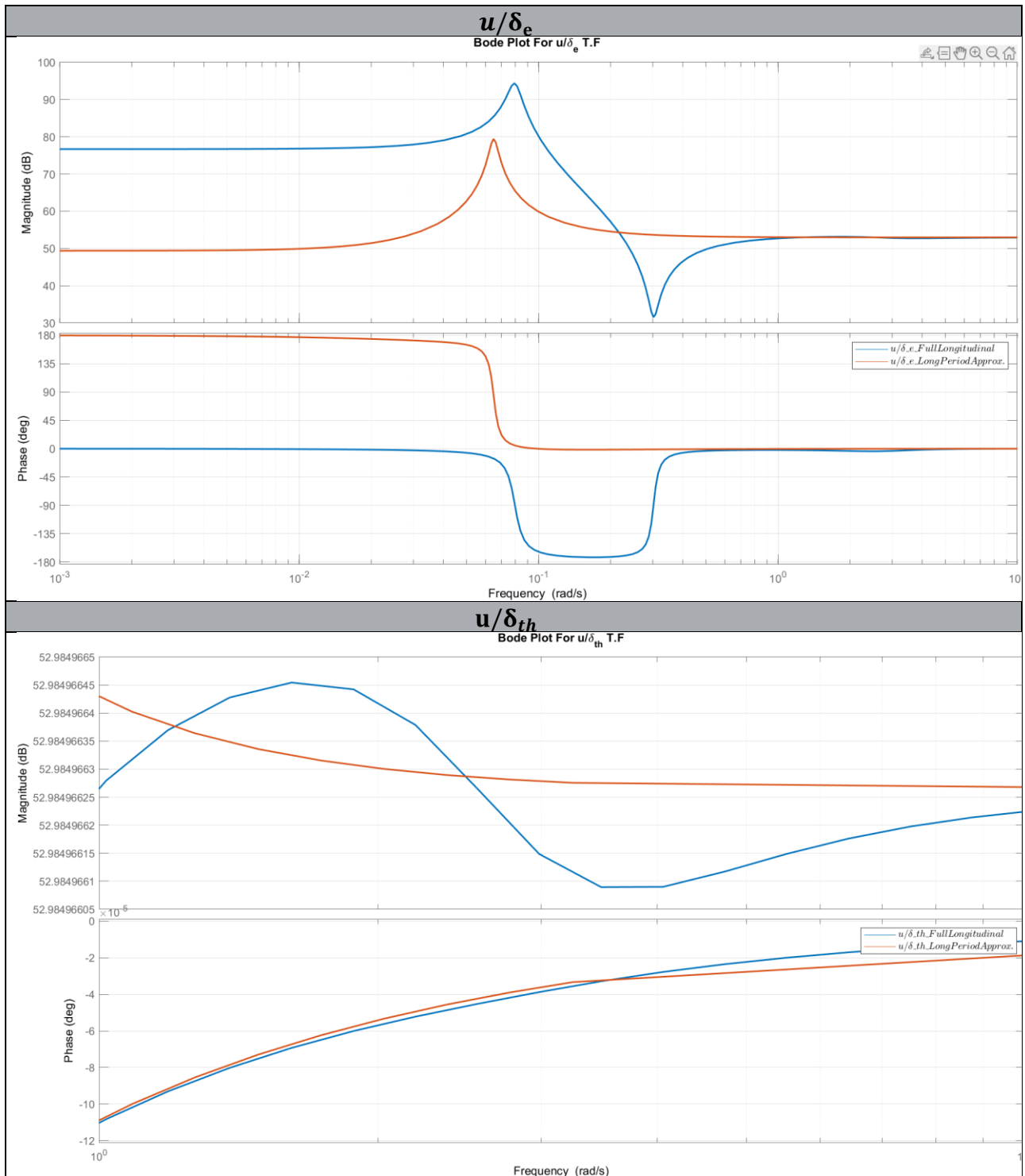
iii. Various δ_{th}



Comments:

- The long period approximation is not accurate for calculation of state (u).
- The accuracy of the linear solution depends on the type of the input which changes (elevator or thrust), the value of this change and the time.
- For elevator: the linear solution gives accurate values for small changes in the angle in range time and this range decreases with the increase of the angle.
- From shown results, we can deduce that for angle of elevator greater than 9 deg, the linear system is not valid as it does not show the nonlinear system dynamics, this dynamics may be due to the coupling of lateral and longitudinal in the nonlinear model.
- For thrust: the accuracy of the linear solution is acceptable for all values of thrust.

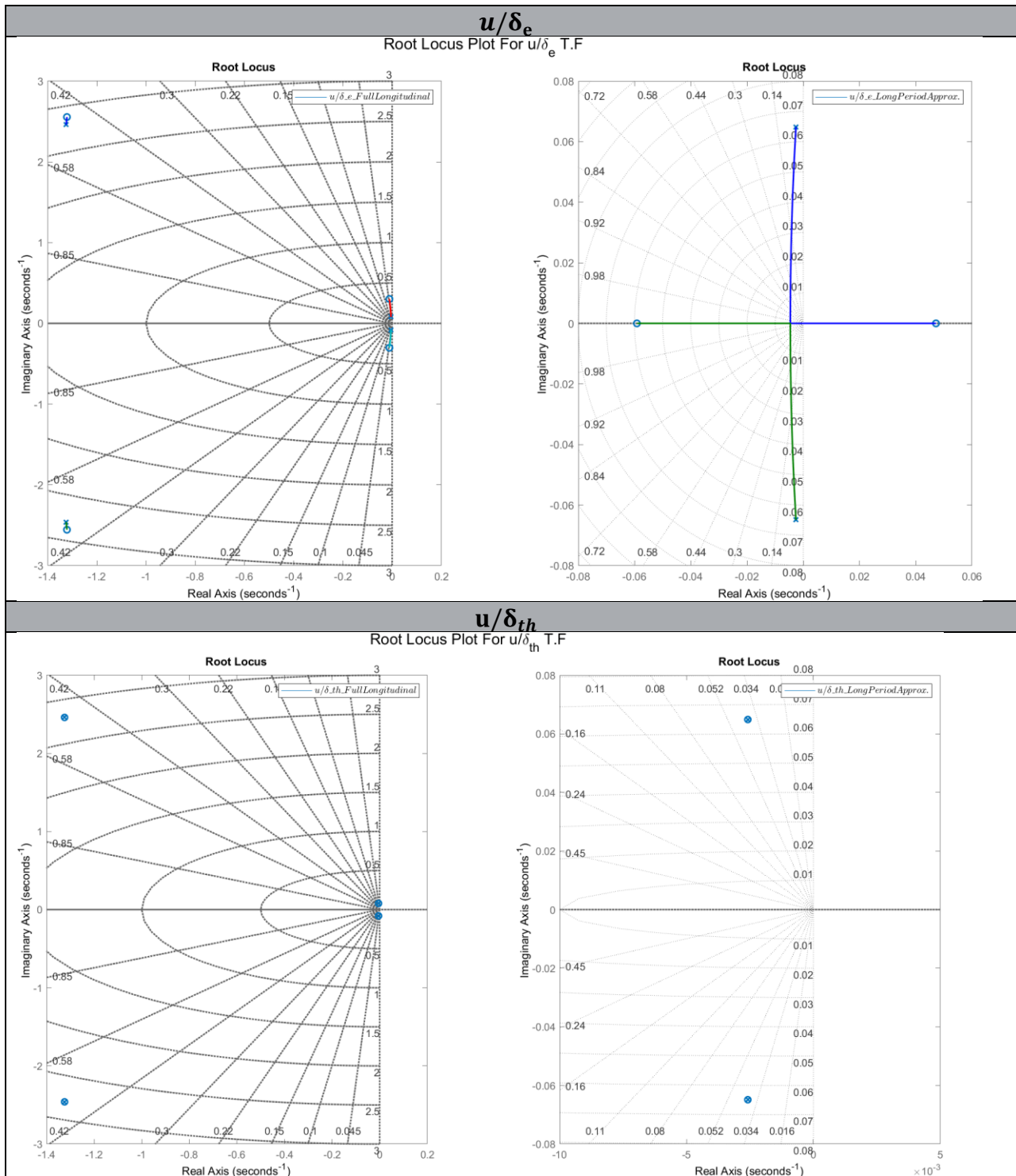
iv. Bode Plot



Comments:

- For δ_e input, The long period approximation is valid compared to the full linear model when the frequency is greater than 1 rad/sec.
- For δ_{th} input, The long period approximation is valid compared to the full linear model when the frequency is greater than 10 rad/sec.

v. Root Locus



Comments:

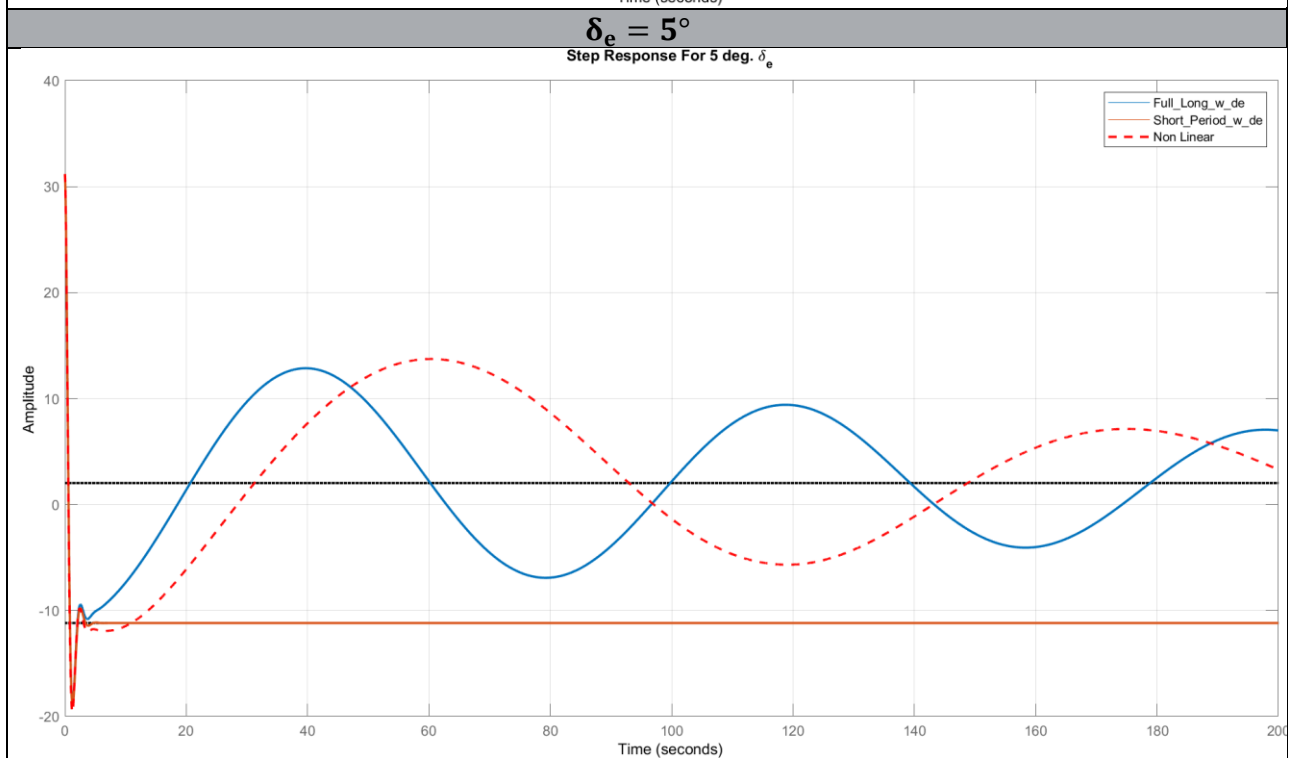
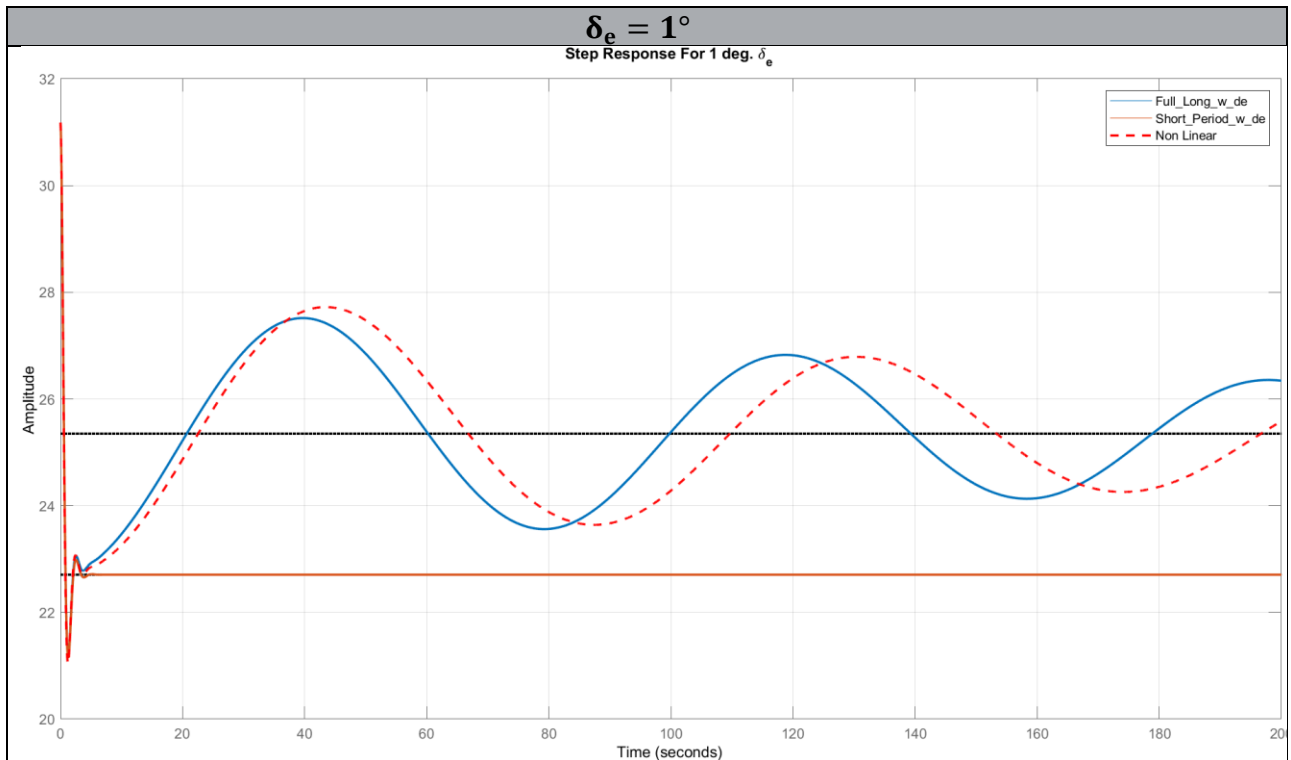
- For delta e input, the transfer function for full linear model is always stable but in the long period approx. the transfer function has instability region.

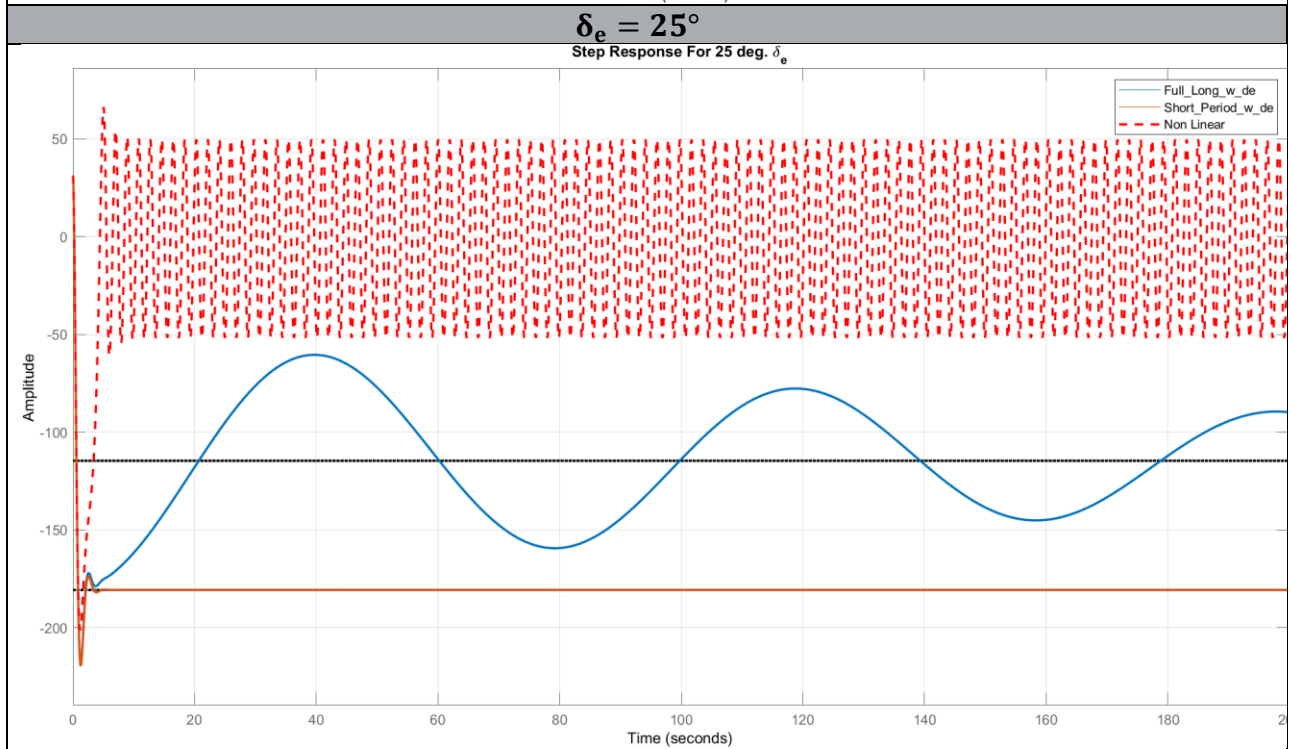
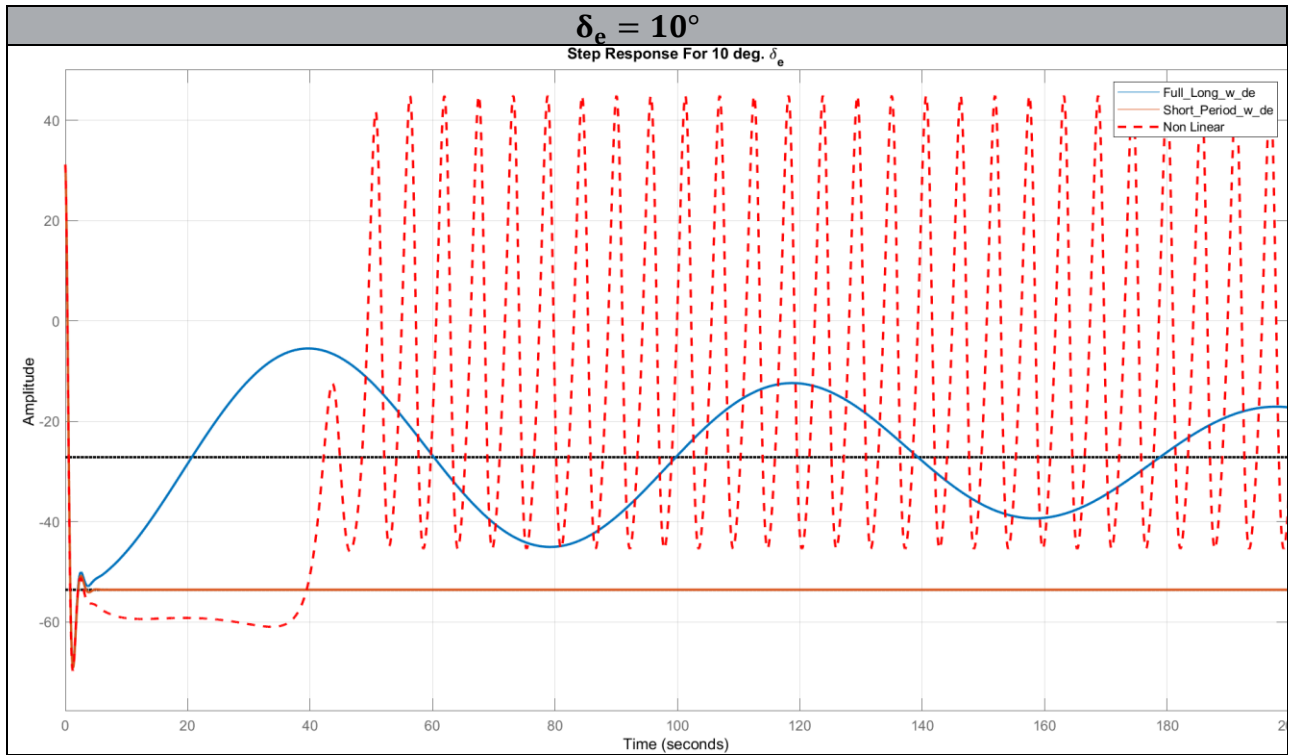
I) Response of state w

i. Transfer Functions

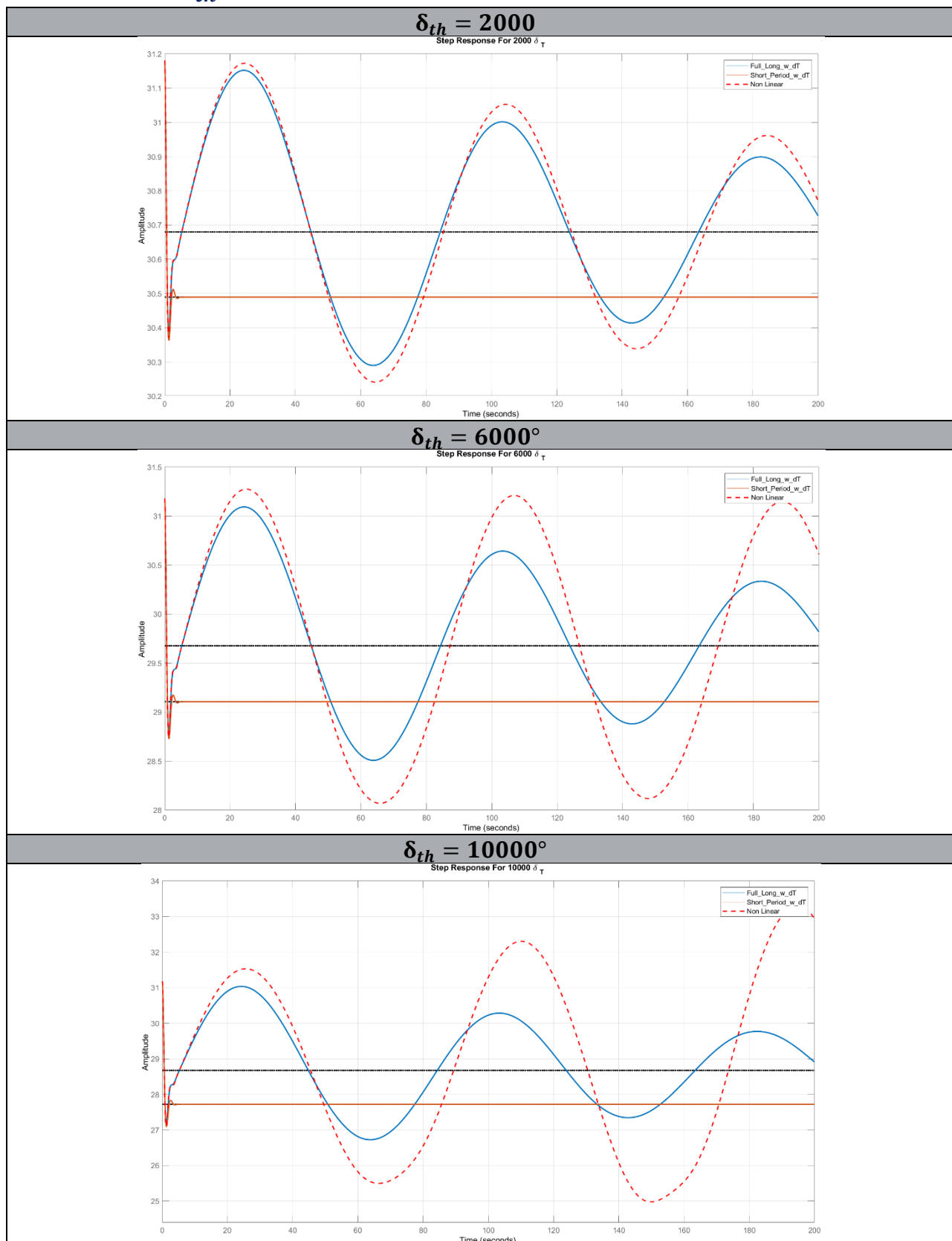
TF	Full Linear	Short Period Mode
w/δ_e	$\frac{-43.2 s^3 - 3782 s^2 - 35.44 s - 16.52}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$	$\frac{-43.2 s - 3781}{s^2 + 2.648 s + 7.788}$
w/δ_{th}	$\frac{-0.002743 s^2 + 0.0001296 s - 1.239e-05}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$	$\frac{0.002693}{s^2 + 2.648 s + 7.788}$

ii. Various δ_e





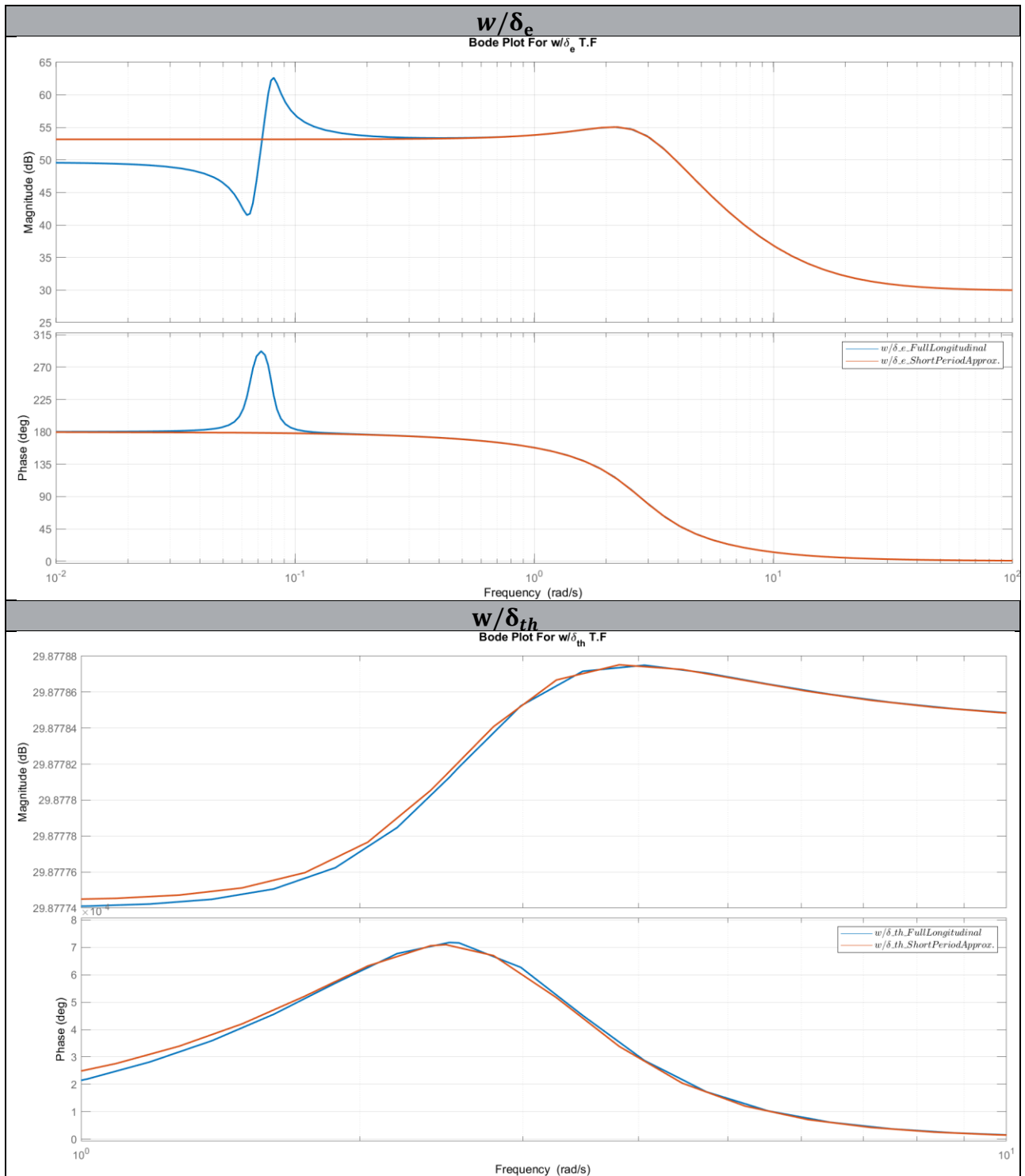
iii. Various δ_{th}



Comments:

- The same notes of state u but the short period approximation is accurate for a range of time (till about 5 sec).

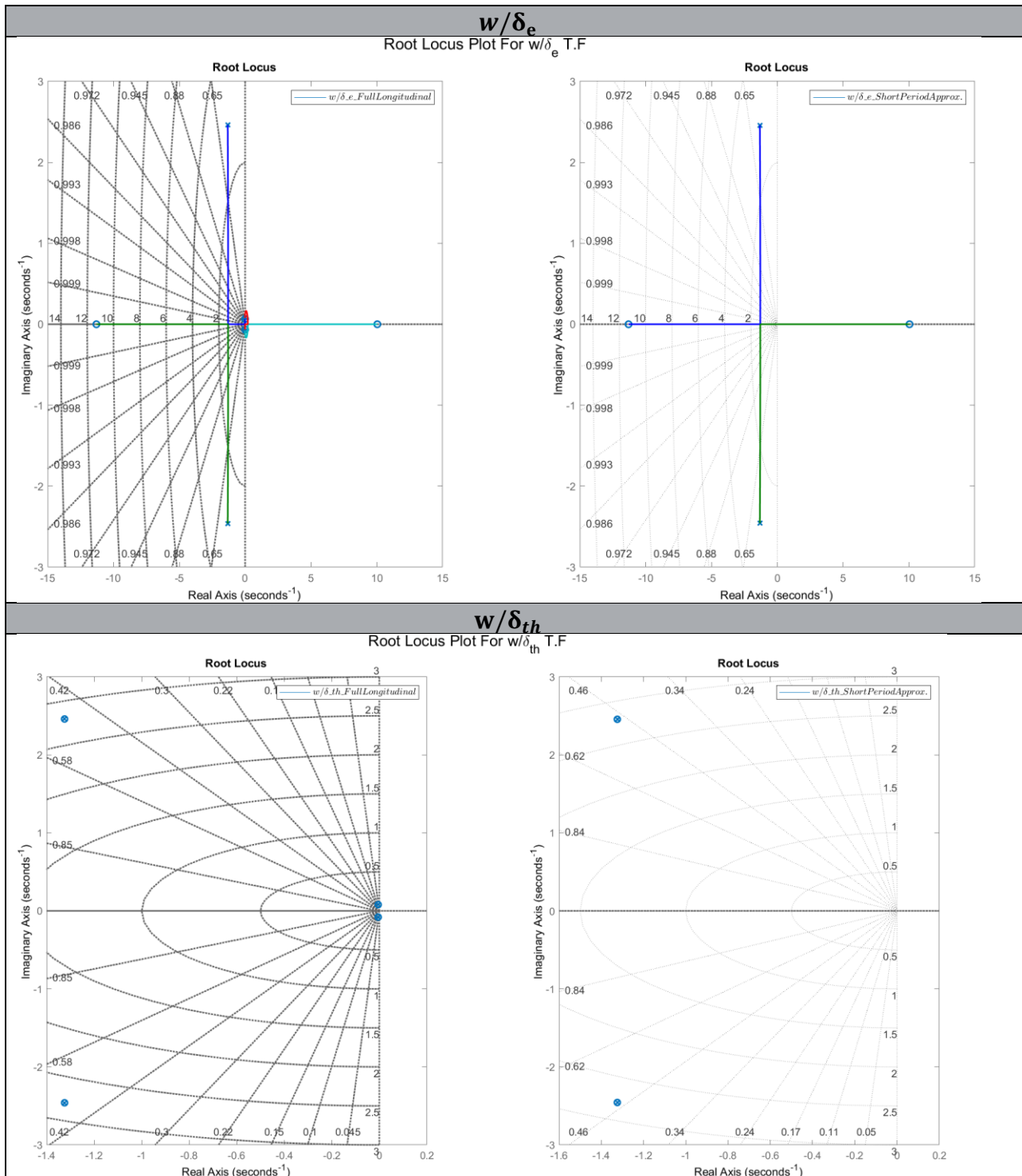
iv. Bode Plot



Comments:

- From bode plot the short period approximation is almost the same as the full linear model.

v. Root Locus



Comments:

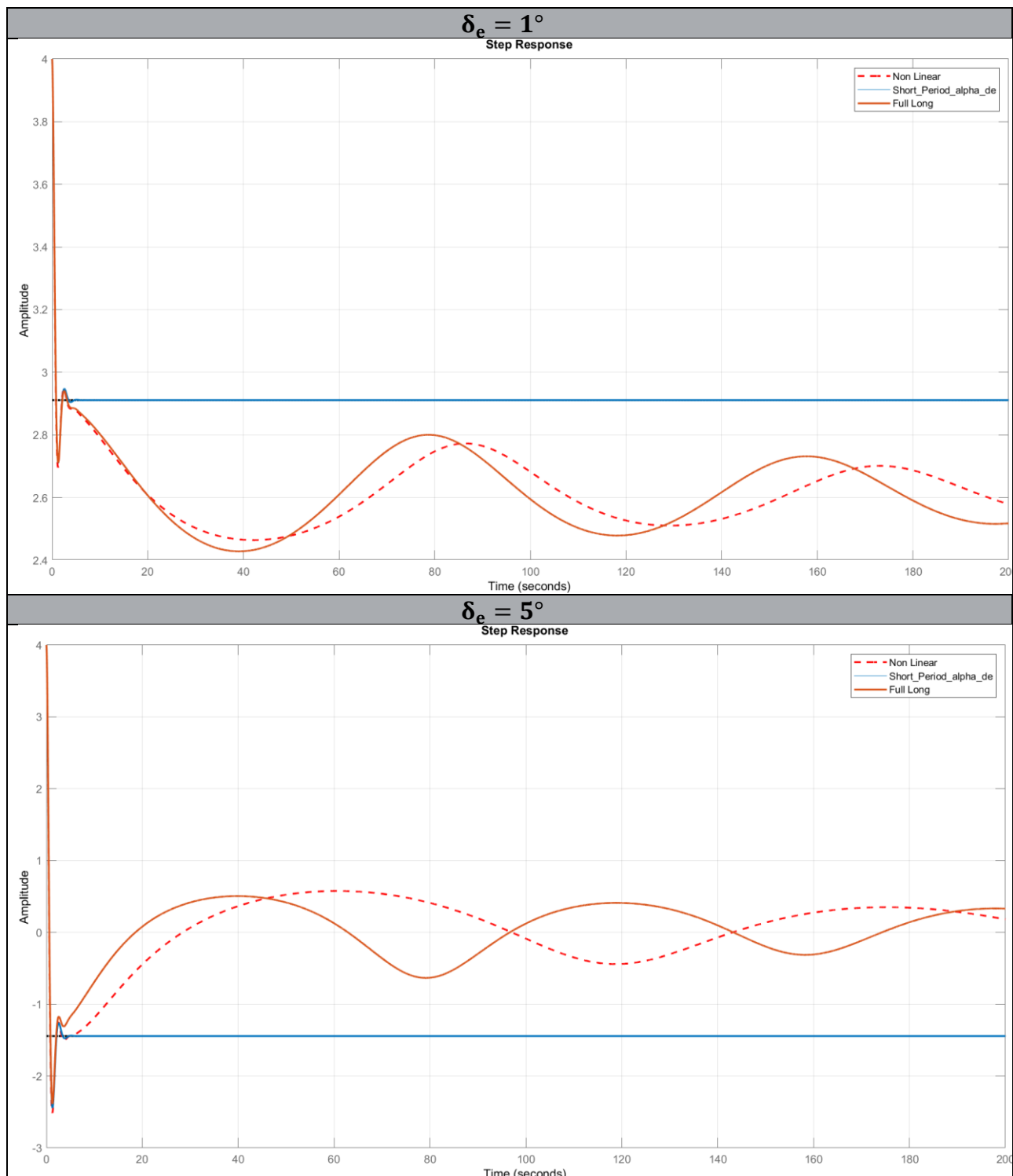
- From the root locus, the transfer function of short period is almost the same dynamics of the fast poles of the full linear model.

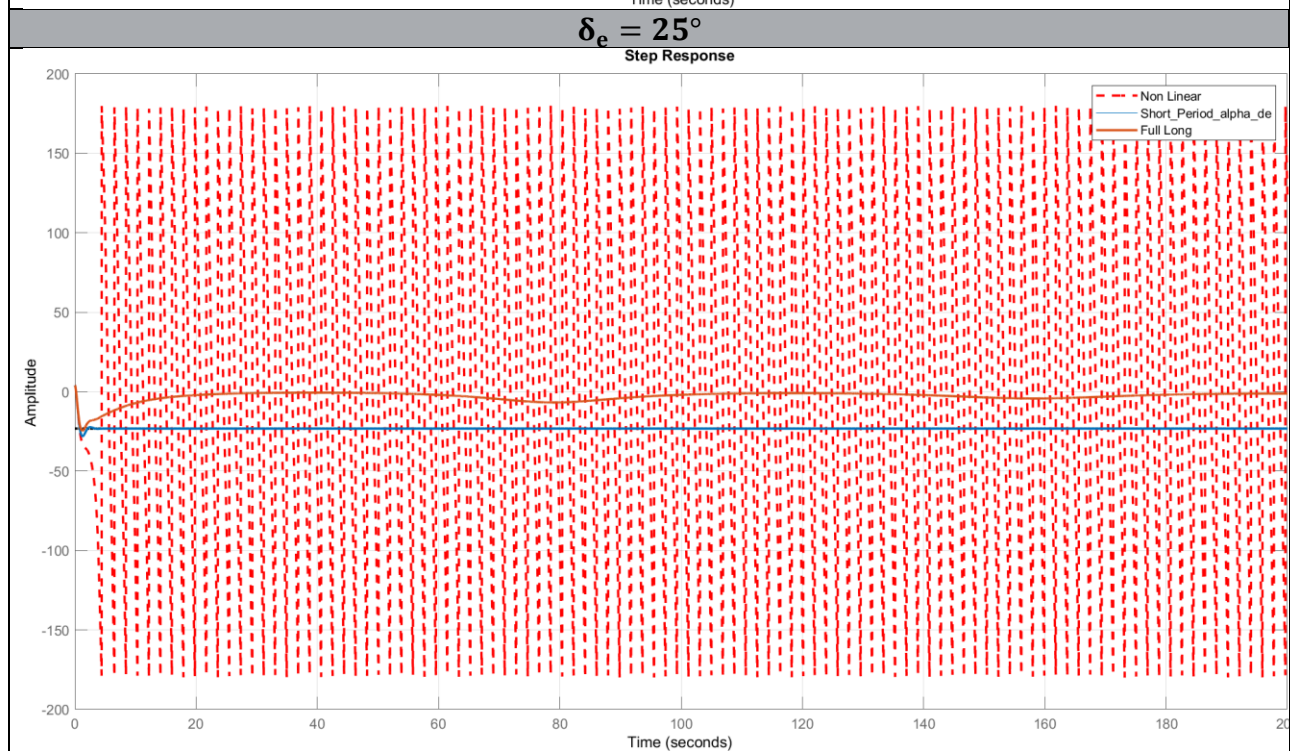
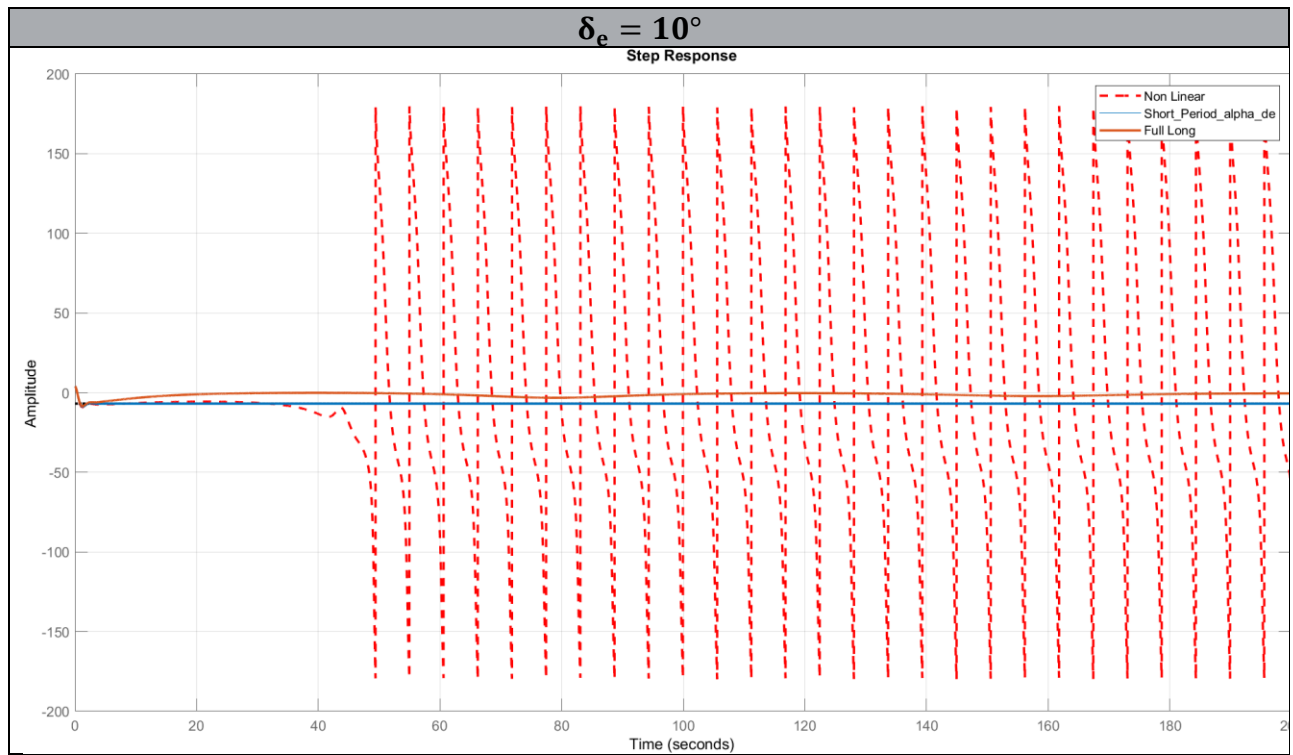
m) Response of state α

i. Transfer Functions

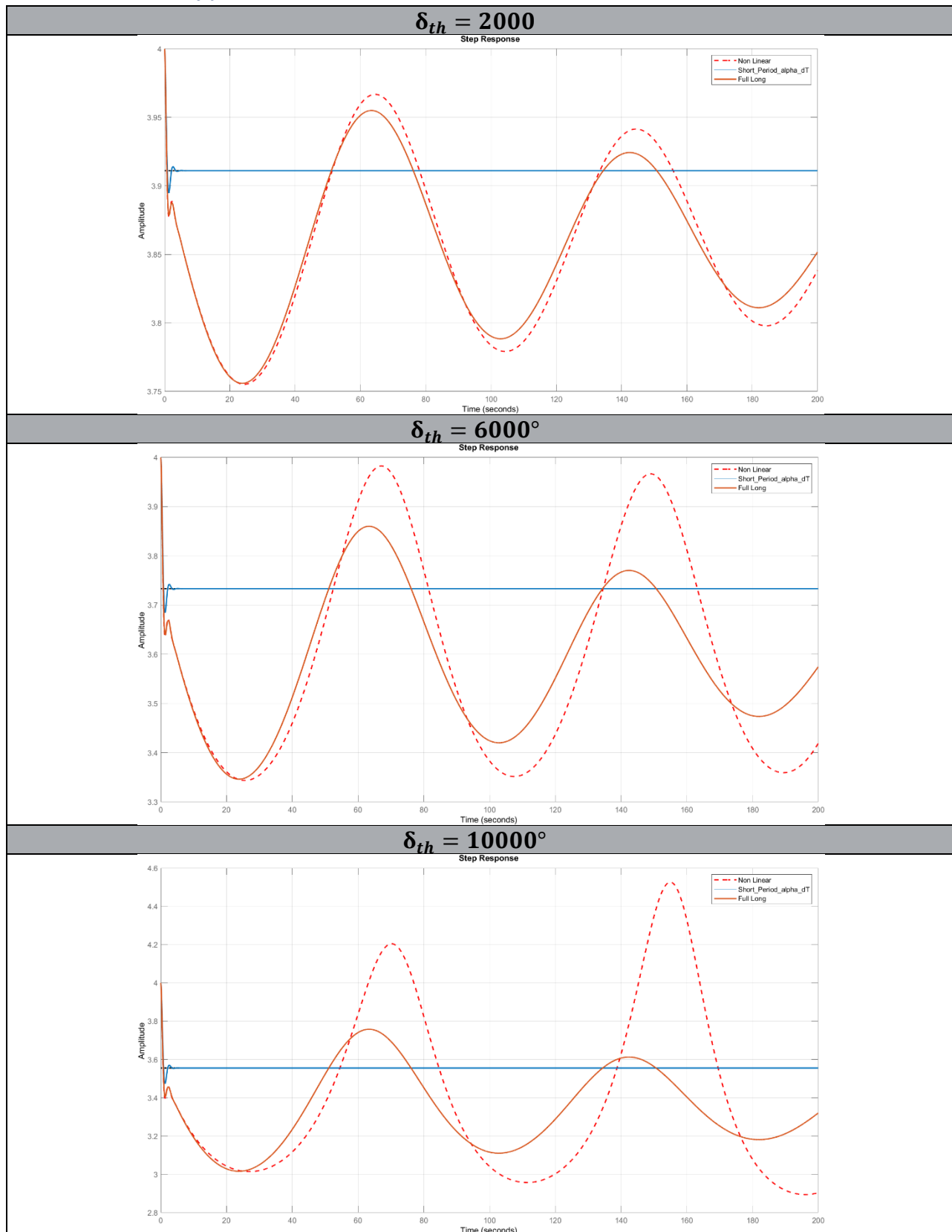
TF	Full Linear	Short Period Mode
α/δ_e	$\frac{-0.09688 s^3 - 8.481 s^2 - 0.1024 s - 0.08609}{-1.314e-07 s^3 - 6.499e-06 s^2 - 7.509e-07 s - 6.112e-08}$	$\frac{-0.09688 s - 8.48}{s^2 + 2.648 s + 7.788}$
α/δ_{th}	$\frac{-1.314e-07 s^3 - 6.499e-06 s^2 - 7.509e-07 s - 6.112e-08}{-1.314e-07 s^3 - 6.499e-06 s^2 - 7.509e-07 s - 6.112e-08}$	$\frac{-6.04e-06}{s^2 + 2.648 s + 7.788}$

ii. Various δ_e





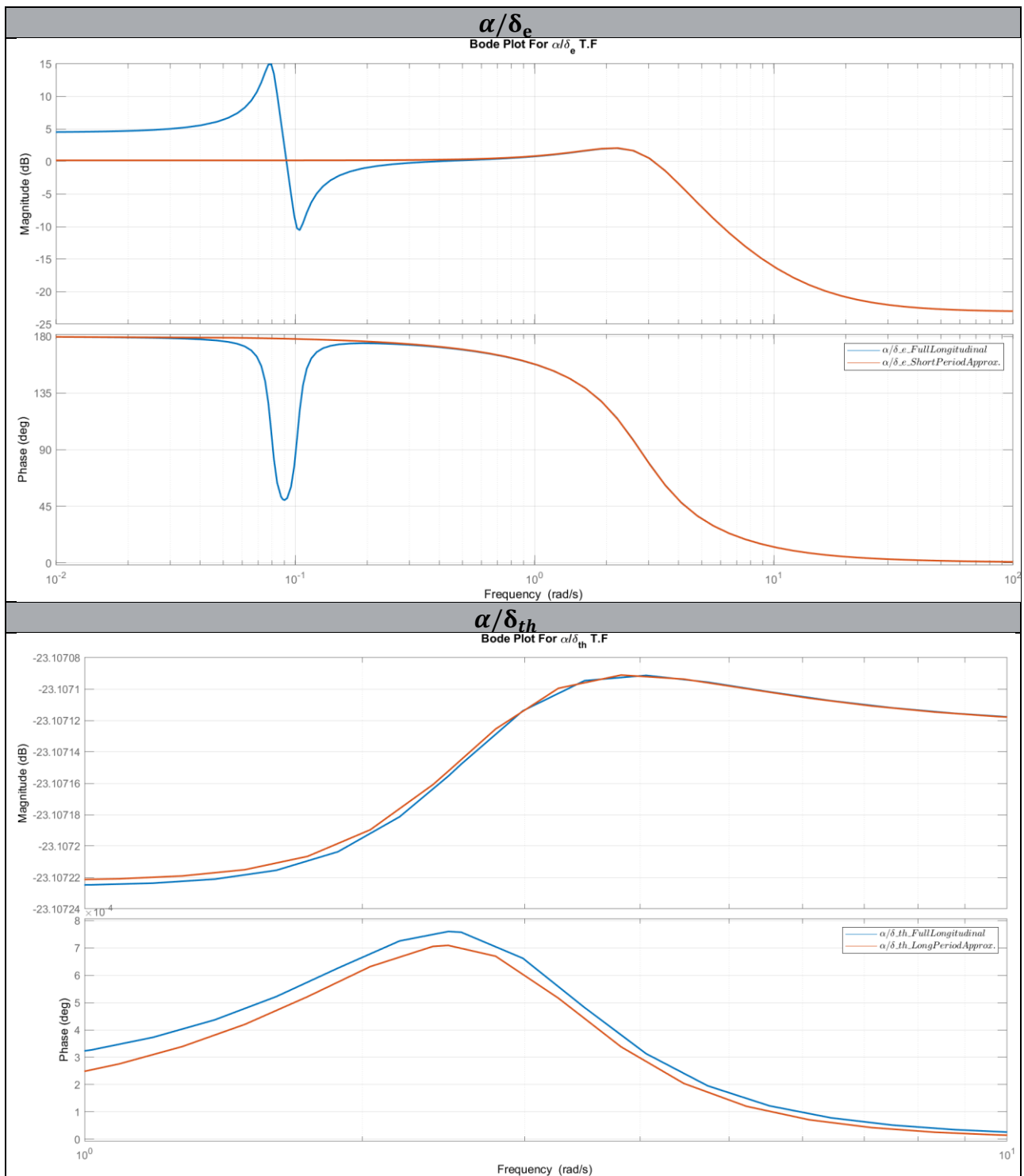
iii. Various δ_{th}



Comments:

- The same notes of state w

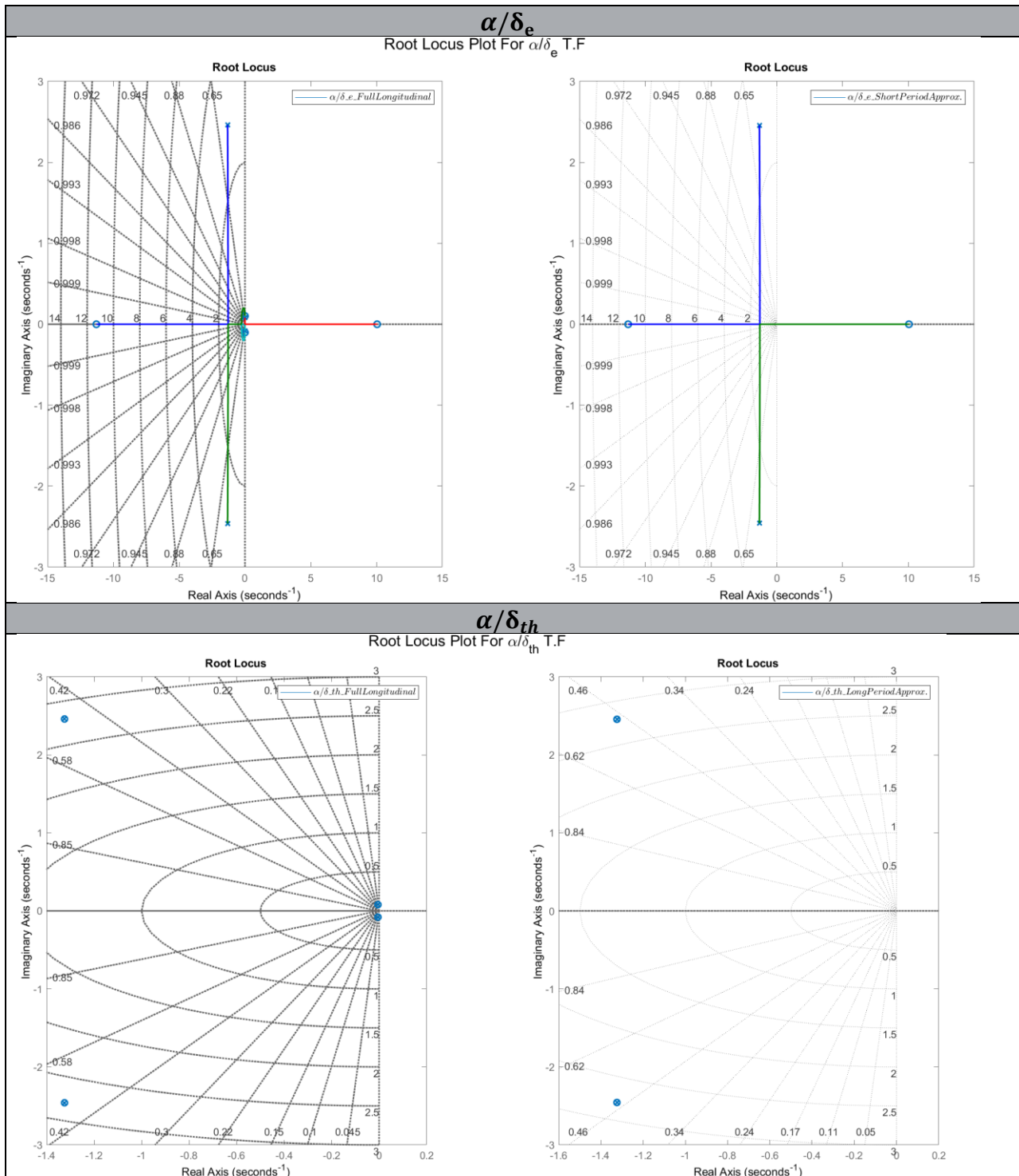
iv. Bode Plot



Comments:

- From bode plot the short period approximation is almost the same as the full linear model.

v. Root Locus



Comments:

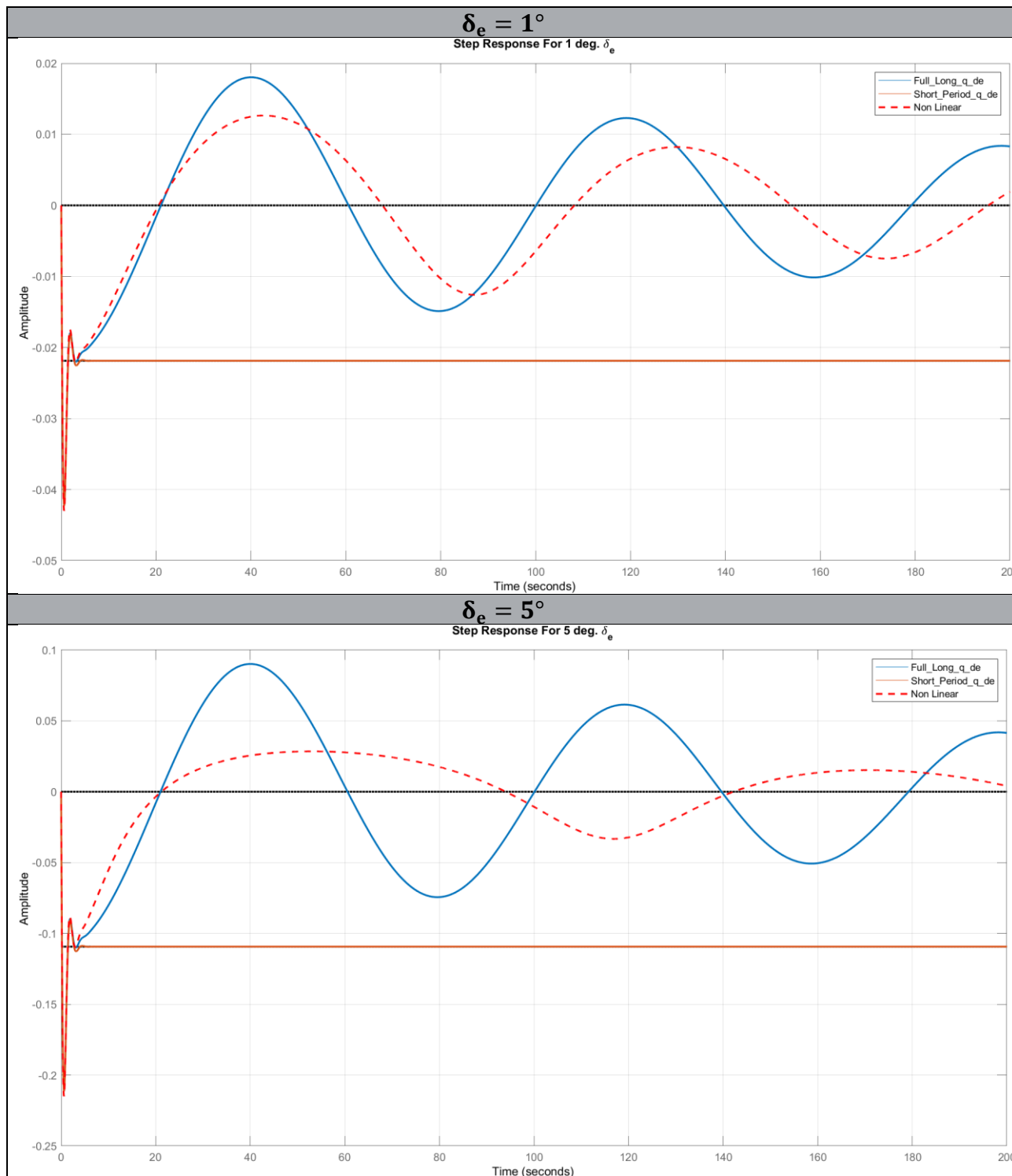
- From the root locus, the transfer function of short period is almost the same dynamics of the fast poles of the full linear model.

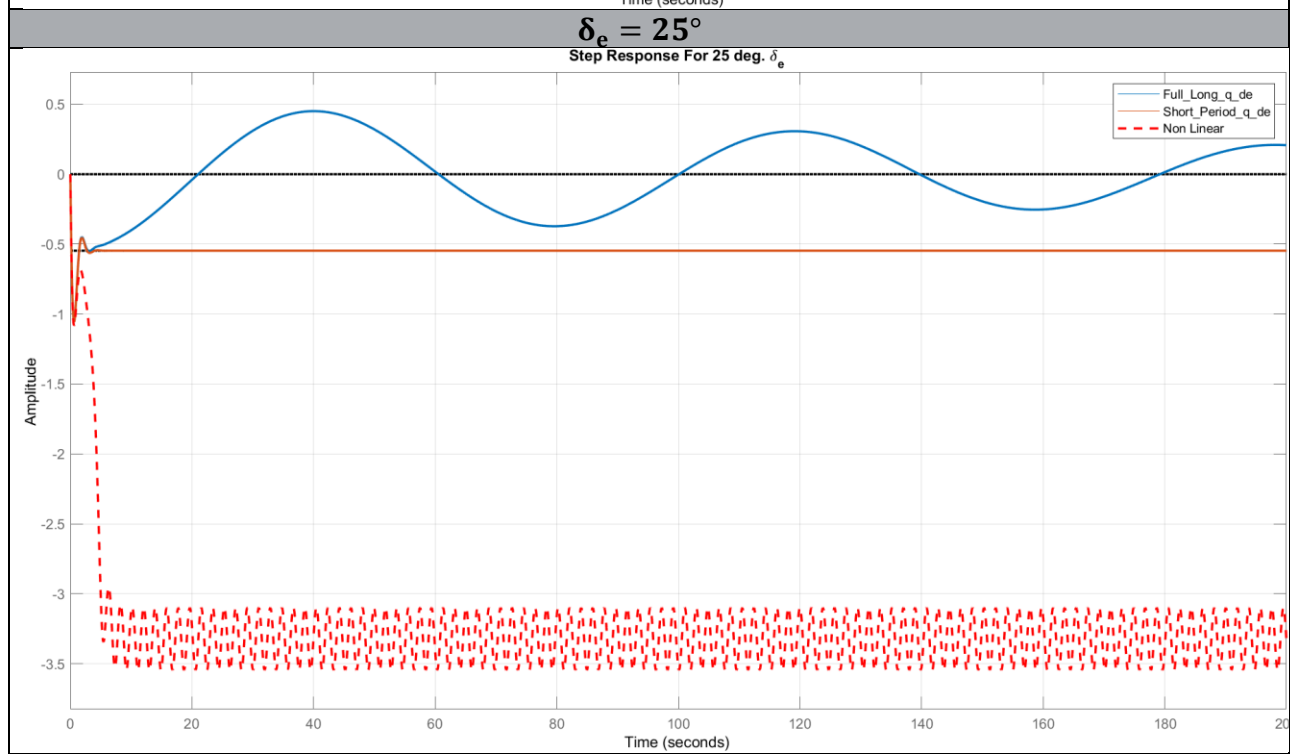
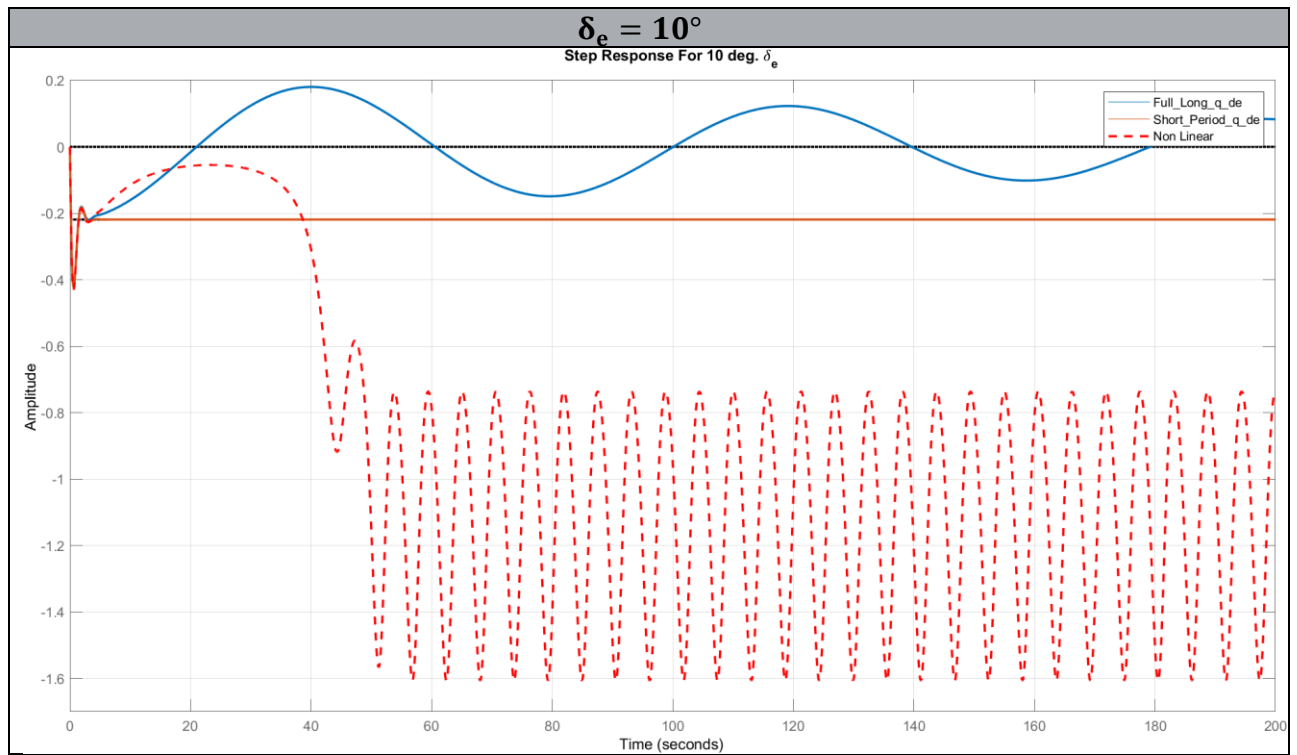
n) Response of state q

i. Transfer Functions

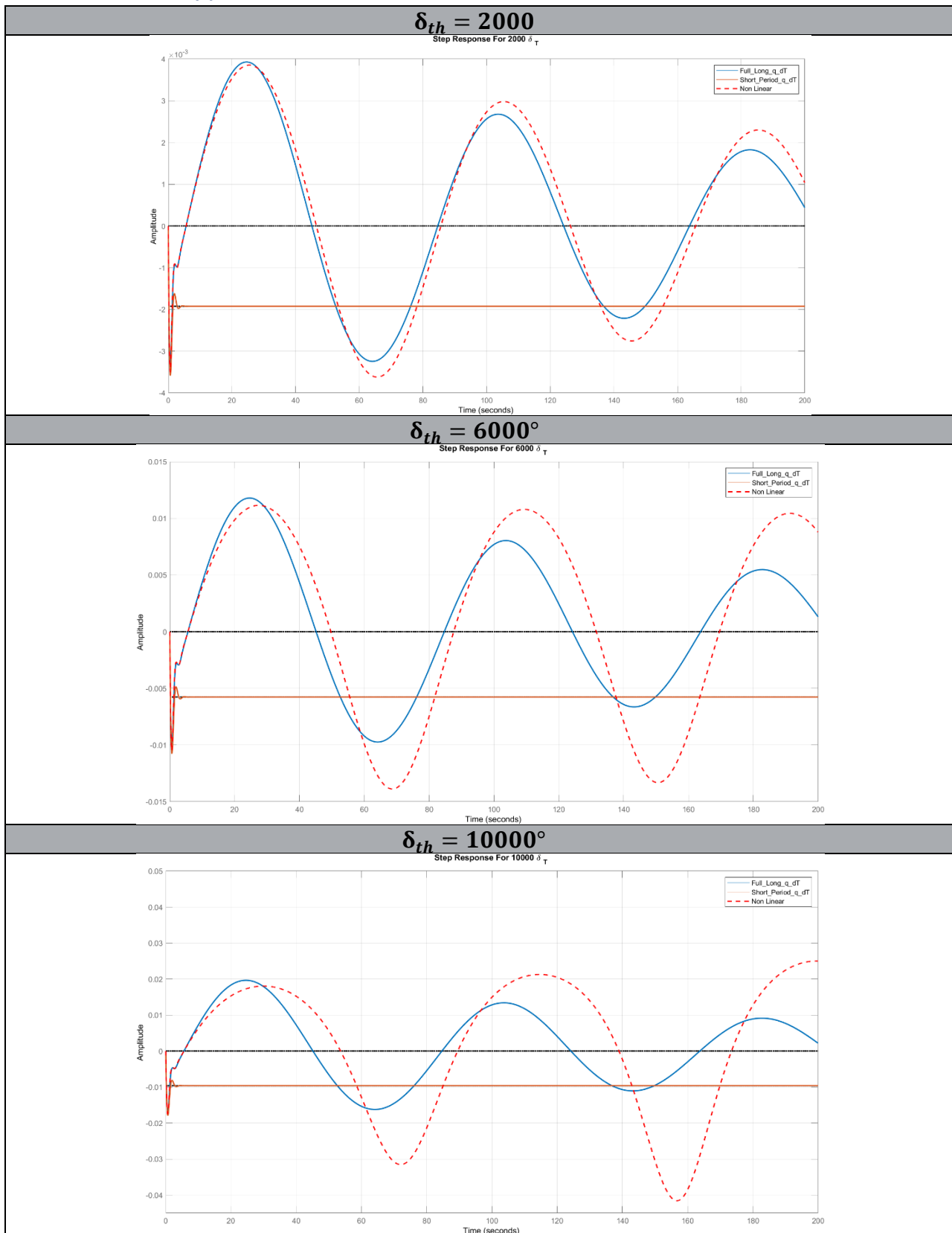
TF	Full Linear	Short Period Mode
q/δ_e	$\frac{-8.343 s^3 - 9.844 s^2 - 0.1563 s - 3.23e-18}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$	$\frac{-8.343 s - 9.76}{s^2 + 2.648 s + 7.788}$
q/δ_{th}	$\frac{-6.04e-06 s^3 - 7.047e-06 s^2 + 1.184e-06 s - 2.863e-24}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$	$\frac{-6.04e-06 s - 7.49e-06}{s^2 + 2.648 s + 7.788}$

ii. Various δ_e





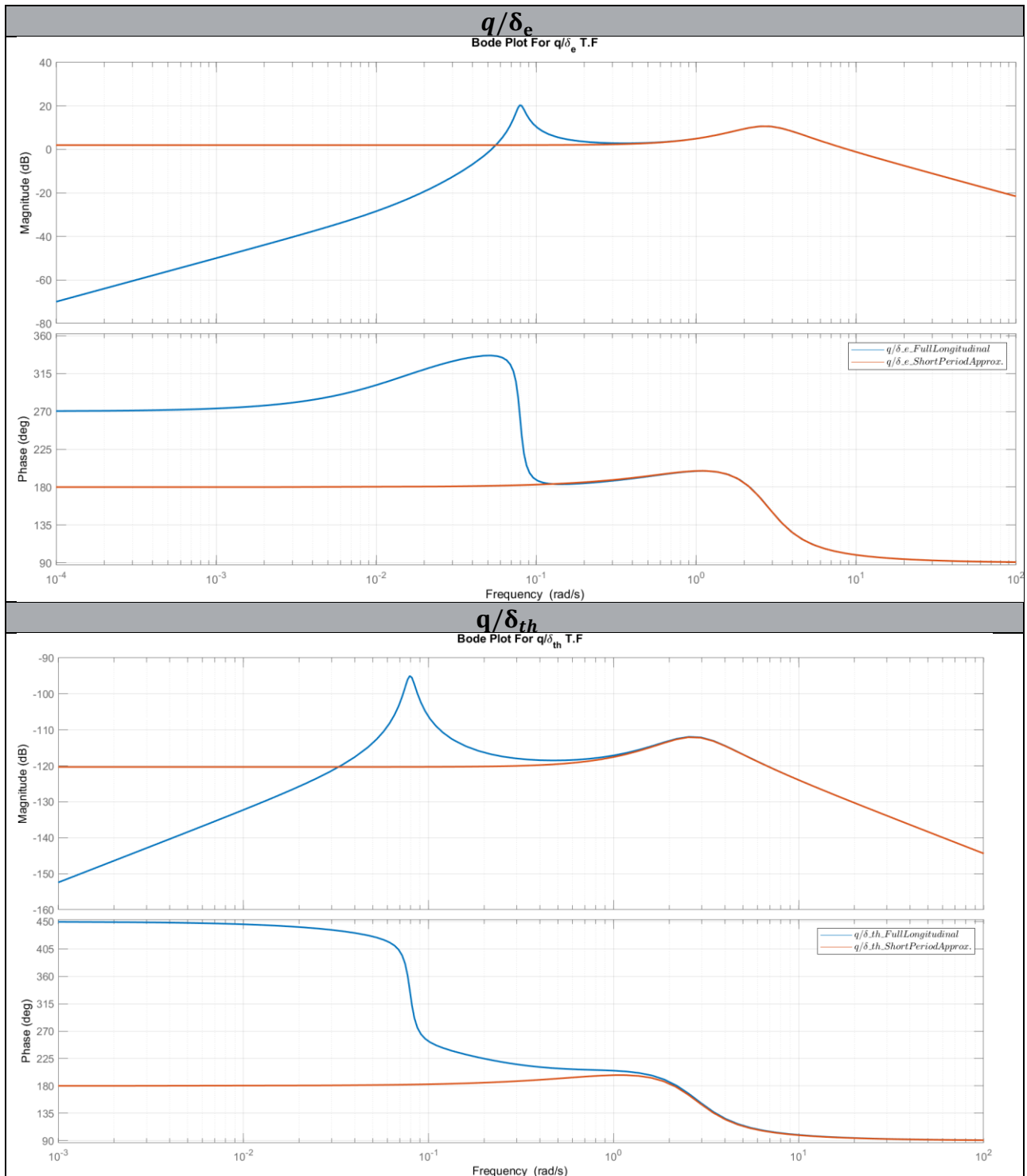
iii. Various δ_{th}



Comments:

- From these results we can deduce that the short period approximation is accurate till 5 sec, and the same previous notes for full linear compared to nonlinear.

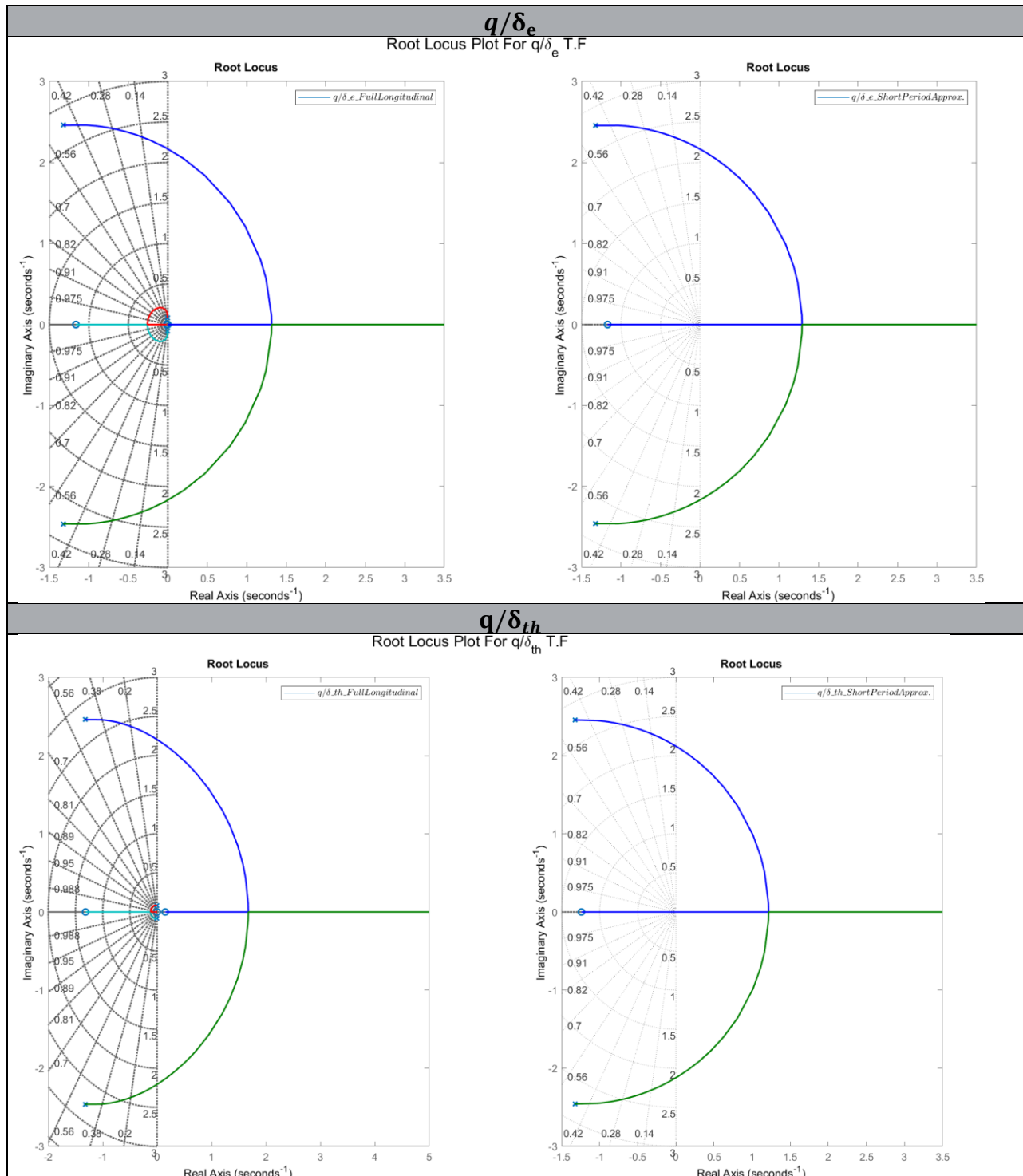
iv. Bode Plot



Comments:

- For frequency greater than 1 rad/sec the short period transfer function is acceptable.

v. Root Locus



Comments:

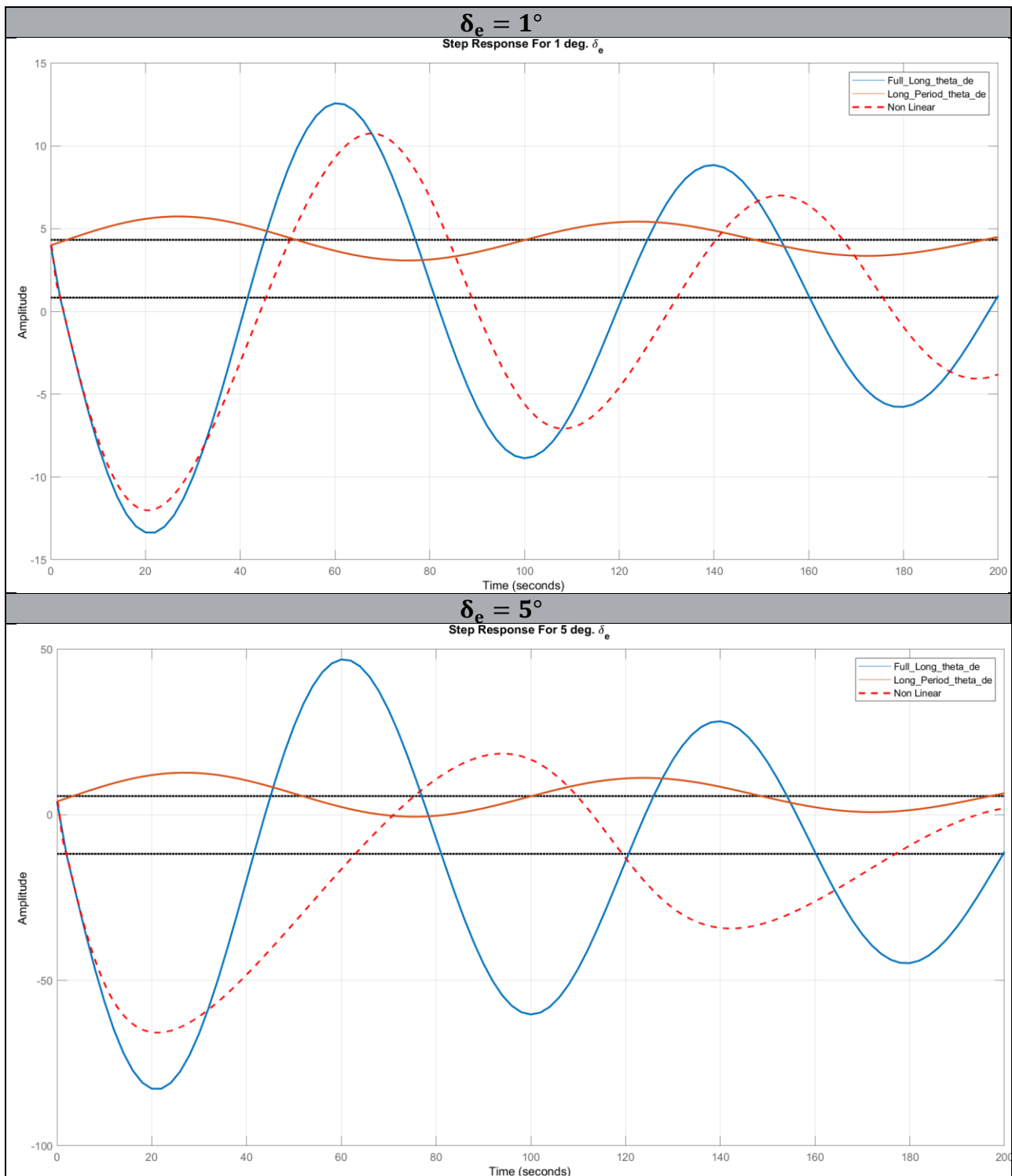
- From the root locus, the transfer function of short period is almost the same dynamics of the fast poles of the full linear model.

o) Response of state θ

i. Transfer Functions

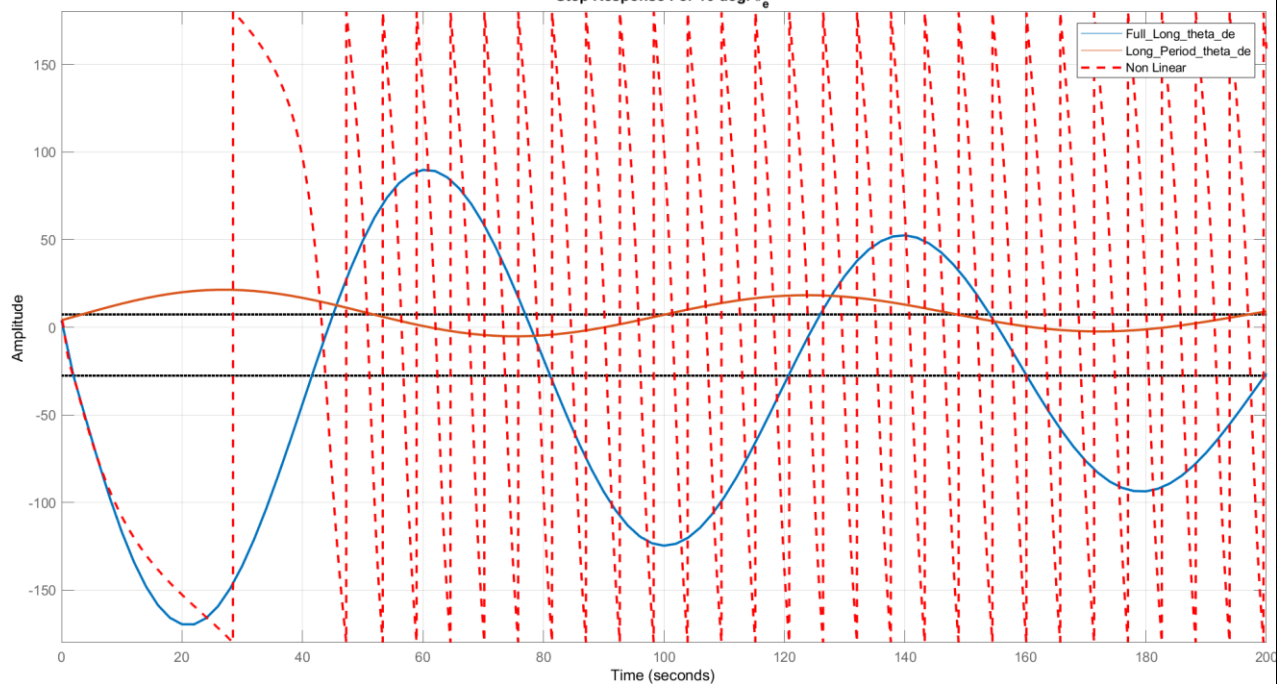
TF	Full Linear	Long Period Mode
θ / δ_e	$\frac{-8.343 s^2 - 9.844 s - 0.1563}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.0924 s + 0.0495}$	$\frac{0.09688 s + 0.00139}{s^2 + 0.005167 s + 0.004217}$
θ / δ_{th}	$\frac{-6.04e-06 s^2 - 7.047e-06 s + 1.184e-06}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.0924 s + 0.04945}$	$\frac{1.12e-07}{s^2 + 0.005167 s + 0.004217}$

ii. Various δ_e



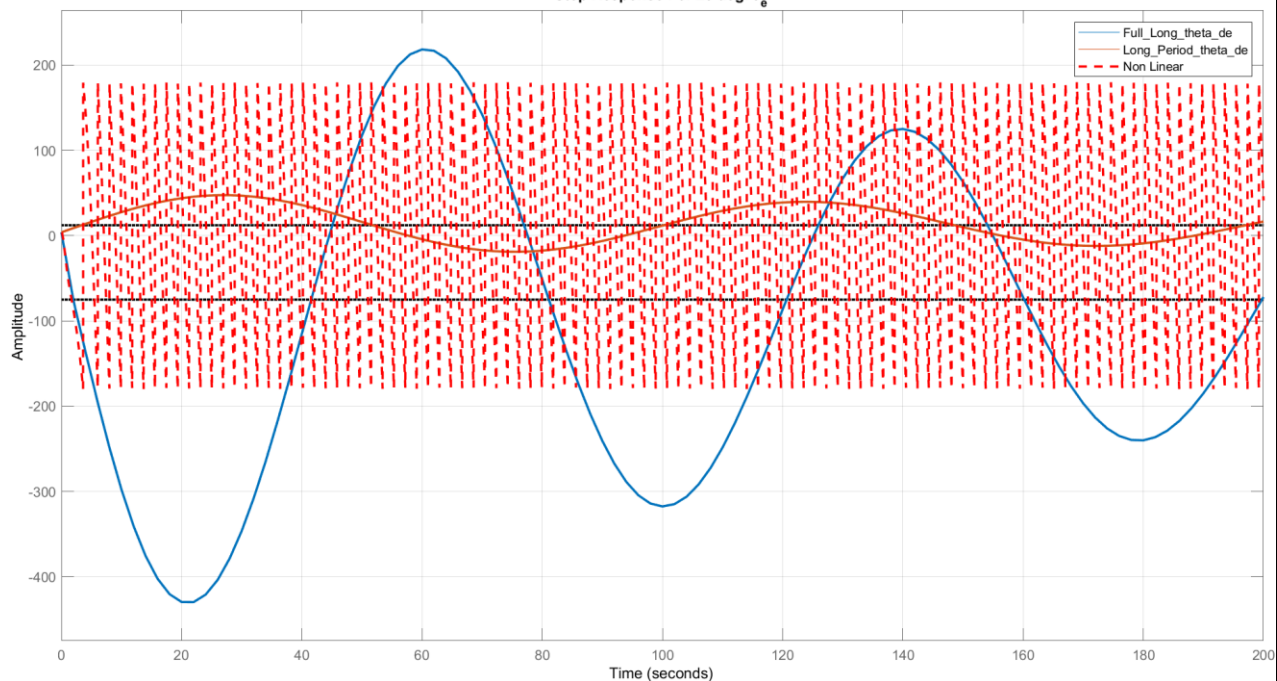
$\delta_e = 10^\circ$

Step Response For 10 deg. δ_e

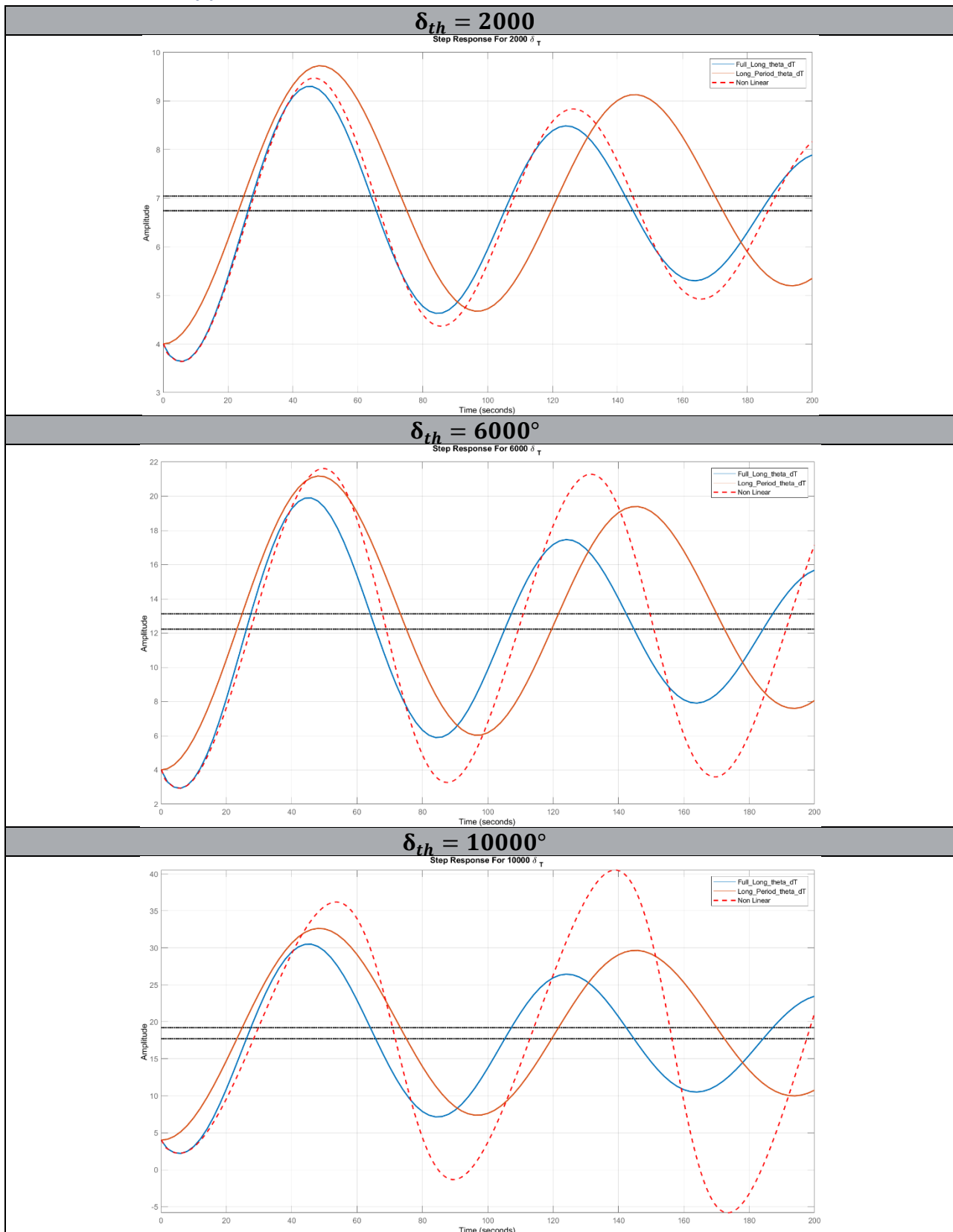


$\delta_e = 25^\circ$

Step Response For 25 deg. δ_e



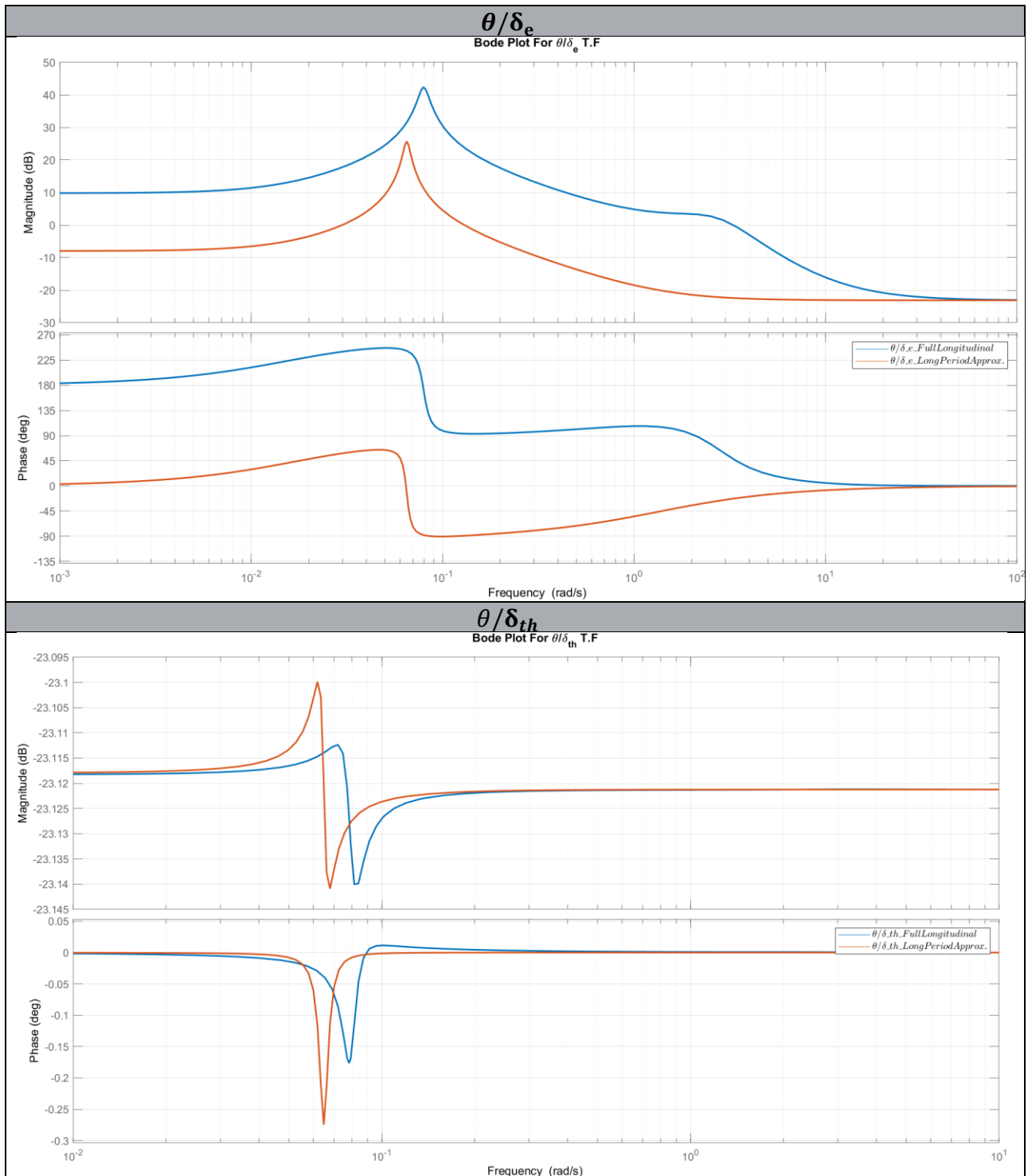
iii. Various δ_{th}



Comments:

- The same notes of u.

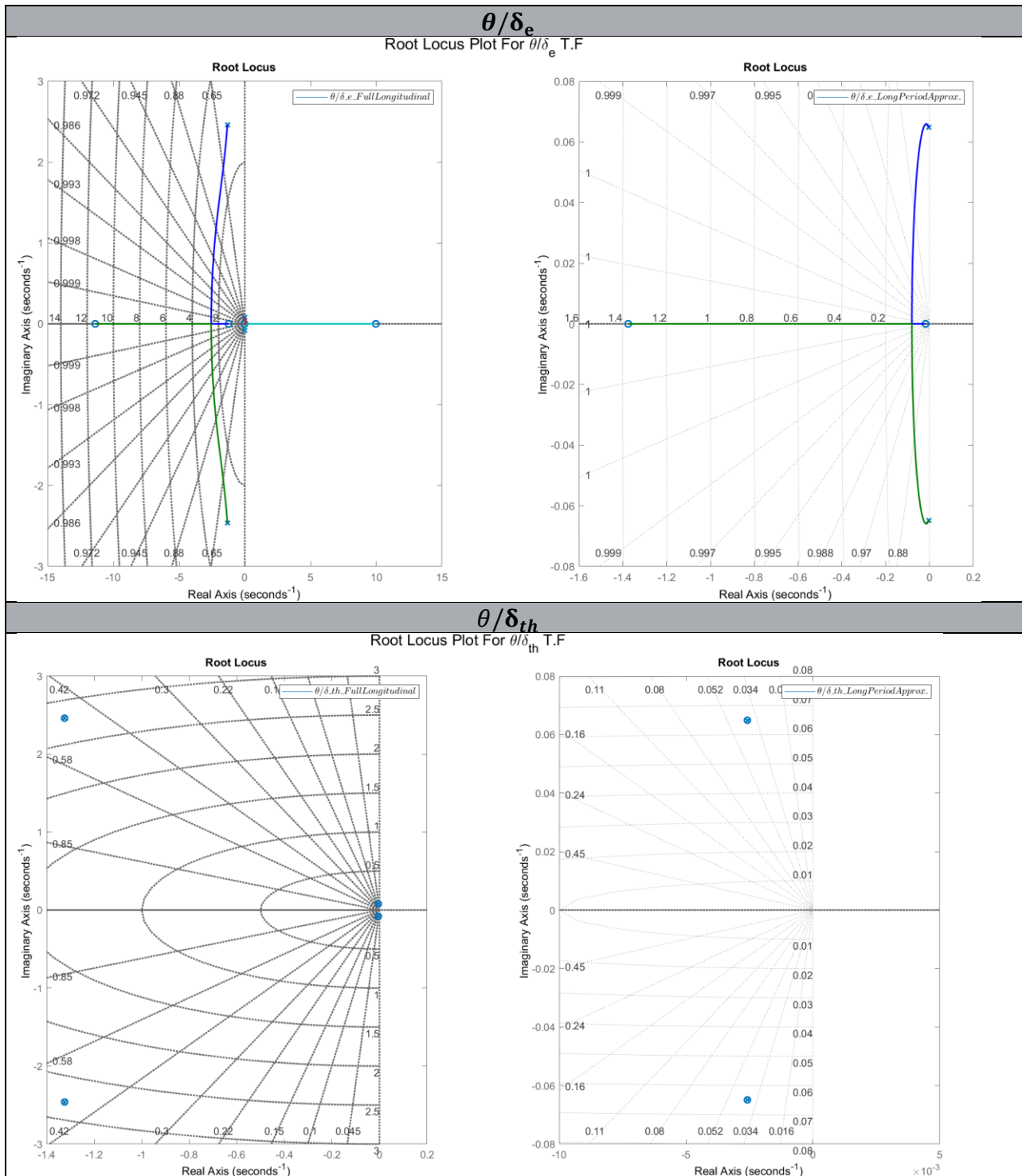
iv. Bode Plot



Comments:

- For δ_e input, the approximation is valid when the frequency is greater than 1 rad/sec.
- For δ_{th} input, the approximation is valid when the frequency is greater than 0.1 rad/sec.

v. Root Locus



Comments:

- For delta e input, the transfer function for long period model is always stable but in the full linear model. the transfer function has instability region.

Simulink Bonus

The Following results are extracted from our SIMULINK model using Model Linearizer and these Transfer Functions are the same as ones obtained from Full Linear Model.

ModelLinearizerTF_u_de =

$$\frac{3.02 s^3 + 263.1 s^2 + 149.5 s + 315.4}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_u_dT =

$$\frac{0.000842 s^3 + 0.002418 s^2 + 0.006665 s + 0.0002144}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_w_de =

$$\frac{-43.2 s^3 - 3782 s^2 - 35.44 s - 16.52}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_w_dT =

$$\frac{-0.002743 s^2 + 0.0001296 s - 1.239e-05}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_alpha_de =

$$\frac{-0.09688 s^3 - 8.481 s^2 - 0.1024 s - 0.08609}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_alpha_dT =

$$\frac{-1.314e-07 s^3 - 6.499e-06 s^2 - 7.509e-07 s - 6.112e-08}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_q_de =

$$\frac{-8.343 s^3 - 9.844 s^2 - 0.1563 s + 8.103e-18}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_q_dT =

$$\frac{-6.04e-06 s^3 - 7.047e-06 s^2 + 1.184e-06 s - 2.1e-23}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_theta_de =

$$\frac{-8.343 s^2 - 9.844 s - 0.1563}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

ModelLinearizerTF_theta_dT =

$$\frac{-6.04e-06 s^2 - 7.047e-06 s + 1.184e-06}{s^4 + 2.658 s^3 + 7.838 s^2 + 0.09242 s + 0.04945}$$

Continuous-time transfer function.

Lateral Results

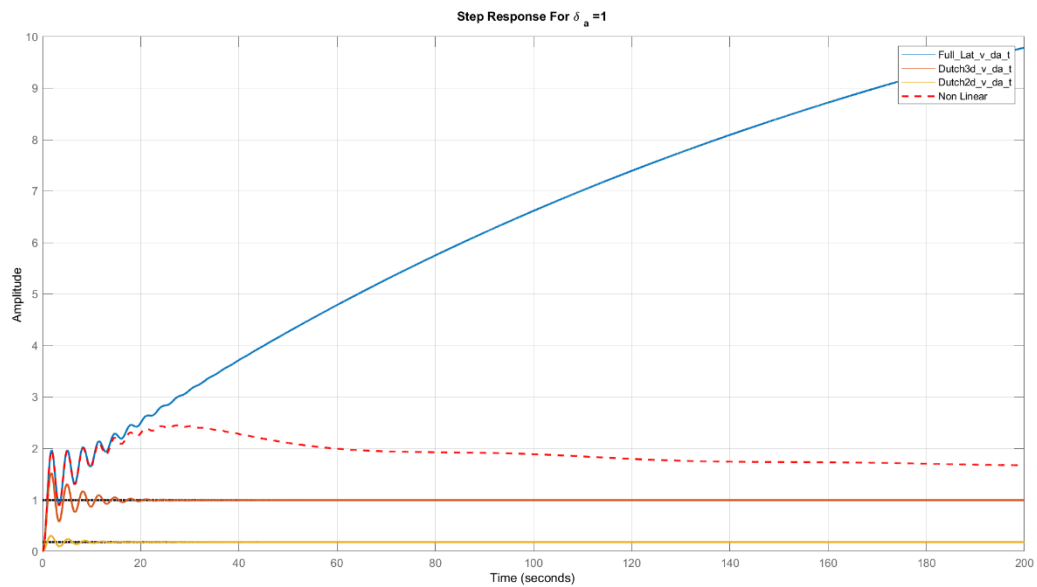
a) Response of state v

i. Transfer Functions

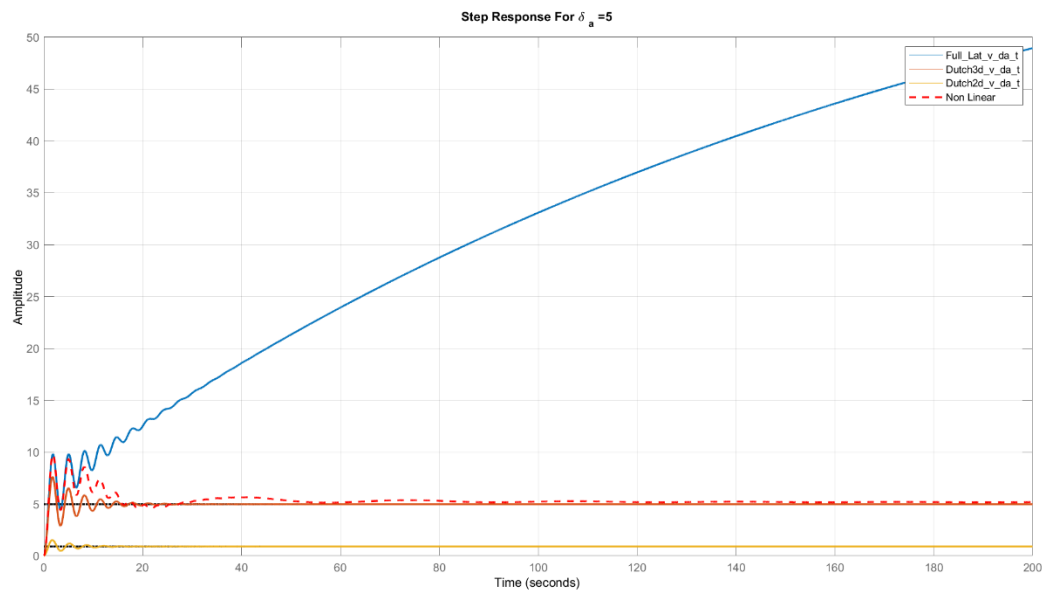
TF	Full Linear	Dutch 3 DOF Mode	Dutch 2 DOF Mode
v/δ_a	$\frac{132.1 s^2 + 399.8 s + 24.48}{s^4 + 1.736 s^3 + 4.295 s^2 + 5.613 s + 0.02994}$	$\frac{132.1 s + 299.2}{ss^3 + 1.736 s^2 + 4.295 s + 5.251}$	$\frac{34.37}{s^2 + 0.424 s + 3.346}$
v/δ_r	$\frac{18.95 s^3 + 886.9 s^2 + 1224 s - 2.901}{s^4 + 1.736 s^3 + 4.295 s^2 + 5.613 s + 0.02994}$	$\frac{18.95 s^2 + 886.9 s + 1177}{s^3 + 1.736 s^2 + 4.295 s + 5.251}$	$\frac{18.95 s + 815.8}{s^2 + 0.424 s + 3.346}$

ii. Various δ_a

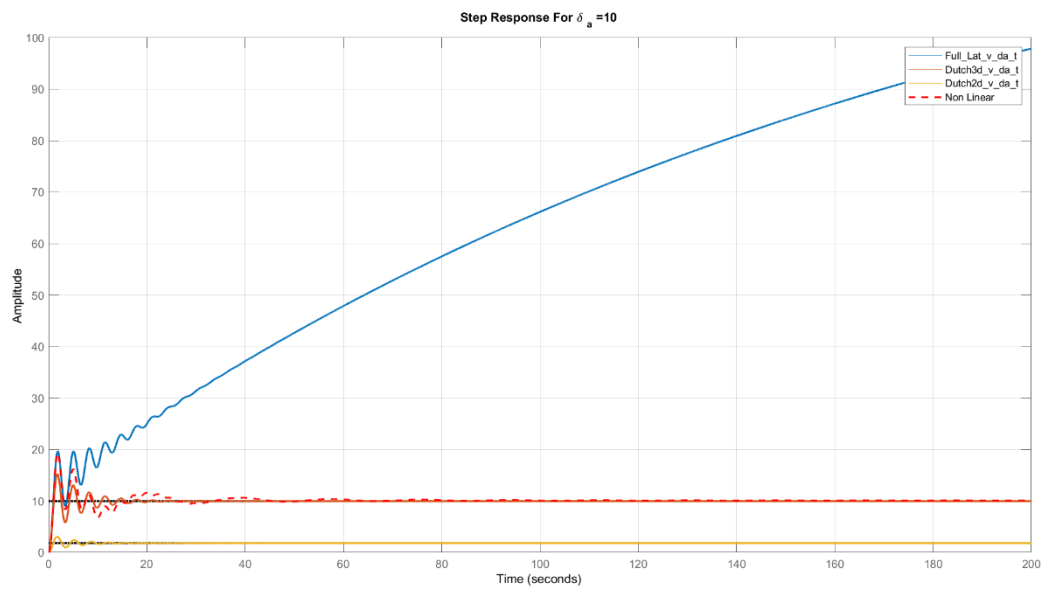
$\delta_a = 1^\circ$



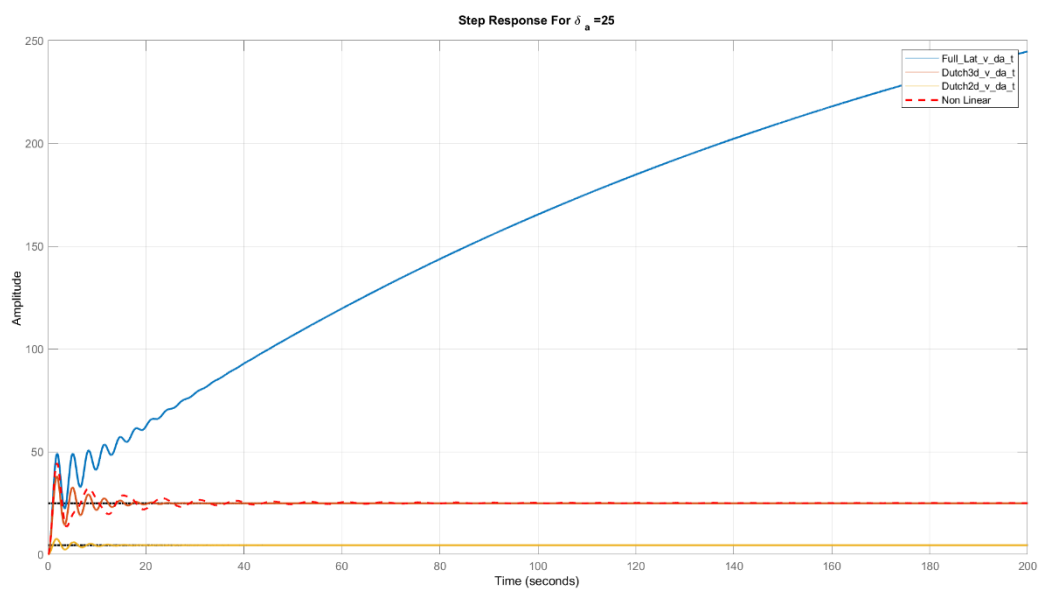
$\delta_a = 5^\circ$



$\delta_a = 10^\circ$

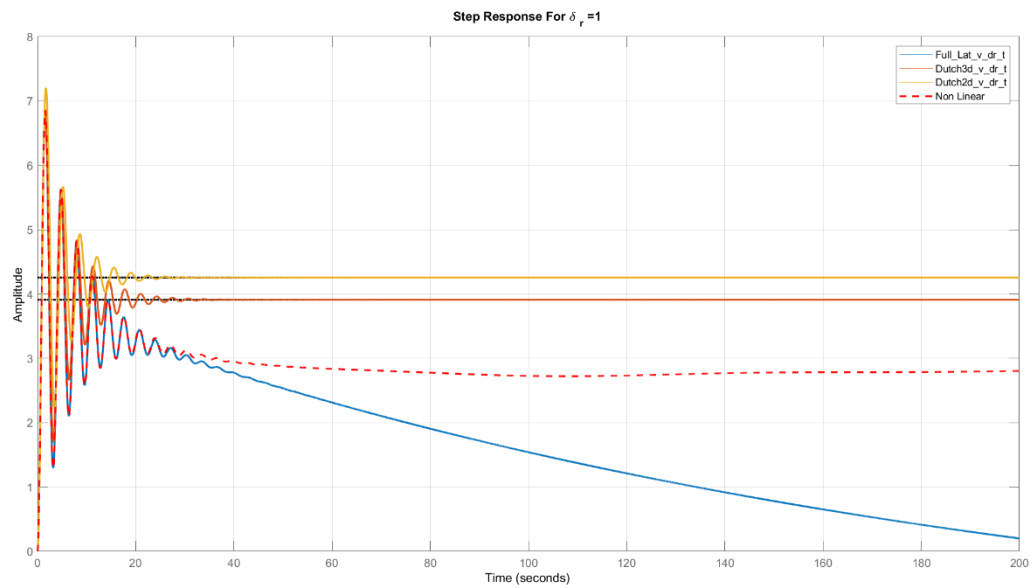


$\delta_a = 25^\circ$

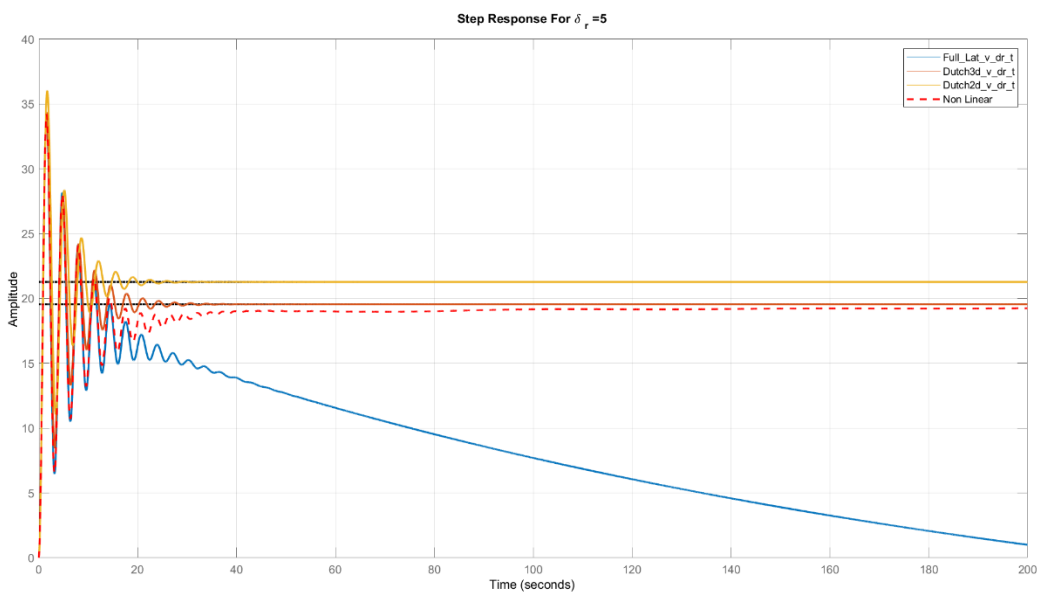


iii. Various δ_r

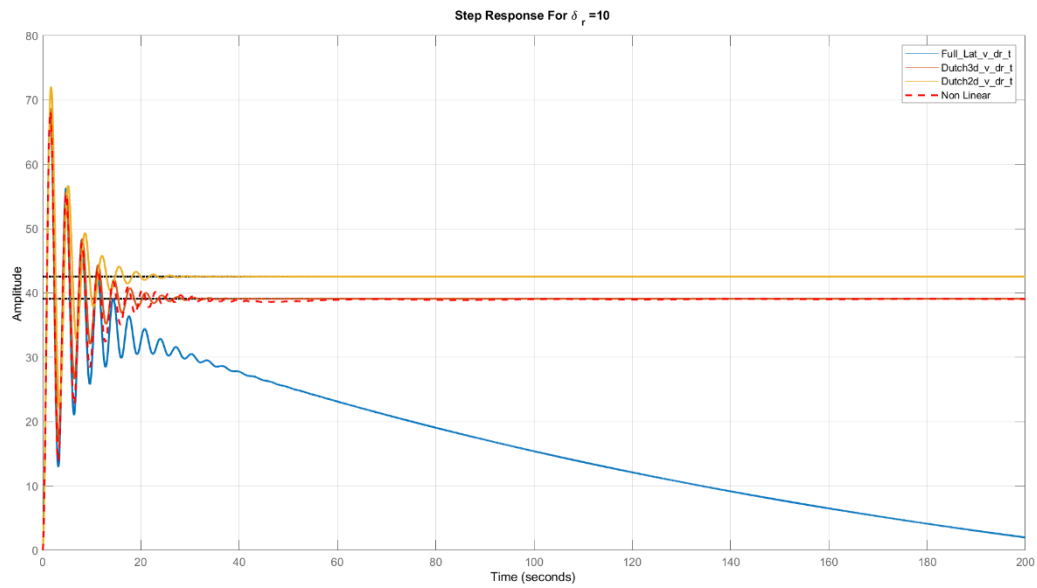
$\delta_r = 1^\circ$



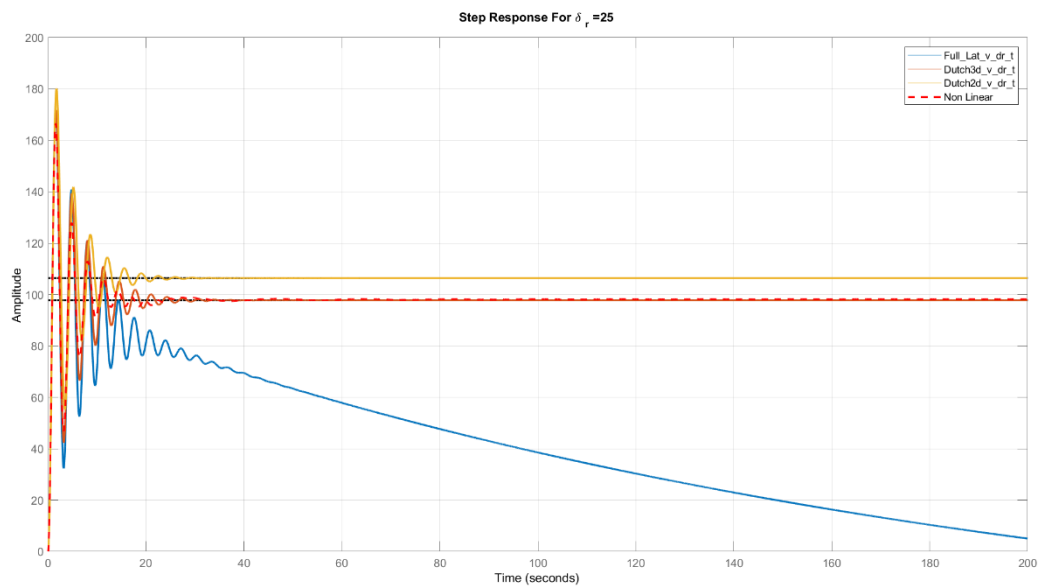
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



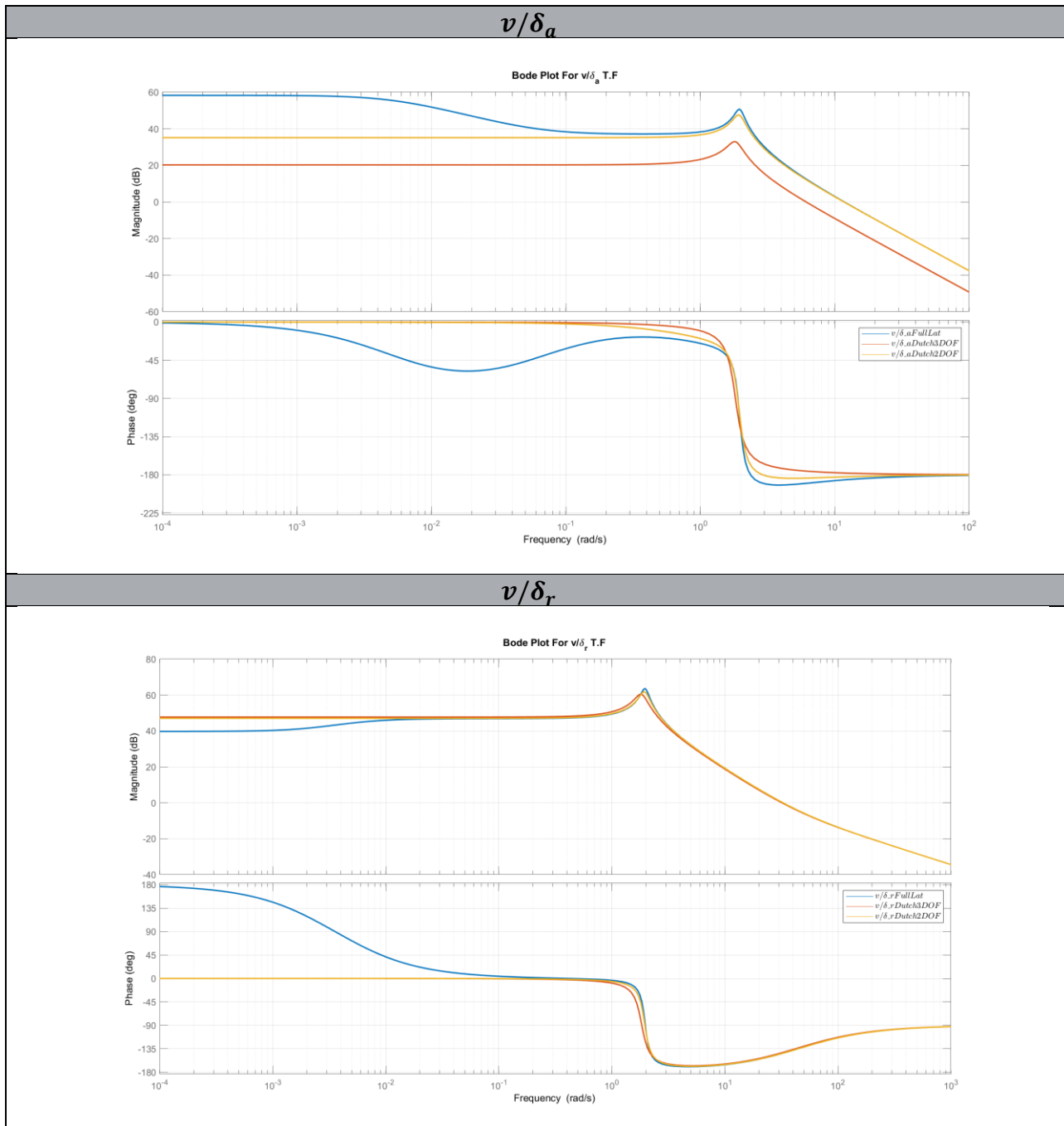
$$\delta_r = 25^\circ$$



Comments:

- The approximations are not accurate for calculation of state (v), but it is noticed that the error is decreased with the increase of angles of aileron and rudder.
- The accuracy of the linear solution depends on the type of the input which changes (aileron or rudder), the value of this change and the time.
- For aileron and rudder: the linear solution gives accurate values for small changes in the angle in range time and this range decreases with the increase of the angle.

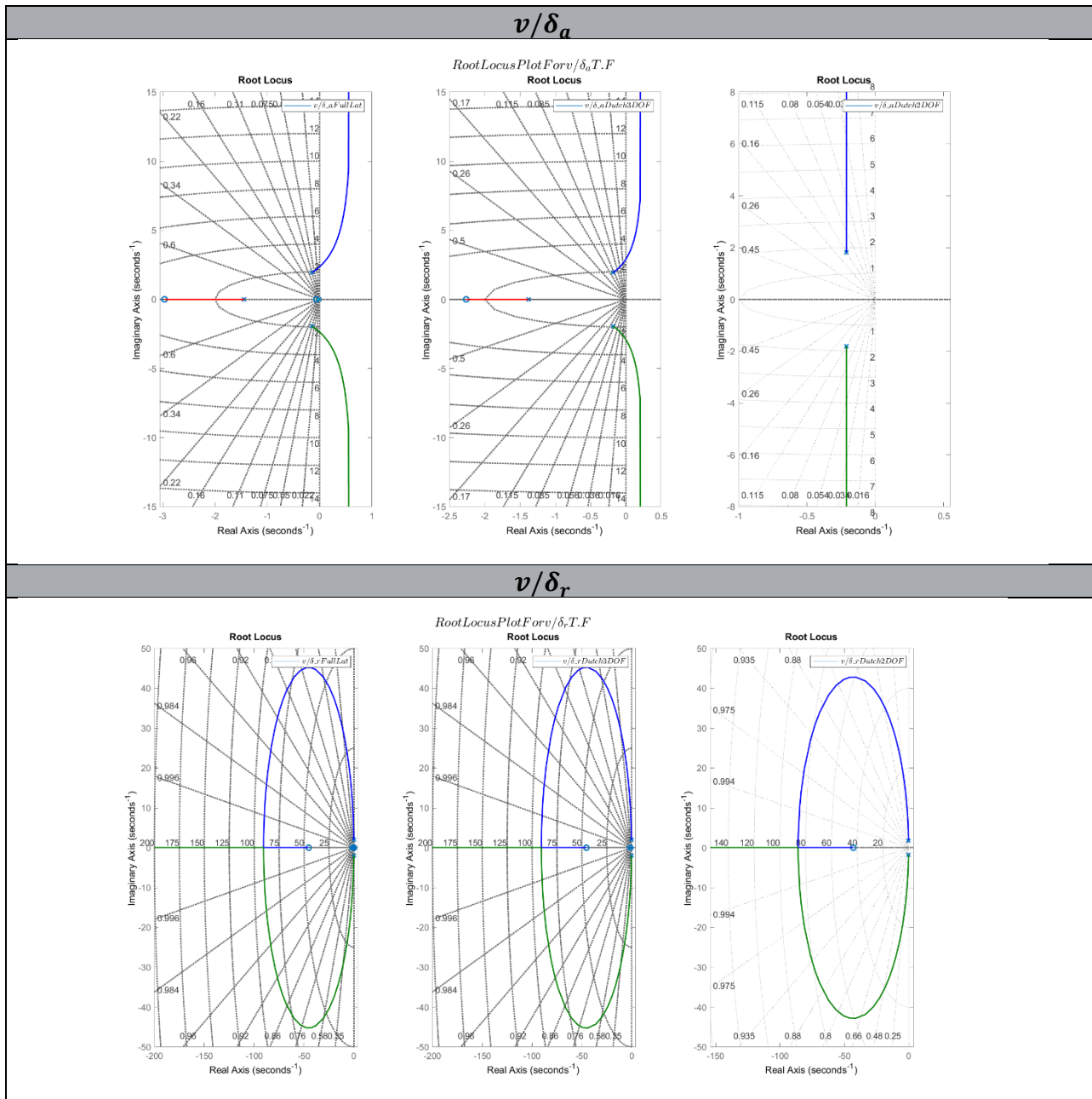
iv. Bode Plot



Comments:

- For δ_a input, The approximations are valid compared to the full linear model when the frequency is greater than 0.4 rad/sec, but the 3 DOF has some error in magnitude.
- For δ_r input, The approximations are valid compared to the full linear model when the frequency is greater than 0.002 rad/sec. (more accurate)

v. Root Locus



Comments:

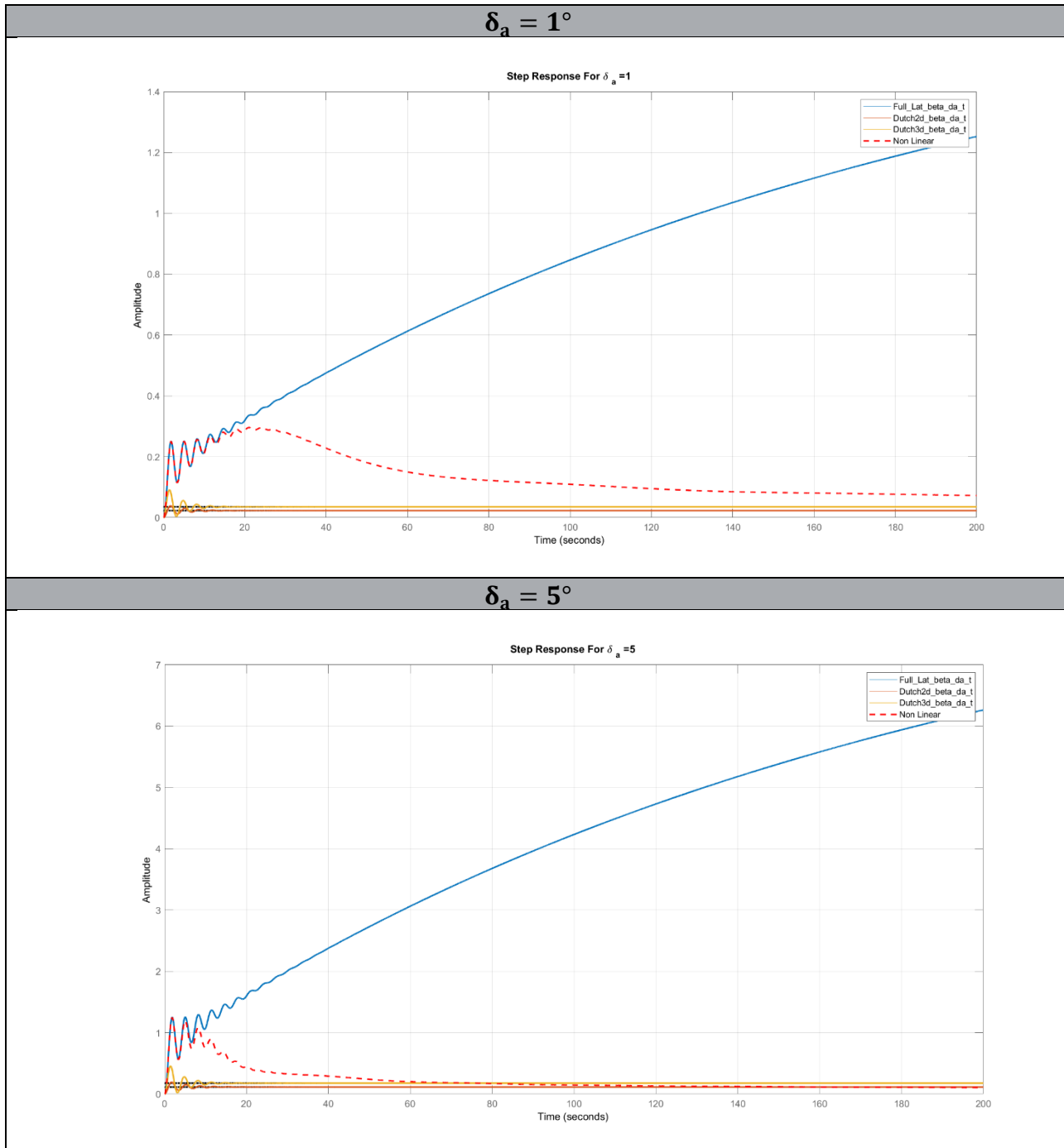
- For delta a input , The full linear and 3 DOF approximation have instability region when gain increases.
- For delta r input , all models are always stable

b) Response of state β

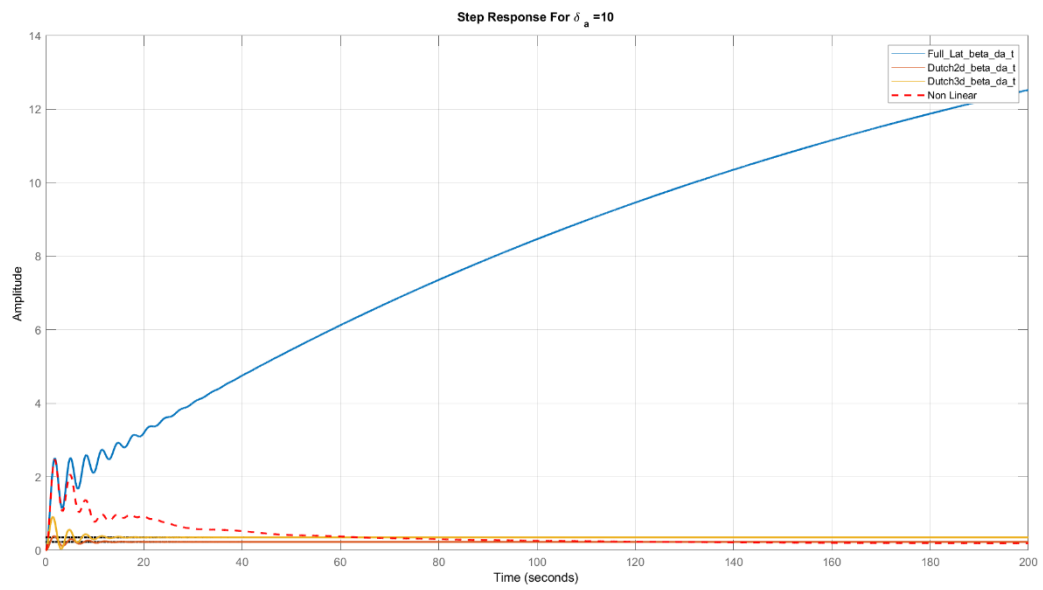
i. Transfer Functions

	Full Linear	Dutch 3 DOF Mode	Dutch 2 DOF Mode
β/δ_a	$\frac{0.2955 s^2 + 0.8944 s + 0.05476}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{0.2955 s + 0.1566}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	$\frac{0.07689}{s^2 + 0.424 s + 3.352}$
β/δ_r	$\frac{0.0424 s^3 + 1.984 s^2 + 2.739 s - 0.006491}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{0.0424 s^2 + 1.984 s + 2.391}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	$\frac{0.0424 s + 1.825}{s^2 + 0.424 s + 3.352}$

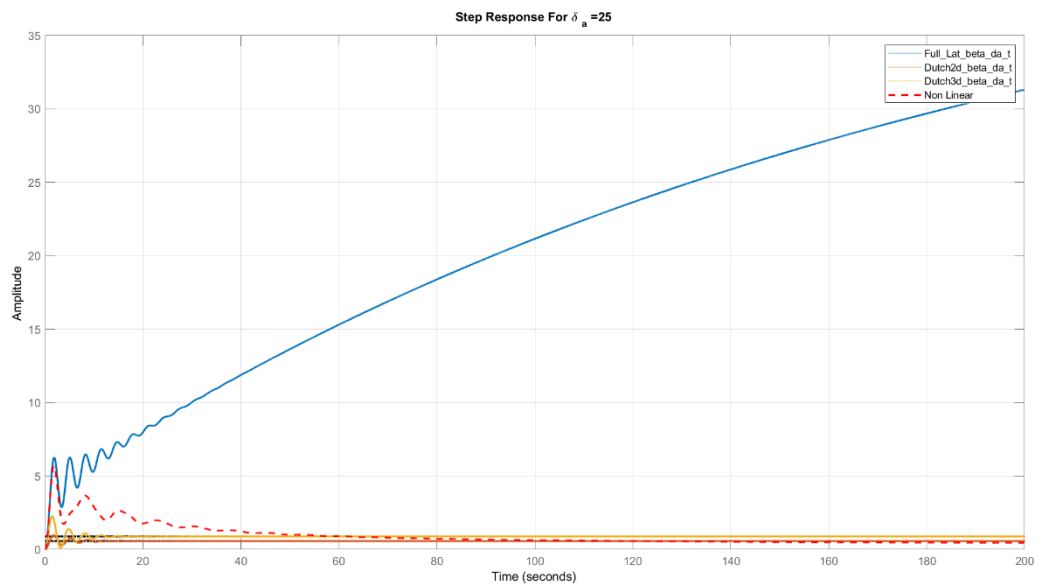
ii. Various δ_a



$\delta_a = 10^\circ$

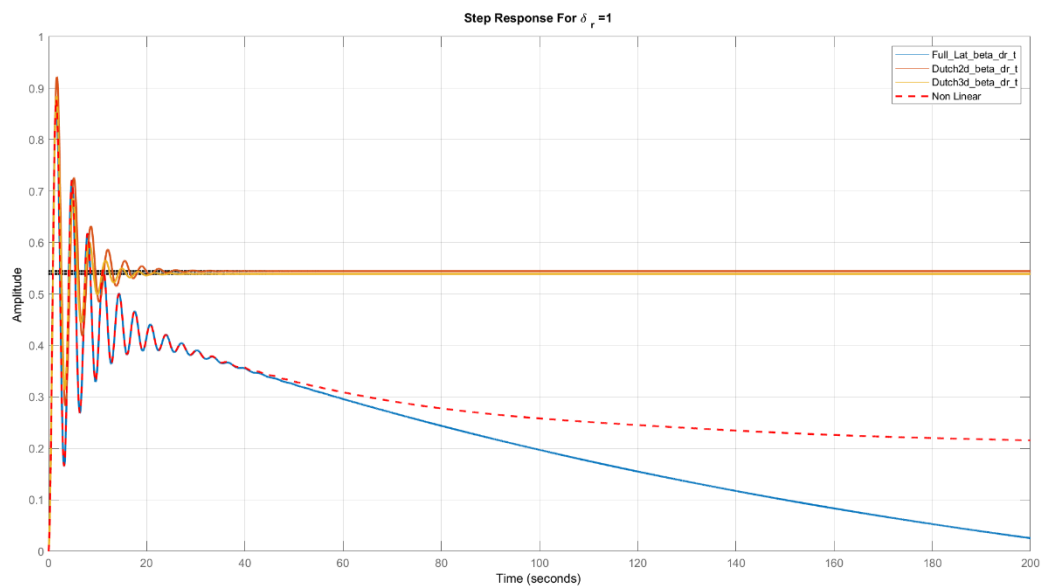


$\delta_a = 25^\circ$

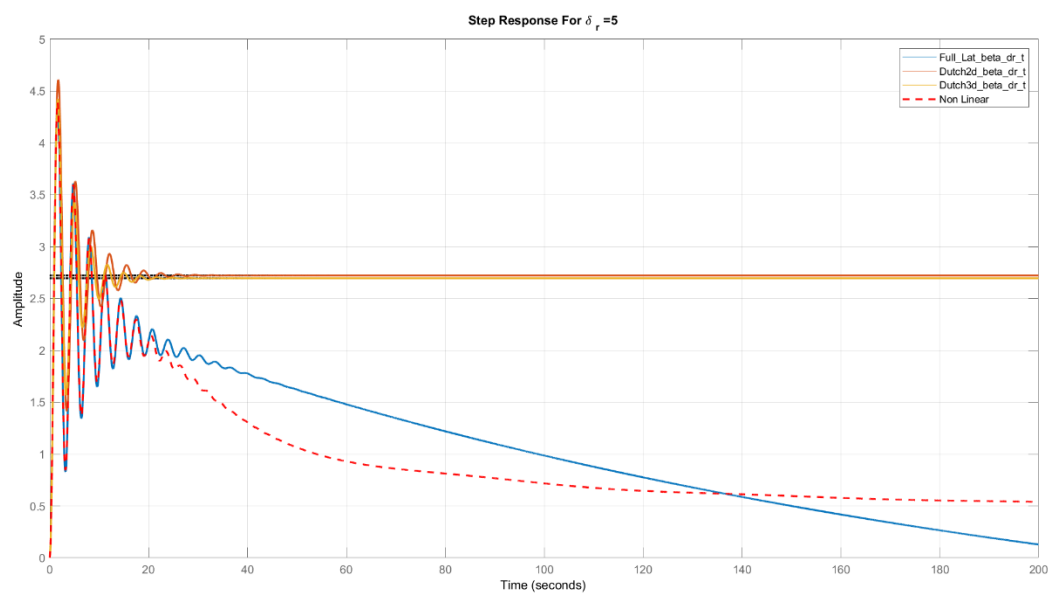


iii. Various δ_r

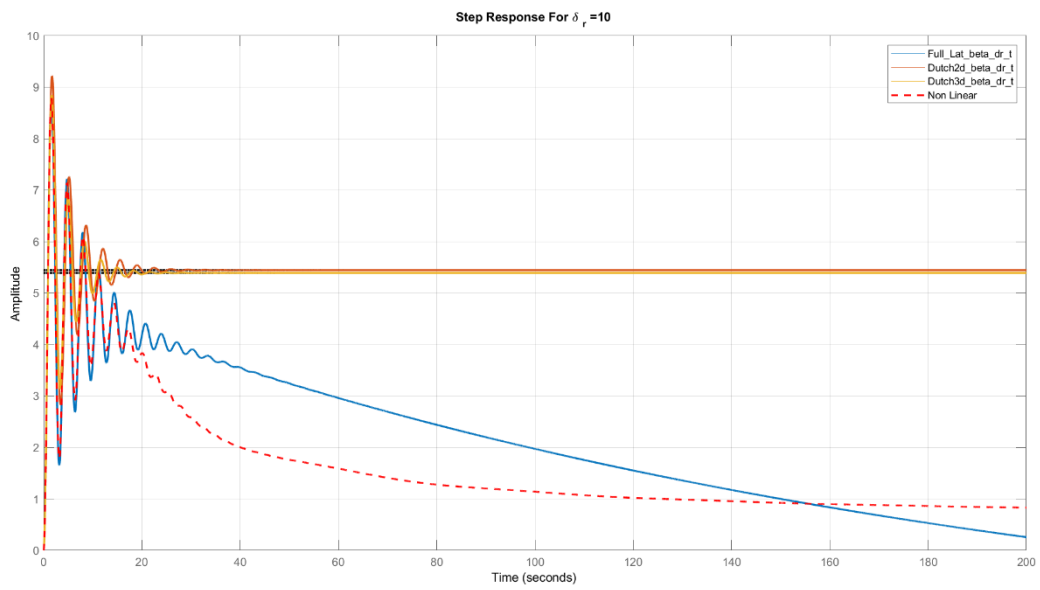
$\delta_r = 1^\circ$



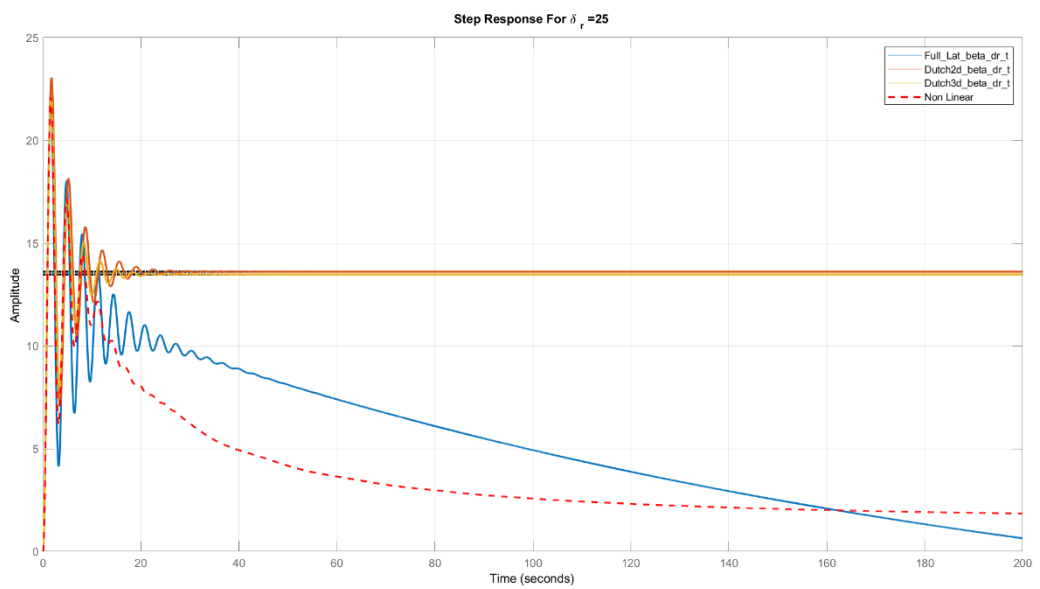
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



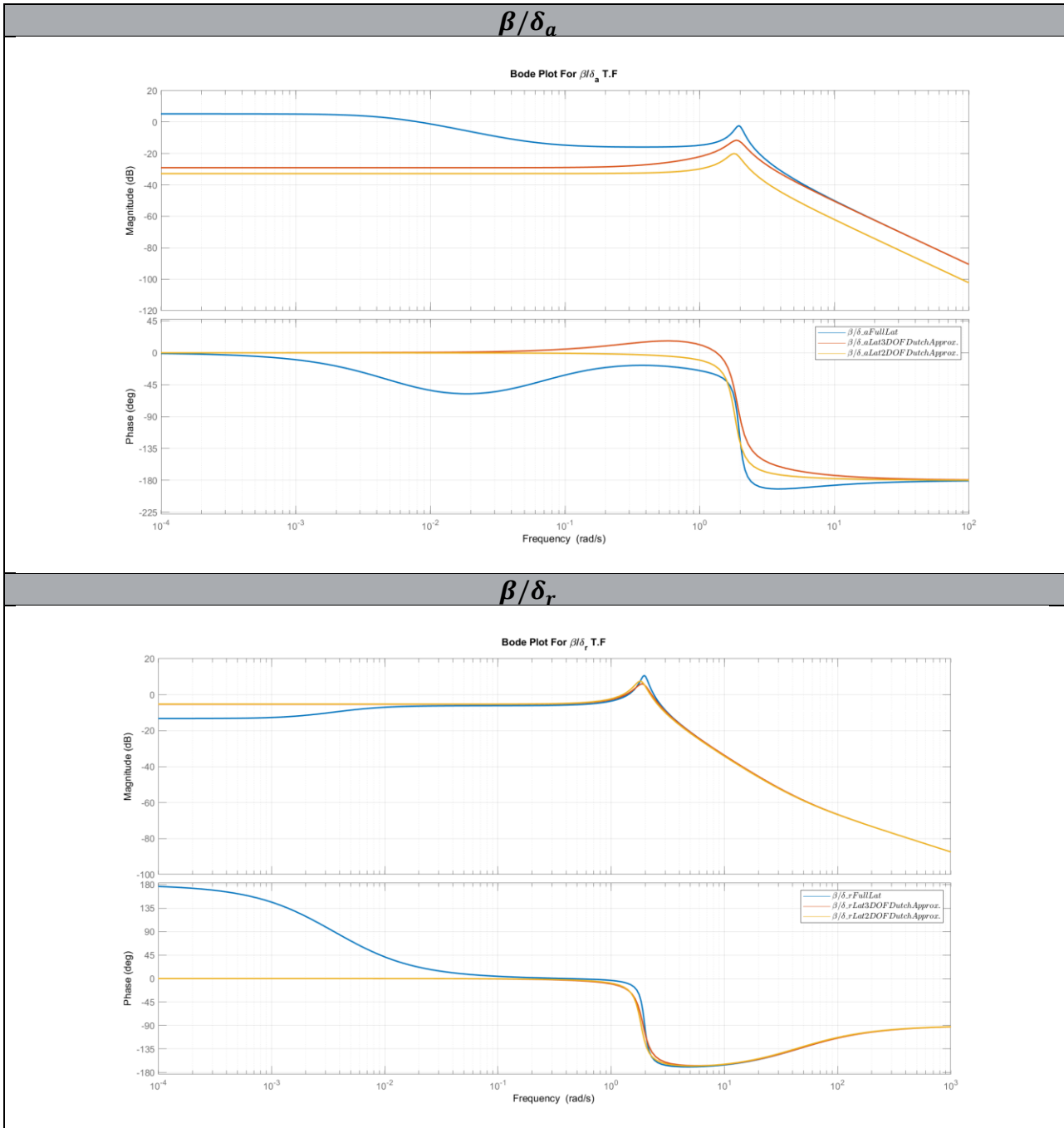
$$\delta_r = 25^\circ$$



Comments:

- Same as notes of v

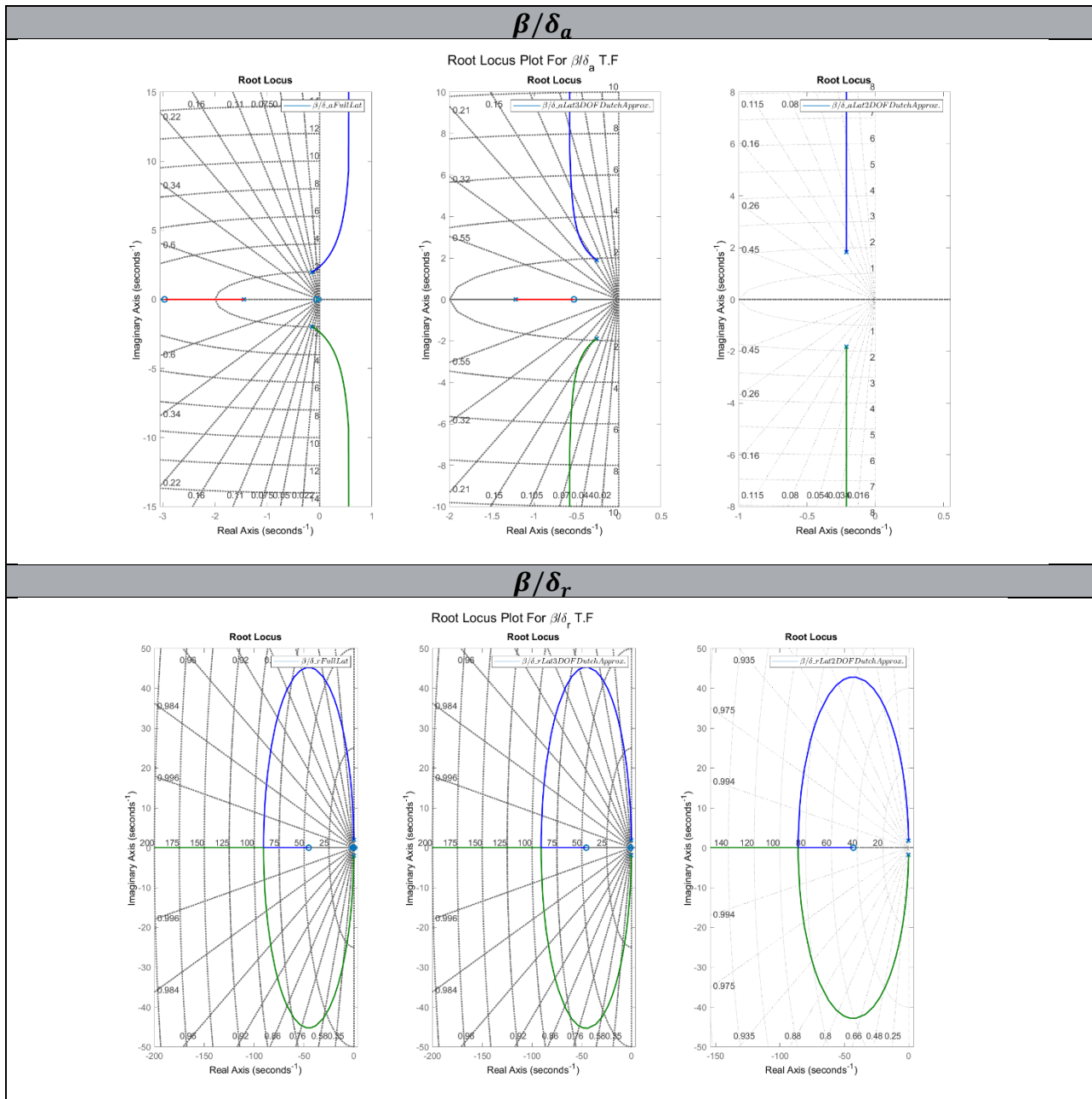
iv. Bode Plot



Comments:

- For delta a input , The approximations have some error compared to the full linear model.
- For delta r input , The approximations are valid compared to the full linear model when the frequency is greater than .002 rad/sec.

v. Root Locus



Comments:

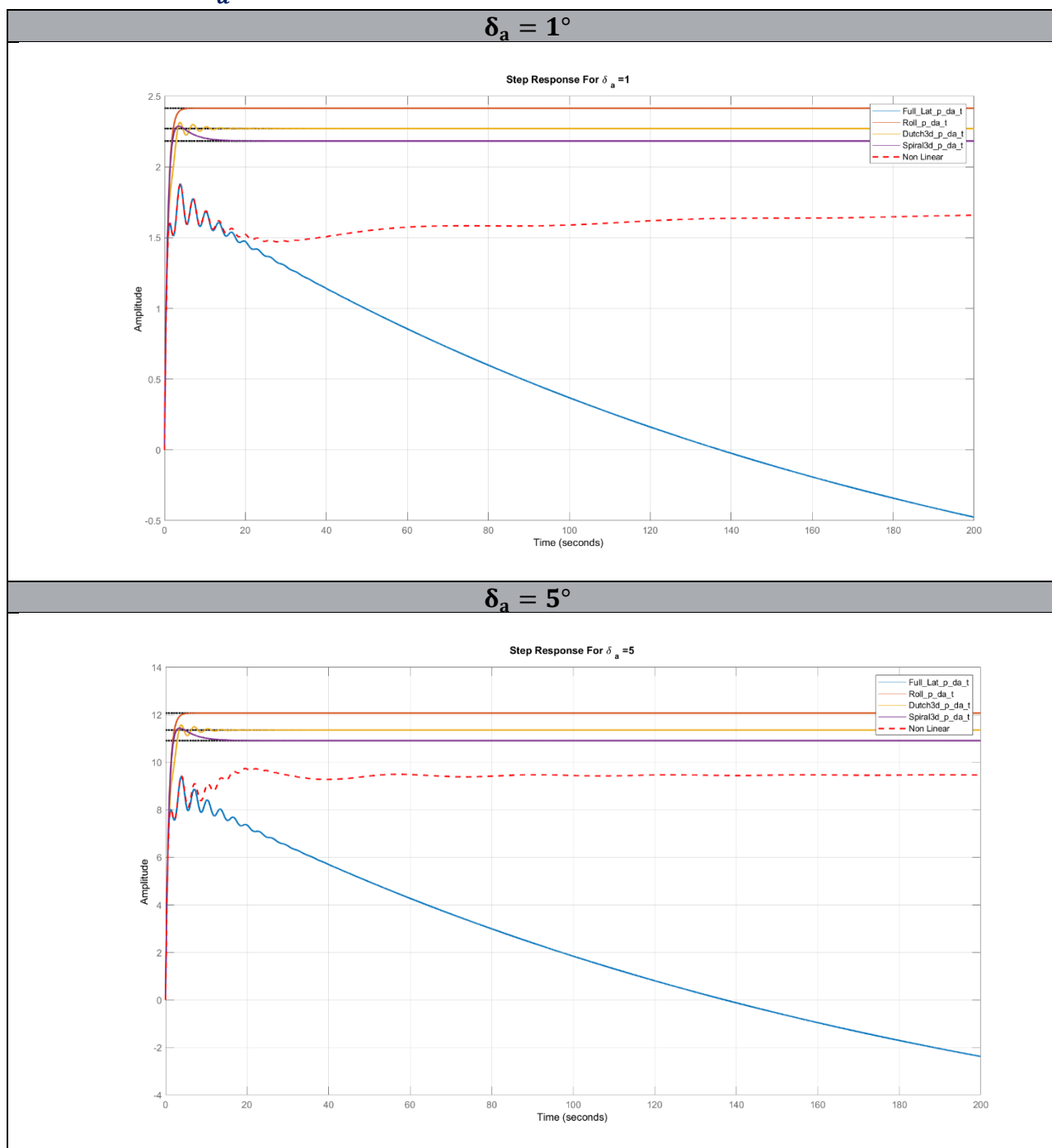
- For delta a input , The full linear has instability region when gain increases.
- For delta r input , all models are always stable.

c) Response of state p

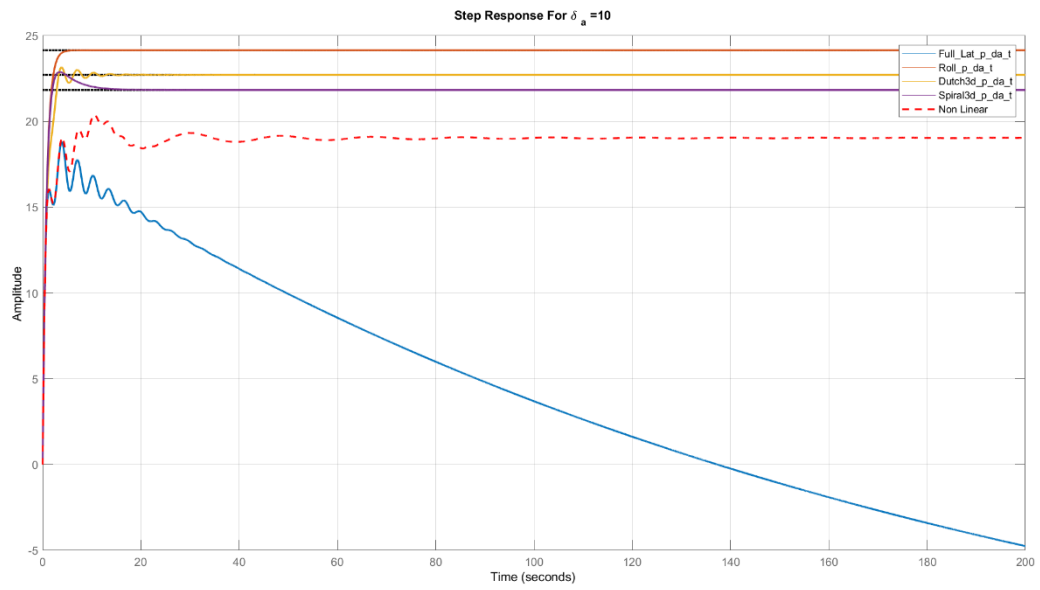
i. Transfer Functions

TF	Full Linear	Spiral 3 DOF Mode	Dutch 3 DOF Mode	Roll 1 DOF Mode
p/δ_a	$\frac{3.14 s^3 + 1.355 s^2 + 10.07 s - 0.05}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{3.14 s + 0.8057}{s^2 + 1.561 s + 0.369}$	$\frac{3.14 s^2 + 1.369 s + 10.08}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	$\frac{3.14}{s + 1.3}$
p/δ_r	$\frac{1.61 s^3 + 0.1509 s^2 - 4.232 s + 0.02122}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{1.61 s + 0.0926}{s^2 + 1.561 s + 0.369}$	$\frac{1.61 s^2 + 0.4785 s - 4.2}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	---

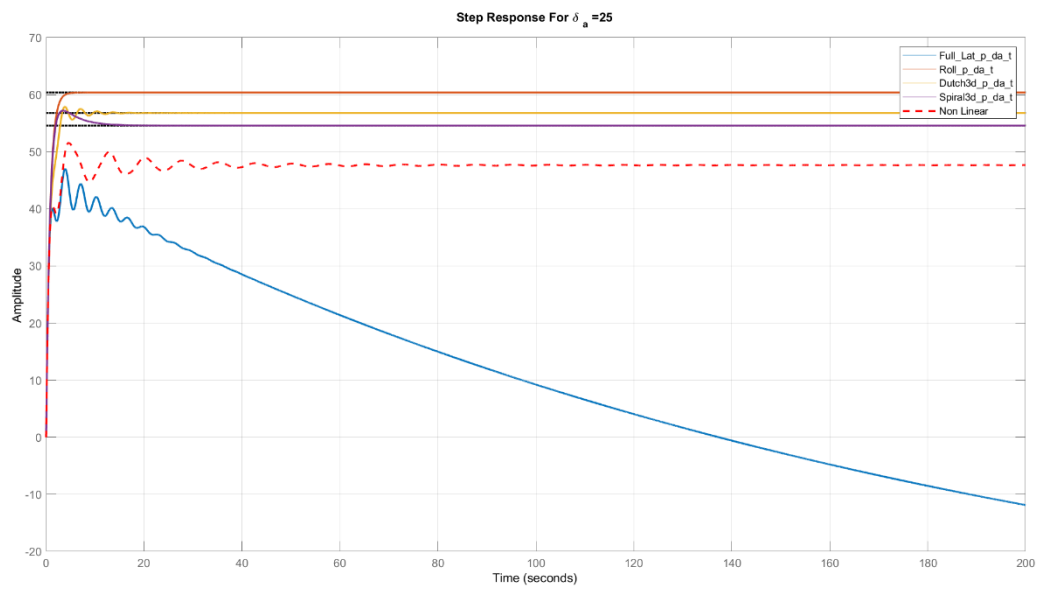
ii. Various δ_a



$\delta_a = 10^\circ$

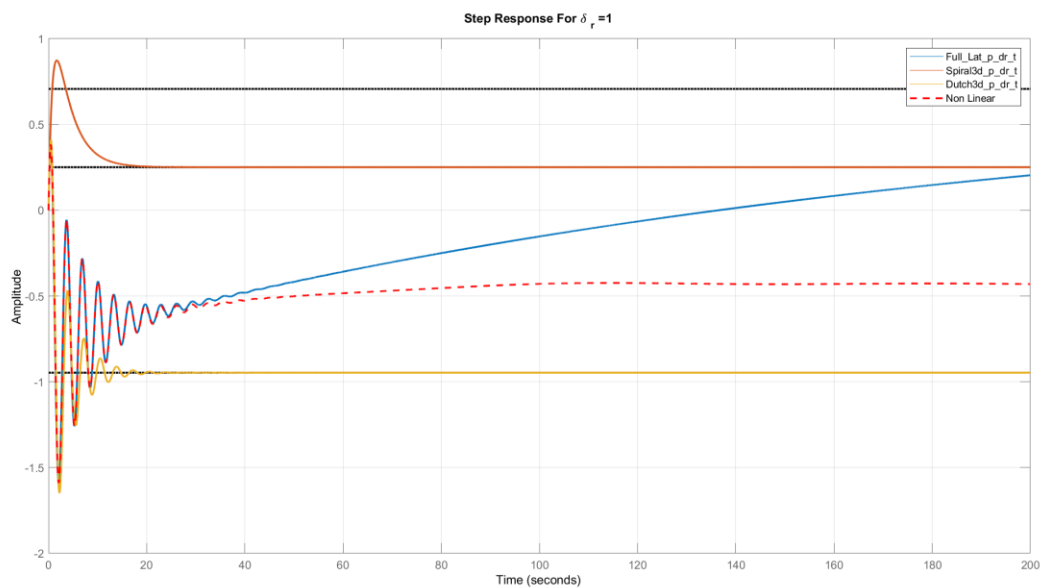


$\delta_a = 25^\circ$

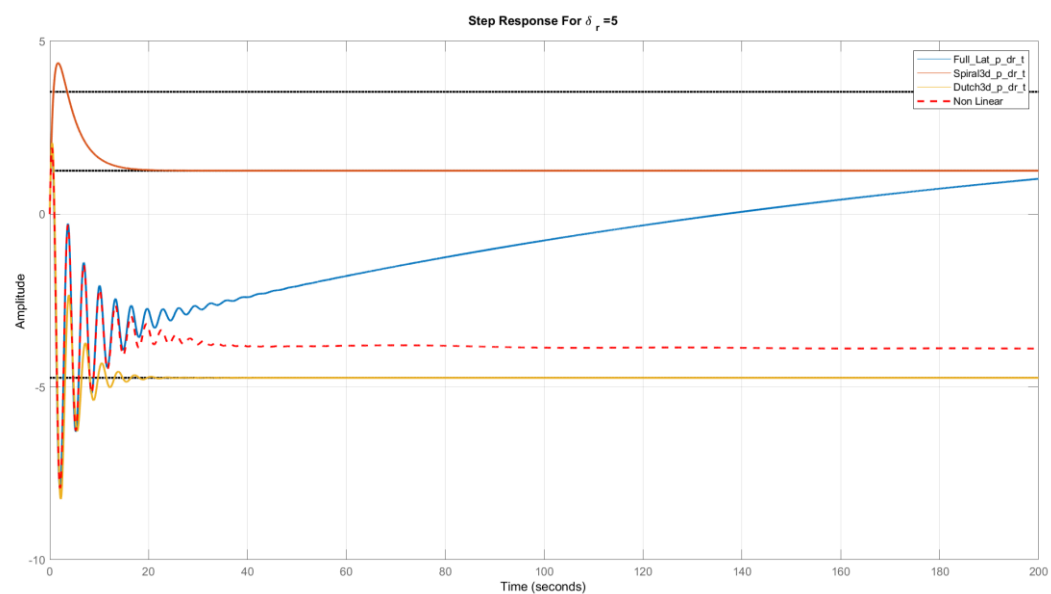


iii. Various δ_r

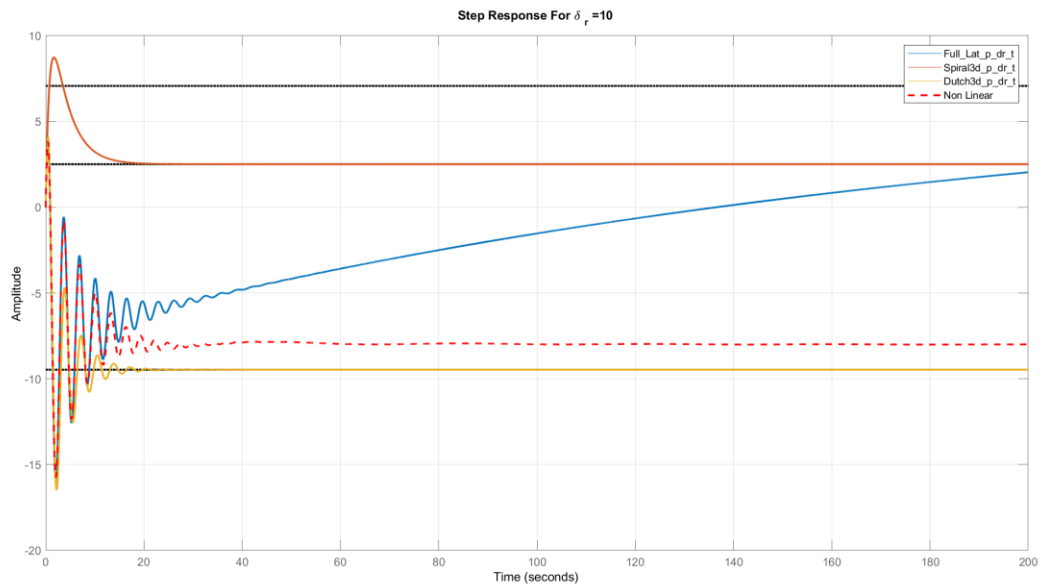
$\delta_r = 1^\circ$



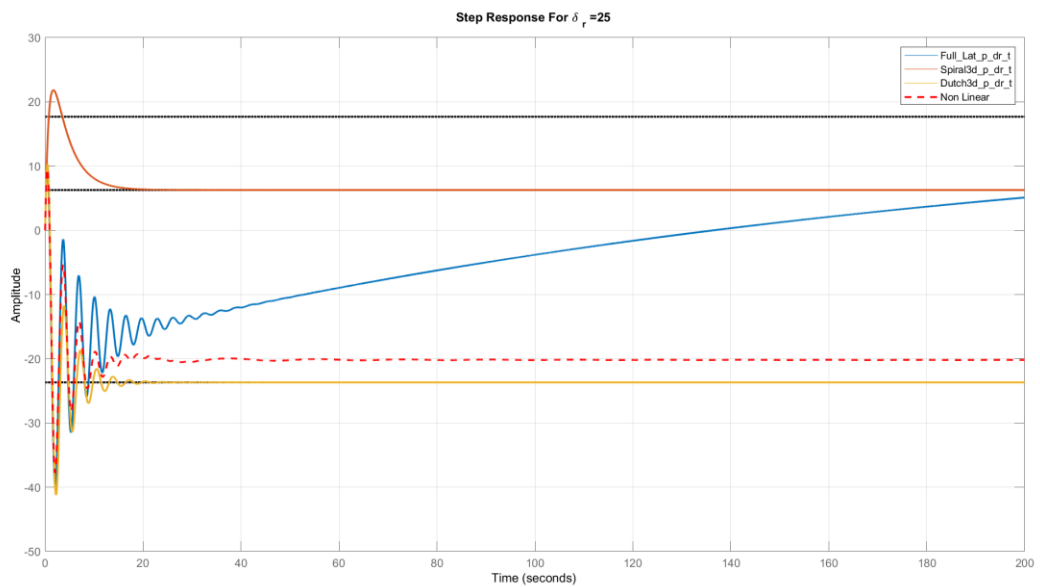
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



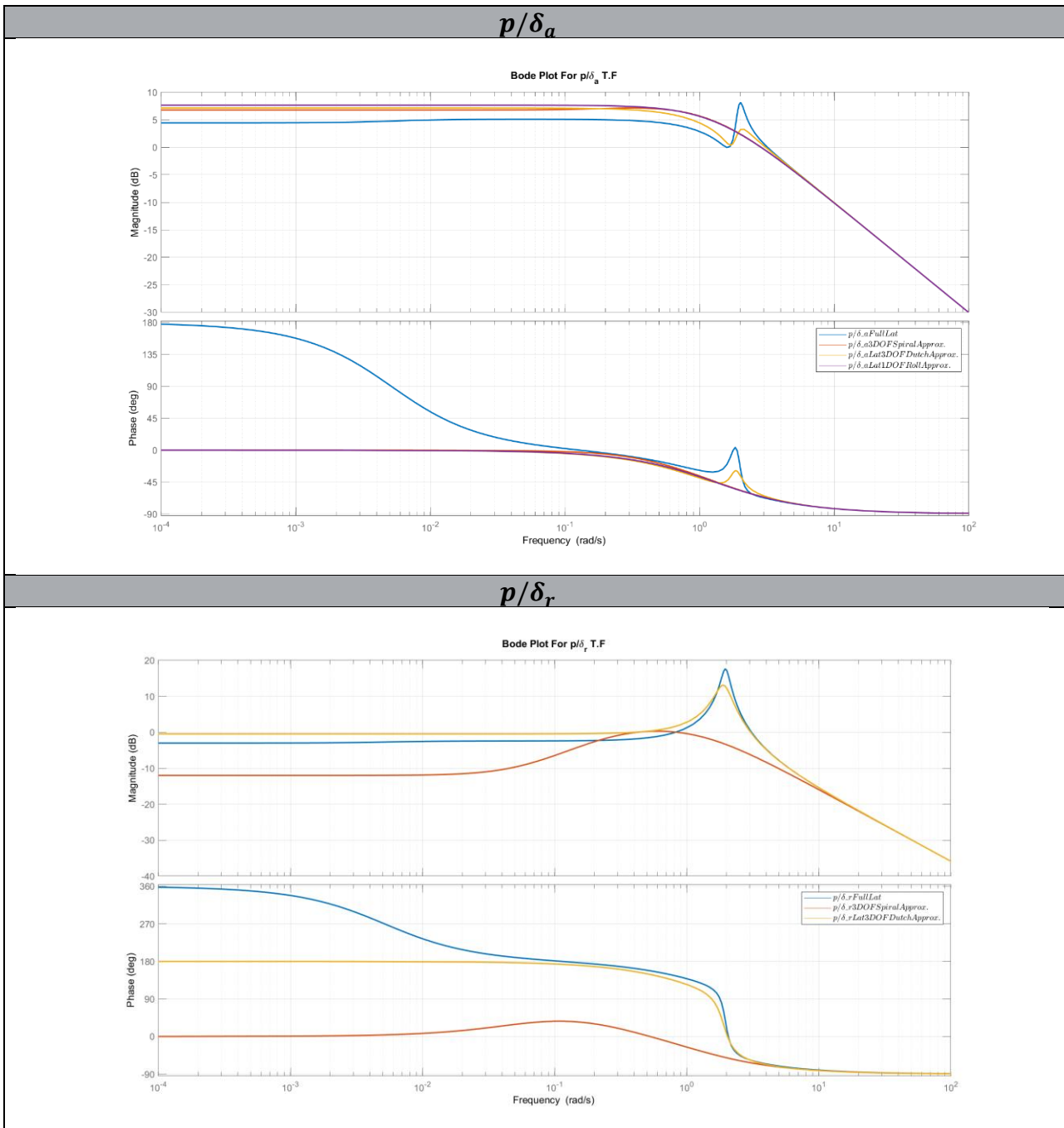
$$\delta_r = 25^\circ$$



Comments:

- The approximations are not accurate for calculation of state (p), but it is noticed that the error is decreased with the increase of angles of aileron and rudder. (except spiral mode)
- The accuracy of the linear solution depends on the type of the input which changes (aileron or rudder), the value of this change and the time.
- For aileron and rudder: the linear solution gives accurate values for small changes in the angle in range time and this range decreases with the increase of the angle.

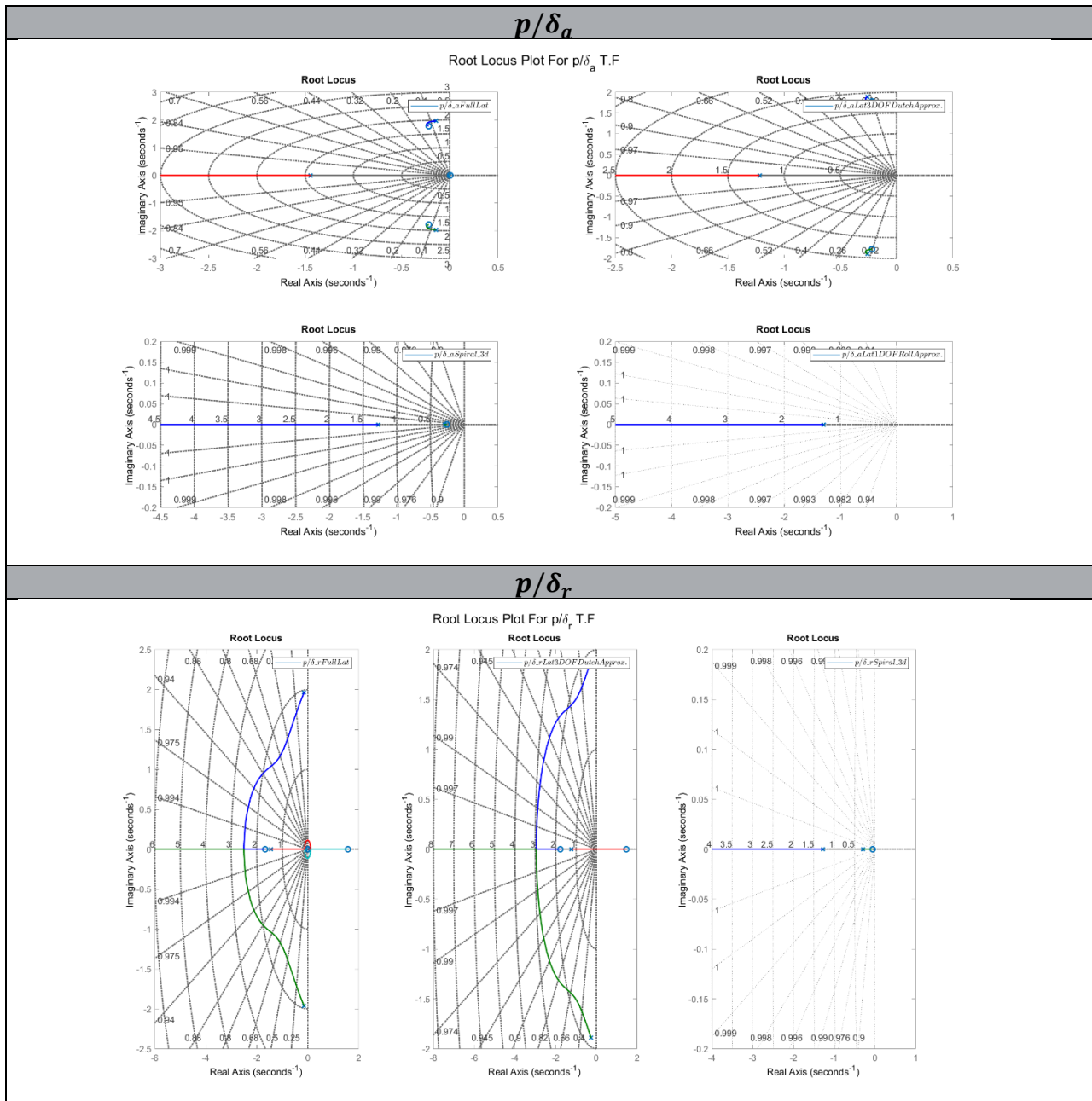
iv. Bode Plot



Comments:

- For δ_a input, The approximations are valid compared to the full linear model when the frequency is greater than .006 rad/sec.
- For δ_r input, The approximations are valid compared to the full linear model when the frequency is greater than 4 rad/sec.

v. Root Locus



Comments:

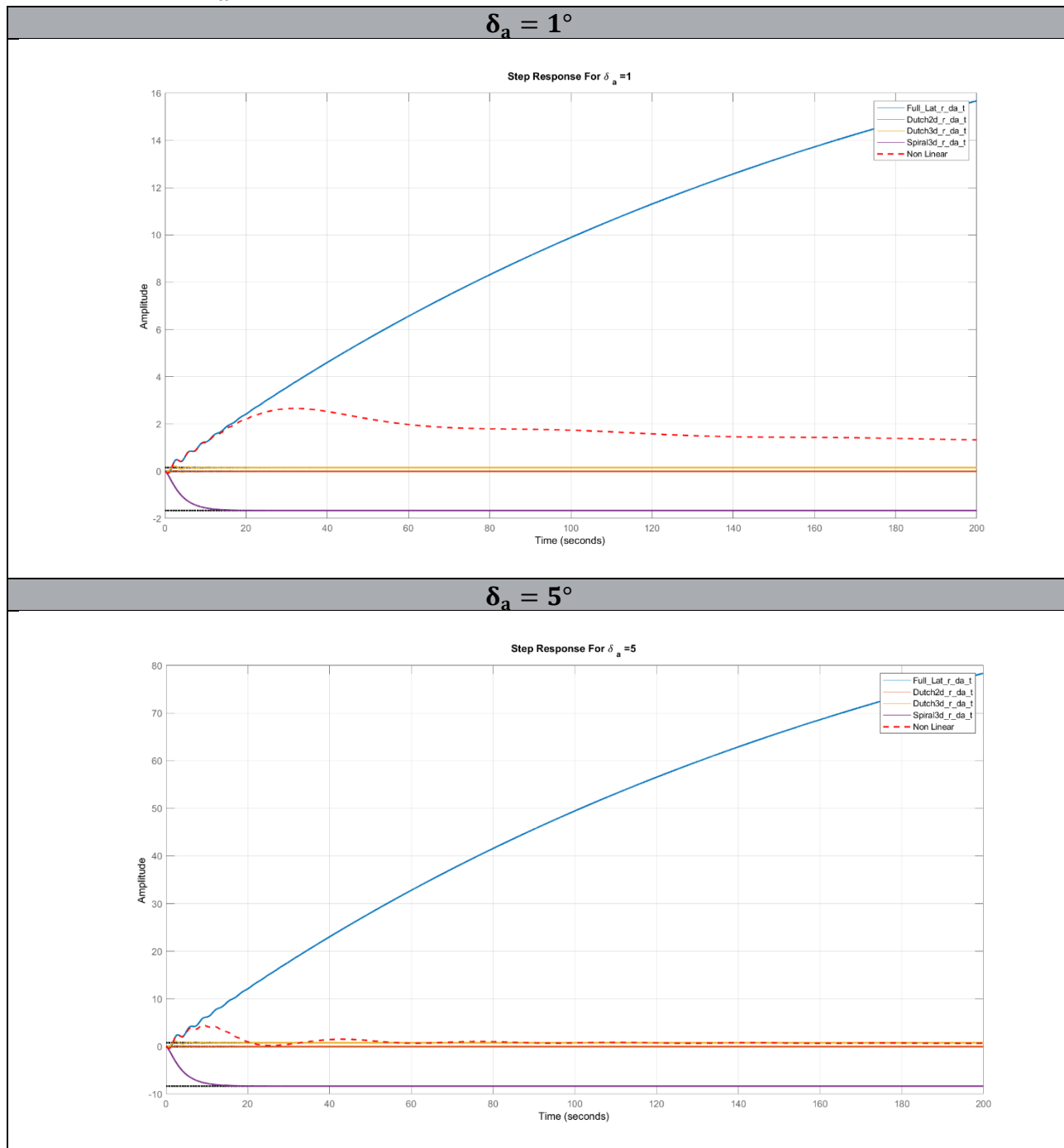
- For delta a input , all models are always stable.
- For delta r input , the full linear and the 3 DOF have instability region.

d) Response of state r

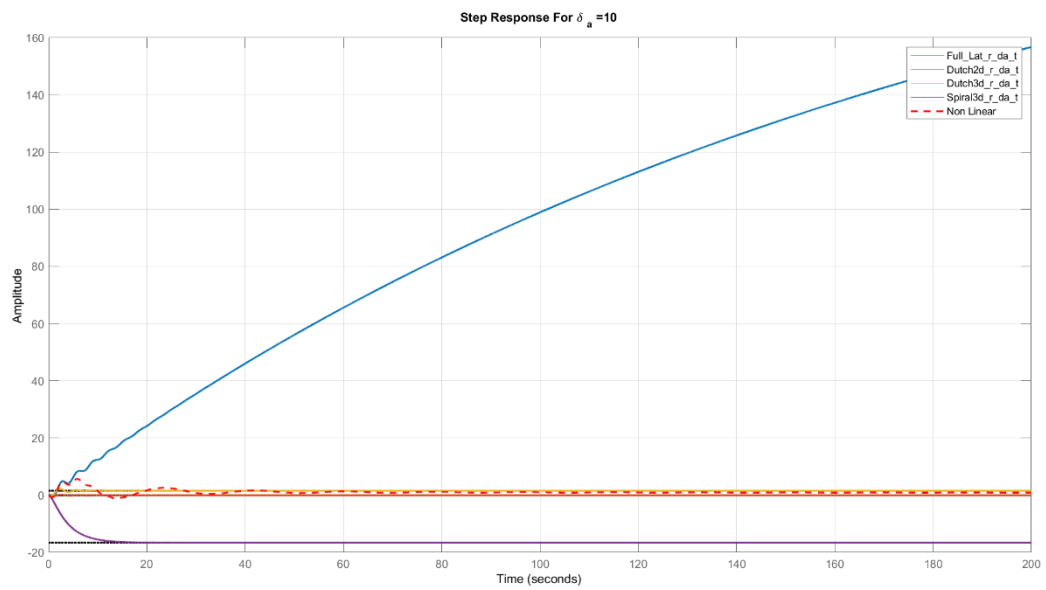
i. Transfer Functions

TF	Full Linear	Spiral 3 DOF Mode	Dutch 3 DOF Mode	Dutch 2 DOF Mode
$\frac{r}{\delta_a}$	$\frac{-0.0767 s^3 - 0.6281 s^2 + 0.5871 s + 0.715}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{-0.0767 s - 0.6147}{s^2 + 1.561 s + 0.369}$	$\frac{-0.0767 s^2 - 0.1131 s + 0.6772}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	$\frac{-0.0767 s - 0.01338}{s^2 + 0.005167 s + 0.004217}$
$\frac{r}{\delta_r}$	$\frac{-1.81 s^3 - 2.794 s^2 - 0.5342 s - 0.3034}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{-1.81 s - 2.617}{s^2 + 1.561 s + 0.369}$	$\frac{-1.81 s^2 - 2.53 s - 0.5246}{s^3 + 1.736 s^2 + 4.272 s + 4.435}$	$\frac{-1.81 s - 0.1759}{s^2 + 0.005167 s + 0.004217}$

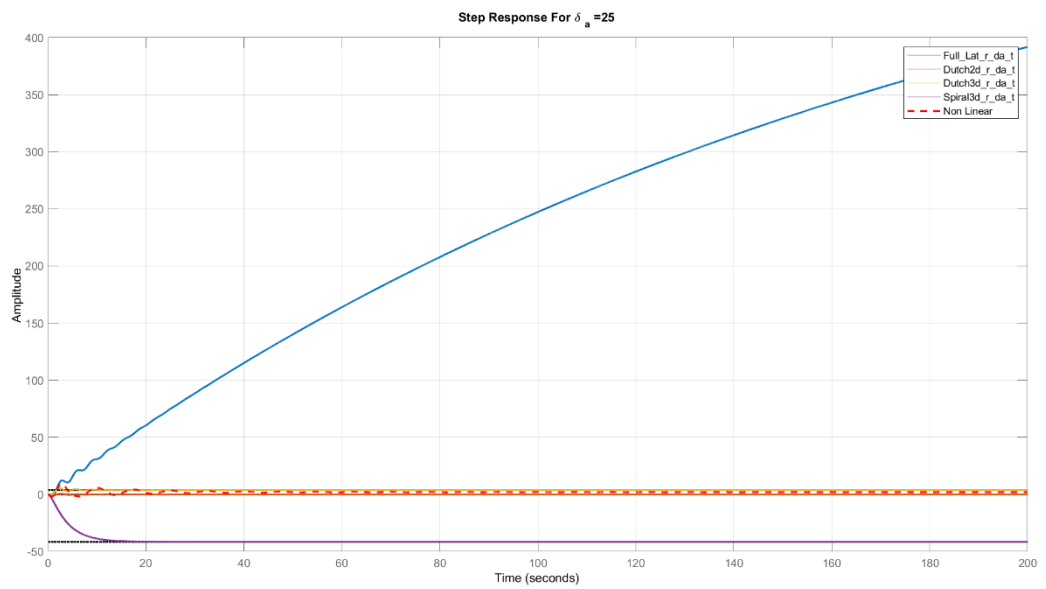
ii. Various δ_a



$\delta_a = 10^\circ$

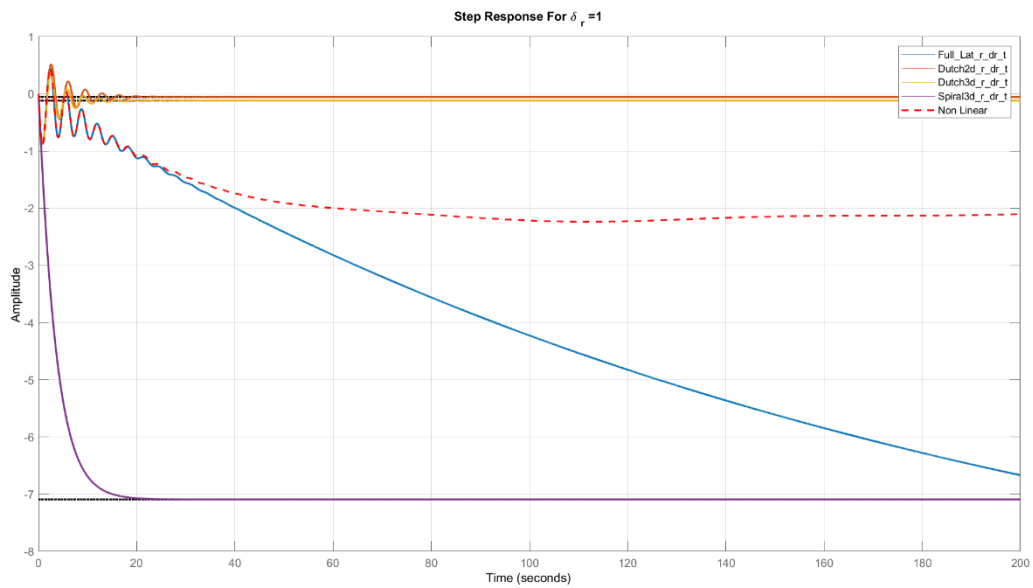


$\delta_a = 25^\circ$

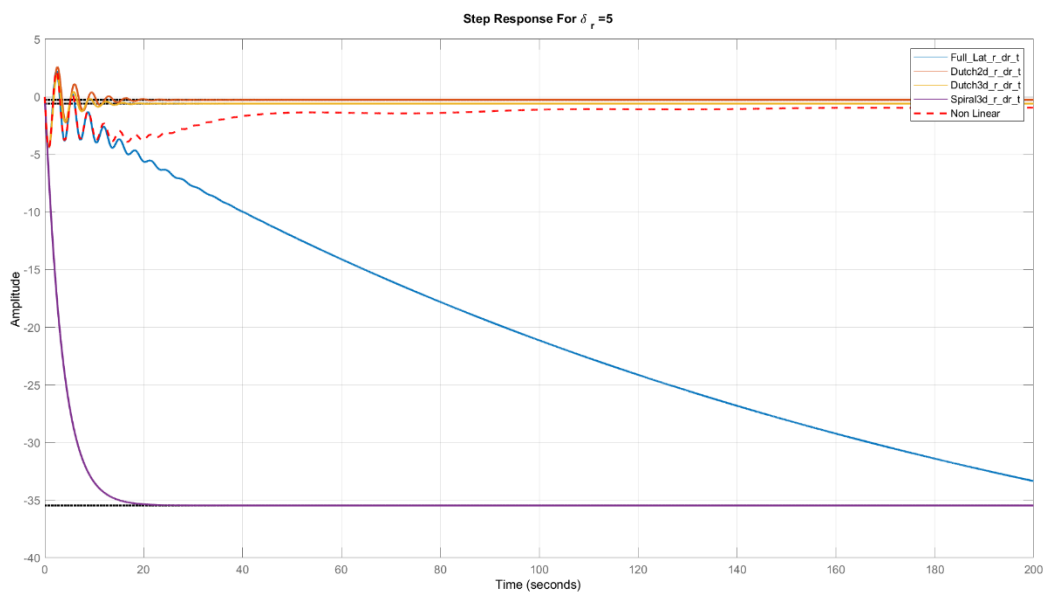


iii. Various δ_r

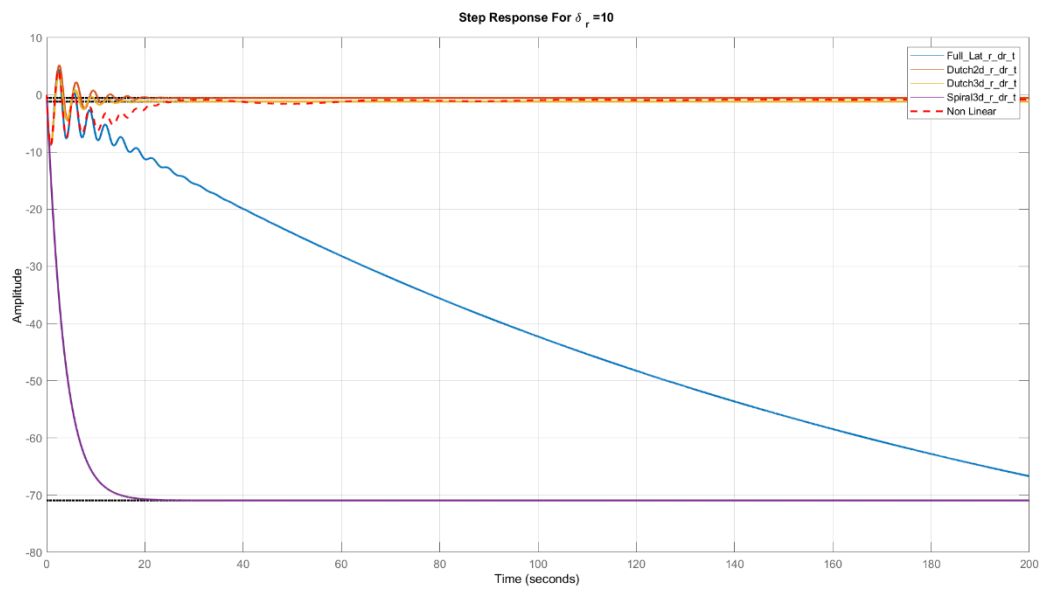
$\delta_r = 1^\circ$



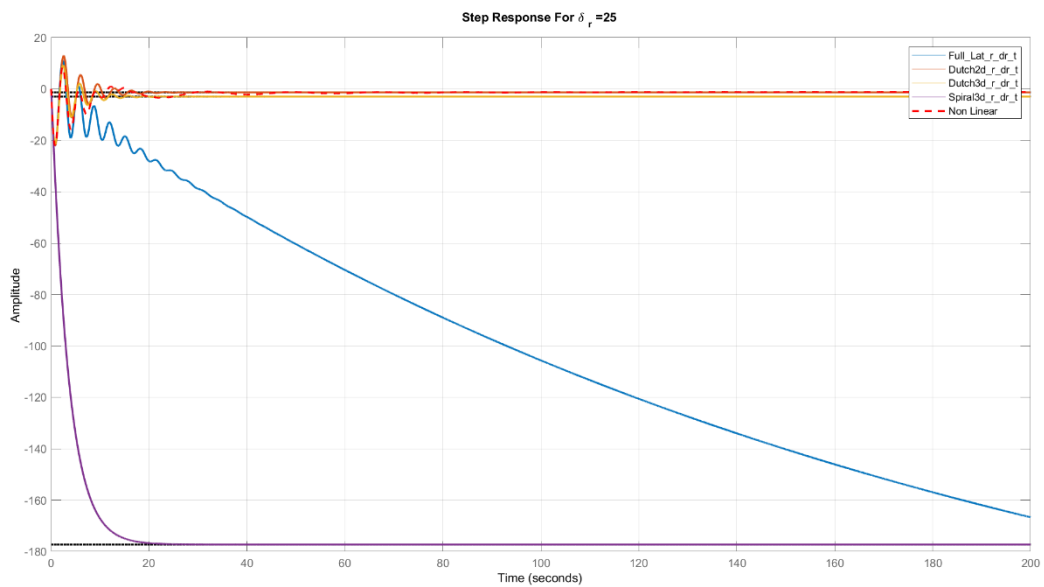
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



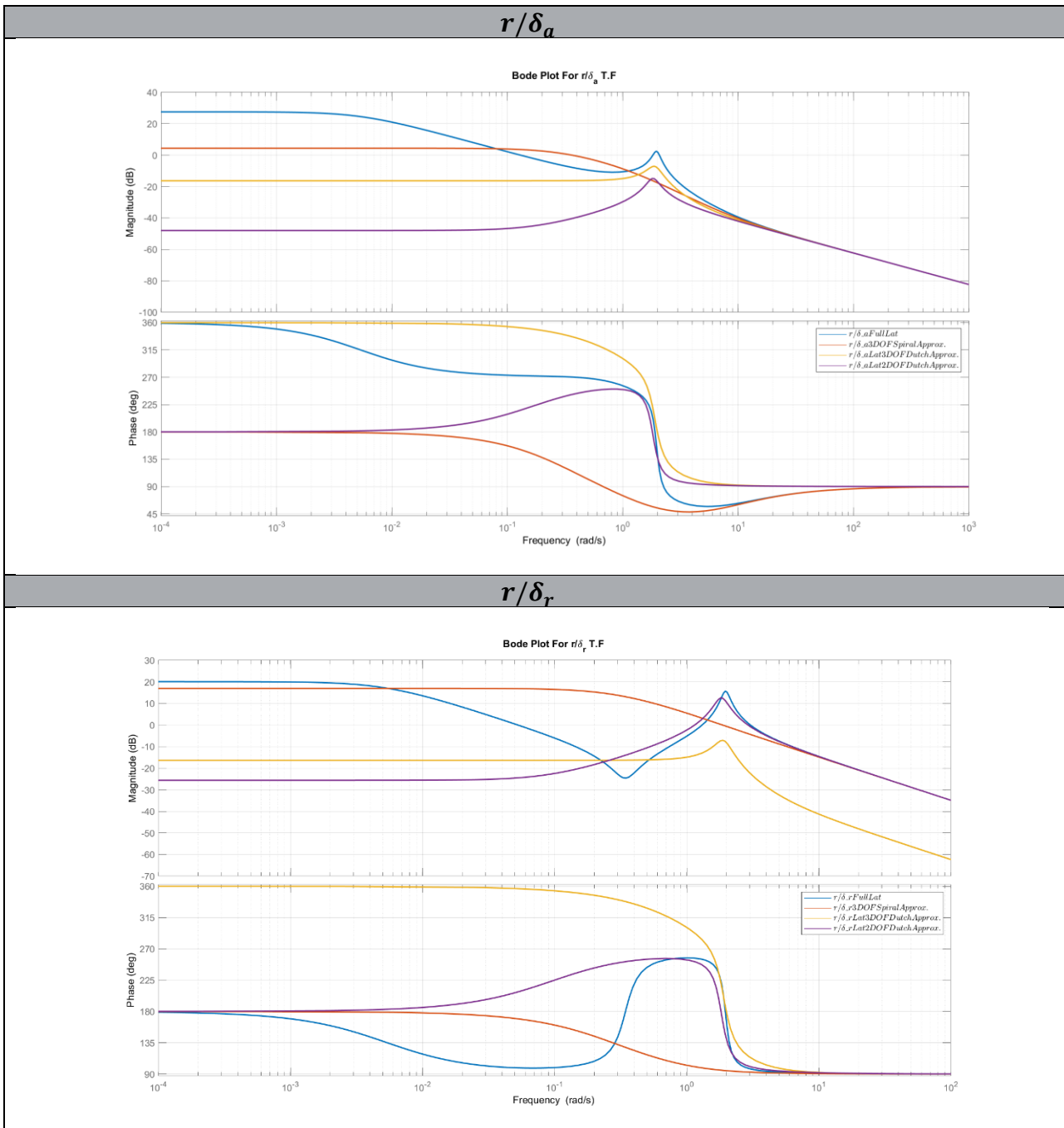
$$\delta_r = 25^\circ$$



Comments:

- Same notes as p.

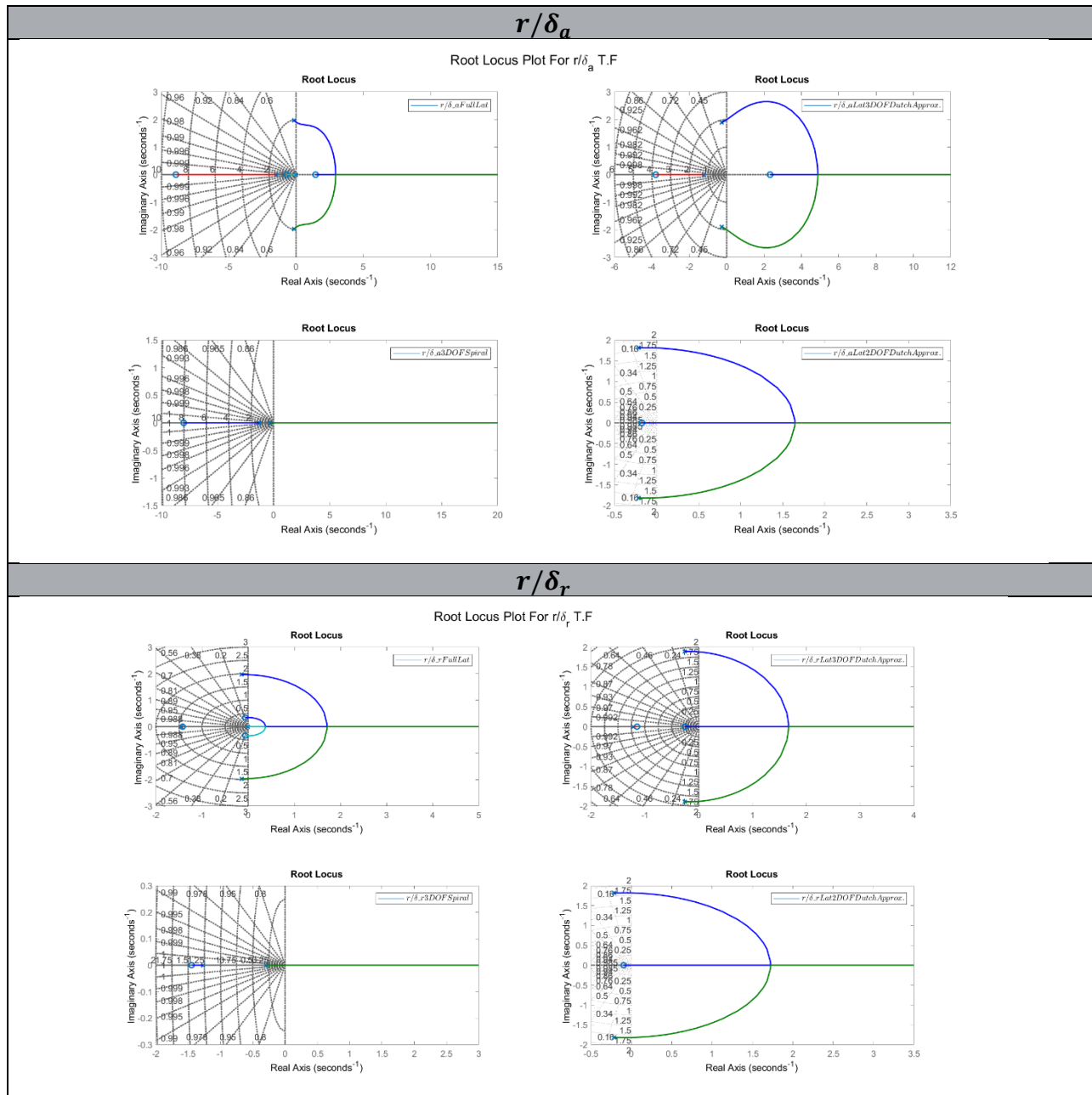
iv. Bode Plot



Comments:

- For δ_a input, The approximations are valid compared to the full linear model when the frequency is greater than 6 rad/sec.
- For δ_r input, The approximations are valid (except spiral) compared to the full linear model when the frequency is greater than 6 rad/sec.

v. Root Locus



Comments:

- All the models have instability regions.

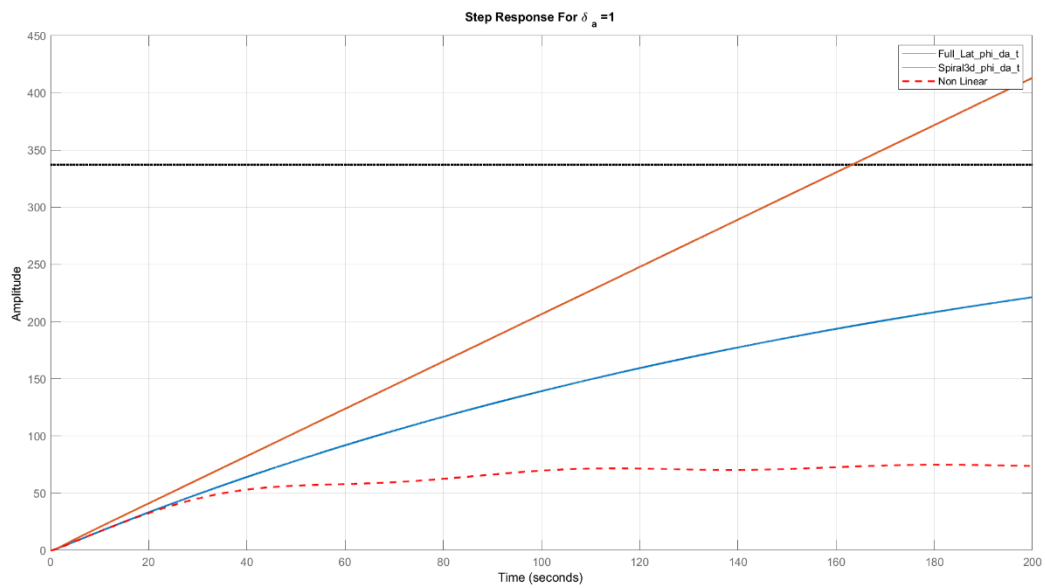
e) Response of state ϕ

i. Transfer Functions

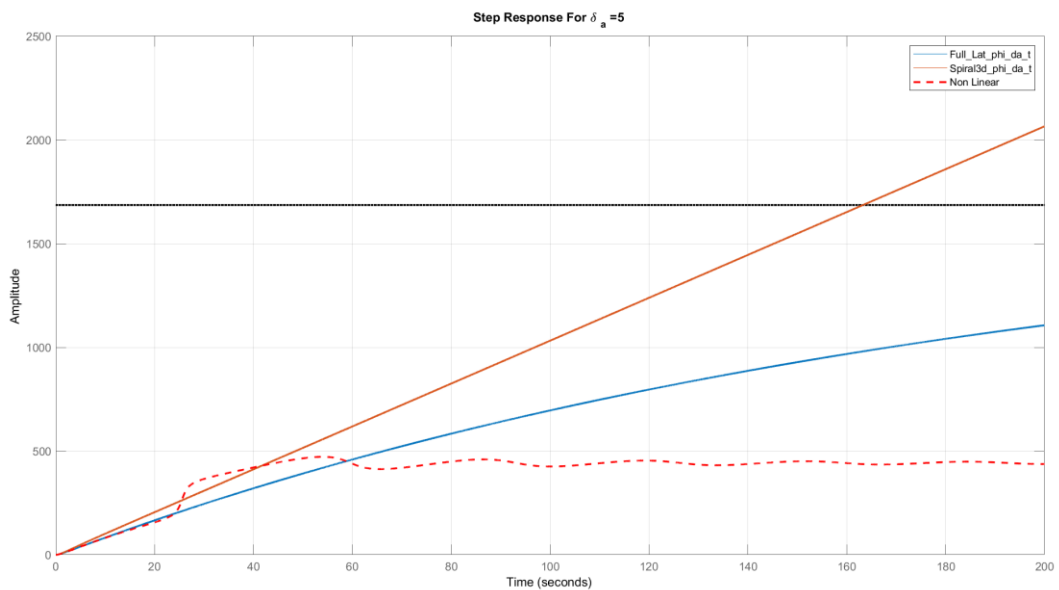
TF	Full Linear	Spiral 3 DOF Mode
ϕ/δ_a	$\frac{3.135 s^2 + 1.311 s + 10.12}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{3.135 s + 0.7627}{s^3 + 1.561 s^2 + 0.369 s}$
ϕ/δ_r	$\frac{1.483 s^2 - 0.04446 s - 4.27}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$	$\frac{1.483 s - 0.0904}{s^3 + 1.561 s^2 + 0.369 s}$

ii. Various δ_a

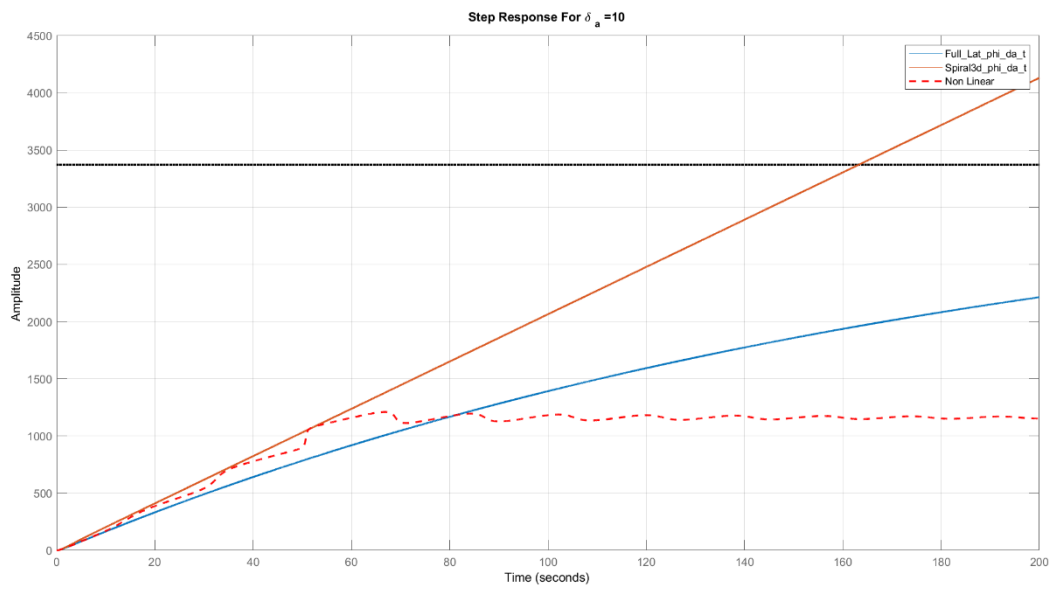
$\delta_a = 1^\circ$



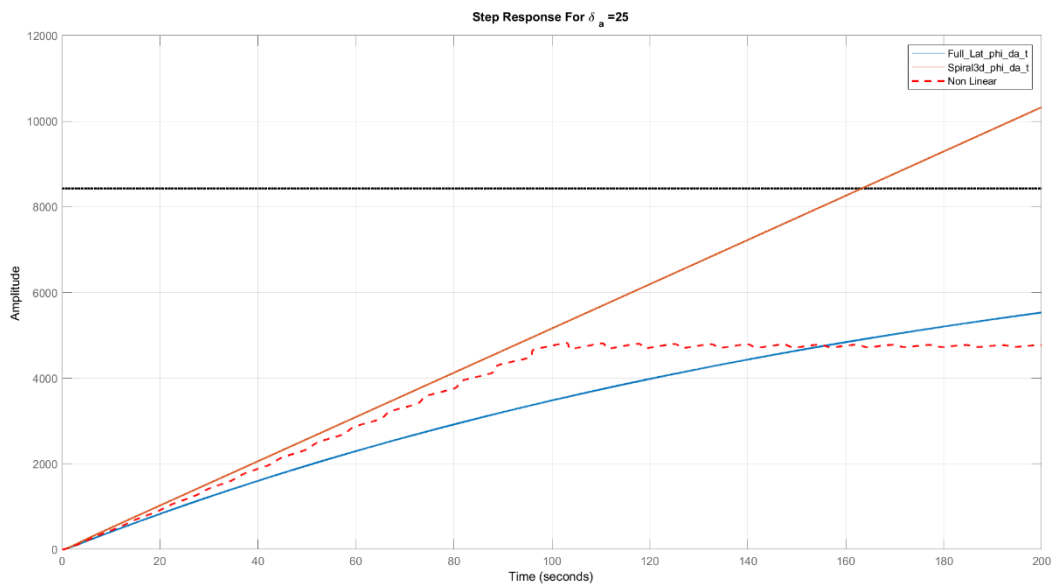
$\delta_a = 5^\circ$



$\delta_a = 10^\circ$

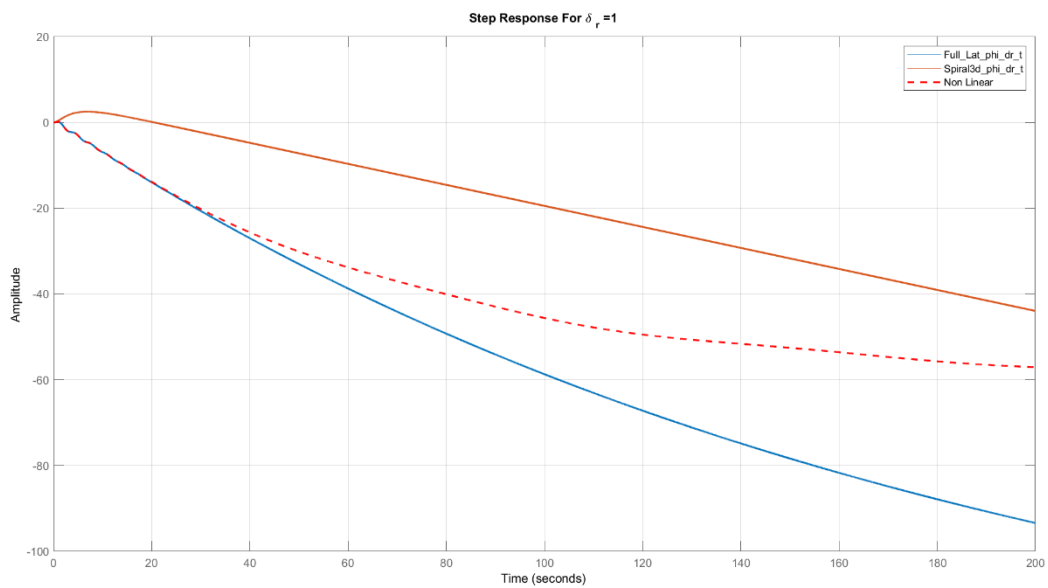


$\delta_a = 25^\circ$

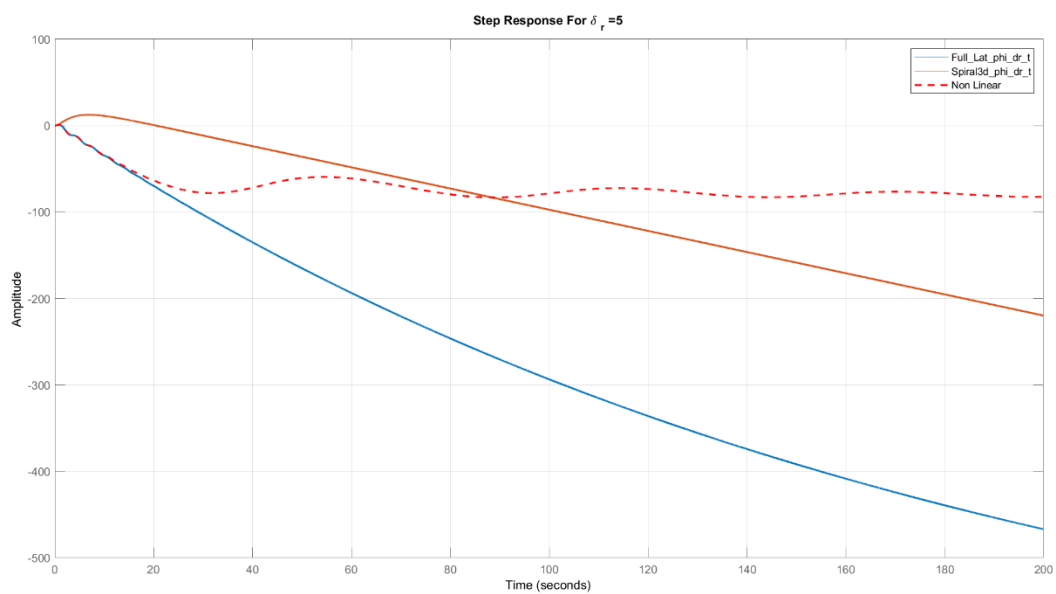


iii. Various δ_r

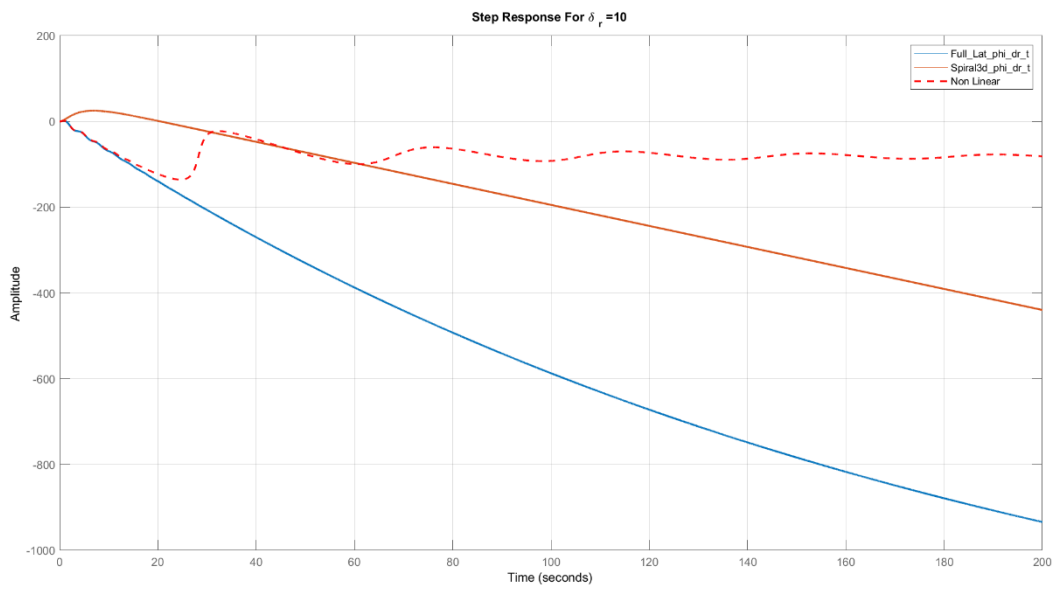
$\delta_r = 1^\circ$



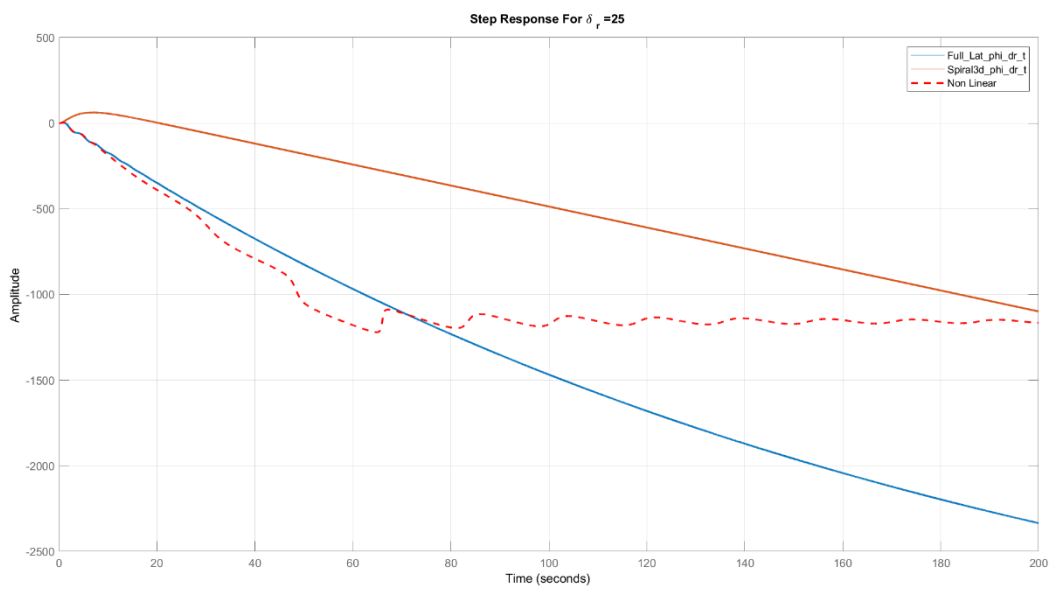
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



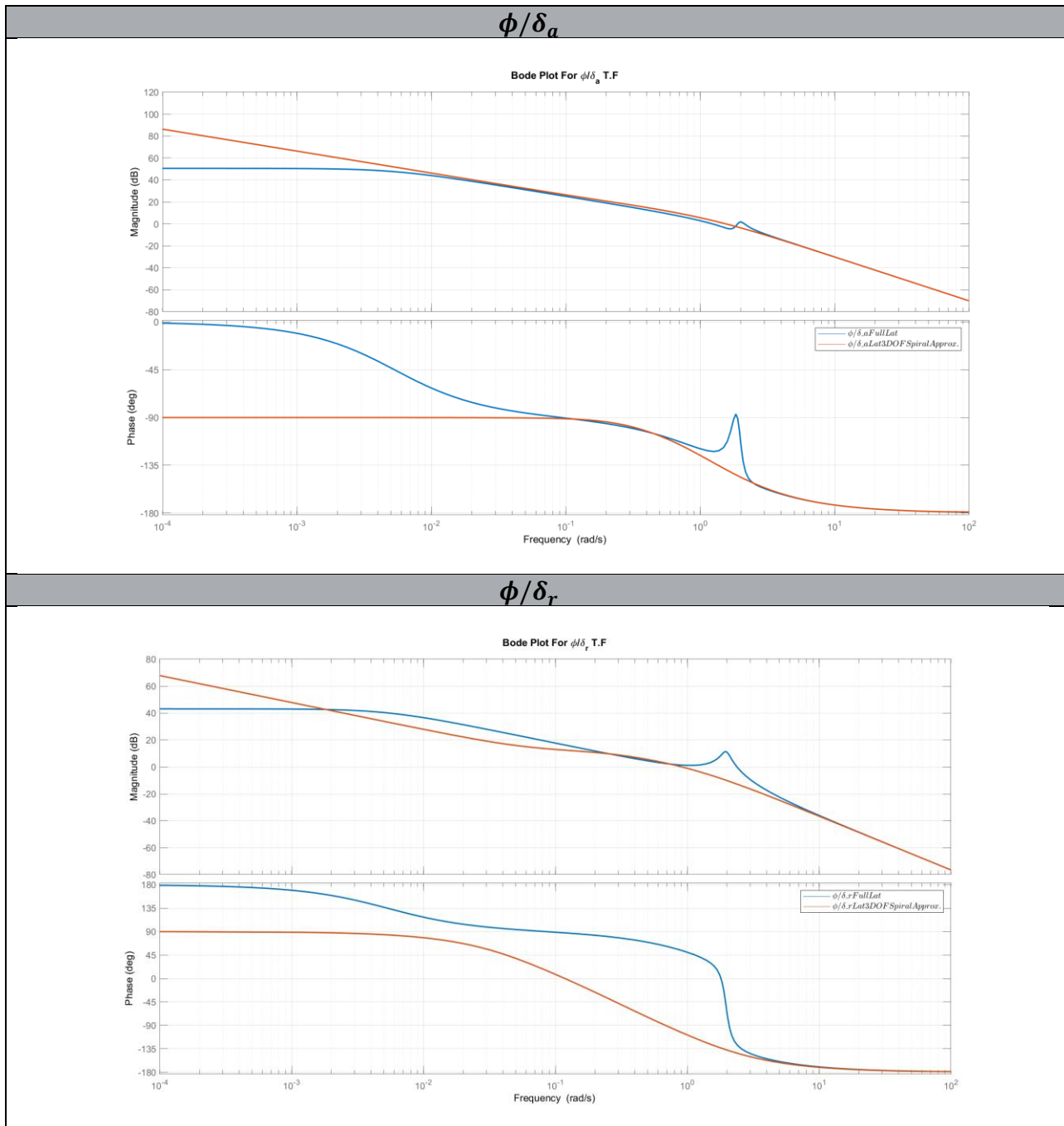
$$\delta_r = 25^\circ$$



Comments:

- Same previous states related to time response.

iv. Bode Plot

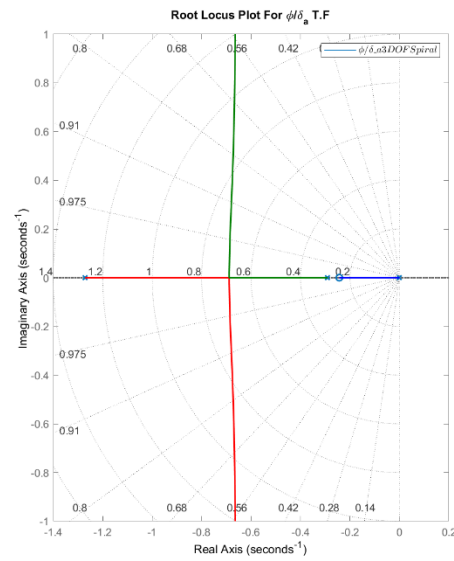
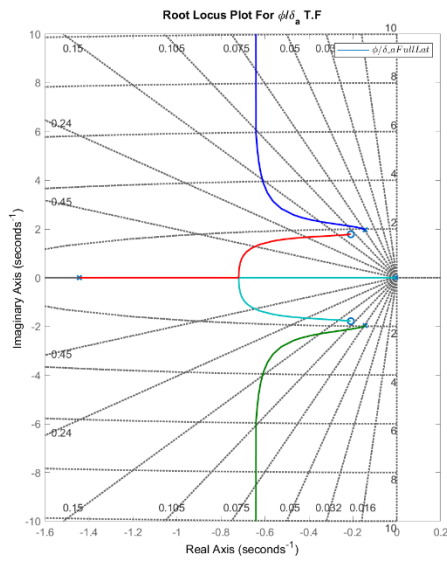


Comments:

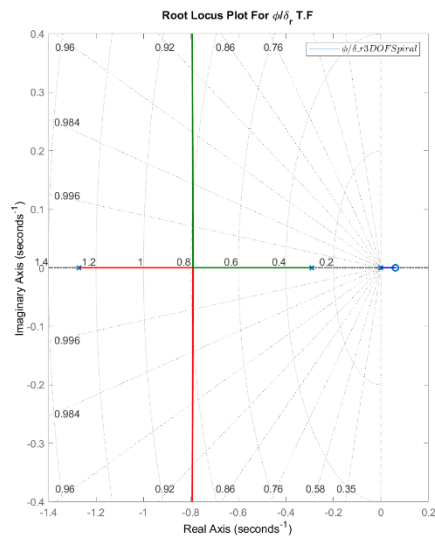
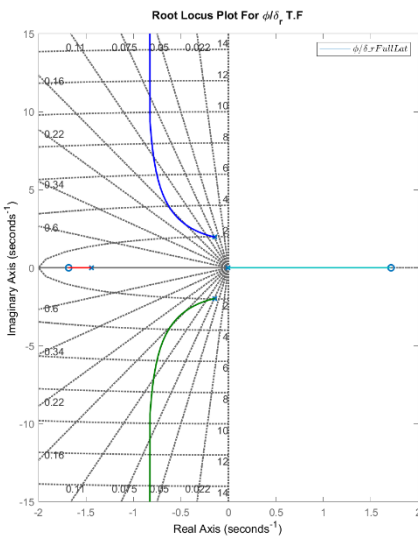
- For δ_a input, The approximation is valid compared to the full linear model when the frequency is greater than .1 rad/sec. (except small region at 2 rad/sec in phase)
- For δ_r input, The approximation is valid compared to the full linear model when the frequency is greater than 5 rad/sec.

v. Root Locus

$$\phi/\delta_a$$



$$\phi/\delta_r$$



Comments:

- For delta a input , two models are always stable.
- For delta r input , the full linear and the 3 DOF have instability region. (large in full linear)

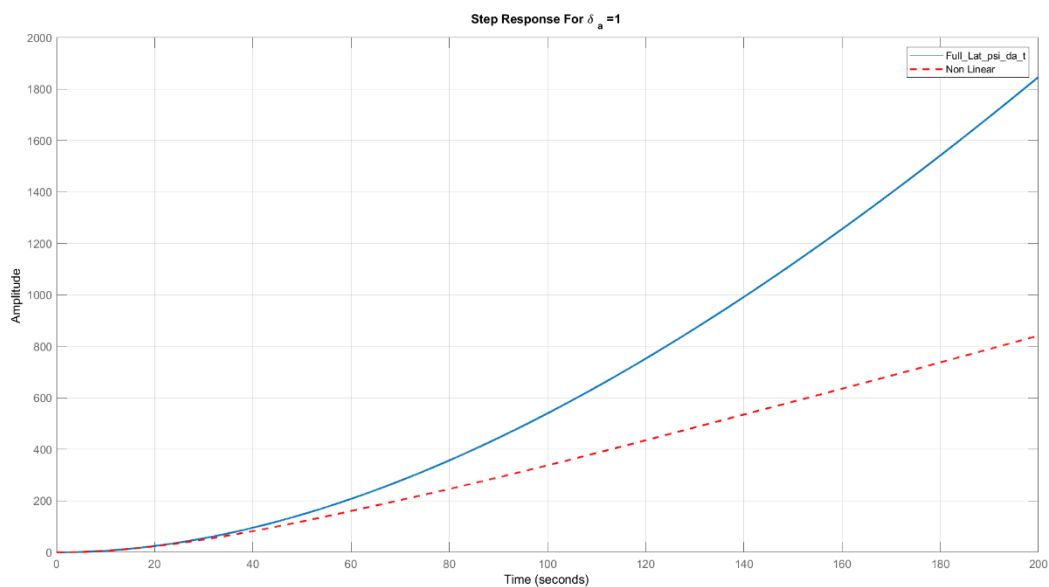
f) Response of state ψ

i. Transfer Functions

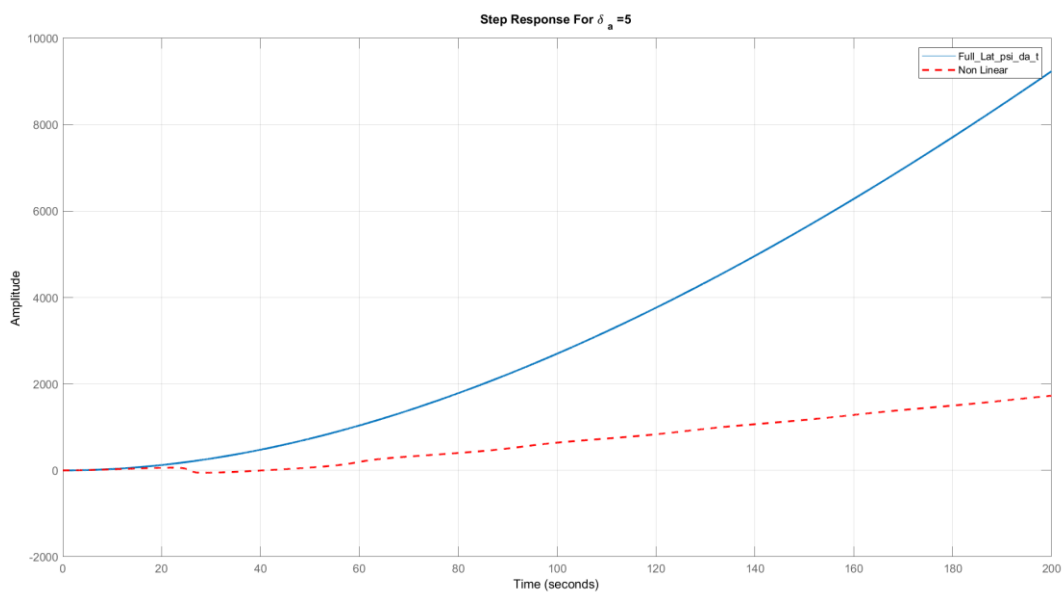
TF	Full Linear
ψ/δ_a	$\frac{-0.07689 s^3 - 0.6296 s^2 + 0.5885 s + 0.7167}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$
ψ/δ_r	$\frac{-1.814 s^3 - 2.801 s^2 - 0.5355 s - 0.3042}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$

ii. Various δ_a

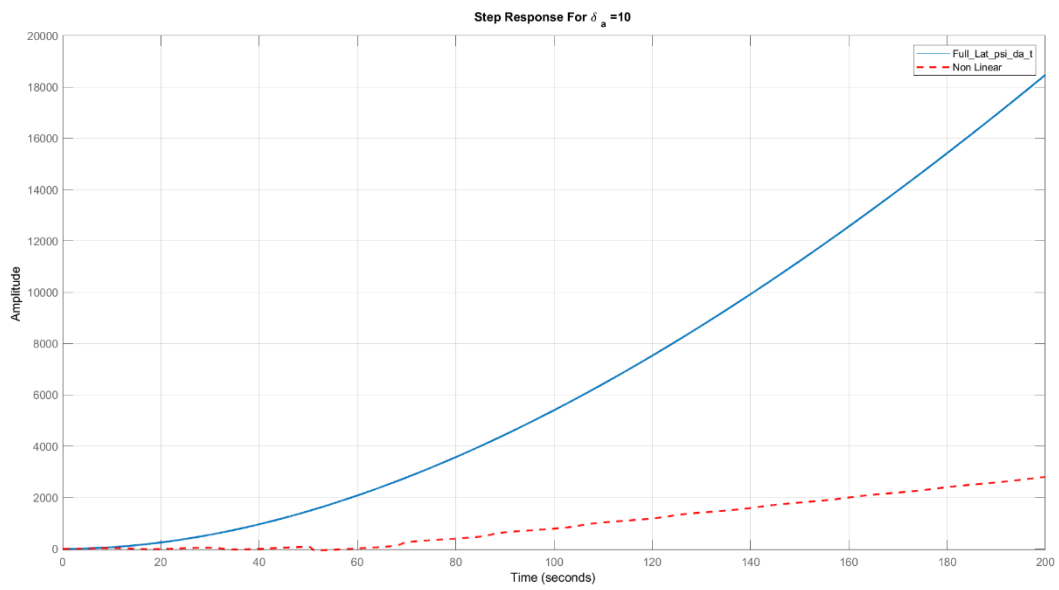
$\delta_a = 1^\circ$



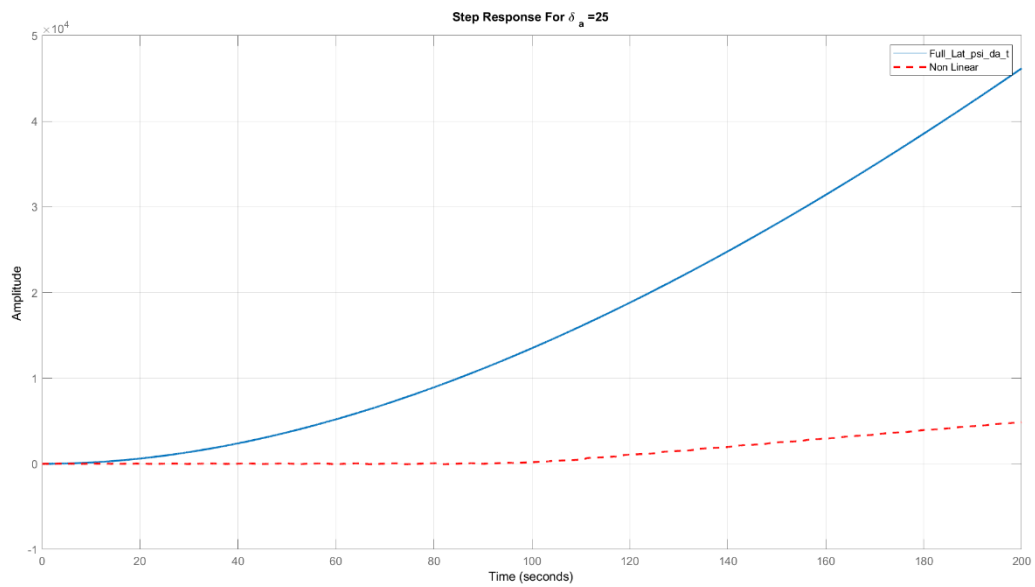
$\delta_a = 5^\circ$



$\delta_a = 10^\circ$

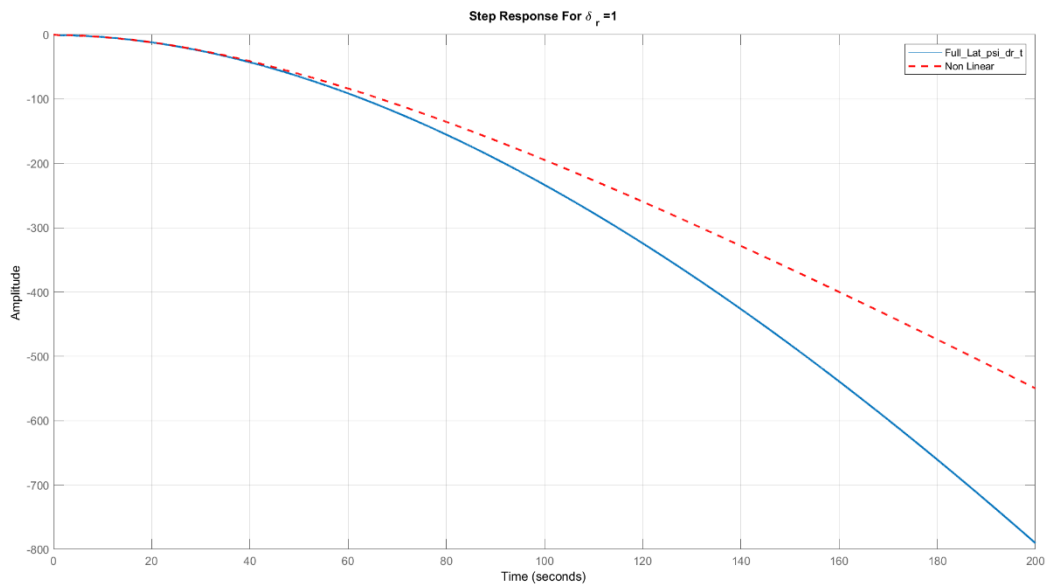


$\delta_a = 25^\circ$

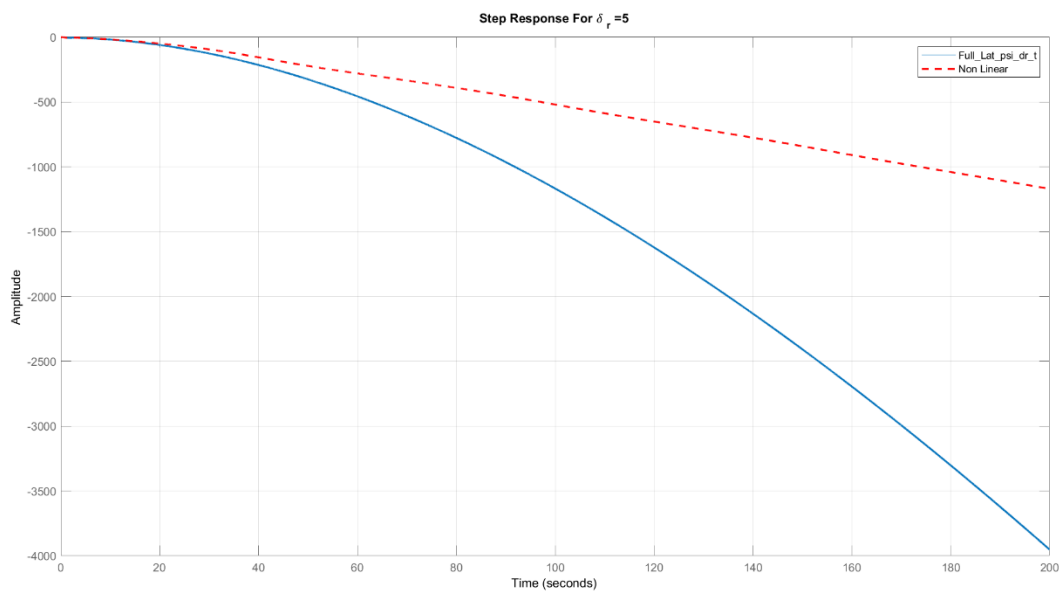


iii. Various δ_r

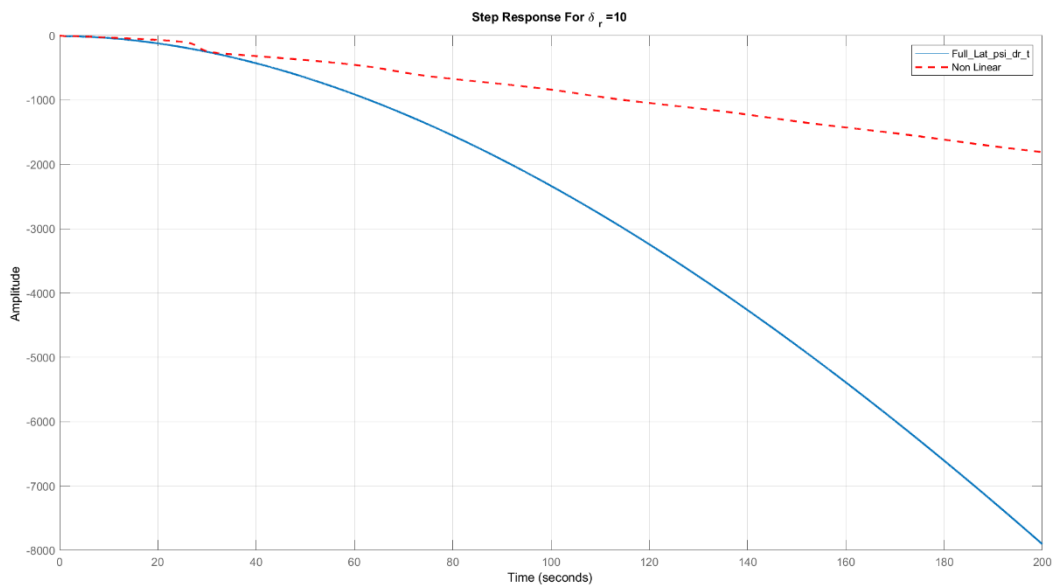
$\delta_r = 1^\circ$



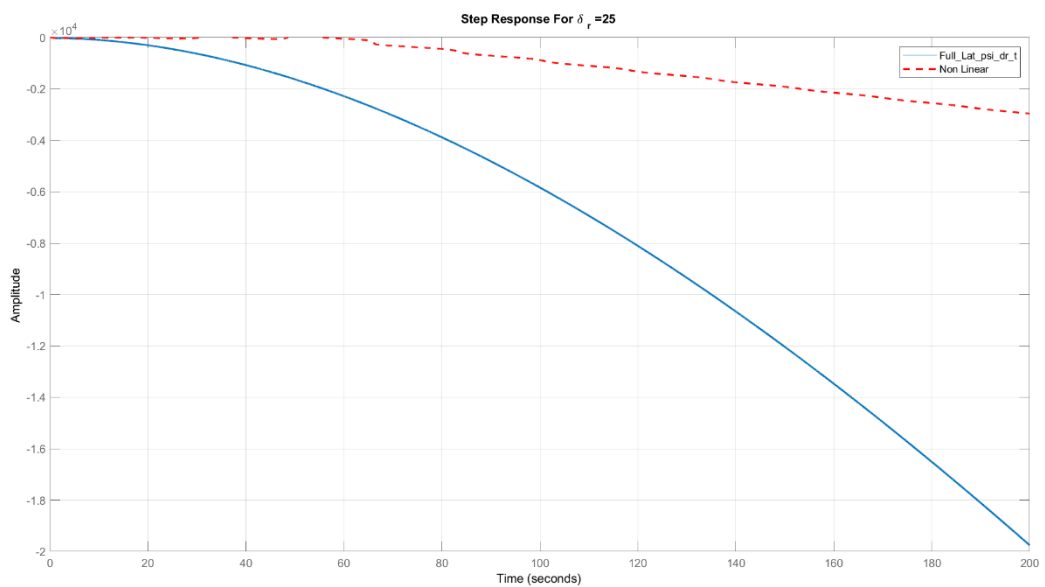
$\delta_r = 5^\circ$



$$\delta_r = 10^\circ$$



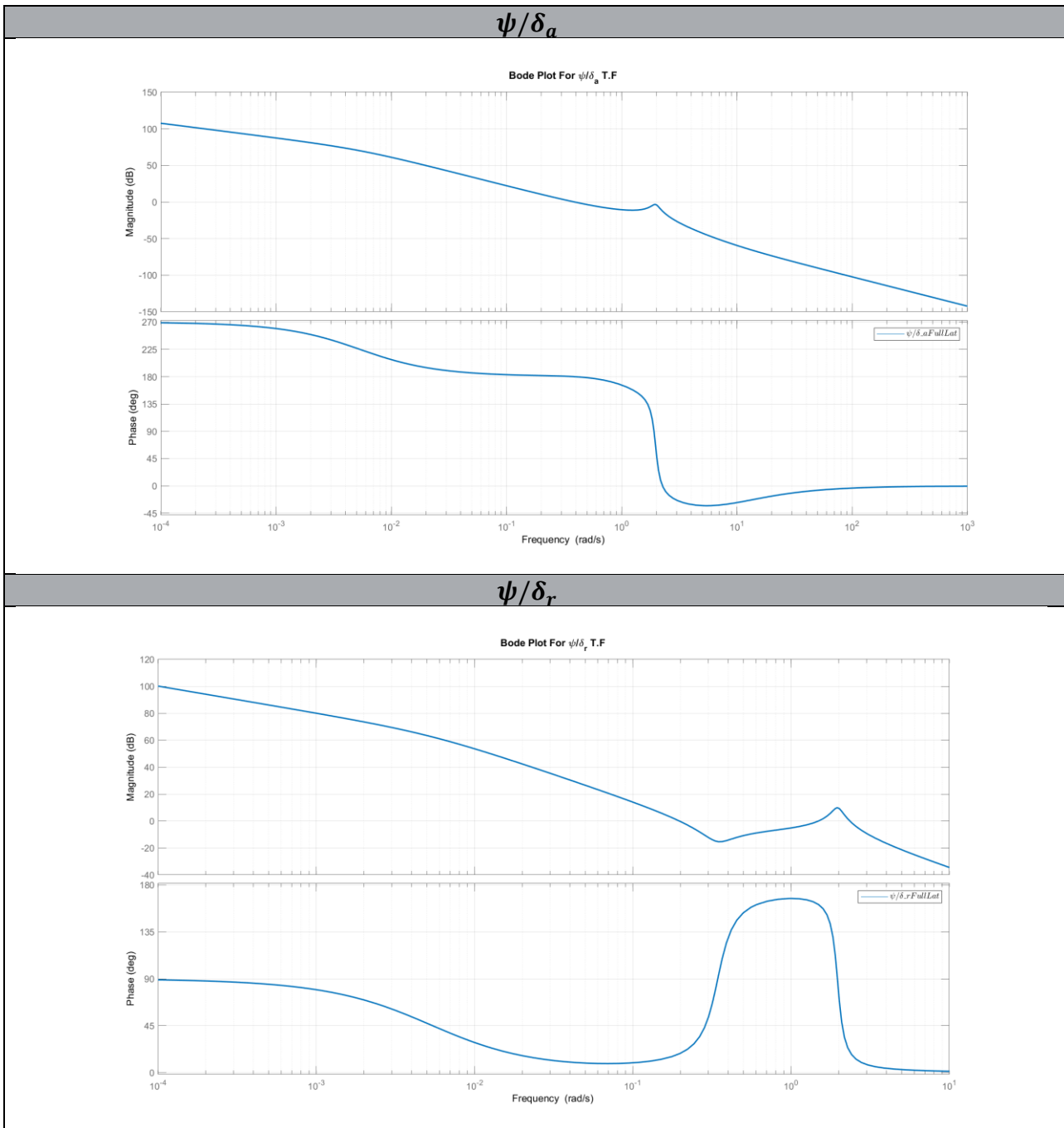
$$\delta_r = 25^\circ$$



Comments:

- The accuracy of the linear solution depends on the type of the input which changes (aileron or rudder), the value of this change and the time.
- For aileron and rudder: the linear solution gives accurate values for small changes in the angle in range time and this range decreases with the increase of the angle

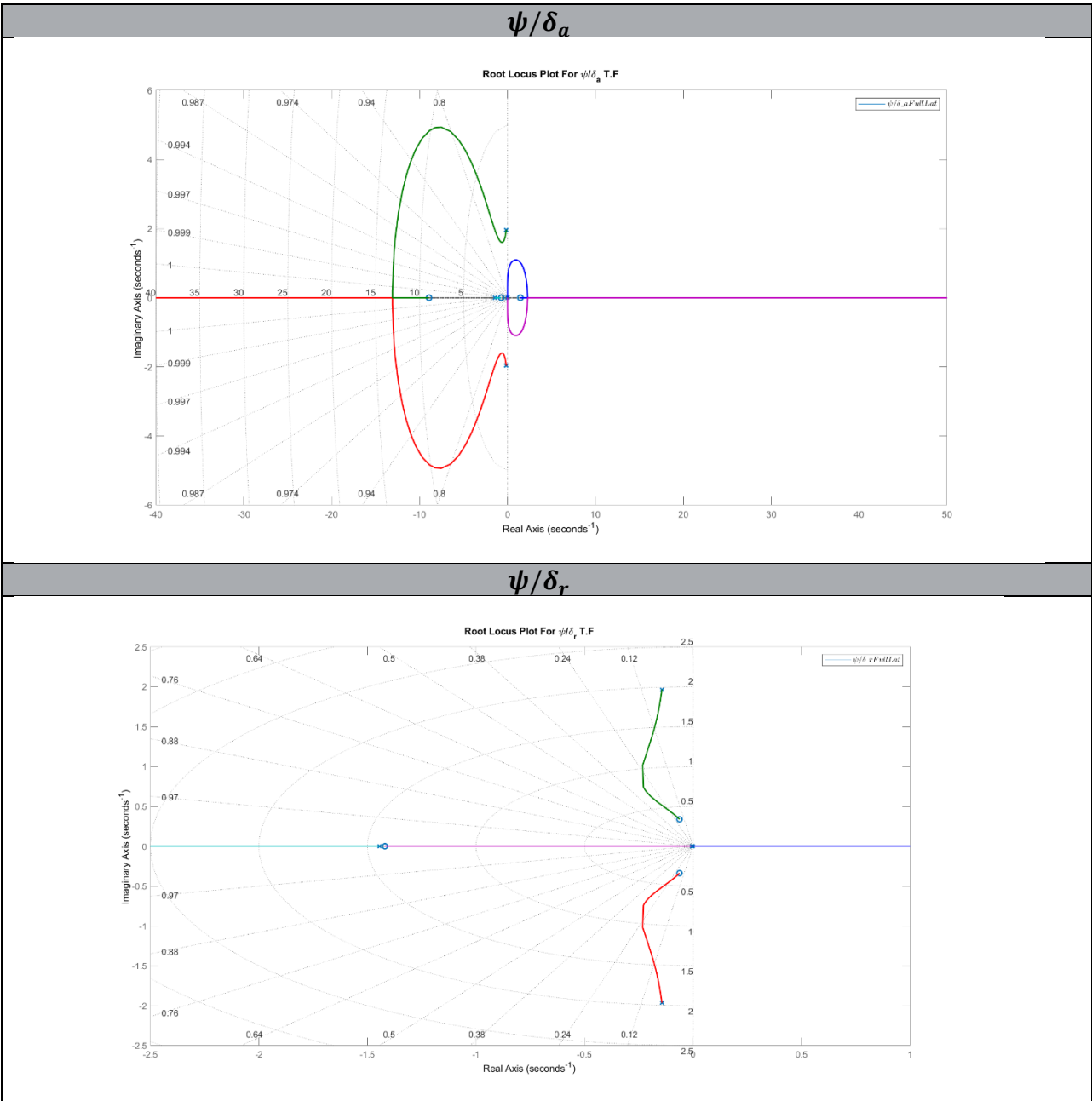
iv. Bode Plot



Comments:

- F

v. Root Locus



Simulink Bonus

The Following results are extracted from our SIMULINK model using Model Linearizer and these Transfer Functions are the same as ones obtained from Full Linear Model.

ModellinearizerTF_v_da =

$$\frac{132.1 s^2 + 399.8 s + 24.48}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_beta_da =

$$\frac{0.2955 s^2 + 0.8944 s + 0.05476}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_p_da =

$$\frac{3.14 s^3 + 1.355 s^2 + 10.07 s - 0.05}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_r_da =

$$\frac{-0.0767 s^3 - 0.6281 s^2 + 0.5871 s + 0.715}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_phi_da =

$$\frac{3.135 s^2 + 1.311 s + 10.12}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_psi_da =

$$\frac{-0.07689 s^3 - 0.6296 s^2 + 0.5885 s + 0.7167}{s^5 + 1.736 s^4 + 4.302 s^3 + 5.622 s^2 + 0.03 s}$$

Continuous-time transfer function.

ModellinearizerTF_v_dr =

$$\frac{18.95 s^3 + 886.9 s^2 + 1224 s - 2.901}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellinearizerTF_beta_dr =

$$\frac{0.0424 s^3 + 1.984 s^2 + 2.739 s - 0.006491}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellLinearizerTF_p_dr =

$$\frac{1.61 s^3 + 0.1509 s^2 - 4.232 s + 0.02122}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellLinearizerTF_r_dr =

$$\frac{-1.81 s^3 - 2.794 s^2 - 0.5342 s - 0.3034}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellLinearizerTF_phi_dr =

$$\frac{1.483 s^2 - 0.04446 s - 4.27}{s^4 + 1.736 s^3 + 4.302 s^2 + 5.622 s + 0.03}$$

Continuous-time transfer function.

ModellLinearizerTF_psi_dr =

$$\frac{-1.814 s^3 - 2.801 s^2 - 0.5355 s - 0.3042}{s^5 + 1.736 s^4 + 4.302 s^3 + 5.622 s^2 + 0.03 s}$$

Continuous-time transfer function.

Published with MATLAB® R2021b

References

- [1]: <https://en.wikipedia.org/wiki/Autopilot>
- [2]: <https://14daypilot.com/aviator-blog/read/the-history-of-autopilot>
- [3]: <https://simpleflying.com/lawrence-sperry-the-man-who-made-the-worlds-first-autopilot>
- [4]: <https://www.aircraftsystemstech.com/2017/05/autopilot-components.html>
- [5]: <https://www.cnbc.com/2015/03/26/autopilot-what-the-system-can-and-cant-do.html>
- [6]: <https://www.flypgs.com/en/travel-glossary/autopilot>
- [7]: <https://www.skybrary.aero/articles/autopilot>
- [8]: <https://www.politicalfunda.com/2022/06/stability-augmentation-systems-sas-difference>
- [9]: <https://www.sciencedirect.com/topics/engineering/stability-augmentation>
- [10]: <https://helicoptermaintenancemagazine.com/article/sas-autopilots-and-flight-directors>
- [11]: https://mitindia.edu/images/pdf/avionics_ppt/stability%20and%20agumentation.pdf
- [12]: <https://www.aircraftsystemstech.com/2017/05/autopilot-components.html>
- [13]: https://en.wikipedia.org/wiki/Inertial_navigation_system
- [14]: https://www5.epsondevice.com/en/information/technical_info/gyro/
- [15]: <https://www.livescience.com/40103-accelerometer-vs-gyroscope.html>
- [16]: https://en.wikipedia.org/wiki/Kalman_filter
- [17]: <https://researchrepository.wvu.edu/cgi/viewcontent.cgi?article=4663&context=etd>
- [18]: <https://www.intechopen.com/chapters/63164>
- [19]: https://www.cs.unc.edu/~tracker/media/pdf/SIGGRAPH2001_CoursePack_08.pdf
- [20]: <https://www.hindawi.com/journals/js/2021/9674015/>
- [21]: <https://en.wikipedia.org/wiki/Fly-by-wire>
- [22]: <https://simpleflying.com/fly-by-wire-control-system-guide/>
- [23]: <https://www.aircraftnerds.com/2019/12/fly-by-wire-flight-control-system.html>
- [24]: <https://ardupilot.org/>
- [25]: <https://px4.io/>
- [26]: Etkin B., Reid L.D. - Dynamics of flight_ Stability and control-Wiley (1996)
- [27]: Flight Stability and Automatic Control - Robert C. Nelson
- [28]: https://ccrma.stanford.edu/~jos/pasp/Body_Fixed_Space_Fixed_Frames_Reference.html
- [29]: <https://www.vectornav.com/resources/inertial-navigation-primer>
- [30]: <https://aircraftflightmechanics.com/EoMs/EulerTransforms.html>
- [31]: <https://courses.cit.cornell.edu/mae5070/DynamicEquations.pdf>
- [32]: https://en.wikipedia.org/wiki/Aircraft_flight_dynamics
- [33]: <https://academicflight.com/articles/aircraft-attitude-and-euler-angles>
- [34]: <https://gamemath.com/book/orient.html>
- [35]: <https://www.mecademic.com/en/how-is-orientation-in-space-represented-with-euler>
- [36]: <https://stevendumble.com/attitude-representations-understanding-direct-cosine>
- [37]: https://www.astro.rug.nl/software/kapteyn-beta/_downloads/attitude.pdf
- [38]: Duffy, D. G. (2016). Advanced Engineering Mathematics with MATLAB. New York: CRC Press.
- [39]: Radi, B., & Hami, A. E. (2018). Advanced Numerical Methods with Matlab. London: ISTE Ltd.
- [40]: https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods
- [41]: <https://www.mathworks.com/help/simulink>
- [42]: <https://www.mathworks.com/help/matlab/ref/rmse.html>
- [43]: <https://www.statisticshowto.com/probability-and-statistics/regression-analysis/rmse-root>
- [44]: <https://stephenallwright.com/good-rmse-value>

Appendix

I. Task 1: Euler, Modified Euler and Runge-Kutta results comparison

%This code is used to compare the results of Euler, Modified Euler and RK4 methods and compare their results to the exact solution.

```
clc;clearvars; close all
```

```
%set up time step increment and number of time steps  
H=[0.1 0.01];
```

```
for i = 1:length(H)  
h = H(i);  
n = 10/h;
```

```
% set up initial conditions  
t=zeros(n+1,1);  
x_euler=zeros(n+1,1); x_euler(1) = 1;  
x_modified=zeros(n+1,1); x_modified(1) = 1;  
x_rk=zeros(n+1,1); x_rk(1) = 1;  
x_exact=zeros(n+1,1); x_exact(1) = 1;
```

```
% set up difference arrays for plotting purposes  
diff1 = zeros(n,1); diff2 = zeros(n,1);  
diff = zeros(n,1); tplot = zeros(n,1);
```

```
% define right side of differential equation  
f=@(t,x) x+t;
```

```
for k = 1:n  
t(k+1) = t(k) + h;
```

```
% compute exact solution  
x_exact(k+1) = 2*exp(t(k+1)) - t(k+1) - 1;
```

```
% compute solution via Euler's method  
k1 = h * f(t(k),x_euler(k));  
x_euler(k+1) = x_euler(k) + k1;  
diff1(k) = x_euler(k+1) - x_exact(k+1);  
diff1(k) = abs(diff1(k) / x_exact(k+1));
```

```
% compute solution via modified Euler method  
k1 = h * f(t(k),x_modified(k));  
k2 = h * f(t(k+1),x_modified(k)+k1);  
x_modified(k+1) = x_modified(k) + 0.5*(k1+k2);  
diff2(k) = x_modified(k+1) - x_exact(k+1);  
diff2(k) = abs(diff2(k) / x_exact(k+1));
```

```
% compute solution via RK4 method  
x_local = x_rk(k);  
t_local = t(k);  
k1 = h * f(t_local,x_local);
```



```

k2 = h * f(t_local + h/2,x_local + k1/2);
k3 = h * f(t_local + h/2,x_local + k2/2);
k4 = h * f(t_local + h,x_local + k3);
t(k+1) = t_local + h;
x_rk(k+1) = x_local + (k1+2*k2+2*k3+k4) / 6;
diff(k) = x_rk(k+1) - x_exact(k+1);
diff(k) = abs(diff(k) / x_exact(k+1));
tplot(k) = t(k);

end

% plot relative errors
semilogy(tplot,diff1,'-',tplot,diff2,'--',tplot,diff,':','Linewidth',3)
hold on
xlabel('TIME','FontSize',20)
ylabel('|RELATIVE ERROR|','FontSize',20)
legend('Euler method','modified Euler method','RK4 method')
legend boxoff;

text(2*i,diff1(0.2*n),['h =',num2str(h)],'FontSize',15,'HorizontalAlignment','right','VerticalAlignment','bottom')
text(3*i,diff2(0.3*n),['h =',num2str(h)],'FontSize',15,'HorizontalAlignment','right','VerticalAlignment','bottom')
text(4*i,diff(0.4*n),['h =',num2str(h)],'FontSize',15,'HorizontalAlignment','right','VerticalAlignment','bottom')

end

```

II. Task 1: ODE problem solution using Runge-Kutta method

```
%This code is used to solve task 1 ODE problem usin Runge-Kutta method.
clc; clearvars; close all;

%% time vector:
tmin=0;
tmax=20;
intervals=100; % number of intervals
h = (tmax-tmin)/intervals;
t = tmin:h:tmax;
n=length(t);

%% Problem
y(:,1) = [-1;1];
F = @(t,y) [sin(t)+cos(y(1))+sin(y(2)) ; cos(t)+sin(y(2))];

%% RK4 Solver Algorithm
for i = 2:n
    k1 = h * F(t(i-1) , y(:,i-1));
    k2 = h * F(t(i-1) + h/2 , y(:,i-1) + k1/2);
    k3 = h * F(t(i-1) + h/2 , y(:,i-1) + k2/2);
    k4 = h * F(t(i-1) + h , y(:,i-1) + k3);

    y(:,i) = y(:,i-1) +1/6 * (k1 + 2*k2 + 2*k3 + k4);

end

%% Solve using matlab function ODE45
[t_matlab,y_matlab] = ode45(F,t,y(:,1));
error = abs(y' - y_matlab)./ y_matlab;

figure(1)
plot(t,y(1,:), '- ',t_matlab,y_matlab(:,1), '--', 'Linewidth',3)
xlabel('TIME','FontSize',20)
ylabel('y1','FontSize',20)
legend('RK4','ODE45')
legend boxoff;

figure(2)
plot(t,y(2,:), '- ',t_matlab,y_matlab(:,2), '--', 'Linewidth',3)
xlabel('TIME','FontSize',20)
ylabel('y2','FontSize',20)
legend('RK4','ODE45')
legend boxoff;
```

III. Task 2: RBD Equations Solver

```
%This code is used to solve airplane ODEs using Runge-Kutta method.
clc; clearvars; close all;
%% time vector:
tmin=0;
tmax=25;
% intervals=15000; % number of intervals
% h = (tmax-tmin)/intervals;
h=0.001;
t = tmin:h:tmax;
n=length(t);
%% Givens
m = 30;
g = 9.81;
F = [10;5;3];
M = [10;15;20];
I = [1 -2 -1;-2 5 -4;-1 -4 0.2];
x0 = [10,2,0, 2*pi/180,1*pi/180,0, 20*pi/180,15*pi/180,30*pi/180,
2,4,7];
y(:,1) = x0;
%% RK4 Solver Algorithm
for i = 2:n
    k1 = h*Airplane_RBD(y(:,i-1),F,M,m,I);
    k2 = h*Airplane_RBD(y(:,i-1) + k1/2,F,M,m,I);
    k3 = h*Airplane_RBD(y(:,i-1) + k2/2,F,M,m,I);
    k4 = h*Airplane_RBD(y(:,i-1) + k3,F,M,m,I);
    y(:,i) = y(:,i-1) +1/6 * (k1 + 2*k2 + 2*k3 + k4);
end
%% Simulink
SimulinkOutput=sim('Simulator_Part_1');
t_sim=SimulinkOutput.tout;
y_sim =
[SimulinkOutput.Velocity,SimulinkOutput.AngularRates,SimulinkOutput.EulerAngles,SimulinkOutput.Position];
%% Plotting:
StatesTitles = {'u','v','w','p','q','r','\phi','\theta','\psi','x','y','z'};
FiguresTitles = {};
for m=1:length(x0)
    figure(m)
    plot(t,y(m,1:n),'r-',t_sim,y_sim(:,m),'k--','linewidth',1)
    xlabel('$t$', 'interpreter','latex','fontsize',14)
    ylabel(['$',StatesTitles{m},'$'], 'interpreter','latex','fontsize',14)
    legend({'Runge-
Kutta','Simulink'}, 'interpreter','latex','location','northwest','fontsize',10)
end
```

IV. Task 2: Function: Airplane Rigid Body Dynamics

```
function Rates = Airplane_RBD(x,F,M,m,I)

    lin_vel = x(1:3);
    ang_vel = x(4:6);
    ang = x(7:9);
    disp = x(10:12);

    %% RBD equations

    %Kinetic
    lin_vel_dot = (F / m) - cross(ang_vel,lin_vel) ;
    ang_vel_dot = I^(-1)*( M - cross(ang_vel,I*ang_vel) );

    %Kinematic
    phi = ang(1);
    theta = ang(2);
    psi = ang(3);

    J = [1,sin(phi)*tan(theta),cos(phi)*tan(theta);...
          0,cos(phi),-sin(phi);...
          0,sin(phi)/cos(theta),cos(phi)/cos(theta)];

    ang_dot= J*ang_vel;

    Euler_Trans = [cos(theta)*cos(psi),...
                    sin(phi)*sin(theta)*cos(psi)-cos(phi)*sin(psi),...
                    cos(phi)*sin(theta)*cos(psi)+sin(phi)*sin(psi);...
                    cos(theta)*sin(psi),...
                    sin(phi)*sin(theta)*sin(psi)+cos(phi)*cos(psi),...
                    cos(phi)*sin(theta)*sin(psi)-sin(phi)*cos(psi);...
                    -sin(theta),sin(phi)*cos(theta),cos(phi)*cos(theta)];

    disp_dot= Euler_Trans*lin_vel;

    Rates = [lin_vel_dot;ang_vel_dot;ang_dot;disp_dot];

end
```

V. Task 3: Airplane's Non-Linear Simulator

```

%% clear variables and close plots
clc;clearvars;close all;
%% Read the derivatives and reference conditions

filename_density_L = 'JETSTAR_FC3.xlsx'; %%put here the location of your excel
sheet

aircraft_data=xlsread(filename_density_L,'B2:B61');%% here B2:B61 means read the
excel sheet from cell B2 to cell B61

%%in the next step we will read from the vector(aircraft_data) but take care of
the order the values in excel sheet is arranged

% Time vector parameters
dt = aircraft_data(1);    tfinal = aircraft_data(2); lengths=tfinal/dt+1;
time_V = (0:dt:tfinal)';

% initial conditions
s0 = aircraft_data(4:15);
sdot0 = zeros(12,1);
w_dot0=0;
Vto = sqrt(s0(1)^2 + s0(2)^2 + s0(3)^2);    % Vto

% control actions values
% da = aircraft_data(57);
% dr = aircraft_data(58);
% de = aircraft_data(59);
% dth = aircraft_data(60);
dc = [ aircraft_data(57:59) * pi/180 ; aircraft_data(60)];

% gravity, mass % inertia
m = aircraft_data(51);
g = aircraft_data(52);
M_0 = [0; 0; 0];
Fg_0 = m*g*[sin(s0(8)); -cos(s0(8))*sin(s0(7)); -cos(s0(8))*cos(s0(7))];

rho = aircraft_data(52);
Ixx = aircraft_data(53);
Iyy = aircraft_data(54);
Izz = aircraft_data(55);
Ixz = aircraft_data(56);    Ixy=0;    Iyz=0;
I = [Ixx , -Ixy , -Ixz ;...
     -Ixy , Iyy , -Iyz ;...
     -Ixz , -Iyz , Izz];
invI=inv(I);

% stability derivatives Longitudinal motion
SD_Long = aircraft_data(21:36);
SD_Long_final = SD_Long;

```

```
% stability derivatives Lateral motion
SD_Lat_dash = aircraft_data(37:50);
run('LateralFunc.m');
SD_Lat_final = SD_Lat;

K = struct('u',s0(1),'v',s0(2),'w',s0(3),'p',s0(4),'q',s0(5),...
          'r',s0(6),'phi',s0(7),'theta',s0(8),'psi',s0(9),'x',s0(10),...
          'y',s0(11),'z',s0(12),'alpha',s0(8),'beta',0,'V_t',Vto,'w_dot',0); % used
in delay box in simulink as initial values to bus selector
%% Matrices
Xu    = SD_Long_final(1);
Xw    = SD_Long_final(4);
Xde   = SD_Long_final(11);
Xdth  = SD_Long_final(14);
Zu    = SD_Long_final(2);
Zw    = SD_Long_final(5);
Zwd   = SD_Long_final(7);
Zq    = SD_Long_final(8);
Zde   = SD_Long_final(12);
Zdth  = SD_Long_final(15);
Mu    = SD_Long_final(3);
Mw    = SD_Long_final(6);
Mwd   = SD_Long_final(9);
Mq    = SD_Long_final(10);
Mde   = SD_Long_final(13);
Mdth  = SD_Long_final(16);

Yv    = SD_Lat_final(1);
Yp    = 0;
Yr    = 0;
Lv    = SD_Lat_final(2);
Nv    = SD_Lat_final(3);
Lp    = SD_Lat_final(4);
Np    = SD_Lat_final(5);
Lr    = SD_Lat_final(6);
Nr    = SD_Lat_final(7);
Yda   = SD_Lat_final(8);
Ydr   = SD_Lat_final(9);
Lda   = SD_Lat_final(10);
Nda   = SD_Lat_final(11);
Ldr   = SD_Lat_final(12);
Ndr   = SD_Lat_final(13);
% delta_states : [d_u d_v d_w d_p d_q d_r d_wdot]
Matrix_states = [Xu    0    Xw    0    0    0    0
                 0     Yv    0     Yp    0     Yr    0
                 Zu    0    Zw    0     Zq    0     Zwd
                 0     Lv    0     Lp    0     Lr    0
                 Mu    0    Mw    0     Mq    0     Mwd
                 0     Nv    0     Np    0     Nr    0];
% delta_controls : [d_aileron d_rudder d_elevator d_thrust]

Matrix_Controllers = [0      0      Xde    Xdth
```

```

        Yda      Ydr      0      0
        0      0      Zde      Zdth
        Lda      Ldr      0      0
        0      0      Mde      Mdth
        Nda      Ndr      0      0    ];

%% RK4 Algorithm Solver
States(:,1) = s0;
    for i = 1:lengths
        if i == 1
            w_dot = w_dot0;
        end
        [F M] =
Airframe_Model(SD_Long_final,SD_Lat_final,[s0;w_dot0],[States(:,i);w_dot],m,I,dc);
        K_1 = RBD_Model(time_V(i)      , States(:,i),F,M,I,m);
        K_2 = RBD_Model(time_V(i)+dt/2 , States(:,i)+dt*K_1/2,F,M,I,m);
        K_3 = RBD_Model(time_V(i)+dt/2 , States(:,i)+dt*K_2/2,F,M,I,m);
        K_4 = RBD_Model(time_V(i)+dt    , States(:,i)+dt*K_3,F,M,I,m);
        States(:,i+1) = States(:,i) + (dt/6)*(K_1+2*K_2+2*K_3+K_4);
        w_dot = K_1(3);
    end
    States=States(:,1:lengths);
    %% State Arrangement
    u = States(1,:);
    v = States(2,:);
    w = States(3,:);
    p_deg = States(4,:)*180/pi;
    q_deg = States(5,:)*180/pi;
    r_deg = States(6,:)*180/pi;
    phi_deg = States(7,:)*180/pi;
    theta_deg = States(8,:)*180/pi;
    psi_deg = States(9,:)*180/pi;
    alpha_deg = atan(w./u)*180/pi;
    beta_deg = asin(v./sqrt(u.^2+v.^2+w.^2))*180/pi;
    x = States(10,:);
    y = States(11,:);
    z = States(12,:);
    %% Simulink Solutuion
    out = sim('Simulator_Part_1');
    %% Plotting
    % Get the screen size
    screen_size = get(0, 'ScreenSize');
    % Define the width and height of the figure
    fig_width = 0.8 * screen_size(3); % Set the width to 80% of the screen width
    fig_height = 0.5 * fig_width; % Set the height to be half the width
    % Create a new figure window and set its size and position
    fig1 = figure();
    fig1.Position(1) = (screen_size(3) - fig_width) / 2; % Center the figure
horizontally
    fig1.Position(2) = (screen_size(4) - fig_height) / 2; % Center the figure
vertically
    fig1.Position(3) = fig_width; % Set the width of the figure
    fig1.Position(4) = fig_height; % Set the height of the figure

```

```

figure(1)
plot3(x,y,z);
hold on
plot3(out.Positions(:,1),out.Positions(:,2),out.Positions(:,3));
title('Trajectory')
legend("RK4", "Sim")
% Create a new figure window and set its size and position
fig2 = figure();
fig2.Position(1) = (screen_size(3) - fig_width) / 2; % Center the figure
horizontally
fig2.Position(2) = (screen_size(4) - fig_height) / 2; % Center the figure
vertically
fig2.Position(3) = fig_width; % Set the width of the figure
fig2.Position(4) = fig_height; % Set the height of the figure
figure(2)
subplot(4,3,1)
plot(time_V,u)
hold on
plot(out.tout,out.Velocities(:,1))
title('u (ft/sec)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,2)
plot(time_V,beta_deg)
hold on
plot(out.tout,out.beta*180/pi)
title('\beta (deg)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,3)
plot(time_V,alpha_deg)
hold on
plot(out.tout,out.alpha*180/pi)
title('\alpha (deg)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,4)
plot(time_V,p_deg)
hold on
plot(out.tout,out.AngularRates(:,1)*180/pi)
title('p (deg/sec)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,5)
plot(time_V,q_deg)
hold on
plot(out.tout,out.AngularRates(:,2)*180/pi)
title('q (deg/sec)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,6)
plot(time_V,r_deg)

```



```
hold on
plot(out.tout,out.AngularRates(:,3)*180/pi)
title('r (deg/sec)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,7)
plot(time_V,phi_deg)
hold on
plot(out.tout,out.EulerAngles(:,1)*180/pi)
title('\phi (deg)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,8)
plot(time_V,theta_deg)
hold on
plot(out.tout,out.EulerAngles(:,2)*180/pi)
title('\theta (deg)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,9)
plot(time_V,psi_deg)
hold on
plot(out.tout,out.EulerAngles(:,3)*180/pi)
title('\psi (deg)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,10)
plot(time_V,x)
hold on
plot(out.tout,out.Positions(:,1))
title('x (ft)')
xlabel('time (sec)')
legend("RK4", "Sim")
subplot(4,3,11)
plot(time_V,y)
hold on
plot(out.tout,out.Positions(:,2))
title('y (ft)')
xlabel('time (sec)')
legend("RK4", "Sim")]
subplot(4,3,12)
plot(time_V,z)
hold on
plot(out.tout,out.Positions(:,3))
title('z (ft)')
xlabel('time (sec)')
legend("RK4", "Sim")
```

VI. Task 3: Function: Airplane's RBD Model

```
function states_dot = RBD_Model(t,states_vec,F,M,I,Mass)

u = states_vec(1);
v = states_vec(2);
w = states_vec(3);

p = states_vec(4);
q = states_vec(5);
r = states_vec(6);

phi = states_vec(7);
theta = states_vec(8);
psi = states_vec(9);

x = states_vec(10);
y = states_vec(11);
z = states_vec(12);

states_dot(1:3,1) = 1/Mass.*F - cross([p;q;r],[u;v;w]);
states_dot(4:6,1) = inv(I)*(M-cross([p;q;r],I*[p;q;r]));
states_dot(7:9,1) = [1 ,sin(phi)*tan(theta),cos(phi)*tan(theta);...
                    0,cos(phi),-sin(phi);...
                    0,sin(phi)/cos(theta),cos(phi)/cos(theta)] * [p;q;r];
states_dot(10:12,1) = eul2rotm([psi theta phi]) * [u;v;w];

end
```

VII. Task 3: Function: Airframe Model

```
function [F M] =
Airframe_Model(long_der,Lat_der,States0,Current_States,Mass,I,input)

    g = 32.17405;
    Ixx = I(1);
    Iyy = I(5);
    Izz = I(9);

    Xu    = long_der(1);
    Xw    = long_der(4);
    Xde   = long_der(11);
    Xdth  = long_der(14);
    Zu    = long_der(2);
    Zw    = long_der(5);
    Zwd   = long_der(7);
    Zq    = long_der(8);
    Zde   = long_der(12);
    Zdth  = long_der(15);
    Mu    = long_der(3);
    Mw    = long_der(6);
    Mwd   = long_der(9);
    Mq    = long_der(10);
    Mde   = long_der(13);
    Mdth  = long_der(16);

    Yv    = Lat_der(1);
    Yp    = 0;
    Yr    = 0;
    Lv    = Lat_der(2);
    Nv    = Lat_der(3);
    Lp    = Lat_der(4);
    Np    = Lat_der(5);
    Lr    = Lat_der(6);
    Nr    = Lat_der(7);
    Yda   = Lat_der(8);
    Ydr   = Lat_der(9);
    Lda   = Lat_der(10);
    Nda   = Lat_der(11);
    Ldr   = Lat_der(12);
    Ndr   = Lat_der(13);

    d_u   = Current_States(1)-States0(1);
    d_v   = Current_States(2)-States0(2);
    d_w   = Current_States(3)-States0(3);
    d_p   = Current_States(4)-States0(4);
    d_q   = Current_States(5)-States0(5);

    d_r   = Current_States(6)-States0(6);
```

```

d_wdot = Current_States(13)-States0(13);
d_aileron = input(1);
d_rudder = input(2);
d_elevator = input(3);
d_thrust = input(4);

delta_states = [d_u d_v d_w d_p d_q d_r d_wdot]';

Matrix_states = [Xu 0 Xw 0 0 0 0
                 0 Yv 0 Yp 0 Yr 0
                 Zu 0 Zw 0 Zq 0 Zwd
                 0 Lv 0 Lp 0 Lr 0
                 Mu 0 Mw 0 Mq 0 Mwd
                 0 Nv 0 Np 0 Nr 0];

delta_controls = [d_aileron d_elevator d_thrust d_rudder]';

Matrix_controls = [0 Xde Xdth 0
                   Yda 0 0 Ydr
                   0 Zde Zdth 0
                   Lda 0 0 Ldr
                   0 Mde Mdth 0
                   Nda 0 0 Ndr];

delta=Matrix_states*delta_states + Matrix_controls*delta_controls;

delta_F = Mass * delta(1:3);
delta_M =[Ixx;Iyy;Izz].*delta(4:6);

gravity0 = [Mass*g*sin(States0(8))
            -Mass*g*cos(States0(8))*sin(States0(7))
            -Mass*g*cos(States0(8))*cos(States0(7))];

gravity = [-Mass*g*sin(Current_States(8))
            Mass*g*cos(Current_States(8))*sin(Current_States(7))
            Mass*g*cos(Current_States(8))*cos(Current_States(7))];

F = delta_F + gravity0 + gravity;

M =delta_M ;
end

```

VIII. Task 3: Function: Lateral

```
% this script convert dash to normal derivitavies

G=1/(1-(Ixz^2/(Ixx*Izz)));

Trans_Mat=[G G*Ixz/Ixx;G*Ixz/Izz G];
Trans_Mat_inv=inv(Trans_Mat);

Y_v = SD_Lat_dash(1);
Y_beta = SD_Lat_dash(2);
Y_da = SD_Lat_dash(9)*Vto;
Y_dr = SD_Lat_dash(10)*Vto;

L_beta_dash = SD_Lat_dash(3);
L_p_dash = SD_Lat_dash(5);
L_r_dash = SD_Lat_dash(7);
L_dr_dash = SD_Lat_dash(13);
L_da_dash = SD_Lat_dash(11);

N_beta_dash = SD_Lat_dash(4);
N_p_dash = SD_Lat_dash(6);
N_r_dash = SD_Lat_dash(8);
N_dr_dash = SD_Lat_dash(14);
N_da_dash = SD_Lat_dash(12);

LN_v = Trans_Mat_inv*[L_beta_dash;N_beta_dash]./Vto;
LN_p = Trans_Mat_inv*[L_p_dash;N_p_dash];
LN_r = Trans_Mat_inv*[L_r_dash;N_r_dash];
LN_dr =Trans_Mat_inv*[L_dr_dash;N_dr_dash];
LN_da =Trans_Mat_inv*[L_da_dash;N_da_dash];

SD_Lat = [Y_v LN_v(1) LN_v(2) LN_p(1) LN_p(2) LN_r(1) LN_r(2) Y_da Y_dr LN_da(1)
LN_da(2) LN_dr(1) LN_dr(2)]';
```

IX. Task 4: Longitudinal main Code

```

clc;clearvars;close all;

addpath('Functions');
addpath('Sub_Scripts');

DerivativesAndData;
Aircraft_Simulator;
LonitudinalStateSpace;

states_0 = s0;

%% Longitudinal
FullSystem = ss(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full);
LongSystem = ss(A_Long,B_Long,C_Long,D_Long);
ShortSystem = ss(A_Short,B_Short,C_Short,D_Short);

%Extract full linearized model transfer functions:
[~,Full_Long_u_de] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,1,1);
[~,Full_Long_w_de] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,1,2);
[~,Full_Long_q_de] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,1,3);
[~,Full_Long_theta_de] =
SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,1,4);
[~,Full_Long_u_dT] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,2,1);
[~,Full_Long_w_dT] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,2,2);
[~,Full_Long_q_dT] = SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,2,3);
[~,Full_Long_theta_dT] =
SS2TF(A_Long_Full,B_Long_Full,C_Long_Full,D_Long_Full,2,4);

%Extract Long Mode Approximated model transfer functions:
[~,Long_Period_u_de] = SS2TF(A_Long,B_Long,C_Long,D_Long,1,1);
[~,Long_Period_theta_de] = SS2TF(A_Long,B_Long,C_Long,D_Long,1,2);
[~,Long_Period_u_dT] = SS2TF(A_Long,B_Long,C_Long,D_Long,2,1);
[~,Long_Period_theta_dT] = SS2TF(A_Long,B_Long,C_Long,D_Long,2,2);

%Extract Short Mode Approximated model transfer functions:
[~,Short_Period_w_de] = SS2TF(A_Short,B_Short,C_Short,D_Short,1,1);
[~,Short_Period_alpha_de] = SS2TF(A_Short2,B_Short2,C_Short,D_Short,1,1);
[~,Short_Period_q_de] = SS2TF(A_Short,B_Short,C_Short,D_Short,1,2);
[~,Short_Period_w_dT] = SS2TF(A_Short,B_Short,C_Short,D_Short,2,1);
[~,Short_Period_alpha_dT] = SS2TF(A_Short2,B_Short2,C_Short,D_Short,2,1);
[~,Short_Period_q_dT] = SS2TF(A_Short,B_Short,C_Short,D_Short,2,2);

%% Plotting:
Plot1; Plot2; Plot3; Plot4;
%% Plotting: % only one colntroller: DE = [1 5 10 25]*pi/180; for i =
1:length(DE) de = DE(i); dc = [0 0 de 0]; out = sim('Simulator_Part_1'); filename
= 'figures'; Plot_de_response end
DT = [2000 6000 10000]; for i = 1:length(DT) dT = DT(i); dc = [0 0 0 dT]; out =
sim('Simulator_Part_1'); filename = 'figures'; Plot_dT_response end
%bode_rlocus_plots;
%% Bonus Part (Compare Full linearized model to Model linearizer)
% Specify the model name
model = 'Simulator_Part_1';

% Create the analysis I/O variable IOs1

```

```

io(1) = linio('Simulator_Part_1/Demux',3,'openinput');%de
io(2) = linio('Simulator_Part_1/Demux',4,'openinput');%dth
io(3) = linio('Simulator_Part_1/Bus Selector',13,'openoutput');%alpha
io(4) = linio('Simulator_Part_1/Bus Selector',11,'openoutput');%theta
io(5) = linio('Simulator_Part_1/Bus Selector',5,'openoutput');%q
io(6) = linio('Simulator_Part_1/Bus Selector',3,'openoutput');%w
io(7) = linio('Simulator_Part_1/Bus Selector',1,'openoutput');%u

% Use the model initial condition as operating point
op = operpoint(model);
sys_ModelLinearizer = linearize(model,io,op);
[~,ModelLinearizerTF_u_de] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,1,5);
[~,ModelLinearizerTF_u_dT] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,2,5);
[~,ModelLinearizerTF_w_de] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,1,4);
[~,ModelLinearizerTF_w_dT] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,2,4);
[~,ModelLinearizerTF_alpha_de] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,1,1);
[~,ModelLinearizerTF_alpha_dT] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,2,1);
[~,ModelLinearizerTF_q_de] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,1,3);
[~,ModelLinearizerTF_q_dT] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,2,3);
[~,ModelLinearizerTF_theta_de] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,1,2);
[~,ModelLinearizerTF_theta_dT] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_Model
Linearizer.D,2,2);
bode_rlocus_plots;
Plot5;

```

X. Task 4: Lateral main Code

```

clc;clearvars;close all;

addpath('Functions');
addpath('Sub_Scripts');

DerivativesAndData;
Aircraft_Simulator;

LateralStateSpace;

states_0 = s0;

%% Laterals Models
Full_Lat_System = ss(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full);
Spiral3dSystem = ss(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3);
Dutch3dSystem = ss(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3);
Dutch2dSystem = ss(A_Dutch2,B_Dutch2,C_Dutch2,D_Dutch2);
RollSystem = ss(A_Roll,B_Roll,C_Roll,D_Roll);

%Extract full linearized model transfer functions:
[~,Full_Lat_v_da] = SS2TF(A_Lat_Full_v,B_Lat_Full_v,C_Lat_Full,D_Lat_Full,1,1);

[~,Full_Lat_beta_da] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,1,1);
[~,Full_Lat_p_da] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,1,2);
[~,Full_Lat_r_da] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,1,3);
[~,Full_Lat_phi_da] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,1,4);
[~,Full_Lat_psi_da] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,1,5);

[~,Full_Lat_v_dr] = SS2TF(A_Lat_Full_v,B_Lat_Full_v,C_Lat_Full,D_Lat_Full,2,1);

[~,Full_Lat_beta_dr] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,2,1);
[~,Full_Lat_p_dr] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,2,2);
[~,Full_Lat_r_dr] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,2,3);
[~,Full_Lat_phi_dr] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,2,4);
[~,Full_Lat_psi_dr] = SS2TF(A_Lat_Full,B_Lat_Full,C_Lat_Full,D_Lat_Full,2,5);

%Extract Spiral3d Mode Approximated model transfer functions:
[~,Spiral3d_p_da] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,1,1);
[~,Spiral3d_r_da] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,1,2);
[~,Spiral3d_phi_da] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,1,3);
[~,Spiral3d_p_dr] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,2,1);
[~,Spiral3d_r_dr] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,2,2);
[~,Spiral3d_phi_dr] = SS2TF(A_Spiral3,B_Spiral3,C_Spiral3,D_Spiral3,2,3);

%Extract Dutch3d Mode Approximated model transfer functions:
[~,Dutch3d_v_da] = SS2TF(A_Dutch3_v,B_Dutch3_v,C_Dutch3,D_Dutch3,1,1);
[~,Dutch3d_beta_da] = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,1,1);
[~,Dutch3d_p_da] = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,1,2);
[~,Dutch3d_r_da] = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,1,3);
[~,Dutch3d_v_dr] = SS2TF(A_Dutch3_v,B_Dutch3_v,C_Dutch3,D_Dutch3,2,1);

```



```
[~,Dutch3d_beta_dr] = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,2,1);
[~,Dutch3d_p_dr]    = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,2,2);
[~,Dutch3d_r_dr]    = SS2TF(A_Dutch3,B_Dutch3,C_Dutch3,D_Dutch3,2,3);
```

```
%Extract Dutch2d Mode Approximated model transfer functions:
```

```
[~,Dutch2d_v_da] = SS2TF(A_Dutch2_v,B_Dutch2_v,C_Dutch2,D_Dutch2,1,1);
[~,Dutch2d_beta_da] = SS2TF(A_Dutch2,B_Dutch2,C_Dutch2,D_Dutch2,1,1);
[~,Dutch2d_r_da]    = SS2TF(A_Dutch2,B_Dutch2,C_Dutch2,D_Dutch2,1,2);
[~,Dutch2d_v_dr] = SS2TF(A_Dutch2_v,B_Dutch2_v,C_Dutch2,D_Dutch2,2,1);
[~,Dutch2d_beta_dr] = SS2TF(A_Dutch2,B_Dutch2,C_Dutch2,D_Dutch2,2,1);
[~,Dutch2d_r_dr]    = SS2TF(A_Dutch2,B_Dutch2,C_Dutch2,D_Dutch2,2,2);
```

```
%Extract Roll1d Mode Approximated model transfer functions:
```

```
[~,Roll_p_da] = SS2TF(A_Roll,B_Roll,C_Roll,D_Roll,1,1);
```

```
%% Plotting:
```

```
% only one controller:
```

```
DA = [1 5 10 25]*pi/180;
```

```
for i = 1:length(DA)
    da = DA(i);
    dc = [da 0 0 0];
    out = sim('Simulator_Part_1');
    filename = 'figures\Lateral Results\delta_a';
    Plot_da_response
end
```

```
DR = [1 5 10 25]*pi/180;
```

```
for i = 1:length(DR)
    dr = DR(i);
    dc = [0 dr 0 0];
    out = sim('Simulator_Part_1');
    filename = 'figures\Lateral Results\delta_r';
    Plot_dr_response
end
```

```
Bode_filename = 'figures\Lateral Results\Bode Plots';
```

```
RL_filename = 'figures\Lateral Results\Root Locus';
```

```
bode_rlocus_plots
```

```
close all;
```

```
%% Bonus Part (Compare Full linearized model to Model linearizer)
```

```
% Specify the model name
```

```
model = 'Simulator_Part_1';
```

```
% Create the analysis I/O variable IOs1
```

```
io(1) = linio('Simulator_Part_1/Demux',2,'openinput'); %dr
io(2) = linio('Simulator_Part_1/Demux',1,'openinput'); %da
io(3) = linio('Simulator_Part_1/Bus Selector',12,'openoutput');%psi
io(4) = linio('Simulator_Part_1/Bus Selector',10,'openoutput'); %phi
```

```

io(5) = linio('Simulator_Part_1/Bus Selector',6,'openoutput'); %r
io(6) = linio('Simulator_Part_1/Bus Selector',4,'openoutput'); %p
io(7) = linio('Simulator_Part_1/Bus Selector',2,'openoutput'); %v
io(8) = linio('Simulator_Part_1/Bus Selector',14,'openoutput'); %beta

% Use the model initial condition as operating point
op = operpoint(model);

sys_ModelLinearizer = linearize(model,io,op);

[~,ModelLinearizerTF_v_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,5);
[~,ModelLinearizerTF_beta_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,6);
[~,ModelLinearizerTF_p_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,4);
[~,ModelLinearizerTF_r_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,3);
[~,ModelLinearizerTF_phi_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,2);
[~,ModelLinearizerTF_psi_da] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,2,1);

[~,ModelLinearizerTF_v_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,5);
[~,ModelLinearizerTF_beta_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,6);
[~,ModelLinearizerTF_p_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,4);
[~,ModelLinearizerTF_r_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,3);
[~,ModelLinearizerTF_phi_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,2);
[~,ModelLinearizerTF_psi_dr] =
SS2TF(sys_ModelLinearizer.A,sys_ModelLinearizer.B,sys_ModelLinearizer.C,sys_ModelLinearizer.D,1,1);

```

I. Task 3: Function: Lateral

```
% this script convert dash to normal derivitavies

G=1/(1-(Ixz^2/(Ixx*Izz)));

Trans_Mat=[G G*Ixz/Ixx;G*Ixz/Izz G];
Trans_Mat_inv=inv(Trans_Mat);

Y_v = SD_Lat_dash(1);
Y_beta = SD_Lat_dash(2);
Y_da = SD_Lat_dash(9)*Vto;
Y_dr = SD_Lat_dash(10)*Vto;

L_beta_dash = SD_Lat_dash(3);
L_p_dash = SD_Lat_dash(5);
L_r_dash = SD_Lat_dash(7);
L_dr_dash = SD_Lat_dash(13);
L_da_dash = SD_Lat_dash(11);

N_beta_dash = SD_Lat_dash(4);
N_p_dash = SD_Lat_dash(6);
N_r_dash = SD_Lat_dash(8);
N_dr_dash = SD_Lat_dash(14);
N_da_dash = SD_Lat_dash(12);

LN_v = Trans_Mat_inv*[L_beta_dash;N_beta_dash]./Vto;
LN_p = Trans_Mat_inv*[L_p_dash;N_p_dash];
LN_r = Trans_Mat_inv*[L_r_dash;N_r_dash];
LN_dr =Trans_Mat_inv*[L_dr_dash;N_dr_dash];
LN_da =Trans_Mat_inv*[L_da_dash;N_da_dash];

SD_Lat = [Y_v LN_v(1) LN_v(2) LN_p(1) LN_p(2) LN_r(1) LN_r(2) Y_da Y_dr LN_da(1)
LN_da(2) LN_dr(1) LN_dr(2)]';
```